AI Resume Builder with ATS - Full Project Source

Generated 2026-09-05

149 files

TOC

1. .agents\skills\prisma-cli\references\db-execute.md
2. .agents\skills\prisma-cli\references\db-pull.md
3. .agents\skills\prisma-cli\references\db-push.md
4. .agents\skills\prisma-cli\references\db-seed.md
5. .agents\skills\prisma-cli\references\debug.md
6. .agents\skills\prisma-cli\references\dev.md
7. .agents\skills\prisma-cli\references\format.md
8. .agents\skills\prisma-cli\references\generate.md
9. .agents\skills\prisma-cli\references\init.md
10. .agents\skills\prisma-cli\references\mcp.md
11. .agents\skills\prisma-cli\references\migrate-deploy.md
12. .agents\skills\prisma-cli\references\migrate-dev.md
13. .agents\skills\prisma-cli\references\migrate-diff.md
14. .agents\skills\prisma-cli\references\migrate-reset.md
15. .agents\skills\prisma-cli\references\migrate-resolve.md
16. .agents\skills\prisma-cli\references\migrate-status.md
17. .agents\skills\prisma-cli\references\studio.md
18. .agents\skills\prisma-cli\references\validate.md
19. .agents\skills\prisma-cli\SKILL.md
20. .agents\skills\prisma-client-api\references\client-methods.md
21. .agents\skills\prisma-client-api\references\constructor.md
22. .agents\skills\prisma-client-api\references\filters.md
23. .agents\skills\prisma-client-api\references\model-queries.md
24. .agents\skills\prisma-client-api\references\query-options.md
25. .agents\skills\prisma-client-api\references\raw-queries.md
26. .agents\skills\prisma-client-api\references\relations.md
27. .agents\skills\prisma-client-api\references\transactions.md
28. .agents\skills\prisma-client-api\SKILL.md
29. .agents\skills\prisma-compute\references\app-deploy-cli.md
30. .agents\skills\prisma-compute\references\compute-config.md
31. .agents\skills\prisma-compute\references\create-prisma.md
32. .agents\skills\prisma-compute\references\frameworks.md
33. .agents\skills\prisma-compute\references\sdk-api.md
34. .agents\skills\prisma-compute\references\troubleshooting.md
35. .agents\skills\prisma-compute\SKILL.md
36. .agents\skills\prisma-database-setup\references\cockroachdb.md
37. .agents\skills\prisma-database-setup\references\mongodb.md
38. .agents\skills\prisma-database-setup\references\mysql.md
39. .agents\skills\prisma-database-setup\references\postgresql.md
40. .agents\skills\prisma-database-setup\references\prisma-client-setup.md

41. .agents\skills\prisma-database-setup\references\prisma-postgres.md
42. .agents\skills\prisma-database-setup\references\sqlite.md
43. .agents\skills\prisma-database-setup\references\sqlserver.md
44. .agents\skills\prisma-database-setup\SKILL.md
45. .agents\skills\prisma-driver-adapter-implementation\SKILL.md
46. .agents\skills\prisma-mongodb-upgrade\references\client-api-mapping.md
47. .agents\skills\prisma-mongodb-upgrade\references\decision-stay-or-migrate.md
48. .agents\skills\prisma-mongodb-upgrade\references\migrations-mapping.md
49. .agents\skills\prisma-mongodb-upgrade\references\schema-contract-mapping.md
50. .agents\skills\prisma-mongodb-upgrade\references\verify-cutover-checklist.md
51. .agents\skills\prisma-mongodb-upgrade\SKILL.md
52. .agents\skills\prisma-postgres-setup\references\api-basics.md
53. .agents\skills\prisma-postgres-setup\references\auth.md
54. .agents\skills\prisma-postgres-setup\references\endpoints.md
55. .agents\skills\prisma-postgres-setup\references\prisma7-client.md
56. .agents\skills\prisma-postgres-setup\SKILL.md
57. .agents\skills\prisma-postgres\references\console-and-connections.md
58. .agents\skills\prisma-postgres\references\create-db-cli.md
59. .agents\skills\prisma-postgres\references\management-api-sdk.md
60. .agents\skills\prisma-postgres\references\management-api.md
61. .agents\skills\prisma-postgres\SKILL.md
62. .agents\skills\prisma-upgrade-v7\references\accelerate-users.md
63. .agents\skills\prisma-upgrade-v7\references\driver-adapters.md
64. .agents\skills\prisma-upgrade-v7\references\env-variables.md
65. .agents\skills\prisma-upgrade-v7\references\esm-support.md
66. .agents\skills\prisma-upgrade-v7\references\prisma-config.md
67. .agents\skills\prisma-upgrade-v7\references\removed-features.md
68. .agents\skills\prisma-upgrade-v7\references\schema-changes.md
69. .agents\skills\prisma-upgrade-v7\SKILL.md
70. .gitignore
71. AGENTS.md
72. backend\app__init__.py
73. backend\app\api__init__.py
74. backend\app\api\ai_routes.py
75. backend\app\api\ats.py
76. backend\app\api\export.py
77. backend\app\api\resume.py
78. backend\app\api\resumes.py
79. backend\app\database.py
80. backend\app\models__init__.py
81. backend\app\models\resume.py
82. backend\app\schemas__init__.py
83. backend\app\schemas\resume.py
84. backend\app\services__init__.py
85. backend\app\services\ai_service.py
86. backend\app\services\ats_scorer.py
87. backend\app\services\export_service.py
88. backend\app\services\resume_parser.py
89. backend\app\services\skill_synonyms.py
90. backend\main.py
91. backend\render.yaml
92. backend\requirements.txt
93. backend\tests\conftest.py
94. backend\tests\test_ai_service.py
95. backend\tests\test_api_resumes.py
96. backend\tests\test_ats_scorer.py
97. backend\tests\test_export.py
98. backend\tests\test_resume_parser.py
99. CLAUDE.md
100. database\docker-compose.yml
101. database\README.md
102. database\seed.py
103. frontend\next-env.d.ts
104. frontend\next.config.ts

105. frontend\package.json
106. frontend\public\file.svg
107. frontend\public\globe.svg
108. frontend\public\next.svg
109. frontend\public\vercel.svg
110. frontend\public\window.svg
111. frontend\src\app\analyzer\page.tsx
112. frontend\src\app\builder\page.tsx
113. frontend\src\app\globals.css
114. frontend\src\app\layout.tsx
115. frontend\src\app\matcher\page.tsx
116. frontend\src\app\page.tsx
117. frontend\src\components\AtsScoreCard.tsx
118. frontend\src\components\Navbar.tsx
119. frontend\src\components\ResumeForm.tsx
120. frontend\src\components\ResumePreview.tsx
121. frontend\src\lib\api-client.ts
122. frontend\src\lib\resume-text.ts
123. frontend\src\types\index.ts
124. frontend\src\types\resume.ts
125. frontend\tsconfig.json
126. mobile\.claude\settings.json
127. mobile\.gitignore
128. mobile\AGENTS.md
129. mobile\app.config.js
130. mobile\App.tsx
131. mobile\CLAUDE.md
132. mobile\eas.json
133. mobile\index.ts
134. mobile\package.json
135. mobile\src\api.ts
136. mobile\src\components.tsx
137. mobile\src\navigation.tsx
138. mobile\src\pdf.ts
139. mobile\src\resumeText.ts
140. mobile\src\screens\Builder.tsx
141. mobile\src\screens\Dashboard.tsx
142. mobile\src\screens\Matcher.tsx
143. mobile\src\screens\Preview.tsx
144. mobile\src\theme.ts
145. mobile\src\types.ts
146. mobile\src\userStore.ts
147. mobile\tsconfig.json
148. README.md
149. skills-lock.json

1. .agents\skills\prisma-cli\references\db-execute.md

````
# prisma db execute

Execute native commands (SQL) to your database.

## Command

```bash
prisma db execute [options]
```

## What It Does

- Connects to your database using the configured datasource
- Executes a script provided via file (`--file`) or stdin (`--stdin`)
- Useful for running raw SQL, maintenance tasks, or applying diffs from `migrate diff`
- Not supported on MongoDB

## Options

| Option | Description |
|--------|-------------|
| `--file` | Path to a file containing the script to execute |
| `--stdin` | Use terminal standard input as the script |
| `--config` | Custom path to your Prisma config file |
````

Current Option Surface

`prisma db execute` uses the datasource configured in `prisma.config.ts`. Use `--config` if you need a separate config file for another environment.

Examples

Execute from file

```bash
prisma db execute --file ./script.sql
```

Execute from stdin

```bash
echo "TRUNCATE TABLE User;" | prisma db execute --stdin
```

Execute `migrate diff` output

Pipe the output of `migrate diff` directly to the database:

```bash
prisma migrate diff \
  --from-empty \
  --to-schema prisma/schema.prisma \
  --script \
| prisma db execute --stdin
```

Configuration

Uses `datasource` from `prisma.config.ts`:

```typescript
export default defineConfig({
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

Use Cases

- **Manual Migrations**: Applying raw SQL changes
- **Data Maintenance**: Truncating tables, cleaning up data
- **Schema Synchronization**: Applying `migrate diff` scripts
- **Debugging**: Running test queries (though typically not for fetching data)

Limitations

- **No Data Return**: The command reports success/failure, not query results (rows). Use Prisma Client or `prisma studio` to view data.
- **SQL Only**: Primarily for SQL databases.

2. .agents\skills\prisma-cli\references\db-pull.md

````
# prisma db pull

Introspects an existing database and updates your Prisma schema to reflect its structure.

## Command

```bash
prisma db pull [options]
```

## What It Does

- Connects to your database
- Reads the database schema (tables, columns, relations, indexes)
- Updates `schema.prisma` with corresponding Prisma models
- For MongoDB, samples data to infer schema

## Options

| Option | Description |
|--------|-------------|
| `--force` | Ignore current Prisma schema file |
| `--print` | Print the introspected Prisma schema to stdout |
| `--schema` | Custom path to your Prisma schema |
| `--config` | Custom path to your Prisma config file |
| `--url` | Override the datasource URL from the Prisma config file |
| `--composite-type-depth` | Specify the depth for introspecting composite types (default: -1 for infinite, 0 = off) |
| `--schemas` | Specify the database schemas to introspect |
| `--local-d1` | Generate a Prisma schema from a local Cloudflare D1 database |

## Examples

### Basic introspection

```bash
prisma db pull
```

### Preview without writing

```bash
prisma db pull --print
```

Outputs schema to terminal for review.

### Force overwrite

```bash
prisma db pull --force
```

Replaces schema file, losing any manual customizations.

## Prerequisites

Configure database connection in `prisma.config.ts`:

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## Workflow

### Starting from existing database

1. Initialize Prisma:
   ```bash
   prisma init
   ```

2. Configure database URL

3. Pull schema:
   ```bash
   prisma db pull
   ```

4. Review and customize generated schema

5. Generate client:
   ```bash
````
````

   prisma generate
   ```

### Syncing changes from database

When database changes are made outside Prisma:

```bash
prisma db pull
prisma generate
```

Generated Schema Example

Database tables become Prisma models:

```sql
-- Database tables
CREATE TABLE users (
  id SERIAL PRIMARY KEY,
  email VARCHAR(255) UNIQUE NOT NULL,
  name VARCHAR(100)
);

CREATE TABLE posts (
  id SERIAL PRIMARY KEY,
  title VARCHAR(255) NOT NULL,
  author_id INTEGER REFERENCES users(id)
);
```

Becomes:

```prisma
model users {
  id    Int     @id @default(autoincrement())
  email String  @unique @db.VarChar(255)
  name  String? @db.VarChar(100)
  posts posts[]
}

model posts {
  id        Int    @id @default(autoincrement())
  title     String @db.VarChar(255)
  author_id Int?
  users     users? @relation(fields: [author_id], references: [id])
}
```

Post-Introspection Cleanup

After `db pull`, consider:

1. **Rename models** to PascalCase:
   ```prisma
   model User {  // Was: users
     @@map("users")
   }
   ```

2. **Rename fields** to camelCase:
   ```prisma
   authorId Int? @map("author_id")
   ```

3. **Add relation names** for clarity:
   ```prisma
   author User? @relation("PostAuthor", fields: [authorId], references: [id])
   ```

4. **Add documentation**:
   ```prisma
   /// User account information
   model User {
     /// Primary email for authentication
     email String @unique
   }
   ```

MongoDB Introspection

For MongoDB, `db pull` samples documents to infer schema:

```bash
prisma db pull
```

May require manual refinement since MongoDB is schemaless.

Warning

`db pull` overwrites your schema file. Always:
- Commit current schema before pulling
```

- Use `--print` to preview first
- Backup customizations you want to keep

3. .agents\skills\prisma-cli\references\db-push.md

````
# prisma db push

Pushes schema changes directly to database without creating migrations. Ideal for prototyping.

## Command

```bash
prisma db push [options]
```

## What It Does

- Syncs your Prisma schema to the database
- Creates database if it doesn't exist
- Does NOT create migration files
- Does NOT track migration history

## Options

| Option | Description |
|--------|-------------|
| `--force-reset` | Force a reset of the database before push |
| `--accept-data-loss` | Ignore data loss warnings |
| `--schema` | Custom path to your Prisma schema |
| `--config` | Custom path to your Prisma config file |
| `--url` | Override the datasource URL from the Prisma config file |

### Follow-up Command

- Run `prisma generate` explicitly when you need refreshed client output

## Examples

### Basic push

```bash
prisma db push
```

### Accept data loss

```bash
prisma db push --accept-data-loss
```

Required when changes would delete data (dropping columns, etc.)

### Force reset

```bash
prisma db push --force-reset
```

Completely resets database and applies schema.

### Full workflow

```bash
prisma db push
prisma generate
```

## When to Use

- **Prototyping** - Rapid schema iteration
- **Local development** - Quick schema changes
- **MongoDB** - Primary workflow (migrations not supported)
- **Testing** - Setting up test databases

## When NOT to Use

- **Production** - Use `migrate deploy`
- **Team collaboration** - Use migrations for trackable changes
- **When you need rollback** - Migrations provide history

## Comparison with migrate dev

| Feature | db push | migrate dev |
|---------|---------|-------------|
| Creates migration files | No | Yes |
| Tracks history | No | Yes |
| Requires shadow database | No | Yes |
| Speed | Faster | Slower |
| Rollback capability | No | Yes |
| Best for | Prototyping | Development |

## MongoDB Workflow

MongoDB doesn't support migrations. Use `db push` exclusively:
````

```bash
# Schema changes for MongoDB
prisma db push
prisma generate
```

Common Patterns

Prototyping workflow

```bash
# Make schema changes
# ...

# Push to database
prisma db push

# Generate client
prisma generate

# Test your changes
# Repeat as needed
```

Reset and start fresh

```bash
prisma db push --force-reset
prisma db seed
```

Handling conflicts

If `db push` can't apply changes safely:

```
Error: The following changes cannot be applied:
  - Removing field `email` would cause data loss

Use --accept-data-loss to proceed
```

Decide whether data loss is acceptable, then:

```bash
prisma db push --accept-data-loss
```

Transition to Migrations

When ready for production, switch to migrations:

```bash
# Create baseline migration from current schema
prisma migrate dev --name init
```

Then use `migrate dev` for future changes.

4. .agents\skills\prisma-cli\references\db-seed.md

````
# prisma db seed

Runs your database seed script to populate data.

## Command

```bash
prisma db seed [options]
```

## What It Does

- Executes your configured seed script
- Populates database with initial/test data
- Runs independently (not auto-run by migrations in v7)

## Options

| Option | Description |
|--------|-------------|
| `--config` | Custom path to your Prisma config file |
| `--` | Pass custom arguments to seed script |

## Configuration

Configure seed script in `prisma.config.ts`:

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
    seed: 'tsx prisma/seed.ts',  // Your seed command
  },
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

### Common seed commands

```typescript
// TypeScript with tsx
seed: 'tsx prisma/seed.ts'

// TypeScript with ts-node
seed: 'ts-node prisma/seed.ts'

// JavaScript
seed: 'node prisma/seed.js'
```

## Seed Script Example

```typescript
// prisma/seed.ts
import { PrismaClient } from '../generated/client'

const prisma = new PrismaClient()

async function main() {
  // Create users
  const alice = await prisma.user.upsert({
    where: { email: 'alice@prisma.io' },
    update: {},
    create: {
      email: 'alice@prisma.io',
      name: 'Alice',
      posts: {
        create: {
          title: 'Hello World',
          published: true,
        },
      },
    },
  })

  const bob = await prisma.user.upsert({
    where: { email: 'bob@prisma.io' },
    update: {},
    create: {
      email: 'bob@prisma.io',
      name: 'Bob',
    },
  })
````
````

```
  console.log({ alice, bob })
}

main()
  .then(async () => {
    await prisma.$disconnect()
  })
  .catch(async (e) => {
    console.error(e)
    await prisma.$disconnect()
    process.exit(1)
  })
```

Examples

Run seed

```bash
prisma db seed
```

With custom arguments

```bash
prisma db seed -- --environment development
```

Arguments after `--` are passed to your seed script.

Current Workflow

Run seeding explicitly after migrations when you need seed data:

```bash
prisma migrate dev --name init
prisma generate
prisma db seed  # Must run explicitly
```

Idempotent Seeding

Use `upsert` to make seeds re-runnable:

```typescript
// Good: Can run multiple times
await prisma.user.upsert({
  where: { email: 'alice@prisma.io' },
  update: {},  // Don't change existing
  create: { email: 'alice@prisma.io', name: 'Alice' },
})

// Bad: Fails on second run
await prisma.user.create({
  data: { email: 'alice@prisma.io', name: 'Alice' },
})
```

Common Patterns

Development reset

```bash
prisma migrate reset --force
prisma db seed
```

Conditional seeding

```typescript
// prisma/seed.ts
const count = await prisma.user.count()
if (count === 0) {
  // Only seed if empty
  await seedUsers()
}
```

Environment-specific seeds

```typescript
// prisma/seed.ts
const env = process.env.NODE_ENV || 'development'

if (env === 'development') {
  await seedDevData()
} else if (env === 'test') {
  await seedTestData()
}
```

Best Practices

1. Use `upsert` for idempotent seeds
2. Keep seeds focused and minimal
3. Use realistic but fake data
4. Document required seed data
5. Version control your seed scripts

5. .agents\skills\prisma-cli\references\debug.md

````
# prisma debug

Prints information helpful for debugging and bug reports.

## Command

```bash
prisma debug [options]
```

## What It Does

Outputs details about your Prisma environment, including:
- Prisma CLI version
- Prisma Client version (if installed)
- Engine binaries (Query Engine, Migration Engine, etc.)
- Platform information (OS, Architecture)
- Node.js version
- Configured datasource provider

## Options

| Option | Description |
|--------|-------------|
| `--schema` | Path to schema file |
| `--config` | Custom path to your Prisma config file |

## Example Output

```
prisma               : 7.3.0
@prisma/client       : 7.3.0
Operating System     : darwin
Architecture         : arm64
Node.js              : v20.10.0
TypeScript           : 5.3.3
Query Compiler       : enabled
PSL                  : ...
Schema Engine        : ...
```

## When to Use

- **Troubleshooting**: Checking version mismatches
- **Reporting Issues**: Including environment info in GitHub issues
- **Verifying Installation**: Ensuring correct binaries are downloaded
````

6. .agents\skills\prisma-cli\references\dev.md

````
# prisma dev

Starts a local Prisma Postgres database for development. Provides a PostgreSQL-compatible database that runs entirely on your machine.

## Command

```bash
prisma dev [options]
```

## What It Does

- Starts a local PostgreSQL-compatible database
- Runs in your terminal or as a background process
- Perfect for development and testing
- Easy migration to Prisma Postgres cloud in production

## Options

| Option | Description | Default |
|--------|-------------|---------|
| `--name` / `-n` | Name for the database instance | `default` |
| `--port` / `-p` | HTTP server port | `51213` |
| `--db-port` / `-P` | Database server port | `51214` |
| `--shadow-db-port` | Shadow database port (for migrations) | `51215` |
| `--detach` / `-d` | Run in background | `false` |
| `--debug` | Enable debug logging | `false` |

## Examples

### Start local database

```bash
prisma dev
```

Interactive mode with keyboard shortcuts:
- `q` - Quit
- `h` - Show HTTP URL
- `t` - Show TCP URLs

### Named instance

```bash
prisma dev --name myproject
```

Useful for multiple projects.

### Background mode

```bash
prisma dev --detach
```

Frees your terminal for other commands.

### Custom ports

```bash
prisma dev --port 5000 --db-port 5432
```

## Instance Management

### List all instances

```bash
prisma dev ls
```

Shows all local Prisma Postgres instances with status.

### Start existing instance

```bash
prisma dev start myproject
```

Starts a previously created instance in background.

### Stop instance

```bash
prisma dev stop myproject
```

### Stop with glob pattern

```bash
````
````

```
prisma dev stop "myproject*"
```

Stops all instances matching pattern.

Remove instance

```bash
prisma dev rm myproject
```

Removes instance data from filesystem.

Force remove (stops first)

```bash
prisma dev rm myproject --force
```

Configuration

Configure your `prisma.config.ts` to use local Prisma Postgres:

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
  },
  datasource: {
    // Local Prisma Postgres URL (from prisma dev output)
    url: env('DATABASE_URL'),
  },
})
```

Workflow

1. Start local database:
   ```bash
   prisma dev
   ```

2. In another terminal, run migrations:
   ```bash
   prisma migrate dev
   ```

3. Generate client:
   ```bash
   prisma generate
   ```

4. Run your application

Production Migration

When ready for production, switch to Prisma Postgres cloud:

```bash
prisma init --db
```

Update your `DATABASE_URL` to the cloud connection string.

7. .agents\skills\prisma-cli\references\format.md

````
# prisma format

Formats your Prisma schema file.

## Command

```bash
prisma format [options]
```

## What It Does

- Fixes formatting (indentation, spacing)
- Adds missing back-relations (e.g., adds the other side of a relation)
- Adds missing relation arguments (e.g., `fields`, `references`)
- Sorts fields and attributes (opinionated)

## Options

| Option | Description |
|--------|-------------|
| `--schema` | Path to schema file |
| `--config` | Custom path to your Prisma config file |

## Examples

### Format default schema

```bash
prisma format
```

### Format specific schema

```bash
prisma format --schema=./custom/schema.prisma
```

## Behavior

`prisma format` modifies the file in place. It is equivalent to "Prettier for Prisma schemas" but also has semantic understanding to fix/add
missing schema definitions.

## Use in Editor

Most Prisma editor extensions (VS Code, WebStorm) run `prisma format` automatically on save. This command is useful for:
- CI pipelines (check formatting)
- CLI-based workflows
- Fixing large schema refactors
````

8. .agents\skills\prisma-cli\references\generate.md

````
# prisma generate

Generates assets based on the generator blocks in your Prisma schema, most commonly Prisma Client.

## Command

```bash
prisma generate [options]
```

## Bun Runtime

If you're using Bun, run Prisma with `bunx --bun` so it doesn't fall back to Node.js:

```bash
bunx --bun prisma generate
```

## What It Does

1. Reads your `schema.prisma` file
2. Generates a customized Prisma Client based on your models
3. Outputs to the directory specified in the generator block

## Options

| Option | Description |
|--------|-------------|
| `--schema` | Custom path to your Prisma schema |
| `--config` | Custom path to your Prisma config file |
| `--sql` | Generate typed sql module |
| `--watch` | Watch the Prisma schema and rerun after a change |
| `--generator` | Generator to use (may be provided multiple times) |
| `--no-hints` | Hides the hint messages but still outputs errors and warnings |
| `--require-models` | Do not allow generating a client without models |

## Examples

### Basic generation

```bash
prisma generate
```

### Watch mode (development)

```bash
prisma generate --watch
```

Auto-regenerates when `schema.prisma` changes.

### Specific generator

```bash
prisma generate --generator client
```

### Multiple generators

```bash
prisma generate --generator client --generator zod_schemas
```

### Typed SQL generation

```bash
prisma generate --sql
```

## Schema Configuration

```prisma
generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

### Current Generator Behavior

- `prisma-client` is the standard generator
- `output` is required when using `prisma-client`
- `prisma-client` supports both ESM and CommonJS via `moduleFormat`
- `compilerBuild` supports `fast` and `small` query compiler artifacts
- Use TypeScript `satisfies` for typed query fragments with `prisma-client`
- Import Prisma Client from your generated output path, for example:

```typescript
import { PrismaClient } from '../generated/prisma/client'
````
````

```

### Compiler Build Tuning

Use `compilerBuild` when you need to trade artifact size against the default build:

```prisma
generator client {
  provider      = "prisma-client"
  output        = "../generated"
  compilerBuild = "small"
}
```

- `fast` is the default build for most targets
- `small` is useful for size-constrained targets
- Prisma defaults `vercel-edge` targets to `small`

Common Patterns

After schema changes

```bash
prisma migrate dev --name my_migration
prisma generate
```

Run `prisma generate` whenever you need refreshed client code after schema-changing commands.

CI/CD pipeline

```bash
prisma generate
```

Run before building your application.

Multiple generators

```prisma
generator client {
  provider = "prisma-client"
  output   = "../generated"
}

generator zod {
  provider = "zod-prisma-types"
  output   = "../generated/zod"
}
```

```bash
prisma generate  # Runs all generators
```

Output Structure

After running `prisma generate`, your output directory contains:

```
generated/
├── browser.ts
├── client.ts
├── commonInputTypes.ts
├── models/
├── enums.ts
├── models.ts
└── ...
```

Import the client:

```typescript
import { PrismaClient, Prisma } from '../generated/prisma/client'
```

Import browser-safe types:

```typescript
import { Prisma } from '../generated/prisma/browser'
import { Role } from '../generated/prisma/enums'
import type { UserModel } from '../generated/prisma/models/User'
```
```

9. .agents\skills\prisma-cli\references\init.md

````
# prisma init

Bootstraps a fresh Prisma ORM project in the current directory.

## Command

```bash
prisma init [options]
```

## Bun Runtime

If you're using Bun, run Prisma with `bunx --bun` so it doesn't fall back to Node.js:

```bash
bunx --bun prisma init
```

## What It Creates

- `prisma/schema.prisma` - Your Prisma schema file
- `prisma.config.ts` - TypeScript configuration for Prisma CLI
- `.env` - Environment variables (DATABASE_URL)
- `.gitignore` - Ensures `.env` is ignored and appends the generated client path

## Options

| Option | Description | Default |
|--------|-------------|---------|
| `--datasource-provider` | Database provider: `postgresql`, `mysql`, `sqlite`, `sqlserver`, `mongodb`, `cockroachdb` | `postgresql` |
| `--db` | Provisions a fully managed Prisma Postgres database on the Prisma Data Platform | - |
| `--url` | Define a custom datasource url | - |
| `--generator-provider` | Define the generator provider to use | `prisma-client` |
| `--output` | Define Prisma Client generator output path to use | - |
| `--preview-feature` | Define a preview feature to use | - |
| `--with-model` | Add example model to created schema file | - |

## Examples

### Basic initialization

```bash
prisma init
```

Creates a PostgreSQL project setup.

### SQLite project

```bash
prisma init --datasource-provider sqlite
```

### MySQL with custom URL

```bash
prisma init --datasource-provider mysql --url "mysql://user:password@localhost:3306/mydb"
```

### Prisma Postgres (cloud)

```bash
prisma init --db
```

Opens browser for authentication, creates cloud database instance.

### Add an example model

```bash
prisma init --with-model
```

Adds a starter model to the generated schema.

### With preview features

```bash
prisma init --preview-feature relationJoins --preview-feature fullTextSearch
```

## Generated Schema

```prisma
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}

datasource db {
````
````

````
  provider = "postgresql"
}
```

## Generated Config (Node.js default)

```typescript
// prisma.config.ts
import "dotenv/config";
import { defineConfig } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
  },
  datasource: {
    url: process.env['DATABASE_URL'],
  },
})
```

## Generated Config (Bun)

```typescript
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
  },
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## Next Steps After Init

1. Configure `DATABASE_URL` in `.env` (and let `prisma.config.ts` read it)
2. Define your models in `prisma/schema.prisma`
3. Run `prisma dev` for local development or connect to remote DB
4. Run `prisma migrate dev` to create migrations
5. Run `prisma generate` to generate Prisma Client
6. Run `prisma db seed` explicitly if you want seed data
````
````

10. .agents\skills\prisma-cli\references\mcp.md

````
# prisma mcp

Starts Prisma's MCP server for AI development tools.

## Command

```bash
prisma mcp
```

## What It Does

- Starts a Model Context Protocol (MCP) server for your Prisma project
- Exposes Prisma schema and database context to compatible AI tools
- Helps AI assistants understand models, generate queries, and suggest migrations

## Usage

```bash
prisma mcp
```

## Typical Use Cases

- Connect Prisma to ChatGPT, Claude, or other MCP-aware tools
- Give an AI assistant access to your Prisma schema structure
- Help an agent propose queries, schema updates, and migration steps with project context

## Notes

- Run this from the project that contains your Prisma schema and `prisma.config.ts`
- The command is separate from Prisma Studio and does not open a browser UI
- The MCP server wraps Prisma CLI commands. For exact behavior of commands like `migrate dev` or `migrate reset`, follow the underlying CLI command docs rather than relying only on the MCP tool descriptions.

## References

- [Prisma CLI `mcp` command](https://docs.prisma.io/docs/cli/mcp)
- [Prisma MCP Server](https://www.prisma.io/docs/ai/tools/chatgpt)
````

11. .agents\skills\prisma-cli\references\migrate-deploy.md

````
# prisma migrate deploy

Applies pending migrations in production/staging environments.

## Command

```bash
prisma migrate deploy
```

## What It Does

- Applies all pending migrations from `prisma/migrations/`
- Updates `_prisma_migrations` table
- Does NOT generate new migrations
- Does NOT run seed scripts
- Safe for CI/CD and production

## Options

| Option | Description |
|--------|-------------|
| `--schema` | Custom path to your Prisma schema |
| `--config` | Custom path to your Prisma config file |

## When to Use

- Production deployments
- Staging environments
- CI/CD pipelines
- Any non-development environment

## Examples

### Basic deployment

```bash
prisma migrate deploy
```

### In CI/CD pipeline

```yaml
# GitHub Actions example
- name: Apply migrations
  run: npx prisma migrate deploy
  env:
    DATABASE_URL: ${{ secrets.DATABASE_URL }}
```

### Docker deployment

```dockerfile
# Run migrations before starting app
CMD npx prisma migrate deploy && node dist/index.js
```

## Comparison with migrate dev

| Feature | migrate dev | migrate deploy |
|---------|-------------|----------------|
| Creates migrations | Yes | No |
| Applies migrations | Yes | Yes |
| Detects drift | Yes | No |
| Prompts for input | Yes | No |
| Uses shadow database | Yes | No |
| Safe for production | No | Yes |
| Resets on issues | Prompts | Fails |

## Production Workflow

1. **Development**: Create migrations locally
   ```bash
   prisma migrate dev --name add_feature
   ```

2. **Commit**: Include migration files in version control
   ```bash
   git add prisma/migrations
   git commit -m "Add feature migration"
   ```

3. **Deploy**: Apply in production
   ```bash
   prisma migrate deploy
   ```

## Error Handling

### Failed migration
````

If a migration fails, `migrate deploy` exits with error. The failed migration is marked as failed in `_prisma_migrations`.

To fix:
1. Resolve the issue (fix SQL, database state, etc.)
2. Mark as resolved: `prisma migrate resolve --applied <migration_name>`
3. Re-run: `prisma migrate deploy`

Check status first

```bash
prisma migrate status
```

Shows pending and applied migrations before deploying.

Configuration

Ensure `prisma.config.ts` has the production database URL:

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

Best Practices

1. Always run `migrate status` before `migrate deploy` in CI
2. Have a rollback plan (backup before migrations)
3. Test migrations in staging first
4. Never use `migrate dev` in production

12. .agents\skills\prisma-cli\references\migrate-dev.md

````
# prisma migrate dev

Creates and applies migrations during development. Requires a shadow database.

## Command

```bash
prisma migrate dev [options]
```

## What It Does

1. Runs existing migrations in shadow database to detect drift
2. Applies any pending migrations
3. Generates new migration from schema changes
4. Applies new migration to development database
5. Updates `_prisma_migrations` table

## Options

| Option | Description |
|--------|-------------|
| `--name` / `-n` | Name the migration |
| `--create-only` | Create a new migration but do not apply it |
| `--schema` | Custom path to your Prisma schema |
| `--config` | Custom path to your Prisma config file |
| `--url` | Override the datasource URL from the Prisma config file |

### Follow-up Commands

- Run `prisma generate` explicitly when you need refreshed client output
- Run `prisma db seed` explicitly when you need seed data

Note: Prisma CLI help for `7.6.0` still says `migrate dev` "trigger[s] generators", but local verification in a temp Prisma 7.6.0 project did not emit generated client files. Treat `prisma generate` as an explicit follow-up step when you need generated artifacts on disk.

## Examples

### Create and apply migration

```bash
prisma migrate dev
```

Prompts for migration name if schema changed.

### Named migration

```bash
prisma migrate dev --name add_users_table
```

### Create without applying

```bash
prisma migrate dev --create-only
```

Useful for reviewing migration SQL before applying.

### Full workflow

```bash
prisma migrate dev --name my_migration
prisma generate
prisma db seed
```

## Migration Files

Created in `prisma/migrations/`:

```
prisma/migrations/
├── 20240115120000_add_users_table/
│   └── migration.sql
├── 20240116090000_add_posts/
│   └── migration.sql
└── migration_lock.toml
```

## Schema Drift Detection

If `migrate dev` detects drift (manual database changes or edited migrations), it prompts to reset:

```
Drift detected: Your database schema is not in sync.

Do you want to reset your database? All data will be lost.
```
````

When to Use

- Local development
- Adding new models/fields
- Changing relations
- Creating indexes

When NOT to Use

- Production deployments (use `migrate deploy`)
- CI/CD pipelines (use `migrate deploy`)
- MongoDB (use `db push` instead)

Common Patterns

After schema changes

```prisma
// schema.prisma - Add new field
model User {
  id        Int      @id @default(autoincrement())
  email     String   @unique
  name      String?
  createdAt DateTime @default(now())  // New field
}
```

```bash
prisma migrate dev --name add_created_at
```

Handling data loss warnings

When a migration would cause data loss:

```bash
prisma migrate dev --name remove_field
# Warning: You are about to delete data...
# Accept with: --accept-data-loss
```

Shadow Database

`migrate dev` requires a shadow database for drift detection. Configure in `prisma.config.ts`:

```typescript
export default defineConfig({
  datasource: {
    url: env('DATABASE_URL'),
    shadowDatabaseUrl: env('SHADOW_DATABASE_URL'),
  },
})
```

For local Prisma Postgres (`prisma dev`), shadow database is handled automatically.

13. .agents\skills\prisma-cli\references\migrate-diff.md

````
# prisma migrate diff

Compares database schemas and generates diffs (SQL or summary).

## Command

```bash
prisma migrate diff [options]
```

## What It Does

- Compares two sources (`--from-...` and `--to-...`)
- Sources can be:
    - Empty (`empty`)
    - Schema file (`schema`)
    - Migrations directory (`migrations`)
    - Database URL (`url`) or Configured Datasource (`config-datasource`)
- Outputs the difference:
    - Human-readable summary (default)
    - SQL script (`--script`)

## Options

| Option | Description |
|--------|-------------|
| `--script` | Render SQL script to stdout |
| `--exit-code` | Exit 2 if changes detected, 0 if empty, 1 if error |
| `--config` | Custom path to your Prisma config file |

### Sources (Must provide one `from` and one `to`)

- `--from-empty`, `--to-empty`
- `--from-schema <path>`, `--to-schema <path>`
- `--from-migrations <path>`, `--to-migrations <path>`
- `--from-url <url>`, `--to-url <url>`
- `--from-config-datasource`, `--to-config-datasource` (uses `prisma.config.ts`)

## Examples

### Generate SQL for a schema change

Compare current production DB to your local schema:

```bash
prisma migrate diff \
  --from-url "$PROD_DB_URL" \
  --to-schema ./prisma/schema.prisma \
  --script
```

### Review pending migrations

Compare database state to migrations directory:

```bash
prisma migrate diff \
  --from-config-datasource \
  --to-migrations ./prisma/migrations
```

### Create baseline migration

Compare empty state to current schema:

```bash
prisma migrate diff \
  --from-empty \
  --to-schema ./prisma/schema.prisma \
  --script > prisma/migrations/0_init/migration.sql
```

### Check for drift (CI)

Check if database matches schema:

```bash
prisma migrate diff \
  --from-config-datasource \
  --to-schema ./prisma/schema.prisma \
  --exit-code
```

## Use Cases

- **Forward-generating migrations**: Creating SQL without `migrate dev`.
- **Drift detection**: Checking if DB is in sync.
- **Baselining**: Creating initial migration from existing DB.
- **Debugging**: Understanding what `migrate dev` would do.
````

14. .agents\skills\prisma-cli\references\migrate-reset.md

````
# prisma migrate reset

Resets your database and re-applies all migrations.

## Command

```bash
prisma migrate reset [options]
```

## What It Does

1. **Drops** the database (if possible) or deletes all data/tables
2. **Re-creates** the database
3. **Applies** all migrations from `prisma/migrations/`
4. Stops there - run seed and generate explicitly if needed

**Warning: All data will be lost.**

## Options

| Option | Description |
|--------|-------------|
| `--force` / `-f` | Skip confirmation prompt |
| `--schema` | Path to schema file |
| `--config` | Custom path to your Prisma config file |

## Examples

### Basic reset

```bash
prisma migrate reset
```

Prompts for confirmation in interactive terminals.

### Force reset (CI/Automation)

```bash
prisma migrate reset --force
```

### With custom schema

```bash
prisma migrate reset --schema=./custom/schema.prisma
```

## When to Use

- **Development**: When you want a fresh start
- **Testing**: Resetting test database before suites
- **Drift Recovery**: When the database is out of sync and you can't migrate

## Follow-up Steps

Run `prisma generate` and `prisma db seed` explicitly when you need refreshed client output or seed data after a reset.

## Configuration

Configure the seed script in `prisma.config.ts`, then run it explicitly after reset:

```typescript
export default defineConfig({
  migrations: {
    seed: 'tsx prisma/seed.ts',
  },
})
```

Typical workflow:

```bash
prisma migrate reset --force
prisma generate
prisma db seed
```
````

15. .agents\skills\prisma-cli\references\migrate-resolve.md

````
# prisma migrate resolve

Resolves issues with database migrations, such as failed migrations or baselining.

## Command

```bash
prisma migrate resolve [options]
```

## What It Does

Updates the `_prisma_migrations` table to manually change the state of a migration. This is a recovery tool.

## Options

You must provide exactly one of `--applied` or `--rolled-back`.

| Option | Description |
|--------|-------------|
| `--applied <name>` | Mark a migration as **applied** (success) |
| `--rolled-back <name>` | Mark a migration as **rolled back** (ignored/failed) |
| `--schema` | Path to schema file |
| `--config` | Custom path to your Prisma config file |

## Examples

### Mark as Applied (Baselining)

If you have existing tables and want to initialize migrations without running the SQL:

```bash
prisma migrate resolve --applied 20240101000000_initial_migration
```

This tells Prisma "Assume this migration has already run".

### Mark as Rolled Back (Fixing Failures)

If a migration failed (e.g., syntax error) and you fixed the SQL or want to retry:

```bash
prisma migrate resolve --rolled-back 20240115120000_failed_migration
```

This tells Prisma "Forget this migration run, let me try applying it again".

## Use Cases

1. **Baselining**: Adopting Prisma Migrate on an existing production database.
2. **Failed Migrations**: Recovering from a failed `migrate deploy` in production.
3. **Hotfixes**: reconciling manual database changes (rare).

## References

- [Baselining](https://www.prisma.io/docs/guides/database/developing-with-prisma-migrate/baselining)
- [Troubleshooting](https://www.prisma.io/docs/guides/database/production-troubleshooting)
````

16. .agents\skills\prisma-cli\references\migrate-status.md

````
# prisma migrate status

Checks the status of your database migrations.

## Command

```bash
prisma migrate status [options]
```

## What It Does

- Connects to the database
- Checks the `_prisma_migrations` table
- Compares applied migrations with local migration files
- Reports:
    - **Status**: Database is up-to-date or behind
    - **Unapplied migrations**: Count of pending migrations
    - **Missing migrations**: Migrations present in DB but missing locally
    - **Failed migrations**: Any migrations that failed to apply

## Options

| Option | Description |
|--------|-------------|
| `--schema` | Path to schema file |
| `--config` | Custom path to your Prisma config file |

## Examples

### Check status

```bash
prisma migrate status
```

Output example (Up to date):
```
Database schema is up to date!
```

Output example (Pending):
```
Following migration have not yet been applied:
  20240115120000_add_user

To apply migrations in development, run:
  prisma migrate dev

To apply migrations in production, run:
  prisma migrate deploy
```

## When to Use

- **Debugging**: Why is `migrate dev` complaining about drift?
- **CI/CD**: Verify database state before deploying
- **Production**: Check if migrations are needed (`migrate deploy`) or if a deployment failed

## Exit Codes

- `0`: Success (may have pending migrations, but command ran successfully)
- `1`: Error

To check for pending migrations programmatically, you might need to parse the output or use `migrate diff` with exit code flags.
````

17. .agents\skills\prisma-cli\references\studio.md

````
# prisma studio

Opens a visual database browser for viewing and editing data.

## Command

```bash
prisma studio [options]
```

## What It Does

- Starts a web-based database GUI
- View all your models and records
- Create, update, and delete records
- Filter and sort data
- Navigate relations

## Options

| Option | Description | Default |
|--------|-------------|---------|
| `--port` / `-p` | Port to start Studio on | `5555` |
| `--browser` / `-b` | Browser to open Studio in | System default |
| `--config` | Custom path to your Prisma config file | - |
| `--url` | Database connection string (overrides the one in your Prisma config) | - |

## Examples

### Open Studio

```bash
prisma studio
```

Opens at http://localhost:5555

### Custom port

```bash
prisma studio --port 3000
```

### Specific browser

```bash
prisma studio --browser firefox
```

### Don't open browser

```bash
BROWSER=none prisma studio
```

Useful for remote servers.

## Features

### View Records

- See all records in table format
- Pagination for large datasets
- Column sorting

### Filter Data

- Filter by any field
- Multiple conditions
- Relation filtering

### Edit Records

- Click to edit inline
- Add new records
- Delete records (with confirmation)

### Navigate Relations

- Click relations to view related records
- See counts of related items
- Follow relation links

## Recent Studio Capabilities

Recent Prisma Studio releases added richer editor workflows:

- multi-cell selection and editing
- full-table search and more intuitive filtering
- command palette shortcuts
````

- dark mode
- copy selections as Markdown
- back-relation navigation
- SQL workflows including raw SQL queries

Some recent builds also expose AI-assisted SQL authoring. Treat these as interactive Studio features rather than a replacement for checked-in migrations or application queries.

Use Cases

- **Development**: Quick data inspection
- **Debugging**: Check data state
- **Testing**: Verify seed data
- **Demo**: Show data to stakeholders

Limitations

- Development tool only
- Not for production use
- Limited to configured database
- Prisma Studio in Prisma 7 currently targets PostgreSQL, MySQL, and SQLite first
- For reproducible application logic, prefer Prisma Client and checked-in SQL scripts

Common Workflow

1. Run migrations:
   ```bash
   prisma migrate dev
   ```

2. Seed data:
   ```bash
   prisma db seed
   ```

3. Open Studio to verify:
   ```bash
   prisma studio
   ```

4. Make manual edits if needed

Security Note

Studio provides direct database access. Only run on:
- Local development machines
- Secure internal networks
- Never expose publicly

18. .agents\skills\prisma-cli\references\validate.md

````
# prisma validate

Validates your Prisma schema file.

## Command

```bash
prisma validate [options]
```

## What It Does

- Parses the `schema.prisma` file
- Checks for syntax errors
- Validates model definitions, relations, and types
- Reports any errors or warnings without generating code

## Options

| Option | Description |
|--------|-------------|
| `--schema` | Path to schema file |
| `--config` | Custom path to your Prisma config file |

## Examples

### Validate default schema

```bash
prisma validate
```

### Validate specific schema

```bash
prisma validate --schema=./custom/schema.prisma
```

### Use in CI

Run `validate` in your CI pipeline to catch schema errors early:

```yaml
- name: Validate Schema
  run: npx prisma validate
```

## Common Errors

- Missing `@relation` fields
- Invalid types
- Duplicate model names
- Syntax errors (missing braces, etc.)
````

19. .agents\skills\prisma-cli\SKILL.md

````
---
name: prisma-cli
description: Prisma ORM CLI commands reference covering init, generate, migrate, db, dev, studio, validate, format, debug, and mcp. Use for ORM/database CLI workflows, not Prisma Compute app deployment. For Prisma Compute, `@prisma/cli app deploy`, `compute:deploy`, `create-prisma --deploy`, apps, deployments, logs, or domains, use the `prisma-compute` skill instead. Triggers on "prisma init", "prisma generate", "prisma migrate", "prisma db", "prisma studio", "prisma mcp".
license: MIT
metadata:
  author: prisma
  version: "7.6.0"
---

# Prisma CLI Reference

Reference for Prisma ORM CLI commands. This skill provides guidance on command usage, options, and best practices for current Prisma ORM releases.

## Boundary: Compute

Do not use this skill for Prisma Compute app deployment. Use `prisma-compute` for `@prisma/cli app deploy`, `compute:deploy`, `create-prisma --deploy`, Compute apps, deployments, logs, domains, and framework deploy readiness.

## When to Apply

Reference this skill when:
- Setting up a new Prisma project (`prisma init`)
- Generating Prisma Client (`prisma generate`)
- Running database migrations (`prisma migrate`)
- Managing database state (`prisma db push/pull`)
- Using local development database (`prisma dev`)
- Debugging Prisma issues (`prisma debug`)

## Rule Categories by Priority

| Priority | Category | Impact | Prefix |
|----------|----------|--------|--------|
| 1 | Setup | HIGH | `init` |
| 2 | Generation | HIGH | `generate` |
| 3 | Development | HIGH | `dev` |
| 4 | Database | HIGH | `db-` |
| 5 | Migrations | CRITICAL | `migrate-` |
| 6 | Utility | MEDIUM | `studio`, `validate`, `format`, `debug`, `mcp` |

## Command Categories

| Category | Commands | Purpose |
|----------|----------|---------|
| Setup | `init` | Bootstrap new Prisma project |
| Generation | `generate` | Generate Prisma Client |
| Validation | `validate`, `format` | Schema validation and formatting |
| Development | `dev` | Local Prisma Postgres for development |
| Database | `db pull`, `db push`, `db seed`, `db execute` | Direct database operations |
| Migrations | `migrate dev`, `migrate deploy`, `migrate reset`, `migrate status`, `migrate diff`, `migrate resolve` | Schema migrations |
| Utility | `studio`, `mcp`, `version`, `debug` | Development and AI tooling |

## Quick Reference

### Project Setup

```bash
# Initialize new project (creates prisma/ folder and prisma.config.ts)
prisma init

# Initialize with specific database
prisma init --datasource-provider postgresql
prisma init --datasource-provider mysql
prisma init --datasource-provider sqlite

# Initialize with Prisma Postgres (cloud)
prisma init --db

# Initialize with an example model
prisma init --with-model
```

### Client Generation

```bash
# Generate Prisma Client
prisma generate

# Watch mode for development
prisma generate --watch

# Generate specific generator only
prisma generate --generator client
```

### Bun Runtime
````

When using Bun, always add the `--bun` flag so Prisma runs with the Bun runtime (otherwise it falls back to Node.js because of the CLI shebang):

```bash
bunx --bun prisma init
bunx --bun prisma generate
```

Local Development Database

```bash
# Start local Prisma Postgres
prisma dev

# Start with specific name
prisma dev --name myproject

# Start in background (detached)
prisma dev --detach

# List all local instances
prisma dev ls

# Stop instance
prisma dev stop myproject

# Remove instance data
prisma dev rm myproject
```

Database Operations

```bash
# Pull schema from existing database
prisma db pull

# Push schema to database (no migrations)
prisma db push

# Seed database
prisma db seed

# Execute raw SQL
prisma db execute --file ./script.sql
```

Migrations (Development)

```bash
# Create and apply migration
prisma migrate dev

# Create migration with name
prisma migrate dev --name add_users_table

# Create migration without applying
prisma migrate dev --create-only

# Reset database and apply all migrations
prisma migrate reset
```

Migrations (Production)

```bash
# Apply pending migrations (CI/CD)
prisma migrate deploy

# Check migration status
prisma migrate status

# Compare schemas and generate diff
prisma migrate diff --from-config-datasource --to-schema schema.prisma --script
```

Utility Commands

```bash
# Open Prisma Studio (database GUI)
prisma studio

# Start Prisma's MCP server for AI tools
prisma mcp

# Show version info
prisma version
prisma -v

# Debug information
prisma debug

# Validate schema
prisma validate
```

```
# Format schema
prisma format
```

Current Prisma CLI Setup

New Configuration File

Use `prisma.config.ts` for CLI configuration:

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
    seed: 'tsx prisma/seed.ts',
  },
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

Current Command Behavior

- Run `prisma generate` explicitly after `migrate dev`, `db push`, or other schema syncs when you need fresh client output
- Run `prisma db seed` explicitly after `migrate dev` or `migrate reset` when you need seed data
- Use `prisma db execute --file ...` for raw SQL scripts

Environment Variables

Load environment variables explicitly in `prisma.config.ts`, commonly with `dotenv`:

```typescript
// prisma.config.ts
import 'dotenv/config'
```

Rule Files

See individual rule files for detailed command documentation:

```
references/init.md           - Project initialization
references/generate.md       - Client generation
references/dev.md            - Local development database
references/db-pull.md        - Database introspection
references/db-push.md        - Schema push
references/db-seed.md        - Database seeding
references/db-execute.md     - Raw SQL execution
references/migrate-dev.md    - Development migrations
references/migrate-deploy.md - Production migrations
references/migrate-reset.md  - Database reset
references/migrate-status.md - Migration status
references/migrate-resolve.md - Migration resolution
references/migrate-diff.md   - Schema diffing
references/studio.md         - Database GUI
references/mcp.md            - Prisma MCP server
references/validate.md       - Schema validation
references/format.md         - Schema formatting
references/debug.md          - Debug info
```

How to Use

Use the command categories above for navigation, then open the specific command reference file you need.

20. .agents\skills\prisma-client-api\references\client-methods.md

````
# Client Methods

Prisma Client instance methods.

## $connect()

Explicitly connect to the database:

```typescript
const prisma = new PrismaClient({ adapter })

// Explicit connection
await prisma.$connect()
```

### When to use

Usually not needed - Prisma connects automatically on first query. Use for:
- Fail fast on startup
- Health checks
- Pre-warming connections

```typescript
async function main() {
  try {
    await prisma.$connect()
    console.log('Database connected')
  } catch (e) {
    console.error('Failed to connect:', e)
    process.exit(1)
  }
}
```

## $disconnect()

Close database connection:

```typescript
await prisma.$disconnect()
```

### Graceful shutdown

```typescript
process.on('beforeExit', async () => {
  await prisma.$disconnect()
})

// Or with SIGTERM
process.on('SIGTERM', async () => {
  await prisma.$disconnect()
  process.exit(0)
})
```

### In tests

```typescript
afterAll(async () => {
  await prisma.$disconnect()
})
```

## $on()

Subscribe to events:

### Query events

```typescript
const prisma = new PrismaClient({
  adapter,
  log: [{ level: 'query', emit: 'event' }]
})

prisma.$on('query', (e) => {
  console.log('Query:', e.query)
  console.log('Params:', e.params)
  console.log('Duration:', e.duration, 'ms')
})
```

### Log events

```typescript
const prisma = new PrismaClient({
  adapter,
  log: [
    { level: 'info', emit: 'event' },
````
````

```
    { level: 'warn', emit: 'event' },
    { level: 'error', emit: 'event' }
  ]
})

prisma.$on('info', (e) => console.log(e.message))
prisma.$on('warn', (e) => console.warn(e.message))
prisma.$on('error', (e) => console.error(e.message))
```

$extends()

Add extensions for custom behavior:

Add custom methods

```typescript
const prisma = new PrismaClient({ adapter }).$extends({
  client: {
    $log: (message: string) => console.log(message)
  }
})

prisma.$log('Hello!')
```

Add model methods

```typescript
const prisma = new PrismaClient({ adapter }).$extends({
  model: {
    user: {
      async findByEmail(email: string) {
        return prisma.user.findUnique({ where: { email } })
      }
    }
  }
})

const user = await prisma.user.findByEmail('alice@prisma.io')
```

Query extensions

```typescript
const prisma = new PrismaClient({ adapter }).$extends({
  query: {
    user: {
      async findMany({ args, query }) {
        // Add default filter
        args.where = { ...args.where, deletedAt: null }
        return query(args)
      }
    }
  }
})
```

Result extensions

```typescript
const prisma = new PrismaClient({ adapter }).$extends({
  result: {
    user: {
      fullName: {
        needs: { firstName: true, lastName: true },
        compute(user) {
          return `${user.firstName} ${user.lastName}`
        }
      }
    }
  }
})

const user = await prisma.user.findFirst()
console.log(user.fullName) // Computed field
```

Chain extensions

```typescript
const prisma = new PrismaClient({ adapter })
  .$extends(loggingExtension)
  .$extends(softDeleteExtension)
  .$extends(computedFieldsExtension)
```

$transaction()

See `transactions.md` for details.

$queryRaw() / $executeRaw()

See `raw-queries.md` for details.

Type utilities

Prisma namespace

```typescript
import { Prisma } from '../generated/client'

// Input types
type UserCreateInput = Prisma.UserCreateInput
type UserWhereInput = Prisma.UserWhereInput

// Output types
type User = Prisma.UserGetPayload<{}>
type UserWithPosts = Prisma.UserGetPayload<{
  include: { posts: true }
}>
```

Type-safe query fragments with satisfies

Type-safe query fragments:

```typescript
import { Prisma } from '../generated/client'

const userSelect = {
  id: true,
  email: true,
  name: true
} satisfies Prisma.UserSelect

const user = await prisma.user.findUnique({
  where: { id: 1 },
  select: userSelect
})
```

With the `prisma-client` generator, use TypeScript `satisfies` for typed query fragments. You may still see older examples that use `Prisma.validator()` with `prisma-client-js`.

21. .agents\skills\prisma-client-api\references\constructor.md

````
# PrismaClient Constructor

Configure Prisma Client when instantiating.

## Basic Instantiation

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
})

const prisma = new PrismaClient({ adapter })
```

## Constructor Options

### adapter (Required for the SQL provider workflow)

Driver adapter instance:

```typescript
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
})

const prisma = new PrismaClient({ adapter })
```

### accelerateUrl (For Accelerate users)

```typescript
import { withAccelerate } from '@prisma/extension-accelerate'

const prisma = new PrismaClient({
  accelerateUrl: process.env.DATABASE_URL,  // prisma:// URL
}).$extends(withAccelerate())
```

### log

Configure logging:

```typescript
const prisma = new PrismaClient({
  adapter,
  log: ['query', 'info', 'warn', 'error'],
})
```

#### Log levels

| Level | Description |
|-------|-------------|
| `query` | All SQL queries |
| `info` | Informational messages |
| `warn` | Warnings |
| `error` | Errors |

#### Log to events

```typescript
const prisma = new PrismaClient({
  adapter,
  log: [
    { level: 'query', emit: 'event' },
    { level: 'error', emit: 'stdout' },
  ],
})

prisma.$on('query', (e) => {
  console.log('Query:', e.query)
  console.log('Duration:', e.duration, 'ms')
})
```

### errorFormat

Control error formatting:

```typescript
const prisma = new PrismaClient({
  adapter,
  errorFormat: 'pretty',  // 'pretty' | 'colorless' | 'minimal'
})
```
````

comments

Attach SQL commenter plugins for observability, tracing, or query insights:

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'
import { prismaQueryInsights } from '@prisma/sqlcommenter-query-insights'
import { queryTags, withQueryTags } from '@prisma/sqlcommenter-query-tags'
import { traceContext } from '@prisma/sqlcommenter-trace-context'

const prisma = new PrismaClient({
  adapter: new PrismaPg(process.env.DATABASE_URL!),
  comments: [prismaQueryInsights(), traceContext(), queryTags()],
})

await withQueryTags({ route: '/api/users', requestId: 'req-123' }, () =>
  prisma.user.findMany(),
)
```

Use `comments` only for SQL providers. This is the clean way to add trace or query-shape metadata without changing your query calls.

transactionOptions

Default transaction settings:

```typescript
const prisma = new PrismaClient({
  adapter,
  transactionOptions: {
    maxWait: 5000,      // Max wait to acquire transaction (ms)
    timeout: 10000,     // Max transaction duration (ms)
    isolationLevel: 'Serializable',
  },
})
```

Singleton Pattern

Prevent multiple client instances in development:

```typescript
// lib/prisma.ts
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined
}

function createPrismaClient() {
  const adapter = new PrismaPg({
    connectionString: process.env.DATABASE_URL!
  })
  return new PrismaClient({ adapter })
}

export const prisma = globalForPrisma.prisma ?? createPrismaClient()

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma
}
```

Next.js Pattern

```typescript
// lib/prisma.ts
import { PrismaClient } from '@/generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const createAdapter = () => new PrismaPg({
  connectionString: process.env.DATABASE_URL!
})

const prismaClientSingleton = () => {
  return new PrismaClient({ adapter: createAdapter() })
}

declare const globalThis: {
  prismaGlobal: ReturnType<typeof prismaClientSingleton>
} & typeof global

const prisma = globalThis.prismaGlobal ?? prismaClientSingleton()

export default prisma

if (process.env.NODE_ENV !== 'production') {
  globalThis.prismaGlobal = prisma
}
```

Query Events

Listen to query events:

```typescript
const prisma = new PrismaClient({
  adapter,
  log: [{ level: 'query', emit: 'event' }],
})

prisma.$on('query', (e) => {
  console.log('Query:', e.query)
  console.log('Params:', e.params)
  console.log('Duration:', e.duration)
})
```

Log Events

```typescript
prisma.$on('info', (e) => console.log(e.message))
prisma.$on('warn', (e) => console.warn(e.message))
prisma.$on('error', (e) => console.error(e.message))
```

22. .agents\skills\prisma-client-api\references\filters.md

````
# Filter Conditions and Operators

Filter operators for the `where` clause.

## Equality

```typescript
// Exact match (implicit)
where: { email: 'alice@prisma.io' }

// Explicit equals
where: { email: { equals: 'alice@prisma.io' } }

// Not equal
where: { email: { not: 'alice@prisma.io' } }
```

## Comparison

```typescript
// Greater than
where: { age: { gt: 18 } }

// Greater than or equal
where: { age: { gte: 18 } }

// Less than
where: { age: { lt: 65 } }

// Less than or equal
where: { age: { lte: 65 } }

// Combined
where: { age: { gte: 18, lte: 65 } }
```

## Lists

```typescript
// In array
where: { role: { in: ['ADMIN', 'MODERATOR'] } }

// Not in array
where: { role: { notIn: ['GUEST', 'BANNED'] } }
```

## String Filters

```typescript
// Contains
where: { email: { contains: 'prisma' } }

// Starts with
where: { email: { startsWith: 'alice' } }

// Ends with
where: { email: { endsWith: '@prisma.io' } }

// Case-insensitive (default for some databases)
where: {
  email: {
    contains: 'PRISMA',
    mode: 'insensitive'
  }
}
```

## Null Checks

```typescript
// Is null
where: { deletedAt: null }

// Is not null
where: { deletedAt: { not: null } }

// Using isSet (for optional fields)
where: { middleName: { isSet: true } }
```

## Logical Operators

### AND (implicit)

```typescript
// Multiple conditions = AND
where: {
  email: { contains: '@prisma.io' },
  role: 'ADMIN'
}
````
````

```

### AND (explicit)

```typescript
where: {
  AND: [
    { email: { contains: '@prisma.io' } },
    { role: 'ADMIN' }
  ]
}
```

OR

```typescript
where: {
  OR: [
    { email: { contains: '@gmail.com' } },
    { email: { contains: '@prisma.io' } }
  ]
}
```

NOT

```typescript
where: {
  NOT: {
    role: 'GUEST'
  }
}

// Multiple NOT conditions
where: {
  NOT: [
    { role: 'GUEST' },
    { verified: false }
  ]
}
```

Combined

```typescript
where: {
  AND: [
    { verified: true },
    {
      OR: [
        { role: 'ADMIN' },
        { role: 'MODERATOR' }
      ]
    }
  ],
  NOT: { deletedAt: { not: null } }
}
```

Relation Filters

some

At least one related record matches:

```typescript
// Users with at least one published post
where: {
  posts: {
    some: { published: true }
  }
}
```

every

All related records match:

```typescript
// Users where all posts are published
where: {
  posts: {
    every: { published: true }
  }
}
```

none

No related records match:

```typescript
// Users with no published posts
```
```

```typescript
where: {
  posts: {
    none: { published: true }
  }
}
```

is / isNot (1-to-1)

```typescript
// Users with profile in specific country
where: {
  profile: {
    is: { country: 'USA' }
  }
}

// Users without profile
where: {
  profile: {
    isNot: null
  }
}
```

Array Field Filters

For fields like `String[]`:

```typescript
// Has element
where: { tags: { has: 'typescript' } }

// Has some elements
where: { tags: { hasSome: ['typescript', 'javascript'] } }

// Has every element
where: { tags: { hasEvery: ['typescript', 'prisma'] } }

// Is empty
where: { tags: { isEmpty: true } }
```

JSON Filters

```typescript
// Path-based filter
where: {
  metadata: {
    path: ['settings', 'theme'],
    equals: 'dark'
  }
}

// String contains in JSON
where: {
  metadata: {
    path: ['bio'],
    string_contains: 'developer'
  }
}
```

Full-Text Search

```typescript
// Requires @@fulltext index
where: {
  content: {
    search: 'prisma database'
  }
}
```

23. .agents\skills\prisma-client-api\references\model-queries.md

````
# Model Queries

CRUD operations for your Prisma models.

## Read Operations

### findUnique

Find a single record by unique field:

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 }
})

const user = await prisma.user.findUnique({
  where: { email: 'alice@prisma.io' }
})
```

#### With composite unique key

```typescript
// Model with @@unique([firstName, lastName])
const user = await prisma.user.findUnique({
  where: {
    firstName_lastName: {
      firstName: 'Alice',
      lastName: 'Smith'
    }
  }
})
```

### findUniqueOrThrow

Same as findUnique but throws if not found:

```typescript
const user = await prisma.user.findUniqueOrThrow({
  where: { id: 1 }
})
// Throws PrismaClientKnownRequestError if not found
```

### findFirst

Find first matching record:

```typescript
const user = await prisma.user.findFirst({
  where: { role: 'ADMIN' },
  orderBy: { createdAt: 'desc' }
})
```

### findFirstOrThrow

```typescript
const user = await prisma.user.findFirstOrThrow({
  where: { role: 'ADMIN' }
})
```

### findMany

Find multiple records:

```typescript
const users = await prisma.user.findMany({
  where: { role: 'USER' },
  orderBy: { name: 'asc' },
  take: 10,
  skip: 0
})
```

## Create Operations

### create

Create a single record:

```typescript
const user = await prisma.user.create({
  data: {
    email: 'alice@prisma.io',
    name: 'Alice'
  }
})
````
````

```

#### With relations

```typescript
const user = await prisma.user.create({
  data: {
    email: 'alice@prisma.io',
    posts: {
      create: [
        { title: 'First Post' },
        { title: 'Second Post' }
      ]
    }
  },
  include: { posts: true }
})
```

createMany

Create multiple records:

```typescript
const result = await prisma.user.createMany({
  data: [
    { email: 'alice@prisma.io', name: 'Alice' },
    { email: 'bob@prisma.io', name: 'Bob' }
  ],
  skipDuplicates: true  // Skip records with duplicate unique fields
})
// Returns { count: 2 }
```

createManyAndReturn

Create multiple and return them:

```typescript
const users = await prisma.user.createManyAndReturn({
  data: [
    { email: 'alice@prisma.io', name: 'Alice' },
    { email: 'bob@prisma.io', name: 'Bob' }
  ]
})
// Returns array of created users
```

Update Operations

update

Update a single record:

```typescript
const user = await prisma.user.update({
  where: { id: 1 },
  data: { name: 'Alice Smith' }
})
```

Atomic operations

```typescript
const post = await prisma.post.update({
  where: { id: 1 },
  data: {
    views: { increment: 1 },
    likes: { decrement: 1 },
    score: { multiply: 2 },
    rating: { divide: 2 },
    version: { set: 5 }
  }
})
```

updateMany

Update multiple records:

```typescript
const result = await prisma.user.updateMany({
  where: { role: 'USER' },
  data: { verified: true }
})
// Returns { count: 42 }
```

updateManyAndReturn

```typescript
const users = await prisma.user.updateManyAndReturn({
  where: { role: 'USER' },
```
```

```
  data: { verified: true }
})
// Returns array of updated users
```

upsert

Update or create:

```typescript
const user = await prisma.user.upsert({
  where: { email: 'alice@prisma.io' },
  update: { name: 'Alice Smith' },
  create: { email: 'alice@prisma.io', name: 'Alice' }
})
```

Delete Operations

delete

Delete a single record:

```typescript
const user = await prisma.user.delete({
  where: { id: 1 }
})
// Returns deleted record
```

deleteMany

Delete multiple records:

```typescript
const result = await prisma.user.deleteMany({
  where: { role: 'GUEST' }
})
// Returns { count: 5 }

// Delete all
const result = await prisma.user.deleteMany({})
```

Aggregation Operations

count

```typescript
const count = await prisma.user.count({
  where: { role: 'ADMIN' }
})
```

aggregate

```typescript
const result = await prisma.post.aggregate({
  _avg: { views: true },
  _sum: { views: true },
  _min: { views: true },
  _max: { views: true },
  _count: { _all: true }
})
```

groupBy

```typescript
const groups = await prisma.user.groupBy({
  by: ['country'],
  _count: { _all: true },
  _avg: { age: true },
  having: {
    age: { _avg: { gt: 30 } }
  }
})
```

Return Types

Method	Returns
`findUnique`	Record | null
`findUniqueOrThrow`	Record (throws if not found)
`findFirst`	Record | null
`findFirstOrThrow`	Record (throws if not found)
`findMany`	Record[]
`create`	Record
`createMany`	{ count: number }
`createManyAndReturn`	Record[]
`update`	Record
`updateMany`	{ count: number }

`delete`	Record
`deleteMany`	{ count: number }
`count`	number
`aggregate`	Aggregate result
`groupBy`	Group result[]

24. .agents\skills\prisma-client-api\references\query-options.md

````
# Query Options

Options for controlling query behavior.

## select

Choose specific fields to return:

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  select: {
    id: true,
    name: true,
    email: true,
    // password: false (excluded by not including)
  }
})
// Returns: { id: 1, name: 'Alice', email: 'alice@prisma.io' }
```

### Select relations

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  select: {
    name: true,
    posts: {
      select: {
        title: true,
        published: true
      }
    }
  }
})
```

### Select with include inside

```typescript
const user = await prisma.user.findMany({
  select: {
    name: true,
    posts: {
      include: {
        comments: true
      }
    }
  }
})
```

### Select relation count

```typescript
const users = await prisma.user.findMany({
  select: {
    name: true,
    _count: {
      select: { posts: true }
    }
  }
})
// Returns: { name: 'Alice', _count: { posts: 5 } }
```

## include

Include related records:

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  include: {
    posts: true,
    profile: true
  }
})
```

### Filtered include

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  include: {
    posts: {
      where: { published: true },
      orderBy: { createdAt: 'desc' },
````
````

```
      take: 5
    }
  }
})
```

Nested include

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  include: {
    posts: {
      include: {
        comments: {
          include: {
            author: true
          }
        }
      }
    }
  }
})
```

Include relation count

```typescript
const users = await prisma.user.findMany({
  include: {
    _count: {
      select: { posts: true, followers: true }
    }
  }
})
```

omit

Exclude specific fields:

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  omit: {
    password: true
  }
})
// Returns all fields except password
```

Omit in relations

```typescript
const users = await prisma.user.findMany({
  omit: { password: true },
  include: {
    posts: {
      omit: { content: true }
    }
  }
})
```

Note: Cannot use `select` and `omit` together.

where

Filter records:

```typescript
const users = await prisma.user.findMany({
  where: {
    email: { contains: '@prisma.io' },
    role: 'ADMIN'
  }
})
```

See `filters.md` for detailed filter operators.

orderBy

Sort results:

```typescript
// Single field
const users = await prisma.user.findMany({
  orderBy: { name: 'asc' }
})

// Multiple fields
const users = await prisma.user.findMany({
```

```
  orderBy: [
    { role: 'desc' },
    { name: 'asc' }
  ]
})
```

Order by relation

```typescript
const users = await prisma.user.findMany({
  orderBy: {
    posts: { _count: 'desc' }
  }
})
```

Null handling

```typescript
const users = await prisma.user.findMany({
  orderBy: {
    name: { sort: 'asc', nulls: 'last' }
  }
})
```

take & skip

Pagination:

```typescript
// First page
const users = await prisma.user.findMany({
  take: 10,
  skip: 0
})

// Second page
const users = await prisma.user.findMany({
  take: 10,
  skip: 10
})
```

Negative take (reverse)

```typescript
const lastUsers = await prisma.user.findMany({
  take: -10,
  orderBy: { id: 'asc' }
})
// Returns last 10 users
```

cursor

Cursor-based pagination:

```typescript
// First page
const firstPage = await prisma.user.findMany({
  take: 10,
  orderBy: { id: 'asc' }
})

// Next page using cursor
const nextPage = await prisma.user.findMany({
  take: 10,
  skip: 1,  // Skip the cursor record
  cursor: { id: firstPage[firstPage.length - 1].id },
  orderBy: { id: 'asc' }
})
```

distinct

Return unique values:

```typescript
const cities = await prisma.user.findMany({
  distinct: ['city'],
  select: { city: true }
})
```

Multiple distinct fields

```typescript
const locations = await prisma.user.findMany({
  distinct: ['city', 'country']
})
```

25. .agents\skills\prisma-client-api\references\raw-queries.md

````
# Raw Queries

Execute raw SQL when Prisma's query API isn't sufficient.

## $queryRaw

Execute SELECT queries and get typed results:

```typescript
const users = await prisma.$queryRaw`
  SELECT * FROM "User" WHERE email LIKE ${'%@prisma.io'}
`
```

### With type

```typescript
type User = { id: number; email: string; name: string | null }

const users = await prisma.$queryRaw<User[]>`
  SELECT id, email, name FROM "User" WHERE role = ${'ADMIN'}
`
```

### Dynamic table/column names

Use `Prisma.raw()` for identifiers (not safe for user input):

```typescript
import { Prisma } from '../generated/client'

const column = 'email'
const users = await prisma.$queryRaw`
  SELECT ${Prisma.raw(column)} FROM "User"
`
```

### With Prisma.sql

Build queries dynamically:

```typescript
import { Prisma } from '../generated/client'

const email = 'alice@prisma.io'
const query = Prisma.sql`SELECT * FROM "User" WHERE email = ${email}`
const users = await prisma.$queryRaw(query)
```

### Join multiple SQL fragments

```typescript
import { Prisma } from '../generated/client'

const conditions = [
  Prisma.sql`role = ${'ADMIN'}`,
  Prisma.sql`verified = ${true}`
]

const users = await prisma.$queryRaw`
  SELECT * FROM "User"
  WHERE ${Prisma.join(conditions, ' AND ')}
`
```

## $executeRaw

Execute INSERT, UPDATE, DELETE (returns affected count):

```typescript
const count = await prisma.$executeRaw`
  UPDATE "User" SET verified = true WHERE email LIKE ${'%@prisma.io'}
`
console.log(`Updated ${count} users`)
```

### Delete example

```typescript
const deleted = await prisma.$executeRaw`
  DELETE FROM "User" WHERE "deletedAt" < ${thirtyDaysAgo}
`
```

### Insert example

```typescript
const inserted = await prisma.$executeRaw`
  INSERT INTO "Log" (message, level, timestamp)
  VALUES (${message}, ${level}, ${new Date()})
````
````

`
```

## $queryRawUnsafe / $executeRawUnsafe

For fully dynamic queries (use with caution!):

```typescript
// ⚠️ SQL injection risk - only use with trusted input
const table = 'User'
const users = await prisma.$queryRawUnsafe(
  `SELECT * FROM "${table}" WHERE id = $1`,
  userId
)
```

Parameterized unsafe query

```typescript
const result = await prisma.$executeRawUnsafe(
  'UPDATE "User" SET name = $1 WHERE id = $2',
  'Alice',
  1
)
```

SQL Injection Prevention

Safe (parameterized)

```typescript
// ✅ User input is parameterized
const email = userInput
const users = await prisma.$queryRaw`
  SELECT * FROM "User" WHERE email = ${email}
`
```

Unsafe (concatenation)

```typescript
// ❌ SQL injection vulnerability!
const email = userInput
const users = await prisma.$queryRawUnsafe(
  `SELECT * FROM "User" WHERE email = '${email}'`
)
```

Database-Specific Features

PostgreSQL

```typescript
// Array operations
const users = await prisma.$queryRaw`
  SELECT * FROM "User" WHERE 'admin' = ANY(roles)
`

// JSON operations
const users = await prisma.$queryRaw`
  SELECT * FROM "User" WHERE metadata->>'theme' = 'dark'
`
```

MySQL

```typescript
// Full-text search
const posts = await prisma.$queryRaw`
  SELECT * FROM Post WHERE MATCH(title, content) AGAINST(${searchTerm})
`
```

Transactions with Raw Queries

```typescript
await prisma.$transaction(async (tx) => {
  await tx.$executeRaw`UPDATE "Account" SET balance = balance - ${amount} WHERE id = ${senderId}`
  await tx.$executeRaw`UPDATE "Account" SET balance = balance + ${amount} WHERE id = ${recipientId}`
})
```

Handling Results

BigInt handling

PostgreSQL returns BigInt for COUNT:

```typescript
const result = await prisma.$queryRaw<[{ count: bigint }]>`
  SELECT COUNT(*) as count FROM "User"
`
const count = Number(result[0].count)
```
```

```

### Date handling

```typescript
type Result = { createdAt: Date }
const users = await prisma.$queryRaw<Result[]>`
  SELECT "createdAt" FROM "User"
`
// createdAt is already a Date object
```
```

26. .agents\skills\prisma-client-api\references\relations.md

````
# Relation Queries

Query and modify related records.

## Include Relations

Load related records:

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  include: {
    posts: true,
    profile: true
  }
})
```

### Filtered include

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  include: {
    posts: {
      where: { published: true },
      orderBy: { createdAt: 'desc' },
      take: 5,
      select: { id: true, title: true }
    }
  }
})
```

### Nested include

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  include: {
    posts: {
      include: {
        comments: {
          include: { author: true }
        }
      }
    }
  }
})
```

## Select Relations

```typescript
const user = await prisma.user.findUnique({
  where: { id: 1 },
  select: {
    name: true,
    posts: {
      select: { title: true }
    }
  }
})
```

## Nested Writes

### Create with relations

```typescript
const user = await prisma.user.create({
  data: {
    email: 'alice@prisma.io',
    posts: {
      create: [
        { title: 'Post 1' },
        { title: 'Post 2' }
      ]
    },
    profile: {
      create: { bio: 'Hello!' }
    }
  }
})
```

### Create or connect

```typescript
const post = await prisma.post.create({
````
````

```
  data: {
    title: 'New Post',
    author: {
      connectOrCreate: {
        where: { email: 'alice@prisma.io' },
        create: { email: 'alice@prisma.io', name: 'Alice' }
      }
    }
  }
})
```

Connect existing

```typescript
const post = await prisma.post.create({
  data: {
    title: 'New Post',
    author: {
      connect: { id: 1 }
    }
  }
})

// Shorthand for foreign key
const post = await prisma.post.create({
  data: {
    title: 'New Post',
    authorId: 1
  }
})
```

Update Relations

Update related records

```typescript
const user = await prisma.user.update({
  where: { id: 1 },
  data: {
    posts: {
      update: {
        where: { id: 1 },
        data: { title: 'Updated Title' }
      }
    }
  }
})
```

Update many related

```typescript
const user = await prisma.user.update({
  where: { id: 1 },
  data: {
    posts: {
      updateMany: {
        where: { published: false },
        data: { published: true }
      }
    }
  }
})
```

Upsert related

```typescript
const user = await prisma.user.update({
  where: { id: 1 },
  data: {
    profile: {
      upsert: {
        create: { bio: 'New bio' },
        update: { bio: 'Updated bio' }
      }
    }
  }
})
```

Disconnect

```typescript
// 1-to-1 optional
const user = await prisma.user.update({
  where: { id: 1 },
  data: {
    profile: { disconnect: true }
  }
})
```

```typescript
// Many-to-many
const post = await prisma.post.update({
  where: { id: 1 },
  data: {
    tags: {
      disconnect: [{ id: 1 }, { id: 2 }]
    }
  }
})
```

Delete related

```typescript
const user = await prisma.user.update({
  where: { id: 1 },
  data: {
    posts: {
      delete: { id: 1 }
    }
  }
})

// Delete many
const user = await prisma.user.update({
  where: { id: 1 },
  data: {
    posts: {
      deleteMany: { published: false }
    }
  }
})
```

Set (replace all)

```typescript
// Replace all related records
const post = await prisma.post.update({
  where: { id: 1 },
  data: {
    tags: {
      set: [{ id: 1 }, { id: 2 }]
    }
  }
})
```

Relation Filters

some

At least one matches:

```typescript
const users = await prisma.user.findMany({
  where: {
    posts: { some: { published: true } }
  }
})
```

every

All match:

```typescript
const users = await prisma.user.findMany({
  where: {
    posts: { every: { published: true } }
  }
})
```

none

None match:

```typescript
const users = await prisma.user.findMany({
  where: {
    posts: { none: { published: true } }
  }
})
```

is / isNot (1-to-1)

```typescript
const users = await prisma.user.findMany({
  where: {
    profile: { is: { country: 'USA' } }
```

```
  }
})
```

Count Relations

```typescript
const users = await prisma.user.findMany({
  select: {
    name: true,
    _count: {
      select: { posts: true, followers: true }
    }
  }
})
// { name: 'Alice', _count: { posts: 5, followers: 100 } }
```

Filter counted relations

```typescript
const users = await prisma.user.findMany({
  select: {
    name: true,
    _count: {
      select: {
        posts: { where: { published: true } }
      }
    }
  }
})
```

27. .agents\skills\prisma-client-api\references\transactions.md

````
# Transactions

Execute multiple operations atomically.

## Sequential Transactions

Array of operations executed in order:

```typescript
const [user, post] = await prisma.$transaction([
  prisma.user.create({ data: { email: 'alice@prisma.io' } }),
  prisma.post.create({ data: { title: 'Hello', authorId: 1 } })
])
```

### All or nothing

If any operation fails, all are rolled back:

```typescript
try {
  await prisma.$transaction([
    prisma.user.create({ data: { email: 'alice@prisma.io' } }),
    prisma.user.create({ data: { email: 'alice@prisma.io' } }) // Duplicate!
  ])
} catch (e) {
  // Both operations rolled back
}
```

## Interactive Transactions

For complex logic and dependent operations:

```typescript
await prisma.$transaction(async (tx) => {
  // Decrement sender balance
  const sender = await tx.account.update({
    where: { id: senderId },
    data: { balance: { decrement: amount } }
  })

  // Check balance
  if (sender.balance < 0) {
    throw new Error('Insufficient funds')
  }

  // Increment recipient balance
  await tx.account.update({
    where: { id: recipientId },
    data: { balance: { increment: amount } }
  })
})
```

### Transaction options

```typescript
await prisma.$transaction(
  async (tx) => {
    // operations
  },
  {
    maxWait: 5000,    // Max wait to acquire lock (ms)
    timeout: 10000,   // Max transaction duration (ms)
    isolationLevel: 'Serializable'  // Isolation level
  }
)
```

### Isolation levels

| Level | Description |
|-------|-------------|
| `ReadUncommitted` | Lowest isolation, can read uncommitted changes |
| `ReadCommitted` | Only read committed changes |
| `RepeatableRead` | Consistent reads within transaction |
| `Serializable` | Highest isolation, serialized execution |

## Nested Writes

Automatic transactions for nested operations:

```typescript
// This is automatically a transaction
const user = await prisma.user.create({
  data: {
    email: 'alice@prisma.io',
    posts: {
      create: [
````
````

```
        { title: 'Post 1' },
        { title: 'Post 2' }
      ]
    },
    profile: {
      create: { bio: 'Hello!' }
    }
  }
})
```

Transaction Client

The `tx` parameter is a Prisma Client scoped to the transaction:

```typescript
await prisma.$transaction(async (tx) => {
  // Use tx instead of prisma
  await tx.user.create({ ... })
  await tx.post.create({ ... })

  // Can call methods
  const count = await tx.user.count()
})
```

OrThrow in Transactions

Use with interactive transactions:

```typescript
await prisma.$transaction(async (tx) => {
  // If not found, throws and rolls back entire transaction
  const user = await tx.user.findUniqueOrThrow({
    where: { id: 1 }
  })

  await tx.post.create({
    data: { title: 'New Post', authorId: user.id }
  })
})
```

Best Practices

Keep transactions short

```typescript
// Good - only DB operations in transaction
const data = prepareData() // Outside transaction
await prisma.$transaction(async (tx) => {
  await tx.user.create({ data })
})
```

Handle errors

```typescript
try {
  await prisma.$transaction(async (tx) => {
    // operations
  })
} catch (e) {
  if (e.code === 'P2002') {
    // Handle unique constraint violation
  }
  throw e
}
```

Use appropriate isolation

```typescript
// Default is fine for most cases
await prisma.$transaction(async (tx) => {
  // operations
})

// Use Serializable for strict consistency
await prisma.$transaction(
  async (tx) => { /* operations */ },
  { isolationLevel: 'Serializable' }
)
```

Sequential vs Interactive

Feature	Sequential	Interactive
Syntax	Array	Async function
Dependent ops	No	Yes
Conditional logic	No	Yes

| Performance | Better | More flexible |
| Use case | Simple batch | Complex logic |

28. .agents\skills\prisma-client-api\SKILL.md

```
---
name: prisma-client-api
description: Prisma Client API reference covering model queries, filters, operators, and client methods. Use when writing database queries,
using CRUD operations, filtering data, or configuring Prisma Client. Triggers on "prisma query", "findMany", "create", "update", "delete",
"$transaction".
license: MIT
metadata:
  author: prisma
  version: "7.6.0"
---

# Prisma Client API Reference

Complete API reference for Prisma Client. This skill provides guidance on model queries, filtering, relations, and client methods for current
Prisma projects.

## When to Apply

Reference this skill when:
- Writing database queries with Prisma Client
- Performing CRUD operations (create, read, update, delete)
- Filtering and sorting data
- Working with relations
- Using transactions
- Configuring client options

## Rule Categories by Priority

| Priority | Category | Impact | Prefix |
|----------|----------|--------|--------|
| 1 | Client Construction | HIGH | `constructor` |
| 2 | Model Queries | CRITICAL | `model-queries` |
| 3 | Query Shape | HIGH | `query-options` |
| 4 | Filtering | HIGH | `filters` |
| 5 | Relations | HIGH | `relations` |
| 6 | Transactions | CRITICAL | `transactions` |
| 7 | Raw SQL | CRITICAL | `raw-queries` |
| 8 | Client Methods | MEDIUM | `client-methods` |

## Quick Reference

- `constructor` - `PrismaClient` setup, adapter wiring, logging, and SQL commenter plugins
- `model-queries` - CRUD operations and bulk operations
- `query-options` - `select`, `include`, `omit`, sort, pagination
- `filters` - scalar and logical filter operators
- `relations` - relation reads and nested writes
- `transactions` - array and interactive transaction patterns
- `raw-queries` - `$queryRaw` and `$executeRaw` safety
- `client-methods` - lifecycle methods, extensions, and `satisfies` patterns for `prisma-client`

## Client Instantiation

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
})

const prisma = new PrismaClient({ adapter })
```

Model Query Methods

Method	Description
`findUnique()`	Find one record by unique field
`findUniqueOrThrow()`	Find one or throw error
`findFirst()`	Find first matching record
`findFirstOrThrow()`	Find first or throw error
`findMany()`	Find multiple records
`create()`	Create a new record
`createMany()`	Create multiple records
`createManyAndReturn()`	Create multiple and return them
`update()`	Update one record
`updateMany()`	Update multiple records
`updateManyAndReturn()`	Update multiple and return them
`upsert()`	Update or create record
`delete()`	Delete one record
`deleteMany()`	Delete multiple records
`count()`	Count matching records
`aggregate()`	Aggregate values (sum, avg, etc.)
`groupBy()`	Group and aggregate

Query Options

Option	Description
`where`	Filter conditions
```

`select`	Fields to include
`include`	Relations to load
`omit`	Fields to exclude
`orderBy`	Sort order
`take`	Limit results
`skip`	Skip results (pagination)
`cursor`	Cursor-based pagination
`distinct`	Unique values only

Client Methods

Method	Description
`$connect()`	Explicitly connect to database
`$disconnect()`	Disconnect from database
`$transaction()`	Execute transaction
`$queryRaw()`	Execute raw SQL query
`$executeRaw()`	Execute raw SQL command
`$on()`	Subscribe to events
`$extends()`	Add extensions

Quick Examples

Find records

```typescript
// Find by unique field
const user = await prisma.user.findUnique({
  where: { email: 'alice@prisma.io' }
})

// Find with filter
const users = await prisma.user.findMany({
  where: { role: 'ADMIN' },
  orderBy: { createdAt: 'desc' },
  take: 10
})
```

Create records

```typescript
const user = await prisma.user.create({
  data: {
    email: 'alice@prisma.io',
    name: 'Alice',
    posts: {
      create: { title: 'Hello World' }
    }
  },
  include: { posts: true }
})
```

Update records

```typescript
const user = await prisma.user.update({
  where: { id: 1 },
  data: { name: 'Alice Smith' }
})
```

Delete records

```typescript
await prisma.user.delete({
  where: { id: 1 }
})
```

Transactions

```typescript
const [user, post] = await prisma.$transaction([
  prisma.user.create({ data: { email: 'alice@prisma.io' } }),
  prisma.post.create({ data: { title: 'Hello', authorId: 1 } })
])
```

Rule Files

Detailed API documentation:

```
references/constructor.md        - PrismaClient constructor options
references/model-queries.md      - CRUD operations
references/query-options.md      - select, include, omit, where, orderBy
references/filters.md            - Filter conditions and operators
references/relations.md          - Relation queries and nested operations
references/transactions.md       - Transaction API
references/raw-queries.md        - $queryRaw, $executeRaw
references/client-methods.md     - $connect, $disconnect, $on, $extends
```

```

## Filter Operators

| Operator | Description |
|----------|-------------|
| `equals` | Exact match |
| `not` | Not equal |
| `in` | In array |
| `notIn` | Not in array |
| `lt`, `lte` | Less than |
| `gt`, `gte` | Greater than |
| `contains` | String contains |
| `startsWith` | String starts with |
| `endsWith` | String ends with |
| `mode` | Case sensitivity |

## Relation Filters

| Operator | Description |
|----------|-------------|
| `some` | At least one related record matches |
| `every` | All related records match |
| `none` | No related records match |
| `is` | Related record matches (1-to-1) |
| `isNot` | Related record doesn't match |

## Resources

- [Prisma Client API Reference](https://www.prisma.io/docs/orm/reference/prisma-client-reference)
- [CRUD Operations](https://www.prisma.io/docs/orm/prisma-client/queries/crud)
- [Filtering and Sorting](https://www.prisma.io/docs/orm/prisma-client/queries/filtering-and-sorting)

## How to Use

Pick the category from the table above, then open the matching reference file for implementation details and examples.
```

29. .agents\skills\prisma-compute\references\app-deploy-cli.md

````
# Prisma Platform CLI App Deploy

Use this reference for existing projects and for generated `compute:deploy` scripts.

## Package and Command

Compute app workflows are exposed through the Prisma Platform CLI package:

```bash
bunx @prisma/cli@latest --help
bunx @prisma/cli@latest app --help
bunx @prisma/cli@latest app deploy --help
bunx @prisma/cli@latest build logs --help
```

The examples in help output may call the binary `prisma-cli`. When using package runners, prefer:

```bash
bunx @prisma/cli@latest app deploy
npx @prisma/cli@latest app deploy
pnpm dlx @prisma/cli@latest app deploy
```

## Agent Skill Installation

`@prisma/cli` can install and refresh Prisma skills for local AI coding agents:

```bash
bunx @prisma/cli@latest agent install
bunx @prisma/cli@latest agent install --skill prisma-compute
bunx @prisma/cli@latest agent update
bunx @prisma/cli@latest agent status --json
```

`agent install` and `agent update` shell out to `skills@latest add prisma/skills` through the detected package runner. Use them when the user wants Prisma's agent context installed or refreshed; they are not a deployment command.

## Typed Compute Config

`prisma.compute.ts` is optional for normal single-app deploys and useful for reusable defaults or multi-app targets. Read [`compute-config.md`](compute-config.md) for config shapes, target selection, precedence, and monorepo rules. This reference only shows how deploy commands consume those settings.

## Auth and Project Binding

Useful commands:

```bash
bunx @prisma/cli@latest auth login
bunx @prisma/cli@latest auth whoami
bunx @prisma/cli@latest project list --json
bunx @prisma/cli@latest project show
bunx @prisma/cli@latest project link <project-id-or-name>
```

`@prisma/cli` can keep multiple local browser-login workspace sessions. Running `auth login` again for a different workspace should add/update that workspace session and make it active; it should not delete the existing workspace session. The active workspace pointer decides which stored OAuth workspace normal commands use.

For agents, prefer this flow before project/app mutations:

```bash
bunx @prisma/cli@latest auth whoami --json
bunx @prisma/cli@latest auth workspace list --json
bunx @prisma/cli@latest auth workspace use <workspace-id>
```

Use workspace ids from `auth workspace list --json` when possible. Names are friendlier for humans but can be ambiguous. Use `auth workspace use` with no argument for a human interactive picker; headless scripts should pass an id because no-argument `use` fails non-interactively when multiple local OAuth workspaces exist.

If the active workspace is logged out or its refresh fails, the CLI intentionally does not auto-select another cached workspace. Choose the next workspace explicitly:

Use `auth workspace list --json`, then `auth workspace use <workspace-id>`.

To clean up one local OAuth workspace without clearing every stored workspace session:

```bash
bunx @prisma/cli@latest auth workspace logout <workspace-id-or-name>
# equivalent:
bunx @prisma/cli@latest auth logout --workspace <workspace-id-or-name>
```

Plain `auth logout` clears all local OAuth workspace sessions. It does not unset `PRISMA_SERVICE_TOKEN`.

For a new linked project:

```bash
bunx @prisma/cli@latest project create my-app --json
````
````

```

For non-interactive or CI work, `@prisma/cli` accepts a workspace service token through `PRISMA_SERVICE_TOKEN`. A non-empty service token takes precedence over stored browser-login credentials, so local OAuth workspace switching does not affect command execution while the env var is set. `auth workspace list --json` may still show local OAuth sessions, but they are not switchable until the service-token env var is unset. Verify auth with `auth whoami` and never print the token value.

If `PRISMA_SERVICE_TOKEN` is set but empty, unset it or provide a real token. The CLI should fail instead of silently falling back to local OAuth credentials.

Local auth storage is useful for debugging but should not be printed verbatim:

- `PRISMA_COMPUTE_AUTH_FILE` can override the auth file path.
- On macOS, the default OAuth credentials file is `~/Library/Application Support/prisma/auth.json`.
- Workspace metadata and the active workspace pointer live beside it as `auth.context.json`.
- Project pins live in `.prisma/local.json`.
- Local CLI state such as selected app and known live deployment lives in `.prisma/cli/state.json`, rooted near `prisma.compute.ts` when a config is discovered.

## Project, Branch, Database, and Env Scope

Compute deploys resolve a target project, app, and branch. Be explicit when the user's intent is not the already linked default project/app:

```bash
bunx @prisma/cli@latest project show --json
bunx @prisma/cli@latest app deploy --project proj_123 --app my-api --branch feature/login --json
```

If `prisma.compute.ts` defines a `name` or an `apps` key, that config can provide the app name. `--app` and `PRISMA_APP_ID` rank above the config value. `[app]` selects a target from `apps`:

```bash
bunx @prisma/cli@latest app deploy api --project proj_123 --branch feature/login --json
```

See [`compute-config.md`](compute-config.md) for no-argument target inference, deploy-all, and build/run target rules.

Branch scope must line up across deploys, databases, and env vars:

- `app deploy --branch <git-name>` creates a deployment for that branch.
- `database create <name> --branch <git-name>` creates a Prisma Postgres database for that branch scope.
- `project env add/update/list/remove --branch <git-name>` manages branch-specific env overrides.
- `project env add/update/list/remove --role production` manages production env.
- `project env add/update/list/remove --role preview` manages preview-template env.

Do not assume a local Git branch was used by the CLI unless the generated script or command output says so. If a user asks for `feature/login`, pass `--branch feature/login` consistently to app, database, and env commands.

Promotion is a separate production action: `app promote <deployment-id>` rebuilds a deployment with production env vars. Do not treat a preview branch deploy as production promotion.

Deployment Story: GitHub vs CLI

When a Compute app is connected to GitHub push-to-deploy, the default branch is the production deploy path. If a PR has been merged into `main` or another configured default branch, the natural answer is that the changes should appear in production after the production deployment completes; use CLI deploys for explicit manual deploys, local-source deploys, or repositories that are not using GitHub push-to-deploy.

`app show`, `app list-deploys`, and `app logs` expose `--app`, `--project`, and for logs `--deployment`, not `--branch`. For branch debugging, capture the deployment id from deploy JSON and inspect that deployment or its logs.

`app deploy --create-project <name>` creates and links a new Project before deploying. Use it only when the user wants a new Project. It conflicts with `--project` and `PRISMA_PROJECT_ID`, and `--yes` alone does not choose Project scope.

`app deploy --region <region>` only applies when deploy creates a new app. Existing apps keep their current region. Use `prisma.compute.ts` `region` for a durable default, and use the flag only for one-off new-app placement.

Database and Env

Create a Prisma Postgres database for the linked project:

```bash
bunx @prisma/cli@latest database create main --branch main --json
```

Manage project env vars:

```bash
bunx @prisma/cli@latest project env list
bunx @prisma/cli@latest project env add --file .env --role production
bunx @prisma/cli@latest project env add --file .env.preview --role preview
bunx @prisma/cli@latest project env add DATABASE_URL=postgresql://... --branch feature/foo
bunx @prisma/cli@latest project env update --file .env --role production
bunx @prisma/cli@latest project env update DATABASE_URL=postgresql://... --branch feature/foo
bunx @prisma/cli@latest project env list --branch feature/foo
bunx @prisma/cli@latest project env remove STRIPE_KEY --role preview
```

`app deploy --env .env` loads environment variables from a file for the deployment. A config-backed deploy can instead load env through `prisma.compute.ts` `env`. Neither path is a migration command or seed command.

Database setup is not part of `prisma.compute.ts`. Keep database intent explicit with `database create` and project env commands. Do not add database setup to deploy examples. Treat any generated connection URL as a one-time secret.
```

Database and env guardrails:

- Deploys do not run migrations, seed data, or schema push. Run the app's own Prisma database command after deploy setup when needed.
- In deploy-all, every target on the same branch shares branch-scoped project env unless you assign app-specific env values yourself.
- Existing database env values supplied through `--env DATABASE_URL=...`, `--env DIRECT_URL=...`, an env file, or project env should be treated as the source of truth.
- Known non-PostgreSQL Prisma schema sources should not be wired to Prisma Postgres automatically.

Project Git, Branch, and Database Operations

These commands are part of the same Platform CLI surface and often matter while preparing Compute deploys:

```bash
bunx @prisma/cli@latest branch list --json
bunx @prisma/cli@latest git connect git@github.com:org/repo.git --project proj_123
bunx @prisma/cli@latest git disconnect --project proj_123
bunx @prisma/cli@latest database list --branch feature/foo --json
bunx @prisma/cli@latest database show db_123 --json
bunx @prisma/cli@latest database remove db_123 --confirm db_123
bunx @prisma/cli@latest database connection list db_123 --json
bunx @prisma/cli@latest database connection create db_123 --name readonly
bunx @prisma/cli@latest database connection remove conn_123 --confirm conn_123
bunx @prisma/cli@latest database connection rotate conn_123 --confirm conn_123
bunx @prisma/cli@latest database usage db_123 --json
bunx @prisma/cli@latest database backup list db_123 --json
bunx @prisma/cli@latest database restore db_123 --backup bkp_123 --confirm db_123
bunx @prisma/cli@latest project rename new-name --project proj_123
bunx @prisma/cli@latest project transfer proj_123 --to-workspace wksp_456 --confirm proj_123
bunx @prisma/cli@latest project remove proj_123 --confirm proj_123
```

Destructive and ownership-changing commands (`remove`, `restore`, `transfer`, `connection rotate`) require exact `--confirm <id>`; `--yes` is not enough.

Git integration connects a Project to a GitHub repository. Console-side GitHub import can create a Compute app and trigger push-to-deploy for the connected repository, including default-branch production deploys. The CLI `git connect` command is setup, not a local deploy command; use `app deploy` for explicit CLI deploys.

For GitHub-driven deploys, inspect the Console/build-runner state, deployment records, build logs, or the `Prisma Compute Deploy` GitHub check run instead of assuming local CLI output exists. The build runner can perform branch-aware database/env wiring: a preview branch with a Prisma schema and no `DATABASE_URL` can get a branch-scoped preview database, while production can wire a missing `DATABASE_URL` template from an existing ready database. GitHub check runs are the guided feedback path; do not promise Vercel-style PR comments.

Database and database-connection commands never print stored secret values in list/show output. `database create` and `database connection create` return a one-time connection URL; treat it as a secret, store it immediately in env if needed, and do not echo it back in summaries. Removal requires exact `--confirm <id>`; `--yes` is not enough.

Build and Run Locally

Before deploy, verify that the app can produce a Compute artifact:

```bash
bunx @prisma/cli@latest app build --build-type auto
bunx @prisma/cli@latest app run --build-type auto --port 3000
```

For Bun/server entrypoints:

```bash
bunx @prisma/cli@latest app build --build-type bun --entry src/index.ts
bunx @prisma/cli@latest app run --build-type bun --entry src/index.ts --port 8080
```

For NestJS, use `app build` to validate the Compute artifact and run the framework's own dev command locally:

```bash
bunx @prisma/cli@latest app build --build-type nestjs
bun run dev
```

With a compute config, pass the target name instead of repeating framework/entry/port flags:

```bash
bunx @prisma/cli@latest app build api
bunx @prisma/cli@latest app run api --port 8080
```

`app run --port` sets `PORT` for local development. It does not rewrite an app's explicit host binding, so a local run is not enough to prove the deployed server is reachable from ingress.

`app run --build-type nestjs` is not supported. If a config-backed NestJS target is selected, run the Nest dev server directly instead.

Deploy

Deploy with prompts:

```bash
bunx @prisma/cli@latest app deploy
```

Agent/script-friendly deploy (do not assume production; add `--prod --yes` only when the user intends a production deploy, and note the first production deploy of an App auto-promotes without `--prod`):

```bash
bunx @prisma/cli@latest app deploy \
  --json \
  --no-interactive \
  --env .env
```

Build-then-verify path for CI: `--no-promote` builds a candidate deployment without changing the live one; it is reachable at its own candidate URL and promoted later with `app promote <deployment-id>`:

```bash
bunx @prisma/cli@latest app deploy --no-promote --json --no-interactive
```

For preview branches, omit `--prod` unless the user explicitly intends a production deploy:

```bash
bunx @prisma/cli@latest app deploy \
  --branch feature/foo \
  --json \
  --no-interactive \
  --env .env.preview
```

After a real deploy, verify the public deployment URL. Do not stop at "deploy succeeded" or a local `app run` check:

```bash
curl -i https://<deployment-url>
```

If the deploy command returns JSON, parse the URL from the result and request that exact public URL. Do not accidentally test `localhost` or `127.0.0.1` instead of public ingress.

Create/link a project during deploy:

```bash
bunx @prisma/cli@latest app deploy \
  --create-project my-app \
  --prod \
  --yes \
  --env .env
```

Deploy with framework and port:

```bash
bunx @prisma/cli@latest app deploy \
  --framework hono \
  --http-port 8080 \
  --prod \
  --yes \
  --env .env
```

Deploy a newly created app in a specific region:

```bash
bunx @prisma/cli@latest app deploy \
  --app my-api \
  --region us-west-1 \
  --prod \
  --yes \
  --env .env
```

`--region` is a new-app placement hint. It does not move an existing app.

Deploy a preview branch with framework and port:

```bash
bunx @prisma/cli@latest app deploy \
  --framework hono \
  --branch feature/foo \
  --http-port 8080 \
  --json \
  --no-interactive \
  --env .env.preview
```

Bun-style app with explicit entrypoint:

```bash
bunx @prisma/cli@latest app deploy \
  --framework bun \
  --entry src/index.ts \
  --http-port 8080 \
  --prod \
  --yes \
  --env .env
```

`--entry <path>` without `--framework` is treated as a Bun app deploy.

Config-backed Bun-style app:

```bash
bunx @prisma/cli@latest app deploy api --prod --yes --env .env
```

Use config for stable app defaults, and flags for one-off project, branch, region, env, and production choices. Keep database setup in explicit database and project-env commands.

Operations

Inspect and open:

```bash
bunx @prisma/cli@latest app show --json
bunx @prisma/cli@latest app open
```

Deployments:

```bash
bunx @prisma/cli@latest app list-deploys --json
bunx @prisma/cli@latest app show-deploy <deployment-id> --json
bunx @prisma/cli@latest app promote <deployment-id> --yes
bunx @prisma/cli@latest app rollback --to <deployment-id> --yes
bunx @prisma/cli@latest app remove --app my-api --yes
```

Logs:

```bash
bunx @prisma/cli@latest app logs
bunx @prisma/cli@latest app logs --deployment <deployment-id>
bunx @prisma/cli@latest app logs --json
```

Build logs for GitHub/Console builds:

```bash
bunx @prisma/cli@latest build logs <build-id>
bunx @prisma/cli@latest build logs <build-id> --follow
bunx @prisma/cli@latest build logs <build-id> --json
```

`build logs` streams build output keyed by a Build id from a GitHub/Console build or check run. It is separate from runtime `app logs`, which are keyed by the current app deployment or a deployment id.

Domains:

```bash
bunx @prisma/cli@latest app domain add shop.example.com
bunx @prisma/cli@latest app domain show shop.example.com
bunx @prisma/cli@latest app domain wait shop.example.com --timeout 15m
bunx @prisma/cli@latest app domain retry shop.example.com
bunx @prisma/cli@latest app domain remove shop.example.com
```

Custom domain commands target production branch runtime. Do not use a preview branch for production domain setup.

Output Handling

When `--json` is available, parse the JSON and summarize:

- project id/name
- branch name
- app id/name
- deployment id/status
- build id when present
- deployment URL
- database id/name if one was created

Do not print secret env var values.

30. .agents\skills\prisma-compute\references\compute-config.md

````
# Prisma Compute Config

Use this reference when creating or updating `prisma.compute.ts`, especially for monorepos, multi-app deploys, reusable framework defaults, env inputs, ports, entrypoints, or build settings.

`prisma.compute.ts` is not required for every deploy. A simple app can deploy with `@prisma/cli app deploy --framework ... --entry ... --http-port ... --env ...`. The config file exists to make those app-level defaults typed and repeatable.

For monorepos or multi-app repositories, use `prisma.compute.ts`: it is the practical way to tell Compute which app target lives at which `root` and which framework/entry/env defaults belong to each target.

## Generating a Config with `init`

Prefer `bunx @prisma/cli@latest init` over hand-writing a fresh single-app config. It detects the framework from the same registry deploy uses, pins `name`, `framework`, and `httpPort` (plus `entry` for Bun and Hono), previews every value with its source, offers the `@prisma/compute-sdk` devDependency for editor types, and offers the Project link. Useful flags: `--framework`, `--entry`, `--http-port`, `--name`, `--no-link`, `--json`.

`--format json` writes a dependency-free static `prisma.compute.json` instead of the TypeScript config; a later explicit `init --format ts` converts it in place when the config needs to become programmatic. `init` fails with `INIT_CONFIG_EXISTS` when any compute config already exists, never scaffolds application code, and never deploys. Multi-app monorepo configs are still written by hand.

## File Names and Discovery

The canonical file is `prisma.compute.ts`. The loader also accepts:

```text
prisma.compute.mts
prisma.compute.js
prisma.compute.mjs
prisma.compute.cjs
prisma.compute.json
```

`prisma.compute.json` is the static, dependency-free variant of the same config; it is discovered and loaded like the others.

Keep exactly one compute config file in a directory. If multiple names exist together, the CLI reports `COMPUTE_CONFIG_INVALID`.

The CLI searches from the invocation directory up to the repository or workspace boundary. Boundaries include `.git`, `pnpm-workspace.yaml`, `bun.lock`, `bun.lockb`, or `package.json#workspaces`. Config-relative paths such as `root` and `env.file` resolve from the config file directory. `--env` flag paths still resolve from the invocation directory.

When a config is discovered, its directory becomes the Compute project directory for local state: `.prisma/local.json` and `.prisma/cli/state.json` live beside that config, not necessarily inside the app root.

## Basic Shape

Import `defineComputeConfig` from `@prisma/compute-sdk/config`. The CLI aliases this helper when loading the config, so the command can evaluate the config without a local SDK install solely for runtime loading.

```typescript
import { defineComputeConfig } from "@prisma/compute-sdk/config";

export default defineComputeConfig({
  app: {
    name: "api",
    framework: "hono",
    httpPort: 8080,
    env: ".env",
  },
});
```

JavaScript configs can default-export a plain object, but prefer `prisma.compute.ts` for type checking.

Define exactly one of:

- `app` for a single deploy target
- `apps` for a monorepo or multi-app repository

Do not define both. Besides `app`/`apps`, the only other allowed top-level key is `region`: a project-level default region applied when deploy creates new apps, overridable per app and by `--region`.

## App Fields

Each app target accepts:

| Field | Meaning |
|---|---|
| `name` | Deployed app name. Defaults to the `apps` key, then CLI inference. |
| `region` | Compute region id used only when deploy creates a new app. Existing apps keep their current region. |
| `root` | App directory relative to the config file. Defaults to the config directory. |
| `framework` | Deploy framework: `nextjs`, `nuxt`, `astro`, `hono`, `nestjs`, `tanstack-start`, `custom`, or `bun`. |
| `entry` | Entrypoint path for Bun/Hono-style deploys, relative to the app root. |
| `httpPort` | Deployed HTTP port. Use this for fixed-port apps. |
| `env` | Dotenv file path string, or `{ file, vars }`. Paths resolve from the config directory. |
| `build` | `{ command, outputDirectory, entrypoint }`. Present means the config owns build settings for that target. |

`env` examples:
````

```typescript
export default defineComputeConfig({
  app: {
    framework: "nextjs",
    env: {
      file: [".env", ".env.production"],
      vars: {
        NODE_ENV: "production",
      },
    },
  },
});
```

Do not put secrets directly in committed `vars`. Keep secret values in platform env, CI secrets, or dotenv files that are intentionally managed outside version control.

`build` examples:

```typescript
export default defineComputeConfig({
  app: {
    framework: "nextjs",
    build: {
      command: "pnpm build",
      outputDirectory: ".next/standalone",
    },
  },
});
```

Use `command: null` to skip the build step only when the app root already contains the deployable artifact.

For a custom or prebuilt artifact, make the deploy target explicit:

```typescript
export default defineComputeConfig({
  app: {
    framework: "custom",
    build: {
      command: "npm run build",
      outputDirectory: "build",
      entrypoint: "handler.js",
    },
  },
});
```

`build.entrypoint` is relative to `build.outputDirectory` when an output directory is set. For Bun/Hono configs without an output directory, an entrypoint-backed build can supply the source entrypoint. Do not set both `entry` and `build.entrypoint` unless they describe the same file.

A config `build` block is accepted for every supported framework: the config-backed build types are `nextjs`, `nuxt`, `astro`, `nestjs`, `tanstack-start`, `custom`, and `bun` (`hono` builds through the `bun` strategy). Only `custom` requires one (`build.outputDirectory` and `build.entrypoint`); for the others it overrides inferred build settings.

Monorepos and Multi-App Repos

For monorepos, put `prisma.compute.ts` at the repo or workspace root and use `apps`. This keeps project binding and local `.prisma/` state at the repo root while each app builds from its own `root`.

```typescript
import { defineComputeConfig } from "@prisma/compute-sdk/config";

export default defineComputeConfig({
  apps: {
    web: {
      root: "apps/web",
      framework: "nextjs",
      env: "apps/web/.env",
    },
    api: {
      root: "apps/api",
      framework: "hono",
      entry: "src/index.ts",
      httpPort: 8080,
      env: {
        file: "apps/api/.env",
        vars: {
          LOG_LEVEL: "info",
        },
      },
    },
    frontend: {
      root: "apps/frontend",
      framework: "custom",
      build: {
        command: "pnpm --filter frontend build",
        outputDirectory: "dist/server",
        entrypoint: "index.mjs",
      },
    },
  },
```

});
```

Target selection:

```bash
bunx @prisma/cli@latest app deploy web
bunx @prisma/cli@latest app deploy api
bunx @prisma/cli@latest app build api
bunx @prisma/cli@latest app run api --port 8080
```

If no `[app]` argument is passed, commands can infer the target from the invocation directory when it is inside a configured `root`. The deepest matching root wins. If no target is inferred from a multi-app config, a bare deploy can deploy all targets in declaration order:

```bash
bunx @prisma/cli@latest app deploy --branch feature/foo --json --no-interactive
```

Deploy-all rejects per-app overrides such as `--app`, `--framework`, `--entry`, `--http-port`, `--region`, `--env`, and `PRISMA_APP_ID`. Project, branch, production, and confirmation flags still apply to the whole run. Keep database setup in explicit database and project-env commands.

`app build` and `app run` still need one target in multi-app configs because a local build/run command cannot operate N apps at once.

Additional target rules:

- A single-entry `apps` map can deploy its only target without an argument.
- With a single `app` config, `[app]` is accepted only when it equals the configured `name`.
- `[app]` without any compute config file is a usage error.

Precedence

Explicit flags win over config values:

- `--framework` overrides `framework`
- `--entry` overrides `entry`
- `--http-port` overrides `httpPort`
- `--region` overrides `region`
- any `--env` flag replaces all config env inputs
- `--app` and `PRISMA_APP_ID` rank above config app names

`region` is not an app selector. Config `region` and `--region` are only used when deploy creates a new app. If the selected app already exists, deploy keeps that app's existing region.

`prisma.compute.ts` never selects Workspace, Project, Branch, or production intent. Keep those in CLI flags, environment variables, `.prisma/local.json`, or CI configuration:

```bash
bunx @prisma/cli@latest app deploy api \
  --project proj_123 \
  --branch feature/foo \
  --prod \
  --yes
```

Database Scope

The config does not declare databases. Keep database intent in `database create`, project env commands, or external automation. Read [`app-deploy-cli.md`](app-deploy-cli.md) for deploy-all, migration, and env-var guardrails.

Relationship to `prisma.config.ts`

Do not put Compute deploy defaults in `prisma.config.ts`. Prisma ORM uses `prisma.config.ts`, while Compute uses `prisma.compute.ts`.
```

31. .agents\skills\prisma-compute\references\create-prisma.md

````
# create-prisma Compute Flow

Use this reference when creating a new app with Prisma and optionally deploying it to Prisma Compute.

Do not use `create-prisma` as the deploy path for an existing app. For existing projects, use the generated `compute:deploy` script when present, or call `bunx @prisma/cli@latest app deploy` directly.

## Reference

Useful scaffold checks:

```bash
bunx create-prisma@latest --help
bunx create-prisma@latest --version
```

Use `create-prisma@latest` for new-project scaffolding.

## Supported Templates

`create-prisma@latest` scaffolds `hono`, `elysia`, `nest`, `next`, `svelte`, `astro`, `nuxt`, `tanstack-start`, and `turborepo`.

Integrated `--deploy` support applies to `hono`, `elysia`, `nest`, `next`, `astro`, `nuxt`, `tanstack-start`, and `turborepo`. For `turborepo`, the generated config target is usually `api`.

The scaffold template name is `nest`, but the Compute deploy framework/config key is `nestjs`.

`svelte` is scaffold-only for Compute because `@prisma/cli app deploy --framework` has no `svelte` key.

## Basic Commands

Interactive creation:

```bash
bunx create-prisma@latest
```

Non-interactive scaffold only:

```bash
bunx create-prisma@latest \
  --name my-api \
  --template hono \
  --provider postgresql \
  --no-install \
  --no-generate \
  --no-migrate-and-seed \
  --no-deploy
```

Create and deploy a supported template:

```bash
bunx create-prisma@latest \
  --name my-api \
  --template hono \
  --provider postgresql \
  --deploy
```

## PostgreSQL and Database Behavior

With PostgreSQL, no explicit `--database-url`, and no `--no-prisma-postgres`, the Compute flow can create:

- a Prisma Compute project
- a `main` Prisma Postgres database on the `main` branch
- a `.env` file containing `DATABASE_URL`
- an initial Compute deployment with env vars loaded from `.env`

`create-prisma` is the new-project path. If the user needs a later preview branch deploy, use the generated `compute:deploy` script or `@prisma/cli app deploy --branch <git-name>` after the app exists. Keep branch names aligned across `app deploy --branch`, `database create --branch`, and `project env ... --branch`.

For unattended local tests, pass `--no-prisma-postgres` unless you intentionally want provisioning:

```bash
bunx create-prisma@latest \
  --name smoke-app \
  --template hono \
  --provider postgresql \
  --no-prisma-postgres \
  --database-url "postgresql://USER:PASSWORD@HOST:PORT/DB" \
  --no-deploy
```

Do not deploy placeholder database URLs. If `DATABASE_URL` came from a placeholder default, omit it from deploy env and ask the user for a real production database.

## Generated Deploy Script
````

When the deploy flow is selected, `create-prisma` can add:

```json
{
  "scripts": {
    "compute:deploy": "bunx @prisma/cli@latest app deploy --prod --yes ..."
  }
}
```

Use the actual generated script from `package.json`; do not reconstruct it from memory. The script redeploys app code using generated flags and/or `prisma.compute.ts`. It does not create a new project, create a new database, run migrations, or seed data. If a scaffolded project does not have `compute:deploy`, use `@prisma/cli app deploy` directly.

Inspect the generated `package.json`, `prisma.compute.ts`, and README before editing deploy behavior.

Generated Files to Preserve

Preserve generated framework runtime files and `prisma.compute.ts` unless you are intentionally changing the deploy target. For framework-specific deploy/runtime details, read [`frameworks.md`](frameworks.md).

All Prisma 7 scaffolds:

- use `prisma.config.ts`
- load `dotenv/config` where the runtime supports it
- generate Prisma Client into a template-local path such as `src/generated/prisma`
- use `@prisma/adapter-pg` with a `DATABASE_URL` connection string for PostgreSQL

Addon Notes

`create-prisma` supports `--skills`, `--mcp`, and `--extension`. Those are separate from Compute deployment. Do not imply that enabling skills or MCP deploys the app.

Failure Handling

If `--deploy` is explicit and setup cannot authenticate, cannot run the Platform CLI, or cannot complete the integrated deploy, report that deploy failed and keep the scaffolded project. Do not delete the user's files.

32. .agents\skills\prisma-compute\references\frameworks.md

````
# Prisma Compute Framework Readiness

Use this reference when deciding whether and how an app can deploy to Prisma Compute.

## CLI-First Model

Treat `@prisma/cli app deploy` as the deployment surface. Treat `create-prisma` as a new-project scaffold that can generate useful defaults
and, for some templates, a `compute:deploy` script.

Compute deploy supports these framework keys:

```text
nextjs
nuxt
astro
hono
nestjs
tanstack-start
custom
bun
```

Auto-detection:

- Next.js: `next.config.*` or `next` dependency
- Nuxt: `nuxt.config.*` or `nuxt` dependency
- Astro: `astro.config.*` or `astro` dependency
- Hono: `hono` dependency
- NestJS: `nest-cli.json` or `@nestjs/core` dependency
- TanStack Start: `@tanstack/react-start` or `@tanstack/solid-start`
- Custom artifact: explicit `framework: "custom"` plus `build.outputDirectory` and `build.entrypoint` in `prisma.compute.ts`
- Bun: explicit `--entry <path>` or `--framework bun`

If detection is ambiguous, set `framework` in `prisma.compute.ts` or pass a supported `--framework` value. If the app is a source-level plain
server, use `framework: "bun"` plus `entry`, or pass `--framework bun --entry <path>`, after verifying the server entrypoint. If the app
already produces a runnable Node artifact, use `framework: "custom"` with `build.outputDirectory` and `build.entrypoint`.

## CLI Matrix

| App shape | Deploy command shape | Auto-detected | Required output/entry | Notes |
|-----------|----------------------|---------------|-----------------------|-------|
| Next.js | `--framework nextjs` | Yes | standalone `server.js` output | Requires `output: "standalone"` |
| Nuxt | `--framework nuxt` | Yes | `.output/server/index.mjs` | Framework strategy supplies build defaults; a config `build` block is
optional |
| Astro | `--framework astro` | Yes | standalone Node server artifact | Framework strategy supplies build defaults; a config `build` block is
optional |
| Hono | `--framework hono` | Yes | Bun entry from `main`, `module`, `--entry`, or `src/index.ts` | Usually fixed port `8080` in generated
config/scripts |
| NestJS | `--framework nestjs` | Yes | NestJS server artifact | Omit host or bind to `0.0.0.0`; a config `build` block is optional |
| TanStack Start | `--framework tanstack-start` | Yes | `.output/server/index.mjs` | Requires Nitro node output |
| Custom artifact | config-backed `framework: "custom"` | No | configured `build.outputDirectory` and `build.entrypoint` | Use for
prebuilt/custom-built Node artifacts |
| Bun / plain server | `--framework bun --entry <path>` | With explicit entry | server entrypoint | Use for Elysia and custom HTTP servers |
| Elysia | `--framework bun --entry src/index.ts` | No dedicated deploy key | Bun entrypoint | Preserve port/host handling |
| SvelteKit | No deploy framework key | No | Node adapter/prebuilt artifact | Do not deploy `vite preview` |
| Turborepo | Deploy concrete app targets | No | app-specific entry/output | Prefer `prisma.compute.ts` with `apps` |

`app build --build-type` uses the framework build type. Build types include `auto`, `nextjs`, `nuxt`, `astro`, `nestjs`, `tanstack-start`,
`custom`, and `bun`.

`app run --build-type` is local-dev oriented and supports `auto`, `bun`, and `nextjs`. It streams the local dev server and is not proof that
the deployed app is reachable through public ingress.

`prisma.compute.ts` can set framework, entrypoint, HTTP port, env inputs, app root, region, and build settings. A config `build` block is
accepted for every supported framework; all build types are config-backed (`nextjs`, `nuxt`, `astro`, `nestjs`, `tanstack-start`, `custom`,
`bun`; `hono` builds through the `bun` strategy). For Nuxt, Astro, and NestJS the framework strategy supplies the default build command and
output, so a `build` block is optional and normally unnecessary, but it overrides those defaults when present. Only `custom` requires one.

Config snippets below assume:

```typescript
import { defineComputeConfig } from "@prisma/compute-sdk/config";
```

## Universal Runtime Requirements

Compute needs a server process:

- It must listen on the deployed HTTP port. `@prisma/cli app deploy` defaults to the framework's default HTTP port (3000 for most frameworks,
4321 for Astro) unless `--http-port` is passed.
- It must bind on all interfaces. Do not hard-code `localhost` or `127.0.0.1` for a deployed server; use `0.0.0.0`, `server.host: true`, or
the framework equivalent.
- It must have a deployable entrypoint or recognized framework output.
- It must not rely on a preview-only command such as `vite preview`.
- It must receive env vars through `--env`, project env, branch env, or external automation.

Check host and port together. A listener on the right port but bound to loopback can appear ready while public ingress cannot reach it.

## Next.js
````

Deploy shape:

```bash
bunx @prisma/cli@latest app deploy --framework nextjs --env .env
```

`next.config.ts` must include standalone output:

```typescript
import type { NextConfig } from "next"

const nextConfig: NextConfig = {
  output: "standalone",
}

export default nextConfig
```

Do not pass `--entry` with `nextjs`; the CLI derives the runtime entrypoint from framework build output.

Do not set `HOSTNAME=localhost` or `HOSTNAME=127.0.0.1` in deploy env. If the standalone server host is overridden, use `0.0.0.0`.

Hono

Deploy shape:

```bash
bunx @prisma/cli@latest app deploy \
  --framework hono \
  --http-port 8080 \
  --env .env
```

Config shape:

```typescript
export default defineComputeConfig({
  app: {
    framework: "hono",
    entry: "src/index.ts",
    httpPort: 8080,
    env: ".env",
  },
});
```

Project expectations:

- `package.json` has `main` or `module` pointing at the entrypoint, or deploy passes `--entry src/index.ts`
- server uses `@hono/node-server`
- code reads `process.env.PORT` and defaults to the same port used by `--http-port`
- code does not set `hostname` to `localhost` or `127.0.0.1`; if hostname is set explicitly, use `0.0.0.0`

Example runtime shape:

```typescript
const rawPort = (process.env.PORT ?? "").trim()
const parsedPort = rawPort.length > 0 ? Number(rawPort) : Number.NaN
const port = Number.isInteger(parsedPort) ? parsedPort : 8080
serve({ fetch: app.fetch, port })
```

NestJS

Deploy shape:

```bash
bunx @prisma/cli@latest app deploy --framework nestjs --env .env
```

Config shape:

```typescript
export default defineComputeConfig({
  app: {
    framework: "nestjs",
    env: ".env",
  },
});
```

Project expectations:

- detection uses `nest-cli.json` or the `@nestjs/core` dependency; pass `--framework nestjs` when neither signal is present
- `src/main.ts` or the compiled runtime must start an HTTP server
- read `process.env.PORT` and default to the same port used by `--http-port`
- omit the host argument in `app.listen(port)` or pass `"0.0.0.0"`; do not pass `"localhost"` or `"127.0.0.1"`
- use `app build --build-type nestjs` for a Compute artifact check; `app run --build-type nestjs` is not supported, so use the Nest dev server locally

Example runtime shape:

```typescript
```

```
const port = Number(process.env.PORT ?? "3000")
await app.listen(port)
```

TanStack Start

Deploy shape:

```bash
bunx @prisma/cli@latest app deploy --framework tanstack-start --env .env
```

Expected `vite.config.ts` shape:

```typescript
import { defineConfig } from "vite"
import viteReact from "@vitejs/plugin-react"
import { tanstackStart } from "@tanstack/react-start/plugin/vite"
import { nitro } from "nitro/vite"

export default defineConfig({
  plugins: [tanstackStart(), nitro(), viteReact()],
})
```

Preserve these details:

- keep `nitro` in `dependencies`
- keep `import { nitro } from "nitro/vite"`
- keep `nitro()` in the Vite plugin list
- keep the React Vite plugin after `tanstackStart()`
- keep Nitro on its default node server preset; do not switch to edge, static, Cloudflare, or another non-Node preset for Compute

The build command is `vite build`. The build must produce `.output/server/index.mjs`, and the production start shape is:

```json
{
  "scripts": {
    "build": "vite build",
    "start": "node .output/server/index.mjs"
  }
}
```

Do not deploy TanStack Start as a Bun entrypoint such as `src/router.tsx`. If `.output/server/index.mjs` is missing, fix the TanStack/Nitro build path.

Make sure Nitro does not bind only to localhost in deployment. If host env/config is customized, use the framework's all-interface host setting rather than `localhost`.

Nuxt

Deploy shape:

```bash
bunx @prisma/cli@latest app deploy --framework nuxt --env .env
```

Config shape:

```typescript
export default defineComputeConfig({
  app: {
    framework: "nuxt",
    env: ".env",
  },
});
```

Nuxt uses Nitro output at `.output/server/index.mjs`. Keep the Nitro preset compatible with a Node server runtime.

Astro

Deploy shape:

```bash
bunx @prisma/cli@latest app deploy --framework astro --env .env
```

Config shape:

```typescript
export default defineComputeConfig({
  app: {
    framework: "astro",
    httpPort: 4321,
    env: ".env",
  },
});
```

Astro Compute-style server output usually needs:

```javascript
import { defineConfig } from "astro/config"
import node from "@astrojs/node"

export default defineConfig({
  output: "server",
  adapter: node({ mode: "standalone" }),
  server: { host: true },
})
```

Bun, Elysia, and Plain Source Servers

Use the Bun deploy key for app shapes without a dedicated `--framework` value:

```bash
bunx @prisma/cli@latest app deploy \
  --framework bun \
  --entry src/index.ts \
  --http-port 8080 \
  --env .env
```

`app deploy` also treats `--entry <path>` without `--framework` as a Bun app deploy.

Requirements:

- pass `--entry` unless `package.json` `main` or `module` points at the runtime entrypoint
- ensure the entrypoint starts an HTTP server, not only exports handlers
- read `process.env.PORT` or align `--http-port` with the fixed listener port
- bind on all interfaces

Elysia example:

```typescript
const port = Number(process.env.PORT ?? "8080")
app.listen({ port, hostname: "0.0.0.0" })
```

Custom Build Artifacts

Use `framework: "custom"` when the app is already built, or when a custom command produces a runnable Node artifact that Compute should stage as-is:

```typescript
export default defineComputeConfig({
  app: {
    framework: "custom",
    build: {
      command: "npm run build",
      outputDirectory: "build",
      entrypoint: "handler.js",
    },
    httpPort: 3000,
    env: ".env",
  },
});
```

Requirements:

- set both `build.outputDirectory` and `build.entrypoint`
- make `build.entrypoint` relative to `build.outputDirectory`
- ensure the artifact starts an HTTP server and binds on all interfaces
- use `command: null` only when the output directory already contains the deployable artifact

SvelteKit and Other Frameworks

`@prisma/cli app deploy --framework` has no `svelte` framework key. Do not claim SvelteKit is directly deployable with that name.

For frameworks without a dedicated deploy key, use one of these paths:

- produce a Node server artifact and deploy with config-backed `framework: "custom"`, or through a supported prebuilt/SDK flow
- if the app has a plain Node/Bun server entrypoint, deploy that entrypoint through `--framework bun --entry <path>`

SvelteKit should use a Node adapter or another production server artifact. Do not use `vite preview` as the deployed runtime.

Turborepo

Deploy concrete app packages, not the monorepo root by default. Prefer `prisma.compute.ts` at the repo root with one `apps` entry per deploy target.

Checklist:

- choose the app directory, such as `apps/api`
- run the workspace build from the correct root/package
- pass the app package's runtime entrypoint or framework
- pass the correct env file, which may live outside the app package
- keep branch env/database scope aligned with the deployed app

Example config:

```typescript
```

```
export default defineComputeConfig({
  apps: {
    web: { root: "apps/web", framework: "nextjs" },
    api: {
      root: "apps/api",
      framework: "bun",
      entry: "src/index.ts",
      httpPort: 3000,
      env: "packages/db/.env",
    },
  },
});
```

Deploy one target:

```bash
bunx @prisma/cli@latest app deploy api --branch feature/foo --json
```

Flag-only shape after confirming output paths:

```bash
bun run build
bunx @prisma/cli@latest app deploy \
  --framework bun \
  --entry apps/api/dist/src/index.js \
  --http-port 3000 \
  --env packages/db/.env
```

Verify the actual output path before using this command.

33. .agents\skills\prisma-compute\references\sdk-api.md

````
# SDK and API Automation

Use this reference when building automation rather than using `create-prisma` or `@prisma/cli app deploy`.

## Prefer the CLI for App Workflows

For normal app deployment:

1. Use generated `compute:deploy` when present.
2. Otherwise use `@prisma/cli app build/run/deploy`.
3. Use SDK/API only for custom automation, platform integrations, or tool builders.

## Compute SDK

Install:

```bash
npm install @prisma/compute-sdk @prisma/management-api-sdk
```

Config helper:

```typescript
import { defineComputeConfig } from "@prisma/compute-sdk/config";
```

Use this import in `prisma.compute.ts` for type checking. The helper is an identity function; the CLI loader aliases the import when it evaluates config files, so a user project does not need the SDK solely to load a Compute config.

Create an authenticated Management API client:

```typescript
import { createManagementApiClient } from "@prisma/management-api-sdk"

const apiClient = createManagementApiClient({
  token: process.env.PRISMA_API_TOKEN,
})
```

Token naming differs by surface. `@prisma/cli app ...` uses `PRISMA_SERVICE_TOKEN` for non-interactive service-token auth. The SDK examples here use `PRISMA_API_TOKEN` as an application convention for passing a token into `createManagementApiClient`; the SDK itself only receives the `token` string.

Deploy a prebuilt artifact:

```typescript
import { ComputeClient, PreBuilt } from "@prisma/compute-sdk"

const compute = new ComputeClient(apiClient)
const databaseUrl = process.env.DATABASE_URL
if (!databaseUrl) throw new Error("DATABASE_URL is required")

const result = await compute.deploy({
  strategy: new PreBuilt({
    appPath: "./dist",
    entrypoint: "index.js",
  }),
  projectId: "proj_abc",
  appName: "my-app",
  region: "us-east-1",
  envVars: { DATABASE_URL: databaseUrl },
  portMapping: { http: 3000 },
})

if (result.isOk()) {
  console.log(result.value.deploymentEndpointDomain)
} else {
  console.error(result.error.message)
}
```

SDK methods return `Result<T, E>`. Check `isOk()` or `isErr()` instead of assuming errors throw. Deploy results expose app/deployment vocabulary including `appId`, `appName`, `projectId`, `region`, `deploymentId`, `deploymentEndpointDomain`, `appEndpointDomain`, `promoted`, `previousDeploymentId`, `previousDeploymentAction`, and `resolvedConfig`.

## SDK Build Strategies

Project Compute SDK strategies:

- `AutoBuild`: tries supported framework strategies such as Next.js, Nuxt, Astro, NestJS, TanStack Start, then Bun
- `NextjsBuild`: requires standalone output and returns `server.js`
- `NuxtBuild`: expects `.output/server/index.mjs`
- `AstroBuild`: expects `dist/server/entry.mjs`
- `NestjsBuild`: builds a NestJS HTTP server artifact
- `TanstackStartBuild`: runs `vite build` and expects a Nitro node server at `.output/server/index.mjs`; keep `tanstackStart()` and `nitro()` in Vite config
- `CustomBuild`: runs optional configured build settings and stages a configured artifact entrypoint
- `BunBuild`: runs `bun build` and needs an explicit entrypoint or `package.json` `main`
- `PreBuilt`: uses an existing artifact directory and relative entrypoint
````

Regions

Known SDK region ids:

```text
us-east-1
us-west-1
eu-west-3
eu-central-1
ap-northeast-1
ap-southeast-1
```

Use `--region` in `@prisma/cli app deploy` or `region` in SDK deploy input only when creating a new Compute app. Existing apps keep their current region.

Management API Concepts

Compute resources map roughly to:

- Project: parent container
- Branch: production or preview scope for env resolution and database/env attachment
- App: stable app endpoint and branch attachment
- Deployment: build artifact plus runtime status and preview URL

Low-level public routes use App/Deployment names:

- list/create apps under a project with `/v1/apps`
- get/update/delete an app
- create/list deployments for an app
- get/start/stop/delete deployments with `/v1/deployments/:deploymentId`
- promote or roll back an app using `deploymentId`
- stream logs with `/v1/deployments/:deploymentId/logs`
- manage custom domains

Internal compatibility aliases may still appear in code. Prefer App/Deployment names in new docs, skills, and automation.

Environment variables are not embedded directly in the low-level deployment create payload. The attached branch's role selects their scope: a preview branch resolves branch-scoped vars, while a production branch (or no branch) resolves project-scoped production vars. Use project/environment-variable APIs or CLI env commands to write env vars first, and keep the branch name consistent across app creation, database creation, and env writes.

When using the CLI alongside SDK automation:

```bash
bunx @prisma/cli@latest project env add --file .env.preview --branch feature/foo
bunx @prisma/cli@latest database create preview-db --branch feature/foo --json
bunx @prisma/cli@latest app deploy --branch feature/foo --json --no-interactive
```

Production promotion is not just "the same branch with another label"; `app promote <deployment-id>` rebuilds with production env vars.

Secrets and Redaction

Management API deployment inspection exposes env var names with redacted values. Treat any value like `[redacted]` as a marker, not as the deployed value.

Do not log:

- service tokens
- OAuth tokens
- full database URLs
- env var values
- pre-signed upload URLs

34. .agents\skills\prisma-compute\references\troubleshooting.md

````
# Troubleshooting Prisma Compute

Use this reference when setup, build, deploy, env, or runtime behavior fails.

## First Checks

Run:

```bash
bunx @prisma/cli@latest --help
bunx @prisma/cli@latest app deploy --help
bunx @prisma/cli@latest auth whoami
bunx @prisma/cli@latest auth workspace list --json
```

Then inspect:

```bash
pwd
cat package.json
find .. -maxdepth 3 \( -name 'prisma.compute.ts' -o -name 'prisma.compute.mts' -o -name 'prisma.compute.js' -o -name 'prisma.compute.mjs' -o -
name 'prisma.compute.cjs' \) -print
test -f .env && sed -n 's/=.*/=<redacted>/p' .env
```

Do not print unredacted secrets.

## `prisma.compute.ts` Not Picked Up

This only matters when the project is supposed to use a config-backed deploy. A simple app without `prisma.compute.ts` can still deploy with
explicit `app deploy` flags.

Symptoms:

- deploy ignores the expected framework, entrypoint, port, env file, or app root
- a monorepo target such as `api` is not recognized
- local state appears in the wrong `.prisma/` directory

Check:

```bash
pwd
find .. -maxdepth 4 \( -name 'prisma.compute.ts' -o -name 'prisma.compute.mts' -o -name 'prisma.compute.js' -o -name 'prisma.compute.mjs' -o -
name 'prisma.compute.cjs' \) -print
bunx @prisma/cli@latest app deploy --help
```

Fix:

- keep exactly one compute config file in the directory where it lives
- put repo-wide or monorepo config at the repository/workspace root
- run commands from inside the repo or workspace boundary so discovery can walk up to the config
- use `[app]` targets from the `apps` keys, such as `bunx @prisma/cli@latest app deploy api`
- remember that config-relative paths such as `root` and `env.file` resolve from the config file directory

## Compute Config Invalid

Symptoms:

- `COMPUTE_CONFIG_INVALID`
- `COMPUTE_CONFIG_TARGET_REQUIRED`
- `COMPUTE_CONFIG_TARGET_UNKNOWN`
- "Multiple compute config files found"

Fix:

- export `defineComputeConfig({ app: ... })` or `defineComputeConfig({ apps: ... })`
- define exactly one of `app` or `apps`
- remove unknown top-level keys
- pass a target for multi-app build/run commands, such as `app build web`
- pass an existing `apps` key for multi-app deploys, such as `app deploy api`
- remove config `build` blocks from `nuxt`, `astro`, and `nestjs` targets
- for `framework: "custom"`, set both `build.outputDirectory` and `build.entrypoint`
- when `build.outputDirectory` is set for a configurable framework, also set `build.entrypoint` if the framework needs a configured runtime
entrypoint

Minimal recovery config:

```typescript
import { defineComputeConfig } from "@prisma/compute-sdk/config";

export default defineComputeConfig({
  app: {
    framework: "hono",
    entry: "src/index.ts",
    httpPort: 8080,
  },
});
```
````

`create-prisma --yes` Did Not Deploy

`--yes` skips prompts and does not opt into deploy. Pass `--deploy` explicitly:

```bash
bunx create-prisma@latest --name my-api --template hono --provider postgresql --deploy
```

If the integrated deploy cannot complete, scaffold succeeds but deploy should be reported as failed.

Accidental Prisma Postgres Provisioning

With PostgreSQL, no `--database-url`, and no `--no-prisma-postgres`, setup can provision Prisma Postgres. For local smoke tests, pass:

```bash
--no-prisma-postgres --database-url "postgresql://USER:PASSWORD@HOST:PORT/DB"
```

Use a disposable real database URL if Prisma commands need to run.

Auth Fails

Symptoms:

- `project list` fails
- `auth whoami` fails
- browser login was not completed
- commands use the wrong workspace after a second login
- another workspace is stored locally but commands behave signed out
- `PRISMA_SERVICE_TOKEN` is missing, empty, expired, or lacks workspace/project permissions

Fix:

```bash
bunx @prisma/cli@latest auth login
bunx @prisma/cli@latest auth whoami
bunx @prisma/cli@latest auth workspace list --json
```

If multiple local OAuth workspaces exist, switch explicitly. Prefer ids from JSON:

```bash
bunx @prisma/cli@latest auth workspace use <workspace-id>
bunx @prisma/cli@latest auth whoami --json
bunx @prisma/cli@latest project list --json
```

For a human terminal, `auth workspace use` with no argument opens an interactive picker or selects the only local OAuth workspace without prompting. In non-interactive or `--json` mode, use `auth workspace use <id-or-name>` instead.

If the active workspace was logged out or its token refresh failed, the CLI intentionally stays signed out for OAuth commands rather than falling through to another cached workspace. Recover by running `auth workspace list --json` and then `auth workspace use <workspace-id>`.

To remove only one local OAuth workspace session:

```bash
bunx @prisma/cli@latest auth workspace logout <workspace-id-or-name>
# or:
bunx @prisma/cli@latest auth logout --workspace <workspace-id-or-name>
```

Use plain `auth logout` only when you want to clear all local OAuth workspace sessions.

For CI, `@prisma/cli` can authenticate with `PRISMA_SERVICE_TOKEN`:

```bash
test -n "${PRISMA_SERVICE_TOKEN:-}" && echo "PRISMA_SERVICE_TOKEN is set"
bunx @prisma/cli@latest auth whoami
bunx @prisma/cli@latest app deploy --json --no-interactive --prod --yes --env .env
```

If `PRISMA_SERVICE_TOKEN` is set and non-empty, it is the active auth source and local OAuth workspace switching is unavailable for command execution. Unset `PRISMA_SERVICE_TOKEN` before using `auth workspace use` to change local OAuth workspace context.

If `PRISMA_SERVICE_TOKEN` is set but empty, the CLI errors before trying browser-login credentials. Unset it or provide a valid workspace service token. Never echo, log, or paste the token value; only check whether it is present.

Local storage hints for debugging:

- Override auth storage with `PRISMA_COMPUTE_AUTH_FILE` when isolating tests.
- Default macOS OAuth credential file: `~/Library/Application Support/prisma/auth.json`.
- Active workspace metadata sidecar: `~/Library/Application Support/prisma/auth.context.json`.
- Project binding: `.prisma/local.json`.
- Local app/project state: `.prisma/cli/state.json`, usually next to the discovered `prisma.compute.ts`.

Do not print credential files or token values into logs.

Project Setup Fails

Symptoms:

- `PROJECT_SETUP_REQUIRED`
- non-interactive deploy cannot choose a Project

- deploy was expected to create a Project but did not

Fix:

```bash
bunx @prisma/cli@latest app deploy --project <id-or-name> --json --no-interactive
bunx @prisma/cli@latest app deploy --create-project <name> --yes
```

Do not rely on `--yes` alone to choose Project scope. `--project`, `--create-project`, and `PRISMA_PROJECT_ID` are mutually exclusive.

Missing or Placeholder `DATABASE_URL`

Symptoms:

- Prisma Client throws `DATABASE_URL is required`
- migration scripts fail immediately
- deploy runs but app fails on database access

Fix:

1. Put a real production-ready `DATABASE_URL` in `.env` or project env.
2. Run `prisma generate`.
3. Run migrations with the project's `db:migrate` or production migration command.
4. Redeploy with `--env .env` or project env configured.

If Prisma Client generation or runtime env loading is the concrete failure, then inspect Prisma-specific config:

```bash
test -f prisma.config.ts && sed -n '1,160p' prisma.config.ts
test -f prisma/schema.prisma && sed -n '1,220p' prisma/schema.prisma
```

Never deploy `postgresql://USER:PASSWORD@HOST:PORT/DATABASE` placeholder values.

Wrong Branch, Env, or Database

Symptoms:

- preview deploy reads production env
- branch deploy cannot find `DATABASE_URL`
- app is deployed to the expected branch but points at the wrong database
- logs are inspected for the current app while the failing URL belongs to a different deployment id

Check:

```bash
bunx @prisma/cli@latest project show --json
bunx @prisma/cli@latest project env list --role production --json
bunx @prisma/cli@latest project env list --role preview --json
bunx @prisma/cli@latest project env list --branch feature/foo --json
bunx @prisma/cli@latest app list-deploys --json
bunx @prisma/cli@latest app logs --deployment <deployment-id> --json
```

Fix:

- pass the same `--branch <git-name>` to `app deploy`, `database create`, and branch-specific `project env` commands
- use `--role production` for production env and `--role preview` for preview-template env
- capture the deployment id and URL from deploy JSON, then inspect logs with `app logs --deployment <deployment-id>`
- `app show`, `app list-deploys`, and `app logs` do not filter by branch; capture and use the deployment id
- treat `app promote <deployment-id>` as a production action because it rebuilds with production env vars
- do not expect `prisma.compute.ts` to select Project, Branch, production, or database scope; it only supplies app deploy defaults

Database Wiring or Schema Did Not Apply

Symptoms:

- deploy runs but the app cannot find `DATABASE_URL`
- database env vars exist but the database is empty
- a deploy-all run points multiple apps at the same branch database

Fix:

- read [`app-deploy-cli.md`](app-deploy-cli.md) `Database and Env` for the database/env guardrails
- create and assign database env vars explicitly for the intended branch/app scope
- run migrations, seed, or schema push yourself after database setup; Compute never applies schema changes for you
- for multi-app deploy-all with app-specific database isolation, create and assign those database env vars explicitly before deploy

Next.js Standalone Missing

Error shape:

```text
Next.js build did not produce standalone output
```

Fix `next.config.ts`:

```typescript
const nextConfig = {
  output: "standalone",
}
```

````
export default nextConfig
```

Then reinstall/build if needed and deploy again.

## Nitro Entry Missing

Nuxt or TanStack Start error shape:

```text
.output/server/index.mjs
```

General fix:

- ensure the correct framework plugins are installed
- run the framework build locally
- avoid custom Nitro presets that produce a non-Node target
- use the default Nitro node server preset

For TanStack Start specifically:

- keep `nitro` in `dependencies`
- keep `import { nitro } from "nitro/vite"` in `vite.config.ts`
- keep `plugins: [tanstackStart(), nitro(), viteReact()]` or the framework-equivalent plugin order
- run `bun run build` and verify `.output/server/index.mjs` exists
- do not replace the production server with `vite preview`

Compute detection selects TanStack Start when it sees `@tanstack/react-start` or `@tanstack/solid-start`. If the Nitro entrypoint is missing after that, fix the TanStack/Nitro build output; do not assume Compute will silently use a Bun deployment.

## Bun Entrypoint Missing

Error shape:

```text
Entrypoint is required
Entrypoint file does not exist
```

Fix either:

```json
{
  "main": "src/index.ts"
}
```

or deploy with:

```bash
bunx @prisma/cli@latest app deploy --framework bun --entry src/index.ts
```

## Port Mismatch

Symptoms:

- deploy succeeds but the app is unreachable
- health checks fail
- logs show the server listening on a different port

Fix:

- read `process.env.PORT`
- pass `--http-port <port>` when the app has a fixed port
- use the generated `compute:deploy` script when it exists
- remember the `@prisma/cli app deploy` default is HTTP `3000`; generated Hono/Elysia projects usually configure `8080` through `prisma.compute.ts` or flag-backed `--http-port 8080` scripts
- use the template defaults: Hono/Elysia `8080`, Next/TanStack/Nuxt `3000`, Astro `4321`

## Public URL Smoke Test Fails

Symptoms:

- deploy command completed
- `app show` or deploy output has a URL
- the public URL times out, returns 5xx, or returns an unexpected page

Check:

```bash
curl -i https://<deployment-url>
curl -i https://<deployment-url>/health
bunx @prisma/cli@latest app logs --json
```

Fix by following the first concrete failure:

- connection timeout or 5xx: check logs, host binding, and port mapping
- unexpected status or body: verify the route path and app framework output
- local URL tested by mistake: rerun against the public deployment URL, not `localhost` or `127.0.0.1`
````
````

Localhost Binding

Symptoms:

- deploy says the app started or the port was observed, but the public URL is unreachable
- logs show a server listening on `localhost` or `127.0.0.1`
- `app run` works locally, but the deployed app cannot receive external traffic

Why this happens:

Compute's boot watcher polls `/proc/net/tcp` and `/proc/net/tcp6` for configured ports entering `LISTEN`. That readiness signal tracks the port, not whether the app bound `127.0.0.1` or all interfaces. A loopback-only listener can therefore look ready while public ingress still cannot reach it.

Fix:

- remove hard-coded `localhost` or `127.0.0.1` server host settings
- bind on `0.0.0.0` or the framework equivalent, such as Astro `server.host: true`
- for Next.js standalone, do not deploy with `HOSTNAME=localhost`; use `HOSTNAME=0.0.0.0` if the host is overridden
- keep port and host fixes together: `0.0.0.0:<deployed-http-port>`

Env Changes Did Not Apply

Generated `compute:deploy` scripts redeploy using the generated flags and/or `prisma.compute.ts`; they do not run migrations or seed data.

After env changes:

```bash
bunx @prisma/cli@latest project env list
bunx @prisma/cli@latest project env list --branch feature/foo
bunx @prisma/cli@latest app deploy --prod --yes --env .env
bunx @prisma/cli@latest app deploy --branch feature/foo --env .env.preview
```

If using branch-specific env, confirm the branch name and role.

Need Logs

Runtime logs for the current app:

```bash
bunx @prisma/cli@latest app logs
```

Specific deployment:

```bash
bunx @prisma/cli@latest app logs --deployment <deployment-id>
```

Machine-readable:

```bash
bunx @prisma/cli@latest app logs --json
```

Build logs for GitHub/Console builds:

```bash
bunx @prisma/cli@latest build logs <build-id>
bunx @prisma/cli@latest build logs <build-id> --follow
bunx @prisma/cli@latest build logs <build-id> --json
```

Use `build logs` for build output keyed by a Build id from a GitHub check run, Console build page, or Management API build record. Use `app logs` for runtime logs keyed by the current app deployment or a deployment id.

Summarize relevant errors. Do not paste secrets.

Report Unresolved CLI Issues

When a CLI failure survives the checks above, or a command crashes with `UNEXPECTED_ERROR`, report it to the Prisma team:

```bash
bunx @prisma/cli@latest feedback "app deploy crashed: <first error line>"
```

Prefer the pre-filled command from a `--json` crash envelope's `nextActions` verbatim. Anonymous; never put secrets, connection URLs, or tokens in the message.

35. .agents\skills\prisma-compute\SKILL.md

````
---
name: prisma-compute
description: Prisma Compute deployment and hosting guide. Use whenever the user mentions Prisma Compute, `prisma.compute.ts`,
`defineComputeConfig`, deploying or hosting a Prisma app, `@prisma/cli app deploy`, `compute:deploy`, `create-prisma --deploy`,
`PRISMA_SERVICE_TOKEN`, `auth workspace`, Compute apps/deployments/build logs/domains, `@prisma/cli agent install`, `@prisma/cli feedback`,
localhost vs `0.0.0.0`, deploy port binding, or framework deploy readiness for Hono, Elysia, Next.js, TanStack Start, Astro, Nuxt, Svelte,
Nest, Turborepo, or custom/prebuilt artifacts.
license: MIT
metadata:
  author: prisma
  version: "1.4.1"
---

# Prisma Compute

Guide agents through Prisma Compute app creation, deployment, operations, and framework-specific deploy readiness.

## Prisma Compute CLI Surface

Use the Prisma Platform CLI for Compute app workflows:

```bash
bunx @prisma/cli@latest app deploy --help
bunx @prisma/cli@latest app --help
bunx @prisma/cli@latest build logs --help
bunx create-prisma@latest --help
```

Use `@prisma/cli@latest` for Compute app deployment. Use `create-prisma@latest` for new-project scaffolding.

## Send Feedback and Report CLI Issues

The CLI has a built-in feedback channel. Use it whenever a command crashes (`UNEXPECTED_ERROR`), a failure survives troubleshooting, or the
user asks to send feedback to the Prisma team:

```bash
bunx @prisma/cli@latest feedback "app deploy crashed: <first error line>"
bunx @prisma/cli@latest feedback "love the deploy flow" --email you@example.com
```

Crash output points here on its own: `--json` crash envelopes carry the exact pre-filled command as a `recover` entry in `nextActions` (run it
verbatim), and human crash output ends with a `Tell us what happened:` hint. Feedback is anonymous unless `--email` is passed and attaches
only the CLI version, node version, and OS platform/arch. Never include secrets, connection URLs, or user data in the message.

## Source-of-Truth Order

Use evidence in this order when deciding what to edit or run:

1. The project's generated scripts and config, especially `prisma.compute.ts`, `compute:deploy`, framework config, and `package.json`.
2. CLI help output from `create-prisma` and `@prisma/cli`.
3. Local installed package code, generated artifacts, and type definitions.
4. Official docs.

## When to Apply

Use this skill for:

- Creating a new app that can deploy to Prisma Compute
- Deploying an existing TypeScript app to Prisma Compute
- Creating or updating a typed `prisma.compute.ts` deploy config
- Deciding whether a framework is Compute-ready
- Debugging `create-prisma --deploy`, `compute:deploy`, or `app deploy`
- Managing Compute app logs, deployments, environment variables, and domains, and listing platform branches (`branch list`; there are no
branch create/remove commands)
- Inspecting GitHub/Console build logs and GitHub push-to-deploy status
- Running non-interactive deploys with browser auth, multiple stored workspaces, or Prisma service tokens
- Switching, selecting, listing, or logging out local Prisma Platform workspaces for `@prisma/cli`
- Installing or updating Prisma skills with `@prisma/cli agent install|update|status`
- Sending feedback or reporting unresolvable CLI failures with `@prisma/cli feedback`
- Programmatic deployments with `@prisma/compute-sdk` or Management API integrations

## Decision Tree

1. Existing project deployment or redeploy:
   Read [`references/app-deploy-cli.md`](references/app-deploy-cli.md).

2. Typed Compute config, monorepos, deploy targets, app roots, or build/env defaults:
   Read [`references/compute-config.md`](references/compute-config.md).

3. Framework-specific build/runtime work:
   Read [`references/frameworks.md`](references/frameworks.md).

4. New project from a scaffold:
   Read [`references/create-prisma.md`](references/create-prisma.md).

5. Programmatic deployment, SDKs, APIs, or low-level App/Deployment concepts:
   Read [`references/sdk-api.md`](references/sdk-api.md).

6. Build, auth, env, deploy, or runtime failures:
   Read [`references/troubleshooting.md`](references/troubleshooting.md).
````

Rules by Priority

Priority	Category	Impact	Prefix
1	Command verification	CRITICAL	`verify-`
2	Auth and workspace selection	CRITICAL	`auth-`
3	Framework readiness	CRITICAL	`framework-`
4	Runtime host and port binding	CRITICAL	`runtime-`
5	Typed Compute config	HIGH	`config-`
6	Branch, environment, and database wiring	HIGH	`env-`
7	Deploy operations	HIGH	`deploy-`
8	SDK and API automation	MEDIUM	`sdk-`

Quick Rules

1. Command Verification

- `verify-help-first` - Use CLI help output to confirm command syntax while working.
- `verify-prisma-vs-platform-cli` - Do not assume `prisma app deploy` exists in the ORM CLI; check whether the task should use `@prisma/cli`.
- `verify-generated-scripts` - Prefer the generated `compute:deploy` script when a project already has one.
- `verify-public-url` - After a real deploy, request the public deployment URL instead of trusting local or readiness-only checks.
- `verify-config-support` - Treat `prisma.compute.ts` as the typed Compute config; inspect the project's config and generated scripts before editing or deploying.
- `verify-auth-workspace-support` - Use `@prisma/cli auth workspace` commands for local workspace list/use/logout flows.

2. Auth and Workspace Selection

- `auth-source-precedence` - A non-empty `PRISMA_SERVICE_TOKEN` is the active auth source for commands and local OAuth workspaces are ignored for execution. If it is set but empty, the CLI should fail instead of falling back to stored OAuth.
- `auth-multi-workspace` - `auth login` can store OAuth sessions for multiple workspaces on the same machine. The active workspace pointer selects which stored OAuth grant normal commands use.
- `auth-list-before-switch` - Use `auth workspace list --json` to inspect local sessions. Agents should prefer workspace ids from JSON over names because names can be ambiguous.
- `auth-switch-explicitly` - Use `auth workspace use <id-or-name>` for non-interactive switching. Use `auth workspace use` with no argument only for an interactive picker or when exactly one local OAuth workspace exists.
- `auth-no-fallthrough` - If the active OAuth workspace is logged out or fails refresh, the CLI should not silently fall through to another cached workspace. Run `auth workspace use <id>` to choose the next workspace.
- `auth-single-workspace-logout` - Use `auth workspace logout <id-or-name>` or `auth logout --workspace <id-or-name>` to remove one local OAuth workspace session. Plain `auth logout` clears all local OAuth workspace sessions.
- `auth-service-token-switching` - While `PRISMA_SERVICE_TOKEN` is set, `auth workspace use` is unavailable because the service token is the active auth source; unset the env var to switch local OAuth workspaces. Workspace logout still only cleans local OAuth state.
- `auth-storage-awareness` - Local OAuth credentials live in the platform auth file, with workspace metadata in a sidecar context file. Project pins live in `.prisma/local.json`, and CLI app/project state lives in `.prisma/cli/state.json` near `prisma.compute.ts` when present.

3. Framework Readiness

- `framework-cli-first` - Evaluate deploy readiness against `@prisma/cli app deploy`, not against what `create-prisma` can scaffold.
- `framework-supported-cli-deploy` - Compute deploy supports `nextjs`, `nuxt`, `astro`, `hono`, `nestjs`, `tanstack-start`, `custom`, and `bun`.
- `framework-create-prisma-defaults-only` - `create-prisma` can provide generated defaults and `compute:deploy`, but it is not the general deploy surface for existing apps.
- `framework-build-output` - Compute needs a server entrypoint or framework artifact, not only static output.

4. Runtime Host and Port Binding

- `runtime-bind-all-interfaces` - Deployed servers must bind on all interfaces (`0.0.0.0` or the framework equivalent), not hard-coded `localhost` or `127.0.0.1`.
- `runtime-match-http-port` - The app must listen on the deployed HTTP port: read `process.env.PORT` when possible, or pass the matching `--http-port`.
- `runtime-readiness-port-only` - Compute readiness watches listening ports; a loopback-only listener can look ready while public ingress cannot reach it.

5. Typed Compute Config

- `config-optional-simple-app` - `prisma.compute.ts` is not required to deploy a normal single app; use flags when there is no durable config.
- `config-init-formalizer` - Generate a fresh config with `bunx @prisma/cli@latest init`: it detects the framework, pins name/framework/httpPort (plus entry for Bun/Hono), and offers the Project link. `--format json` writes a dependency-free `prisma.compute.json` instead. `init` refuses when any config already exists, never scaffolds code, and never deploys.
- `config-use-prisma-compute-ts` - Put reusable deploy defaults in `prisma.compute.ts` with `defineComputeConfig`, not in `prisma.config.ts`.
- `config-app-vs-apps` - Use `app` for a single deploy target and `apps` for monorepos or multi-app repos; define exactly one.
- `config-monorepo-roots` - For monorepos, use `prisma.compute.ts` to declare app targets, roots, framework defaults, entrypoints, ports, and env inputs.
- `config-targets` - In multi-app configs, `@prisma/cli app deploy web` selects the `apps.web` target. Without `[app]`, commands can infer the target from the current directory; otherwise deploy can run all targets while build/run require one.
- `config-region-new-app-only` - A config `region` is only a default for newly created apps; deploys to existing apps keep the app's current region.
- `config-custom-artifact` - Use `framework: "custom"` with `build.outputDirectory` and `build.entrypoint` for prebuilt or custom-built artifacts.
- `config-no-project-branch-secrets` - Do not commit Workspace, Project, Branch, production intent, service tokens, or secret values in `prisma.compute.ts`; keep those in flags, `.prisma/local.json`, env storage, or CI secrets. App-level defaults such as `region`, `root`, `framework`, `entry`, `httpPort`, and non-secret env file paths belong in config.
- `config-flags-win` - Explicit deploy flags such as `--framework`, `--entry`, `--http-port`, `--region`, and `--env` override matching config values.

6. Branch, Environment, and Database

- `env-do-not-leak-secrets` - Never print full `DATABASE_URL`, service tokens, or secret values.
- `env-deploy-loads-dotenv` - Generated deploy scripts may load env via `prisma.compute.ts` or `--env .env`; inspect the actual script/config before redeploy.
- `env-migrations-separate` - Redeploy scripts do not run migrations or seed data. Run the appropriate Prisma database scripts separately.
- `env-cli-token-name` - `@prisma/cli` uses `PRISMA_SERVICE_TOKEN` for service-token auth.
- `env-branch-scope` - Branch deploys, branch env vars, and branch databases must use the same branch name; pass `--branch <git-name>`

explicitly when targeting a preview branch.
- `env-production-vs-preview` - Use `--role production` for production env, `--role preview` for preview template env, and `--branch <git-name>` for branch-specific overrides.
- `env-db-explicit` - Keep database and env wiring explicit through database and project env commands; deploy examples should not add database setup, and deploys do not run migrations, seed data, or create one database per app automatically.

7. Deploy Operations

- `deploy-prod-intent` - Use `--prod --yes` only when the user intends a production deploy. The first production deploy of an App auto-promotes without `--prod`; the flag gates subsequent production-branch deploys.
- `deploy-no-promote` - Use `app deploy --no-promote` for build-then-verify: it builds a candidate reachable at its own URL without touching the live deployment, promoted later with `app promote <deployment-id>`.
- `deploy-github-default-branch` - When a Compute app is connected to GitHub push-to-deploy, a merge to the default branch is the production deploy path; check deployment records or GitHub check runs instead of telling users to redeploy the merged PR branch or run a default-branch preview deploy.
- `deploy-build-logs` - Use `@prisma/cli build logs <build-id>` for GitHub/Console build output. Use `app logs` for runtime deployment logs; the two ids are different.
- `deploy-noninteractive-auth` - Non-interactive deploys need either the correct active stored OAuth workspace or a supported service token env var; never print the token.
- `deploy-json-for-agents` - Use `--json --no-interactive` for scripts and agent-readable output.
- `deploy-create-project` - Use `--create-project <name>` only when the user wants deploy to create and link a new project; it conflicts with `--project` and `PRISMA_PROJECT_ID`.
- `deploy-ops-targets` - App show/open/logs/list-deploys/promote/rollback/remove and domain commands can also accept `[app]` targets from `prisma.compute.ts`.
- `deploy-report-cli-bugs` - On `UNEXPECTED_ERROR` or an unresolvable failure, report it with the feedback command; see "Send Feedback and Report CLI Issues" above.

8. SDK and API

- `sdk-use-cli-first` - Prefer `@prisma/cli app deploy` for app workflows; use `create-prisma` only to scaffold a new app unless the user is building lower-level automation.
- `sdk-result-handling` - `@prisma/compute-sdk` returns `Result` values; check `isOk()`/`isErr()` instead of relying on exceptions.

Preferred Workflow

1. Inspect the project: package manager, template/framework, `package.json` scripts, Prisma version, Prisma client location, `prisma.compute.ts`, and existing `compute:deploy`.
2. Verify CLI help output for the package actually being used.
3. Verify auth context before project/app mutations: `auth whoami --json`, and when multiple local sessions may exist, `auth workspace list --json`.
4. Choose the path:
   - existing app deploy: config-backed target when present, generated `compute:deploy`, or `@prisma/cli app build/run/deploy` flags
   - new app scaffold: `create-prisma`, then generated `compute:deploy` or `@prisma/cli app deploy`
   - low-level automation: `@prisma/compute-sdk` or Management API
5. Check framework readiness plus host/port/env/runtime requirements, including project and branch scope.
6. Run a local build or `app build` before deploying when feasible.
7. Deploy with JSON output when automating, then request the public URL and summarize app URL, app id, deployment id, project id, workspace id, and follow-up steps.
8. For GitHub/Console builds, inspect the `Prisma Compute Deploy` check run or `build logs <build-id>` before guessing why a build failed.

Avoid

- Do not bury Compute deployment guidance in the generic `prisma-cli` skill.
- Do not run `create-prisma` inside an existing app just to deploy it; use the generated `compute:deploy` script or `@prisma/cli app deploy`.
- Do not tell users that every `create-prisma` template can auto-deploy.
- Do not deploy with placeholder `DATABASE_URL` values.
- Do not assume `next start` is the Compute runtime path; Next.js deploys need standalone output.

36. .agents\skills\prisma-database-setup\references\cockroachdb.md

````
# CockroachDB Setup

Configure Prisma with CockroachDB.

## Prerequisites

- CockroachDB cluster

## 1. Schema Configuration

In `prisma/schema.prisma`:

```prisma
datasource db {
  provider = "cockroachdb"
}

generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

## 2. Config Configuration

In `prisma.config.ts`:

```typescript
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## 3. Environment Variable

In `.env`:

```env
DATABASE_URL="postgresql://user:password@host:26257/db?sslmode=verify-full"
```

Note: CockroachDB uses the PostgreSQL wire protocol, so the URL often looks like postgresql, but the provider **MUST** be `cockroachdb` in the schema to handle specific CRDB features correctly.

## Driver Adapter

Use a driver adapter for the standard SQL workflow. CockroachDB is PostgreSQL-compatible, so use the PostgreSQL adapter.

1. Install adapter and driver:
   ```bash
   npm install @prisma/adapter-pg pg
   ```

2. Instantiate Prisma Client with the adapter:
   ```typescript
   import 'dotenv/config'
   import { PrismaClient } from '../generated/client'
   import { PrismaPg } from '@prisma/adapter-pg'

   const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL })
   const prisma = new PrismaClient({ adapter })
   ```

## ID Generation

CockroachDB uses `BigInt` or `UUID` for IDs efficiently.

```prisma
model User {
  id BigInt @id @default(autoincrement()) // Uses unique_rowid()
}
```

Or using string UUIDs:

```prisma
model User {
  id String @id @default(dbgenerated("gen_random_uuid()")) @db.Uuid
}
```

## Common Issues

### Schema Introspection
Always use `provider = "cockroachdb"` to ensure correct type mapping during `db pull`.
````

37. .agents\skills\prisma-database-setup\references\mongodb.md

````
# MongoDB Setup

MongoDB projects should stay on the latest Prisma 6.x release. Do not upgrade a MongoDB app to Prisma 7's SQL client path.

## Prerequisites

- MongoDB 4.2+
- Replica Set configured (required for transactions)
- Latest Prisma 6.x release, or your team's pinned Prisma 6 version
- Node.js 20.19.0+
- TypeScript 5.4.0+

## 1. Schema Configuration

Use the standard Prisma 6 MongoDB setup with `prisma-client-js`.

In `prisma/schema.prisma`:

```prisma
datasource db {
  provider = "mongodb"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}
```

### Driver Adapters

Do **not** apply the Prisma 7 SQL adapter setup here. MongoDB does not use a SQL `@prisma/adapter-*` package.

### ID Field Requirement

MongoDB models **must** have a mapped `_id` field using `@id` and `@map("_id")`, usually of type `String` with `auto()` and `db.ObjectId`.

```prisma
model User {
  id    String @id @default(auto()) @map("_id") @db.ObjectId
  email String @unique
  name  String?
}
```

### Relations

Relations in MongoDB expect IDs to be `db.ObjectId` type.

```prisma
model Post {
  id       String @id @default(auto()) @map("_id") @db.ObjectId
  author   User   @relation(fields: [authorId], references: [id])
  authorId String @db.ObjectId
}
```

## 2. Environment Variable

In `.env`:

```env
DATABASE_URL="mongodb+srv://user:password@cluster.mongodb.net/mydb?retryWrites=true&w=majority"
```

## Migrations vs Introspection

- **No Migrations**: MongoDB is schema-less. `prisma migrate` commands **do not work**.
- **db push**: Use `prisma db push` to sync indexes and constraints.
- **db pull**: Use `prisma db pull` to generate schema from existing data (sampling).

## Current Verification Notes

- `prisma init --datasource-provider mongodb` is still implemented in Prisma's CLI source.
- Prisma's upstream repo still contains MongoDB fixtures and tests.
- Local verification shows Prisma 7 can still recognize MongoDB inputs, but the generated client path does not provide a supported MongoDB upgrade path.
- Local verification shows Prisma 6.x works end to end with `prisma-client-js`, `prisma db push`, and `new PrismaClient()` against a MongoDB replica set.

## Version Guidance

- For MongoDB, stay on the latest available Prisma 6.x release.
- Treat Prisma 7 MongoDB migration attempts as unsupported until Prisma ships a real MongoDB upgrade path.

## Common Issues

### "Transactions not supported"
Ensure your MongoDB instance is a **Replica Set**. Standalone instances do not support transactions. Atlas clusters are replica sets by default.
````

"Invalid ObjectID"
Ensure fields referencing IDs are decorated with `@db.ObjectId` if the target is an ObjectID.

38. .agents\skills\prisma-database-setup\references\mysql.md

````
# MySQL Setup

Configure Prisma with MySQL (or MariaDB).

## Prerequisites

- MySQL or MariaDB database
- Connection string

## 1. Schema Configuration

In `prisma/schema.prisma`:

```prisma
datasource db {
  provider = "mysql"
}

generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

## 2. Config Configuration

In `prisma.config.ts`:

```typescript
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## 3. Environment Variable

In `.env`:

```env
DATABASE_URL="mysql://user:password@localhost:3306/mydb"
```

### Connection String Format

```
mysql://USER:PASSWORD@HOST:PORT/DATABASE
```

- **USER**: Database user
- **PASSWORD**: Password
- **HOST**: Hostname
- **PORT**: Port (default 3306)
- **DATABASE**: Database name

## Driver Adapter

Use a driver adapter for the standard SQL workflow.

1. Install adapter and driver:
   ```bash
   npm install @prisma/adapter-mariadb mariadb
   ```

2. Instantiate Prisma Client with the adapter:
   ```typescript
   import 'dotenv/config'
   import { PrismaClient } from '../generated/client'
   import { PrismaMariaDb } from '@prisma/adapter-mariadb'

   const adapter = new PrismaMariaDb({
     host: 'localhost',
     port: 3306,
     connectionLimit: 5,
     user: process.env.MYSQL_USER,
     password: process.env.MYSQL_PASSWORD,
     database: process.env.MYSQL_DATABASE,
   })

   const prisma = new PrismaClient({ adapter })
   ```

### Text protocol option

If you need the MariaDB driver's text protocol instead of the default binary `execute()` path, enable `useTextProtocol` explicitly:
````

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaMariaDb } from '@prisma/adapter-mariadb'

const adapter = new PrismaMariaDb(process.env.DATABASE_URL!, {
  useTextProtocol: true,
})

const prisma = new PrismaClient({ adapter })
```

Use this only when you specifically need text-protocol compatibility for your MariaDB setup.

PlanetScale Setup

PlanetScale uses MySQL but requires specific settings because it doesn't support foreign key constraints.

In `prisma/schema.prisma`:

```prisma
datasource db {
  provider     = "mysql"
  relationMode = "prisma" // Emulate foreign keys in Prisma
}
```

Common Issues

"Too many connections"
MySQL has a connection limit. Adjust connection pool size in URL:
```env
DATABASE_URL="mysql://...?connection_limit=5"
```

JSON Support
MySQL 5.7+ supports JSON. MariaDB 10.2+ supports JSON (as an alias for LONGTEXT with check constraints). Prisma handles this, but verify your version.

39. .agents\skills\prisma-database-setup\references\postgresql.md

````
# PostgreSQL Setup

Configure Prisma with PostgreSQL.

## Prerequisites

- PostgreSQL database (local or cloud)
- Connection string

## 1. Schema Configuration

In `prisma/schema.prisma`:

```prisma
datasource db {
  provider = "postgresql"
}

generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

## 2. Config Configuration

In `prisma.config.ts`:

```typescript
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## 3. Environment Variable

In `.env`:

```env
DATABASE_URL="postgresql://user:password@localhost:5432/mydb?schema=public"
```

### Connection String Format

```
postgresql://USER:PASSWORD@HOST:PORT/DATABASE?schema=SCHEMA
```

- **USER**: Database user
- **PASSWORD**: Password (URL encoded if special chars)
- **HOST**: Hostname (localhost, IP, or domain)
- **PORT**: Port (default 5432)
- **DATABASE**: Database name
- **SCHEMA**: Schema name (default `public`)

## Driver Adapter

Use a driver adapter for the standard SQL workflow.

1. Install adapter and driver:
   ```bash
   npm install @prisma/adapter-pg pg
   ```

2. Instantiate Prisma Client with the adapter:
   ```typescript
   import 'dotenv/config'
   import { PrismaClient } from '../generated/client'
   import { PrismaPg } from '@prisma/adapter-pg'

   const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL })
   const prisma = new PrismaClient({ adapter })
   ```

## Common Issues

### "Can't reach database server"
- Check host and port
- Check firewall settings
- Ensure database is running

### "Authentication failed"
- Check user/password
- Special characters in password must be URL-encoded
````

"Schema does not exist"
- Ensure `?schema=public` (or your schema) is in the URL

40. .agents\skills\prisma-database-setup\references\prisma-client-setup.md

````
# Prisma Client Setup

Generate and instantiate Prisma Client for Prisma's standard SQL provider workflow. For MongoDB, follow the provider-specific notes in
`references/mongodb.md` instead of copying the SQL adapter example below.

## 1. Install dependencies

```bash
npm install prisma --save-dev
npm install @prisma/client
```

## 2. Add generator block

In `prisma/schema.prisma`:

```prisma
generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

`prisma-client` requires an explicit `output` path and does not generate into `node_modules` by default.

## 3. Generate Prisma Client

```bash
npx prisma generate
```

Re-run `prisma generate` after every schema change to keep the client in sync.

## 4. Instantiate Prisma Client

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL })
const prisma = new PrismaClient({ adapter })
```

If you change the generator `output`, update the import path to match. For the SQL provider workflow, replace `PrismaPg` with the adapter for
your database.

## 5. Use a single instance

Each `PrismaClient` instance creates a connection pool. Reuse a single instance per app process to avoid exhausting database connections.
````

41. .agents\skills\prisma-database-setup\references\prisma-postgres.md

````
# Prisma Postgres Setup

Configure Prisma with Prisma Postgres (Managed).

## Overview

Prisma Postgres is a serverless, managed PostgreSQL database optimized for Prisma.

## Setup via CLI

You can provision a Prisma Postgres instance directly via the CLI:

```bash
prisma init --db
```

This will:
1. Log you into Prisma Data Platform.
2. Create a new project and database instance.
3. Update your `.env` with the connection string.

## Connection String

For Prisma CLI flows and Accelerate-style usage, you may see a `prisma+postgres://` URL.

For Prisma Client with a driver adapter in Node.js, prefer the direct TCP connection string from the Prisma Postgres dashboard:

```env
DATABASE_URL="postgres://identifier:key@db.prisma.io:5432/postgres?sslmode=require"
```

## 1. Schema Configuration

In `prisma/schema.prisma`:

```prisma
datasource db {
  provider = "postgresql" // Use postgresql provider
}

generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

## 2. Config Configuration

In `prisma.config.ts`:

```typescript
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## Driver Adapter

Use a driver adapter for Prisma Postgres in the standard SQL workflow.

### Recommended for standard Node.js apps

1. Install adapter and driver:
   ```bash
   npm install @prisma/adapter-pg pg
   ```

2. Use the direct TCP connection string from Prisma Console:
   ```typescript
   import 'dotenv/config'
   import { PrismaClient } from '../generated/client'
   import { PrismaPg } from '@prisma/adapter-pg'

   const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL })
   const prisma = new PrismaClient({ adapter })
   ```

`PrismaPg` also accepts the connection string directly:

```typescript
const adapter = new PrismaPg(process.env.DATABASE_URL!)
const prisma = new PrismaClient({ adapter })
```

For PostgreSQL prepared statement naming, pass adapter options as the second argument:
````

```typescript
import { createHash } from 'node:crypto'

const adapter = new PrismaPg(process.env.DATABASE_URL!, {
  statementNameGenerator: ({ sql }) =>
    `prisma_${createHash('sha1').update(sql).digest('hex').slice(0, 16)}`,
})
```

Edge/serverless option

Use the Prisma Postgres serverless driver only when you need HTTP/WebSocket transport in environments like Workers or Edge Functions:

```bash
npm install @prisma/adapter-ppg @prisma/ppg
```

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaPostgresAdapter } from '@prisma/adapter-ppg'

const prisma = new PrismaClient({
  adapter: new PrismaPostgresAdapter({
    connectionString: process.env.PRISMA_DIRECT_TCP_URL,
  }),
})
```

This serverless driver is the specialized path for HTTP/WebSocket-based edge and serverless runtimes, not the default recommendation for standard Node.js apps.

Features

- **Serverless**: Scales to zero.
- **Caching**: Integrated query caching (Accelerate).
- **Real-time**: Database events (Pulse).

Using with Prisma Client

Use the Prisma Postgres adapter shown above when instantiating Prisma Client.

42. .agents\skills\prisma-database-setup\references\sqlite.md

````
# SQLite Setup

Configure Prisma with SQLite.

## Prerequisites

- None (file-based)

## 1. Schema Configuration

In `prisma/schema.prisma`:

```prisma
datasource db {
  provider = "sqlite"
}

generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

## 2. Config Configuration

In `prisma.config.ts`:

```typescript
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## 3. Environment Variable

In `.env`:

```env
DATABASE_URL="file:./dev.db"
```

### Connection String Format

```
file:PATH
```

- **PATH**: Relative path to the database file. Check `prisma.config.ts` if you need to confirm how your app resolves it.

## Driver Adapter

Use a driver adapter for the standard SQL workflow.

1. Install adapter and driver:
   ```bash
   npm install @prisma/adapter-better-sqlite3 better-sqlite3
   ```

2. Instantiate Prisma Client with the adapter:
   ```typescript
   import { PrismaClient } from '../generated/client'
   import { PrismaBetterSqlite3 } from '@prisma/adapter-better-sqlite3'

   const adapter = new PrismaBetterSqlite3({
     url: process.env.DATABASE_URL ?? 'file:./dev.db',
   })

   const prisma = new PrismaClient({ adapter })
   ```

## Using Driver Adapter (LibSQL / Turso)

For edge compatibility or Turso:

1. Install:
   ```bash
   npm install @prisma/adapter-libsql @libsql/client
   ```

2. Instantiate:
   ```typescript
   import { PrismaClient } from '../generated/client'
   import { PrismaLibSql } from '@prisma/adapter-libsql'

   const adapter = new PrismaLibSql({
````
````

```
    url: process.env.TURSO_DATABASE_URL,
    authToken: process.env.TURSO_AUTH_TOKEN,
  })
  const prisma = new PrismaClient({ adapter })
  ```

Limitations

- **No Enums**: SQLite doesn't support enums (Prisma polyfills them or treats as String).
- **No Scalar Lists**: `String[]` is not supported directly.
- **Concurrency**: Write operations lock the file.

Common Issues

"Database file not found"
Ensure the path in `DATABASE_URL` is correct relative to where Prisma is running or the schema file. `file:./dev.db` creates it next to schema.

43. .agents\skills\prisma-database-setup\references\sqlserver.md

````
# SQL Server Setup

Configure Prisma with Microsoft SQL Server.

## Prerequisites

- SQL Server 2017, 2019, 2022, or Azure SQL
- TCP/IP enabled

## 1. Schema Configuration

In `prisma/schema.prisma`:

```prisma
datasource db {
  provider = "sqlserver"
}

generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

## 2. Config Configuration

In `prisma.config.ts`:

```typescript
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## 3. Environment Variable

In `.env`:

```env
DATABASE_URL="sqlserver://localhost:1433;database=mydb;user=sa;password=Password123;encrypt=true;trustServerCertificate=true"
```

### Connection String Format

```
sqlserver://HOST:PORT;database=DB;user=USER;password=PASS;encrypt=true;trustServerCertificate=true
```

- **encrypt**: Required for Azure (true).
- **trustServerCertificate**: True for self-signed certs (local dev).

## Driver Adapter

Use a driver adapter for the standard SQL workflow.

1. Install adapter and driver:
   ```bash
   npm install @prisma/adapter-mssql mssql
   ```

2. Instantiate Prisma Client with the adapter:
   ```typescript
   import 'dotenv/config'
   import { PrismaClient } from '../generated/client'
   import { PrismaMssql } from '@prisma/adapter-mssql'

   const adapter = new PrismaMssql({
     server: 'localhost',
     port: 1433,
     database: 'mydb',
     user: process.env.SQLSERVER_USER,
     password: process.env.SQLSERVER_PASSWORD,
     options: {
       encrypt: true,
       trustServerCertificate: true,
     },
   })

   const prisma = new PrismaClient({ adapter })
   ```

## Common Issues

### "Login failed for user"
- SQL Server auth vs Windows auth. Prisma typically uses SQL Server authentication (username/password).
````

- Ensure TCP/IP is enabled in SQL Server Configuration Manager.

"Table not found" (dbo schema)
Prisma assumes `dbo` schema by default. If using another schema, update the model or connection string? SQL Server provider mostly sticks to default schema.

44. .agents\skills\prisma-database-setup\SKILL.md

```
---
name: prisma-database-setup
description: Guides for configuring Prisma with different database providers (PostgreSQL, MySQL, SQLite, MongoDB, etc.). Use when setting up a
new project, changing databases, or troubleshooting connection issues. Triggers on "configure postgres", "connect to mysql", "setup mongodb",
"sqlite setup".
license: MIT
metadata:
  author: prisma
  version: "7.6.0"
---

# Prisma Database Setup

Comprehensive guides for configuring Prisma ORM with various database providers.

## When to Apply

Reference this skill when:
- Initializing a new Prisma project
- Switching database providers
- Configuring connection strings and environment variables
- Troubleshooting database connection issues
- Setting up database-specific features
- Generating and instantiating Prisma Client

## Rule Categories by Priority

| Priority | Category | Impact | Prefix |
|----------|----------|--------|--------|
| 1 | Provider Guides | CRITICAL | provider names |
| 2 | Prisma Postgres | HIGH | `prisma-postgres` |
| 3 | Client Setup | CRITICAL | `prisma-client-setup` |

## System Prerequisites

- **Node.js 20.19.0+**
- **TypeScript 5.4.0+**

## Bun Runtime

If you're using Bun, run Prisma CLI commands with `bunx --bun prisma ...` so Prisma uses the Bun runtime instead of falling back to Node.js.

## Supported Databases

| Database | Provider String | Notes |
|----------|-----------------|-------|
| PostgreSQL | `postgresql` | Default, full feature support |
| MySQL | `mysql` | Widespread support, some JSON diffs |
| SQLite | `sqlite` | Local file-based, no enum/scalar lists |
| MongoDB | `mongodb` | Mongo-specific workflow; do not apply SQL driver-adapter guidance |
| SQL Server | `sqlserver` | Microsoft ecosystem |
| CockroachDB | `cockroachdb` | Distributed SQL, Postgres-compatible |
| Prisma Postgres | `postgresql` | Managed serverless database |

## Configuration Files

Your configuration shape depends on the provider and Prisma major version:

1. **All providers** use **`prisma/schema.prisma`**.
2. **Prisma 7 SQL setups** typically use **`prisma.config.ts`** for datasource URLs.
3. **MongoDB projects should stay on Prisma 6.x**, keep `url = env("DATABASE_URL")` in the schema, and continue using the classic MongoDB
setup.

## Driver Adapters

The standard SQL workflow uses a driver adapter. Choose the adapter and driver for your database and pass the adapter to `PrismaClient`.

| Database | Adapter | JS Driver |
|----------|---------|-----------|
| PostgreSQL | `@prisma/adapter-pg` | `pg` |
| CockroachDB | `@prisma/adapter-pg` | `pg` |
| Prisma Postgres (Node.js) | `@prisma/adapter-pg` | `pg` |
| Prisma Postgres (edge/serverless) | `@prisma/adapter-ppg` | `@prisma/ppg` |
| MySQL / MariaDB | `@prisma/adapter-mariadb` | `mariadb` |
| SQLite | `@prisma/adapter-better-sqlite3` | `better-sqlite3` |
| SQLite (Turso/LibSQL) | `@prisma/adapter-libsql` | `@libsql/client` |
| SQL Server | `@prisma/adapter-mssql` | `node-mssql` |

MongoDB should not follow the Prisma 7 SQL adapter workflow. Use the latest Prisma 6.x release for MongoDB projects and do not install a SQL
`@prisma/adapter-*` package for it.

Example (PostgreSQL):

```ts
import 'dotenv/config'
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL })
const prisma = new PrismaClient({ adapter })
```
```

```

## Prisma Client Setup (Required)

Prisma Client must be installed and generated for any database.

1. Install Prisma CLI and Prisma Client:
   ```bash
   npm install prisma --save-dev
   npm install @prisma/client
   ```

1. Add a generator block (`prisma-client` requires an explicit output path):
   ```prisma
   generator client {
     provider = "prisma-client"
     output   = "../generated"
   }
   ```

1. Generate Prisma Client:
   ```bash
   npx prisma generate
   ```

1. For SQL providers, instantiate Prisma Client with the database-specific driver adapter:
   ```typescript
   import { PrismaClient } from '../generated/client'
   import { PrismaPg } from '@prisma/adapter-pg'

   const adapter = new PrismaPg({ connectionString: process.env.DATABASE_URL })
   const prisma = new PrismaClient({ adapter })
   ```

1. Re-run `prisma generate` after every schema change.

Quick Reference

PostgreSQL
```prisma
datasource db {
  provider = "postgresql"
}

generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

MySQL
```prisma
datasource db {
  provider = "mysql"
}

generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

SQLite
```prisma
datasource db {
  provider = "sqlite"
}

generator client {
  provider = "prisma-client"
  output   = "../generated"
}
```

MongoDB
```prisma
datasource db {
  provider = "mongodb"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}
```

For MongoDB, stay on the latest Prisma 6.x line and keep the connection URL in `schema.prisma`. Do not move a MongoDB project to the Prisma 7 SQL adapter setup. If a MongoDB project asks about upgrading Prisma versions, route to the `prisma-mongodb-upgrade` skill (stay-on-v6 vs Prisma Next is the real decision; Prisma 7 is not an option).

Rule Files

See individual rule files for detailed setup instructions:
```

```
references/postgresql.md
references/mysql.md
references/sqlite.md
references/mongodb.md
references/sqlserver.md
references/cockroachdb.md
references/prisma-postgres.md
references/prisma-client-setup.md
```

How to Use

Choose the provider reference file for your database, then apply `references/prisma-client-setup.md` to complete client generation and adapter setup. For MongoDB, use `references/mongodb.md` instead of copying the SQL adapter examples or Prisma 7 config pattern.

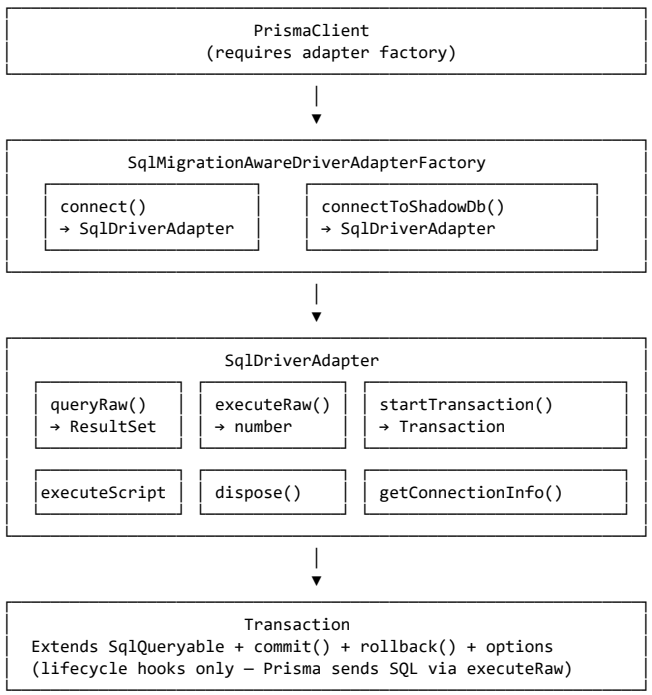
45. .agents\skills\prisma-driver-adapter-implementation\SKILL.md

```
---
name: prisma-driver-adapter-implementation
description: Required reference for Prisma v7 driver adapter work. Use when implementing or modifying adapters, adding database drivers, or
touching SqlDriverAdapter/Transaction interfaces. Contains critical contract details not inferable from code examples — including the
transaction lifecycle protocol, error mapping requirements, and verification checklist. Existing implementations do not replace this skill.
license: MIT
metadata:
  author: Tyler Benfield
  version: "7.6.0"
---

# Prisma 7 Driver Adapter Implementation Guide

This skill provides everything needed to implement a Prisma ORM v7 driver adapter for any database.

## Architecture Overview

```
```


```

Required Interfaces

Import from `@prisma/driver-adapter-utils`:

```typescript
import type {
  ColumnType,
  IsolationLevel,
  SqlDriverAdapter,
  SqlMigrationAwareDriverAdapterFactory,
  SqlQuery,
  SqlQueryable,
  SqlResultSet,
  Transaction,
  TransactionOptions,
  ArgType,
  ConnectionInfo,
  MappedError,
} from "@prisma/driver-adapter-utils";
import {
  ColumnTypeEnum,
  DriverAdapterError,
} from "@prisma/driver-adapter-utils";
```

Interface Definitions

SqlQuery (input to queryRaw/executeRaw)

```typescript
type SqlQuery = {
  sql: string; // Parameterized SQL with placeholders
  args: Array<unknown>; // Bound parameter values
  argTypes: Array<ArgType>; // Type hints for each argument
};

type ArgType = {
  scalarType: ArgScalarType; // 'string' | 'int' | 'bigint' | 'float' | 'decimal' | 'boolean' | 'enum' | 'uuid' | 'json' | 'datetime' |
```

```
'bytes' | 'unknown'
  dbType?: string;
  arity: "scalar" | "list";
};
```

SqlResultSet (output from queryRaw)

```typescript
interface SqlResultSet {
  columnNames: Array<string>; // Column names in order
  columnTypes: Array<ColumnType>; // Column types matching columnNames
  rows: Array<Array<unknown>>; // Row data as arrays
  lastInsertId?: string; // For INSERT without RETURNING
}
```

ColumnTypeEnum values

```typescript
const ColumnTypeEnum = {
  Int32: 0,
  Int64: 1,
  Float: 2,
  Double: 3,
  Numeric: 4,
  Boolean: 5,
  Character: 6,
  Text: 7,
  Date: 8,
  Time: 9,
  DateTime: 10,
  Json: 11,
  Enum: 12,
  Bytes: 13,
  Set: 14,
  Uuid: 15,
  Int32Array: 64,
  Int64Array: 65,
  FloatArray: 66,
  DoubleArray: 67,
  NumericArray: 68,
  BooleanArray: 69,
  CharacterArray: 70,
  TextArray: 71,
  DateArray: 72,
  TimeArray: 73,
  DateTimeArray: 74,
  JsonArray: 75,
  EnumArray: 76,
  BytesArray: 77,
  UuidArray: 78,
  UnknownNumber: 128,
} as const;
```

SqlDriverAdapter

```typescript
interface SqlDriverAdapter extends SqlQueryable {
  executeScript(script: string): Promise<void>;
  startTransaction(isolationLevel?: IsolationLevel): Promise<Transaction>;
  getConnectionInfo?(): ConnectionInfo;
  dispose(): Promise<void>;
}
```

Transaction

```typescript
interface Transaction extends SqlQueryable {
  readonly options: TransactionOptions;
  commit(): Promise<void>;
  rollback(): Promise<void>;
}

type TransactionOptions = { usePhantomQuery: boolean };
```

SqlMigrationAwareDriverAdapterFactory

```typescript
interface SqlMigrationAwareDriverAdapterFactory {
  readonly provider: "mysql" | "postgres" | "sqlite" | "sqlserver";
  readonly adapterName: string;
  connect(): Promise<SqlDriverAdapter>;
  connectToShadowDb(): Promise<SqlDriverAdapter>;
}
```

Implementation Steps

Step 1: Create the Queryable base class

```typescript
class MyQueryable<TClient> implements SqlQueryable {
  readonly provider = "postgres" as const; // or 'sqlite' | 'mysql' | 'sqlserver'
  readonly adapterName = "@my-org/adapter-mydb" as const;

  constructor(protected readonly client: TClient) {}

  async queryRaw(query: SqlQuery): Promise<SqlResultSet> {
    try {
      const args = query.args.map((arg, i) =>
        mapArg(arg, query.argTypes[i] ?? { scalarType: "unknown", arity: "scalar" })
      );

      // Execute query with your driver
      const result = await this.client.query(query.sql, args);

      // Extract column metadata
      const columnNames = /* get from result */;
      const columnTypes = /* map to ColumnTypeEnum */;

      // Map rows to ResultValue arrays
      const rows = result.map(row => mapRow(row, columnTypes));

      return { columnNames, columnTypes, rows };
    } catch (e) {
      this.onError(e);
    }
  }

  async executeRaw(query: SqlQuery): Promise<number> {
    try {
      const args = query.args.map((arg, i) =>
        mapArg(arg, query.argTypes[i] ?? { scalarType: "unknown", arity: "scalar" })
      );
      const result = await this.client.query(query.sql, args);
      return result.affectedRows ?? 0;
    } catch (e) {
      this.onError(e);
    }
  }

  protected onError(error: unknown): never {
    throw new DriverAdapterError(convertDriverError(error));
  }
}
```

Step 2: Create the Transaction class

Critical: `commit()` and `rollback()` are **lifecycle hooks only**. They must NOT issue SQL. Prisma sends `COMMIT`/`ROLLBACK` via `executeRaw` on the transaction object.

```typescript
class MyTransaction extends MyQueryable<TClient> implements Transaction {
  readonly options: TransactionOptions;
  readonly #release: () => void;

  constructor(
    client: TClient,
    options: TransactionOptions,
    release: () => void,
  ) {
    super(client);
    this.options = options;
    this.#release = release;
  }

  commit(): Promise<void> {
    // DO NOT issue COMMIT SQL here — Prisma does it via executeRaw
    this.#release(); // Release connection/resources
    return Promise.resolve();
  }

  rollback(): Promise<void> {
    // DO NOT issue ROLLBACK SQL here — Prisma does it via executeRaw
    this.#release();
    return Promise.resolve();
  }
}
```

Step 3: Create the Adapter class

```typescript
class MyAdapter extends MyQueryable<TClient> implements SqlDriverAdapter {
  #transactionDepth = 0;

  constructor(client: TClient) {
    super(client);
  }

  async executeScript(script: string): Promise<void> {
```

```
    // For SQLite: split on ';' and run each statement
    // For Postgres: use multi-statement execution
    try {
      // Implementation depends on driver capabilities
    } catch (e) {
      this.onError(e);
    }
  }

  async startTransaction(
    isolationLevel?: IsolationLevel,
  ): Promise<Transaction> {
    // Validate isolation level for your database
    const validLevels = new Set<IsolationLevel>([
      "READ UNCOMMITTED",
      "READ COMMITTED",
      "REPEATABLE READ",
      "SERIALIZABLE",
    ]);

    if (isolationLevel !== undefined && !validLevels.has(isolationLevel)) {
      throw new DriverAdapterError({
        kind: "InvalidIsolationLevel",
        level: isolationLevel,
      });
    }

    const options: TransactionOptions = { usePhantomQuery: false };

    this.#transactionDepth += 1;
    const depth = this.#transactionDepth;

    try {
      if (depth === 1) {
        // Issue BEGIN (with isolation level if specified)
        const beginSql = isolationLevel
          ? `BEGIN ISOLATION LEVEL ${isolationLevel}`
          : "BEGIN";
        await this.client.query(beginSql);
      } else {
        // Nested: use savepoints
        await this.client.query(`SAVEPOINT sp_${depth}`);
      }
    } catch (e) {
      this.#transactionDepth -= 1;
      this.onError(e);
    }

    const release = () => {
      this.#transactionDepth -= 1;
    };
    return new MyTransaction(this.client, options, release);
  }

  getConnectionInfo(): ConnectionInfo {
    return { supportsRelationJoins: true };
  }

  async dispose(): Promise<void> {
    await this.client.close();
  }
}
```

Step 4: Create the Factory class

```typescript
export type MyAdapterConfig = {
  url: string;
};

export type MyAdapterOptions = {
  shadowDatabaseUrl?: string;
};

export class MyAdapterFactory implements SqlMigrationAwareDriverAdapterFactory {
  readonly provider = "postgres" as const;
  readonly adapterName = "@my-org/adapter-mydb" as const;

  constructor(
    private readonly config: MyAdapterConfig,
    private readonly options?: MyAdapterOptions,
  ) {}

  connect(): Promise<SqlDriverAdapter> {
    return Promise.resolve(new MyAdapter(openConnection(this.config.url)));
  }

  connectToShadowDb(): Promise<SqlDriverAdapter> {
    const url = this.options?.shadowDatabaseUrl ?? this.config.url;
    return Promise.resolve(new MyAdapter(openConnection(url)));
  }
}
```

```

## Conversion Helpers

### Argument Mapping (input)

Convert Prisma argument values to driver-native types:

```typescript
function mapArg(arg: unknown, argType: ArgType): unknown {
  if (arg === null || arg === undefined) return null;

  // String → number for int columns
  if (typeof arg === "string" && argType.scalarType === "int")
    return Number.parseInt(arg, 10);

  // String → number for float columns
  if (typeof arg === "string" && argType.scalarType === "float")
    return Number.parseFloat(arg);

  // String → BigInt for bigint columns
  if (typeof arg === "string" && argType.scalarType === "bigint")
    return BigInt(arg);

  // Base64 string → Buffer for bytes columns
  if (typeof arg === "string" && argType.scalarType === "bytes")
    return Buffer.from(arg, "base64");

  // Boolean → 0/1 for SQLite
  if (typeof arg === "boolean" && /* SQLite */)
    return arg ? 1 : 0;

  return arg;
}
```

Row Mapping (output)

Convert driver result values to Prisma-expected types:

```typescript
function mapRow(row: unknown[], columnTypes: ColumnType[]): ResultValue[] {
  const result: ResultValue[] = [];

  for (let i = 0; i < row.length; i++) {
    const value = row[i] ?? null;
    const colType = columnTypes[i];

    if (value === null) {
      result.push(null);
      continue;
    }

    // bigint → string for Int64 (JSON-safe)
    if (typeof value === "bigint") {
      result.push(value.toString());
      continue;
    }

    // Date → ISO 8601 string for DateTime
    if (value instanceof Date) {
      result.push(value.toISOString());
      continue;
    }

    // JSON objects → stringified
    if (colType === ColumnTypeEnum.Json && typeof value === "object") {
      result.push(JSON.stringify(value));
      continue;
    }

    result.push(value as ResultValue);
  }

  return result;
}
```

Column Type Inference

When the driver doesn't provide type metadata, infer from JS values:

```typescript
function inferColumnType(value: NonNullable<unknown>): ColumnType {
  if (typeof value === "boolean") return ColumnTypeEnum.Boolean;
  if (typeof value === "bigint") return ColumnTypeEnum.Int64;
  if (value instanceof Uint8Array) return ColumnTypeEnum.Bytes;
  if (value instanceof Date) return ColumnTypeEnum.DateTime;
  if (Array.isArray(value)) return ColumnTypeEnum.Text; // fallback
  if (typeof value === "object") return ColumnTypeEnum.Json;
  if (typeof value === "number") return ColumnTypeEnum.UnknownNumber;
  return ColumnTypeEnum.Text;
}
```
```

```

## Error Handling

Map driver errors to `MappedError` for Prisma to handle correctly:

```typescript
function convertDriverError(error: unknown): MappedError {
  if (error instanceof Error) {
    // Database-specific error mapping
    const dbError = error as Error & { code?: string; errno?: number };

    // PostgreSQL example
    if (dbError.code === "23505") {
      return { kind: "UniqueConstraintViolation" };
    }
    if (dbError.code === "23502") {
      return { kind: "NullConstraintViolation" };
    }
    if (dbError.code === "23503") {
      return { kind: "ForeignKeyConstraintViolation" };
    }
    if (dbError.code === "42P01") {
      return { kind: "TableDoesNotExist" };
    }

    // SQLite example
    if (error.name === "SQLiteError") {
      return {
        kind: "sqlite",
        extendedCode: dbError.errno ?? 1,
        message: error.message,
      };
    }

    // PostgreSQL raw error
    if (dbError.code) {
      return {
        kind: "postgres",
        code: dbError.code,
        severity: "ERROR",
        message: error.message,
        detail: undefined,
        column: undefined,
        hint: undefined,
      };
    }
  }

  return { kind: "GenericJs", id: 0 };
}
```

Database-Specific Notes

SQLite

- Set `safeIntegers: true` when opening the database to get `bigint` for large integers
- Only `SERIALIZABLE` isolation level is valid
- `executeScript`: split on `;` and run each statement individually
- Boolean values: store as 0/1, return as boolean

PostgreSQL

- All standard isolation levels are valid
- For connection pooling (PgBouncer), use `prepare: false`
- Transactions require a dedicated connection (`reserve()` pattern)
- `executeScript`: use multi-statement execution (`.simple()` in some drivers)
- `int8` columns may return as string (already stringified by driver)
- `numeric` columns return as string to preserve precision

MySQL/MariaDB

- Supports `READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`
- Use `?` placeholders for parameters
- Handle `BIGINT` as string for large values

Testing Strategy

Unit Tests (no PrismaClient)

Test the adapter directly with the raw database driver:

```typescript
describe("queryRaw", () => {
  test("returns column names and types", async () => {
    const adapter = new MyAdapter(createTestConnection());
    const result = await adapter.queryRaw({
      sql: "SELECT id, name FROM users",
      args: [],
      argTypes: [],
    });
    expect(result.columnNames).toEqual(["id", "name"]);
```
```

```
    expect(result.columnTypes[0]).toBe(ColumnTypeEnum.Int32);
  });
});

describe("startTransaction", () => {
  test("commit persists changes", async () => {
    const adapter = new MyAdapter(createTestConnection());
    const tx = await adapter.startTransaction();
    await tx.executeRaw({
      sql: "INSERT INTO users (name) VALUES (?)",
      args: ["Alice"],
      argTypes: [],
    });
    // Prisma sends COMMIT via executeRaw
    await tx.executeRaw({ sql: "COMMIT", args: [], argTypes: [] });
    await tx.commit(); // lifecycle hook only
    // Verify data persisted
  });
});
```

E2E Tests (with PrismaClient)

Test the full integration:

```typescript
describe("E2E", () => {
  let prisma: PrismaClient;

  beforeEach(async () => {
    const factory = new MyAdapterFactory({ url: TEST_DB_URL });
    prisma = new PrismaClient({ adapter: factory });
  });

  test("CRUD operations", async () => {
    const user = await prisma.user.create({ data: { name: "Alice" } });
    expect(user.id).toBeGreaterThan(0);

    const found = await prisma.user.findUnique({ where: { id: user.id } });
    expect(found?.name).toBe("Alice");
  });

  test("transactions roll back on error", async () => {
    await expect(
      prisma.$transaction(async (tx) => {
        await tx.user.create({ data: { name: "Bob" } });
        throw new Error("Rollback!");
      }),
    ).rejects.toThrow();

    expect(await prisma.user.count()).toBe(0);
  });
});
```

Usage Example

```typescript
import { PrismaClient } from "./generated/prisma/client";
import { MyAdapterFactory } from "@my-org/adapter-mydb";

const factory = new MyAdapterFactory({
  url: process.env.DATABASE_URL!,
});

const prisma = new PrismaClient({ adapter: factory });

// Use prisma normally
const users = await prisma.user.findMany();
```

Checklist

Before considering the adapter complete:

- [ ] `SqlMigrationAwareDriverAdapterFactory` implemented with `connect()` and `connectToShadowDb()`
- [ ] `SqlDriverAdapter` implements `queryRaw`, `executeRaw`, `executeScript`, `startTransaction`, `dispose`
- [ ] `Transaction` implements `queryRaw`, `executeRaw`, `commit`, `rollback` with `options: { usePhantomQuery: false }`
- [ ] `commit()` and `rollback()` are lifecycle hooks only (no SQL issued)
- [ ] `startTransaction` issues `BEGIN` (depth 1) or `SAVEPOINT sp_N` (nested)
- [ ] Argument mapping handles: string→int, string→bigint, string→float, base64→bytes
- [ ] Row mapping handles: bigint→string, Date→ISO string, JSON→string
- [ ] Column types correctly mapped to `ColumnTypeEnum`
- [ ] Errors wrapped in `DriverAdapterError` with proper `MappedError` kind
- [ ] Isolation level validation for the target database
- [ ] Unit tests pass for queryRaw, executeRaw, executeScript, transactions
- [ ] E2E tests pass with real PrismaClient

46. .agents\skills\prisma-mongodb-upgrade\references\client-api-mapping.md

````
# client-api-mapping

How v6 Prisma Client calls map to Prisma Next's Mongo client — names map, parity does not.

## Priority

CRITICAL

## Why It Matters

The v6 and Next client APIs look superficially similar, but none of the v6 MongoDB raw
methods exist under their old names, aggregation moved to a different lane entirely, and
transactions go through the driver rather than a façade wrapper. Assuming parity produces
code that does not compile — or, in the transactions case, code that silently loses
atomicity.

## The mapping

| v6 call | Prisma Next equivalent | Notes |
|---------|------------------------|-------|
| `prisma.user.findMany(...)` | `db.orm.users.where(...).all()` | Fluent ORM lane; storage-name keys (see `schema-contract-mapping.md`) |
| `prisma.user.findFirst(...)` | `db.orm.users.where(...).first()` | |
| `create` / `update` / `upsert` / `delete` / `updateMany` / `deleteMany` | `create` / `update` / `upsert` / `delete` / `updateAll` / `deleteAll` on `db.orm.<collection>` | See Prisma Next's `prisma-next-queries` skill |
| `prisma.user.aggregate(...)`, `groupBy(...)` | **No ORM equivalent.** Use the typed aggregation-pipeline builder: `db.query.from(...).match(...).group(...).build()` | Prisma Next's `prisma-next-queries` skill covers the builder lane |
| `$runCommandRaw(...)` ([v6 docs](https://www.prisma.io/docs/orm/prisma-client/using-raw-sql/raw-queries#runcommandraw)) | **Name does not exist in Next.** Raw lane is `mongoRaw(...)` → a raw collection with `aggregate`, `insertOne/Many`, `updateOne/Many`, `deleteOne/Many`, `findOneAndUpdate/Delete`. For arbitrary database commands, use the underlying `mongodb` driver directly — it is a user-supplied peer dependency and fully accessible | Check the installed version's raw surface |
| `<model>.findRaw(...)` ([v6 docs](https://www.prisma.io/docs/orm/prisma-client/using-raw-sql/raw-queries#findraw)) | `mongoRaw(...)` collection reads (e.g. `aggregate` with a `$match` stage) | No direct `findRaw` name |
| `<model>.aggregateRaw(...)` ([v6 docs](https://www.prisma.io/docs/orm/prisma-client/using-raw-sql/raw-queries#aggregateraw)) | `mongoRaw(...).aggregate(...)` or the typed pipeline builder | |
| `$transaction(...)` — works on v6 with a replica set ([v6 docs](https://www.prisma.io/docs/orm/overview/databases/mongodb#replica-set-configuration)) | The façade does not wrap `db.transaction(...)` yet, **but the underlying `mongodb` driver is directly available** (user-supplied peer dependency): multi-document atomicity works today via driver sessions (`client.startSession()` / `session.withTransaction(...)`) on a replica set | A façade wrapper is expected soon; this row will be updated when it merges |
| `$connect` / `$disconnect` | `connect()` / `close()` on the Mongo façade client | |

## Bad

```typescript
// Assuming v6 names exist in Prisma Next:
await db.user.$runCommandRaw({ collStats: 'users' }); // no such method
await db.transaction(async (tx) => { ... });          // no such method on the Mongo façade
```

## Good

```typescript
// Raw lane under its Next name:
const raw = mongoRaw(db);
await raw.users.aggregate([{ $match: { status: 'active' } }]);

// Aggregation through the typed pipeline builder:
const stats = await db.query.from('users').group({ _id: '$role', n: { $count: {} } }).build();

// Multi-document atomicity today: the mongodb driver (a direct dependency of the
// project) exposes sessions and transactions as usual:
const session = mongoClient.startSession();
await session.withTransaction(async () => {
  // ...writes...
});
```

## References

- [v6 MongoDB raw queries](https://www.prisma.io/docs/orm/prisma-client/using-raw-sql/raw-queries#raw-queries-with-mongodb)
- [v6 replica set requirement for transactions](https://www.prisma.io/docs/orm/overview/databases/mongodb#replica-set-configuration)
- Prisma Next queries + runtime skills (`skills/prisma-next-queries`, incl. its dedicated `mongo.md`; `skills/prisma-next-runtime`) — authoritative for the Next side; verified @ `a2791c5dd59d579b4b3052942ae7f8fe5e2ee852`
````

47. .agents\skills\prisma-mongodb-upgrade\references\decision-stay-or-migrate.md

````
# decision-stay-or-migrate

How to decide between migrating a MongoDB project to Prisma Next and staying on Prisma v6.

## Priority

CRITICAL

## Why It Matters

MongoDB projects cannot follow the general "upgrade Prisma" advice: Prisma 7 has no MongoDB
connector, so the forward path is Prisma Next. Advising an impossible v7 upgrade, or
silently rewriting the app onto SQL, are both serious failure modes. The encouraged path is
migrating to Prisma Next — its MongoDB support is Early Access and the Prisma team wants
early adopters' feedback — with a deliberate stay on v6 where a hard blocker applies.

## The facts the decision rests on

Prisma Next side (verified against prisma/prisma-next @ `a2791c5dd59d579b4b3052942ae7f8fe5e2ee852`;
status confirmed by the Prisma team 2026-07):

- **MongoDB support is Early Access**, actively developed, with GA planned after Postgres.
- The implementation is deep, not a stub: a full package family (ORM, typed
  aggregation-pipeline builder, raw lane, driver over the official `mongodb` package),
  first-class contract-driven migrations, and extensive tests against real in-memory MongoDB.
- **The Mongo client façade does not wrap `db.transaction(...)` yet** — multi-document
  atomicity is done through the MongoDB driver's session API, which is directly available
  (the `mongodb` package is a user-supplied peer dependency). A façade wrapper is expected;
  this skill will be updated when it merges.
- Early Access means pre-1.0 minors can carry breaking changes, with published upgrade
  recipes (e.g. 0.11→0.12 changed Mongo validator emission and made `mongodb` a
  user-supplied peer dependency). Floor: MongoDB 8.0 and `mongodb@^7`.

Prisma v6 side:

- v6 fully supports MongoDB, including transactions on replica sets — "MongoDB only allows
  you to start a transaction on a replica set. Prisma ORM uses transactions internally"
  ([replica set configuration](https://www.prisma.io/docs/orm/overview/databases/mongodb#replica-set-configuration)).
- v6 MongoDB has no Prisma Migrate; the workflow is `db push`
  ([no support for Prisma Migrate](https://www.prisma.io/docs/orm/overview/databases/mongodb#no-support-for-prisma-migrate)).

## Blocker checks before migrating

Run these checks yourself — from the codebase, not by asking the user:

- **Search the codebase for `$transaction` usage** (grep for `$transaction`). If present,
  plan the raw-driver session equivalents before migrating (see `client-api-mapping.md`) —
  or stay on v6 until the façade wrapper lands.
- **Check the MongoDB server version** (must be 8.0+ for Next; v6 tolerated older).
- **Confirm the team can absorb pre-1.0 upgrades.** Next publishes versioned upgrade recipes
  between minors; someone has to run them. For a production app, confirm the user accepts
  Early Access status before migrating.

## Bad

```text
User: "We're on Prisma 6 with MongoDB. Should we upgrade to Prisma 7?"
Agent: "Yes — here's the v7 upgrade guide. Step 1: install a driver adapter..."
```

Prisma 7 has no MongoDB connector; this migration is impossible and the SQL driver-adapter
steps corrupt a working v6 setup.

## Good

```text
User: "We're on Prisma 6 with MongoDB. Should we upgrade to Prisma 7?"
Agent: "Prisma 7 does not support MongoDB — v6 is the last classic-ORM
major for MongoDB. The path forward is Prisma Next, the successor: its MongoDB support is
Early Access and migrating is encouraged. Let me check the codebase for blockers first —
searching for $transaction usage and checking the MongoDB server version..."
```

## Stay-on-v6 hygiene

Staying is a decision, not a default-by-neglect:

- Pin `prisma` and `@prisma/client` to the latest 6.x and keep taking 6.x patches.
- Watch Prisma release notes and security advisories for the 6.x maintenance line.
- Keep the classic setup (`url = env("DATABASE_URL")` in the schema; `db push`; no SQL
  driver adapters).
- Re-evaluate when Prisma Next's MongoDB is GA, or when blockers for trying EA are resolved.

## References

- [Prisma Next repository](https://github.com/prisma/prisma-next)
- [Prisma v6 MongoDB documentation](https://www.prisma.io/docs/orm/overview/databases/mongodb)
````

48. .agents\skills\prisma-mongodb-upgrade\references\migrations-mapping.md

````
# migrations-mapping

How the v6 MongoDB "no migrations" story maps onto Prisma Next's first-class migration flow.

## Priority

HIGH

## Why It Matters

This is the largest workflow change in the migration — in v6, MongoDB explicitly has no
Prisma Migrate, while in Prisma Next MongoDB participates in the full migration lifecycle.
Teams porting a `db push` habit into Next without understanding the plan/verify/sign flow
will fight the tooling or bypass its safety rails.

## v6: `db push` only

MongoDB on v6 has no Prisma Migrate and no plans to add it — "MongoDB projects do not rely
on internal schemas" ([no support for Prisma Migrate](https://www.prisma.io/docs/orm/overview/databases/mongodb#no-support-for-prisma-
migrate)).
The workflow is `prisma db push` to sync indexes and unique constraints, with no migration
history on disk.

## Prisma Next: first-class, contract-driven migrations (Mongo included)

Migration authoring in Next is first-class for Postgres **and Mongo** (prisma-next
`skills/prisma-next-migrations/SKILL.md`) — MongoDB is not a push-only special case:

- **Flow:** contract *emit* → diff → *plan* (writes a content-hashed migration package) →
  *migrate* (apply in graph order) → *verify* (live schema vs destination contract) →
  *sign* (advance the marker after a verify pass).
- **Mongo migration ops** come from dedicated factories: `createCollection`,
  `dropCollection`, `validatedCollection`, `setValidation`, `createIndex`, `dropIndex`,
  `collMod`, and `dataTransform` for data backfills.
- **Marker storage:** Next records migration state in a document in the
  `_prisma_migrations` collection (per space) — the same collection name family v6 users
  know from SQL, repurposed for Mongo state.
- **DDL is not transactional on Mongo:** the runner applies operations, verifies the live
  schema against the destination contract, and only advances the marker on a verify pass —
  making interrupted runs resumable rather than atomic (see Prisma Next's
  `prisma-next-migrations` skill).
- **Push-style alternative still exists:** `db update` diffs the live database against the
  contract and applies directly without writing a migration directory — the closest
  analogue to the v6 `db push` habit, at the cost of no history.
- Validators: Next emits closed `$jsonSchema` validators by default since 0.12 (prisma-next
  `CHANGELOG.md`) — collections gain schema enforcement v6 never applied.

## Bad

```text
Porting the v6 habit: run the Next equivalent of `db push` for every change in production,
accumulating no migration history, and hand-editing collections when verification fails.
```

## Good

```text
Adopt the Next lifecycle: emit the contract, plan a migration package, apply it with
migrate, let verify gate the marker, and sign. Reserve `db update` for local prototyping,
mirroring how `db push` was used on v6.
```

## References

- [v6: no Prisma Migrate for MongoDB](https://www.prisma.io/docs/orm/overview/databases/mongodb#no-support-for-prisma-migrate)
- Prisma Next migrations skill (`skills/prisma-next-migrations`) — authoritative for the Next side; verified @
`a2791c5dd59d579b4b3052942ae7f8fe5e2ee852`
````

49. .agents\skills\prisma-mongodb-upgrade\references\schema-contract-mapping.md

```
# schema-contract-mapping

How v6 MongoDB schema concepts map onto Prisma Next's contract model.

## Priority

HIGH

## Why It Matters

Prisma Next does not consume the v6 `schema.prisma` as-is: the schema becomes a *contract*
(authored in PSL or TypeScript via the contract builder), and several v6 MongoDB idioms have
different — or deliberately absent — equivalents. Translating mechanically without knowing
the mapping produces contracts that fail verification or, worse, silently change collection
addressing.

## The mapping

| v6 concept | Prisma Next equivalent | Notes |
|------------|------------------------|-------|
| `datasource db { provider = "mongodb" }` + `url = env(...)` ([v6 docs](https://www.prisma.io/docs/orm/overview/databases/mongodb#example)) | `defineConfig` from `@prisma-next/mongo/config` wiring the mongo family/target/adapter/driver descriptors | Next selects MongoDB by importing the `@prisma-next/mongo` façade, not by a provider string in the schema; `prisma-next init` accepts `mongodb` as a target name |
| `@id @default(auto()) @map("_id") @db.ObjectId` ([using ObjectId](https://www.prisma.io/docs/orm/overview/databases/mongodb#using-objectid)) | ObjectId-typed id field in the Next contract (PSL or TS builder) | Verify the exact attribute surface against the installed Next version's `prisma-next-contract` skill — the contract builder also exposes `index` and `valueObject` |
| Composite (embedded) types — MongoDB-only in v6 ([composite types](https://www.prisma.io/docs/orm/prisma-client/special-fields-and-types/composite-types)) | Value objects / embedded shapes in the Next contract (`valueObject` in the Mongo contract builder) | Same conceptual role: documents embedded in a parent document |
| Model names address the client (`prisma.user`) | **Collection storage names** address the ORM: `db.orm.users`, i.e. the `@@map(...)` name or the lowercased model name — not `db.orm.User` | prisma-next `skills/prisma-next/SKILL.md`, `skills/prisma-next-quickstart/SKILL.md`; the most common porting mistake |
| Indexes declared in schema, applied by `db push` | Indexes are contract-declared and applied through migrations (`createIndex`/`dropIndex` factories) | See `migrations-mapping.md` |
| No native polymorphism | No schema-layer polymorphism on Mongo either: `@@base`/`@@discriminator` are SQL-only in Next; model an explicit `discriminator` field | prisma-next `skills/prisma-next-contract/SKILL.md` |

## Bad

```typescript
// Ported from v6 and addressed by model name:
const user = await db.orm.User.first(); // undefined — Mongo ORM keys are storage names
```

Good

```typescript
// Mongo ORM keys are collection storage names (@@map or lowercased model name):
const user = await db.orm.users.first();
```

Environment requirements

Prisma Next's Mongo target requires MongoDB 8.0+ and `mongodb@^7` installed by the user as a
peer dependency (prisma-next `CHANGELOG.md`, 0.11→0.12). v6 supports older MongoDB servers,
so check the server version before planning a migration.

References

- [v6 MongoDB schema documentation](https://www.prisma.io/docs/orm/overview/databases/mongodb)
- [v6 composite types (MongoDB-only)](https://www.prisma.io/docs/orm/prisma-client/special-fields-and-types/composite-types)
- Prisma Next contract skill (`skills/prisma-next-contract`) in the prisma-next repository — authoritative for the Next side
```

50. .agents\skills\prisma-mongodb-upgrade\references\verify-cutover-checklist.md

```
# verify-cutover-checklist

Verification checklist for a v6 → Prisma Next cutover: the data never moves — only the code does.

## Priority

CRITICAL

## Why It Matters

A v6 → Next migration is a *client and workflow* migration against the **same MongoDB
database** — there is no data export/import step, and introducing one (or pointing the new
stack at a fresh database) turns a code migration into an outage. The checklist below keeps
the cutover observable and reversible.

## Ground rules

- **No data moves.** The Next contract is authored to describe the existing collections;
  both stacks read the same database during the staged phase.
- **v6 stays runnable until cutover is verified.** Do not delete the v6 client, schema, or
  dependencies until the checklist passes.

## Checklist

1. **Same database, verified:** the Next config points at the same connection string /
   database name the v6 app uses (minus v6-specific URL parameters that the `mongodb@^7`
   driver rejects — validate the URL with the driver first).
2. **Server floor:** MongoDB server is 8.0+ (Next's requirement; v6 tolerated older).
   Confirm before authoring any contract.
3. **Contract round-trip on a copy:** on a staging copy (or `mongodb-memory-server`), emit
   the contract, run plan → migrate → verify → sign, and confirm `verify` passes against
   data copied from production shape. Verification failures here are contract-mapping bugs,
   not database problems.
4. **Index parity:** enumerate indexes on every collection (`db.collection.getIndexes()`)
   and confirm the Next contract declares the same set — v6 `db push` may have created
   indexes the new contract must re-declare, or verification and query performance will
   diverge.
5. **Validator impact assessed:** Next emits closed `$jsonSchema` validators by default;
   confirm legacy documents (extra fields, drifted shapes) pass them on the staging copy
   before applying to production.
6. **Storage-name addressing audited:** every ported call site uses collection storage
   names (`db.orm.users`), not model names (see `schema-contract-mapping.md`).
7. **Transaction inventory mapped:** grep the v6 app for `$transaction`; each hit gets a
   driver-session equivalent (the `mongodb` driver is directly available; the façade wrapper
   is expected soon — see `client-api-mapping.md`).
8. **Raw call inventory mapped:** every `$runCommandRaw` / `findRaw` / `aggregateRaw` call
   has an explicit Next-side replacement (`mongoRaw(...)` lane or pipeline builder).
9. **Staged read-only soak:** run the Next stack read-only against staging/production data
   alongside v6 and compare outputs before allowing writes.
10. **Cutover + rollback:** switch writes to Next only after the soak; keep the v6 branch
    deployable as the rollback path. Rolling back is a code rollback — the data was never
    moved.

After cutover, install and follow Prisma Next's own skills for ongoing work (see the
hand-off rule in `SKILL.md`).

## References

- [v6 MongoDB documentation](https://www.prisma.io/docs/orm/overview/databases/mongodb)
- Prisma Next migrations + queries skills — authoritative for the Next side; verified @ `a2791c5dd59d579b4b3052942ae7f8fe5e2ee852`
```

51. .agents\skills\prisma-mongodb-upgrade\SKILL.md

```
---
name: prisma-mongodb-upgrade
description: Decision and migration guide for Prisma ORM MongoDB projects on v6, which have no upgrade path to v7. Use when a MongoDB project
asks about upgrading Prisma, when "upgrade to prisma 7" comes up in a project with provider = "mongodb", or when evaluating a move to Prisma
Next. Triggers on "upgrade prisma mongodb", "prisma 7 mongodb", "mongodb prisma migration", "prisma next mongodb".
license: MIT
metadata:
  author: prisma
  version: "0.1.0"
---

# Prisma MongoDB Upgrade Path

MongoDB projects are the one Prisma cohort with no road into Prisma 7: **v6 is the terminal
classic-ORM major for MongoDB, and v7 never ships a MongoDB connector**. The successor path
is [Prisma Next](https://github.com/prisma/prisma-next), where MongoDB support is in Early
Access with GA planned after Postgres. This skill frames the real decision — migrate to
Prisma Next (the encouraged path), or stay on v6 where a hard blocker applies — and carries
the migration mechanics.

**Never do either of these:**

- Never advise a MongoDB project to "upgrade to Prisma 7". The connector does not exist
  there. The `prisma-upgrade-v7` guide does not apply to MongoDB projects.
- Never solve the version question by rewriting the app onto a SQL database. Changing the
  database engine is a separate, much larger decision that is not yours to make implicitly.

## The version landscape

| Version | MongoDB status |
|---------|----------------|
| Prisma ORM v6 | Fully supported (`mongodb` provider); latest 6.x is the current stable path; maintenance line |
| Prisma ORM v7 | **No MongoDB connector — not an option, ever** |
| Prisma Next | MongoDB support in **Early Access**, actively developed, GA planned after Postgres — the successor path for MongoDB projects |

## The decision, up front

**Migrating to Prisma Next is the encouraged path.** MongoDB support in Prisma Next is Early
Access: functional and moving quickly, with GA planned after Postgres — and the Prisma team
wants MongoDB users to migrate early and share feedback. The migration mechanics are
detailed in the references.

**Staying on the latest v6 remains a legitimate choice where a hard blocker applies** —
stated plainly: the Next Mongo façade does not wrap transactions yet (the underlying driver
is available directly; this is expected to change soon), and pre-1.0 minors can carry
breaking changes with published upgrade recipes.

### Decision table

| Signal | Direction |
|--------|-----------|
| No blockers below apply | Migrate to Next; run the `verify-cutover-checklist` and share feedback with the Prisma team |
| Greenfield / prototype / internal tool | Migrate to Next |
| Codebase uses multi-document transactions (`$transaction`) — check with grep, do not ask | Plan raw-driver session equivalents first (see
`client-api-mapping`), or stay on v6 until the façade wrapper lands |
| Team cannot absorb pre-1.0 breaking upgrades between minors | Stay on v6 until GA |
| Risk-averse but interested | Run a staged Next round-trip on a copy (see `verify-cutover-checklist`), then migrate |

Note: the transactions gap is expected to close soon — this section will be updated when
façade transactions merge in Prisma Next.

### If staying on v6: hygiene (a deliberate stay, not neglect)

- Pin the Prisma packages to the latest 6.x line and keep taking 6.x patch releases.
- Track Prisma release notes and security advisories for the 6.x line.
- Keep the classic v6 MongoDB setup: `url = env("DATABASE_URL")` in the schema, `db push`
  workflow, no SQL driver adapters (see `prisma-database-setup` for the v6 MongoDB shape).
- Re-evaluate when Prisma Next's MongoDB is GA, or when blockers for trying EA are resolved.

## Reference files

| Reference | What it covers |
|-----------|----------------|
| `references/decision-stay-or-migrate.md` | The full decision framing, blocker checks, and stay-hygiene detail |
| `references/schema-contract-mapping.md` | v6 schema (`mongodb` provider, `@db.ObjectId`, composite types) → Next contract concepts |
| `references/client-api-mapping.md` | v6 client calls → Next equivalents, incl. raw escape hatches and transactions — names map, parity does
not |
| `references/migrations-mapping.md` | v6 `db push`-only story → Next's plan/migrate/verify/sign flow |
| `references/verify-cutover-checklist.md` | No-data-moves verification: same DB, index parity, staged round-trip before cutover |

## Verified against

Behavioral claims about Prisma Next in this skill were verified against
[prisma/prisma-next](https://github.com/prisma/prisma-next) at commit
`a2791c5dd59d579b4b3052942ae7f8fe5e2ee852` (pre-1.0, ~v0.14/0.15 line). Prisma Next moves
quickly in Early Access: **before acting on any Next-side claim, verify it against the
version actually installed** (check the project's `@prisma-next/*` versions and the
prisma-next skills installed with it). Next's Mongo target requires MongoDB 8.0+ and expects
`mongodb@^7` as a user-supplied peer dependency.
```

Hand-off rule

This skill is the **discovery bridge**, not a replacement for Prisma Next's own documentation. After a project switches to Prisma Next, run Prisma Next's `init`/skill installation and follow its own skills (quickstart, contract, queries, migrations, runtime) for day-to-day work — do not keep working from this skill's summaries.

52. .agents\skills\prisma-postgres-setup\references\api-basics.md

````
# api-basics

Core conventions for the Prisma Management API. All three `prisma-postgres-*` skills share these patterns.

## Base URL

```
https://api.prisma.io/v1
```

API documentation: https://api.prisma.io/v1/doc

## Response Envelope

### Single resource

```json
{
  "data": {
    "id": "proj_clx7abc123def456",
    "type": "project",
    "name": "My Project",
    "createdAt": "2025-06-15T10:30:00.000Z"
  }
}
```

### Collection

```json
{
  "data": [
    { "id": "proj_aaa", "type": "project", "name": "Alpha" },
    { "id": "proj_bbb", "type": "project", "name": "Beta" }
  ],
  "pagination": {
    "hasMore": true,
    "nextCursor": "clx7cursor123"
  }
}
```

## Resource ID Prefixes

Every resource ID carries a type prefix:

| Prefix | Resource |
|---|---|
| `proj_` | Project |
| `db_` | Database |
| `con_` | Connection |
| `wksp_` | Workspace |

Always include the prefix when sending IDs in API requests.

## Pagination

Collection endpoints use cursor-based pagination:

```
GET /v1/projects?limit=10
GET /v1/projects?cursor=clx7abc123&limit=10
```

| Parameter | Type | Default | Description |
|---|---|---|---|
| `cursor` | string | — | Opaque cursor from `nextCursor` |
| `limit` | number | 100 | Maximum items per page |

Continue fetching while `pagination.hasMore` is `true`, using `pagination.nextCursor` as the `cursor` parameter.

## Error Responses

All errors follow this shape:

```json
{
  "error": {
    "code": "resource-not-found",
    "message": "database with id db_abc not found"
  }
}
```

### Error codes by HTTP status

| HTTP Status | Error Code | Meaning |
|---|---|---|
| 400 | `client-error` | Malformed request |
| 401 | `authentication-failed` | Missing or invalid token |
````

403	`permission-denied`	Token lacks required access
404	`resource-not-found`	Resource does not exist or is not accessible
422	`validation-error`	Request body failed validation
429	`rate-limit-exceeded`	Too many requests
500	`internal-server-error`	Server error — retry after a delay

Self-correction patterns

- **401**: Token is invalid or expired. Create a new service token in Console → Workspace Settings → Service Tokens.
- **404**: Verify the resource ID includes the correct prefix (`proj_`, `db_`, `con_`). Use `GET /v1/projects` or `GET /v1/databases` to list available resources.
- **422**: Check the request body against the endpoint schema. Common issues: missing required fields, invalid region ID, empty `name`.
- **429**: Wait 2–5 seconds and retry. If repeated, increase the backoff interval.

53. .agents\skills\prisma-postgres-setup\references\auth.md

````
# auth

How to authenticate with the Prisma Management API using service tokens.

## Service Tokens

Service tokens authenticate server-to-server requests. They are scoped to a workspace and grant access to all resources within it.

### Creating a service token

1. Open https://console.prisma.io
2. Navigate to **Workspace Settings** → **Service Tokens**
3. Click **Create Token**
4. Copy the token immediately — it is only shown once

### Using a service token

Set the token as an environment variable:

```bash
export PRISMA_SERVICE_TOKEN="eyJ..."
```

Include it in the `Authorization` header of every API request:

```bash
curl -H "Authorization: Bearer $PRISMA_SERVICE_TOKEN" \
  https://api.prisma.io/v1/projects
```

### Token scope

Service tokens are workspace-scoped. A single token grants access to all projects, databases, and connections within the workspace. There are no project-scoped tokens at this time.

### Security practices

- Store tokens in environment variables or secret managers, never in source code
- Add `.env` to `.gitignore` to prevent accidental commits
- Rotate tokens periodically via Console → Workspace Settings → Service Tokens
- In CI/CD, store tokens as encrypted secrets (e.g., GitHub Secrets)

## OAuth 2.0 (for user-scoped access)

OAuth is used when acting on behalf of a user, typically in partner/integrator flows. See the `prisma-postgres-integrator` skill for OAuth details.

For standard database setup, service tokens are the recommended authentication method.
````

54. .agents\skills\prisma-postgres-setup\references\endpoints.md

````
# endpoints

Management API endpoint details for database setup workflows.

## List regions

```
GET /v1/regions/postgres
```

No request body. Returns available Prisma Postgres regions.

**Response:**

```json
{
  "data": [
    {
      "id": "us-east-1",
      "type": "region",
      "name": "US East (N. Virginia)",
      "status": "available"
    },
    {
      "id": "eu-west-1",
      "type": "region",
      "name": "EU West (Ireland)",
      "status": "available"
    }
  ]
}
```

Only use regions where `status` is `available`.

## Create project (with database)

```
POST /v1/projects
```

**Request body:**

```json
{
  "name": "my-project",
  "region": "us-east-1",
  "createDatabase": true
}
```

| Field | Type | Required | Default | Description |
|---|---|---|---|---|
| `name` | string | No | Auto-generated | Project display name |
| `region` | string | No | `us-east-1` | Region for the database |
| `createDatabase` | boolean | No | `true` | Create a default database with the project |

**Response** (with `createDatabase: true`):

```json
{
  "data": {
    "id": "proj_clx7abc123",
    "type": "project",
    "url": "https://api.prisma.io/v1/projects/proj_clx7abc123",
    "name": "my-project",
    "createdAt": "2025-06-15T10:30:00.000Z",
    "defaultRegion": "us-east-1",
    "workspace": {
      "id": "wksp_xyz789",
      "url": "https://api.prisma.io/v1/workspaces/wksp_xyz789",
      "name": "My Workspace"
    },
    "database": {
      "id": "db_def456",
      "type": "database",
      "url": "https://api.prisma.io/v1/databases/db_def456",
      "name": "my-project",
      "status": "ready",
      "createdAt": "2025-06-15T10:30:00.000Z",
      "isDefault": true,
      "defaultConnectionId": "con_ghi789",
      "connections": [
        {
          "id": "con_ghi789",
          "type": "connection",
          "url": "https://api.prisma.io/v1/connections/con_ghi789",
          "name": "Default",
          "createdAt": "2025-06-15T10:30:00.000Z",
          "kind": "postgres",
````
````

```
          "endpoints": {
            "direct": {
              "host": "db.prisma.io",
              "port": 5432,
              "connectionString": "postgres://user:pass@db.prisma.io:5432/postgres?sslmode=require"
            }
          }
        }
      ],
      "region": {
        "id": "us-east-1",
        "name": "US East (N. Virginia)"
      }
    }
  }
}
```

Key field to extract:

- `data.database.connections[0].endpoints.direct.connectionString` → use as `DATABASE_URL`

The response also includes `pooled` and `accelerate` endpoints — ignore these for new projects. The direct connection string is all you need.

If `data.database.status` is `provisioning`, poll `GET /v1/databases/{id}` until `status` is `ready`.

Get database

```
GET /v1/databases/{databaseId}
```

Use to check database status after creation or to retrieve database details.

Response:

```json
{
  "data": {
    "id": "db_def456",
    "type": "database",
    "url": "https://api.prisma.io/v1/databases/db_def456",
    "name": "my-project",
    "status": "ready",
    "createdAt": "2025-06-15T10:30:00.000Z",
    "isDefault": true,
    "defaultConnectionId": "con_ghi789",
    "connections": [],
    "project": {
      "id": "proj_clx7abc123",
      "url": "https://api.prisma.io/v1/projects/proj_clx7abc123",
      "name": "my-project"
    },
    "region": {
      "id": "us-east-1",
      "name": "US East (N. Virginia)"
    }
  }
}
```

Create connection

```
POST /v1/databases/{databaseId}/connections
```

Creates a new named connection string for a database. Use for per-developer or per-environment connections.

Request body:

```json
{
  "name": "dev"
}
```

Field	Type	Required	Description
`name`	string	Yes	Display name for the connection

Response:

```json
{
  "data": {
    "id": "con_newcon123",
    "type": "connection",
    "url": "https://api.prisma.io/v1/connections/con_newcon123",
    "name": "dev",
    "createdAt": "2025-06-15T10:31:00.000Z",
    "kind": "postgres",
    "endpoints": {
```

```
      "direct": {
        "host": "db.prisma.io",
        "port": 5432,
        "connectionString": "postgres://user:pass@db.prisma.io:5432/postgres?sslmode=require"
      }
    },
    "database": {
      "id": "db_def456",
      "url": "https://api.prisma.io/v1/databases/db_def456",
      "name": "my-project"
    }
  }
}
```

Extract: `data.endpoints.direct.connectionString` → use as `DATABASE_URL`.

Delete database

```
DELETE /v1/databases/{databaseId}
```

Permanently deletes a database and all its connections. Returns `204 No Content` on success.

List projects

```
GET /v1/projects
```

Returns all projects in the workspace. Supports cursor-based pagination (`?cursor=...&limit=...`).

Delete project

```
DELETE /v1/projects/{projectId}
```

Permanently deletes a project and all its databases. Returns `204 No Content` on success.

55. .agents\skills\prisma-postgres-setup\references\prisma7-client.md

````
# Prisma 7 Client Instantiation

Prisma 7 changed how PrismaClient connects to databases. The CLI (`prisma db push`, `prisma migrate`) reads the URL from `prisma.config.ts`.
But at **runtime**, you must provide a driver adapter to PrismaClient explicitly.

## Required packages

```bash
npm install @prisma/client @prisma/adapter-pg pg
```

- `@prisma/adapter-pg` — the Prisma adapter for the `pg` PostgreSQL driver
- `pg` — the underlying Node.js PostgreSQL driver

## Basic instantiation

```typescript
import 'dotenv/config'
import pg from 'pg'
import { PrismaPg } from '@prisma/adapter-pg'
import { PrismaClient } from './generated/prisma/client.js'

const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL })
const adapter = new PrismaPg(pool)
const prisma = new PrismaClient({ adapter })
```

## Key rules

1. **Import path**: Always `./generated/prisma/client.js` — not `./generated/prisma` and not `@prisma/client`.

2. **Adapter is mandatory**: `new PrismaClient()` with no arguments throws. `new PrismaClient({ datasourceUrl: '...' })` also throws —
`datasourceUrl` does not exist in Prisma 7.

3. **ESM required**: The generated client uses ESM. Ensure `package.json` has `"type": "module"`.

4. **Pool lifecycle**: Call `await pool.end()` when shutting down (after `prisma.$disconnect()`).

## Usage in application code

```typescript
import 'dotenv/config'
import pg from 'pg'
import { PrismaPg } from '@prisma/adapter-pg'
import { PrismaClient } from './generated/prisma/client.js'

const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL })
const adapter = new PrismaPg(pool)
const prisma = new PrismaClient({ adapter })

// Create
const user = await prisma.user.create({
  data: { email: 'alice@example.com', name: 'Alice' },
})

// Read with relations
const posts = await prisma.post.findMany({
  where: { published: true },
  include: { author: true },
})

// Update
await prisma.post.update({
  where: { id: 1 },
  data: { published: true },
})

// Delete
await prisma.post.delete({ where: { id: 1 } })

// Cleanup
await prisma.$disconnect()
await pool.end()
```

## Common mistakes

| Mistake | Error | Fix |
|---|---|---|
| `import { PrismaClient } from './generated/prisma'` | `Cannot find module` | Use `./generated/prisma/client.js` |
| `new PrismaClient()` | `PrismaClient needs non-empty options` | Pass `{ adapter }` |
| `new PrismaClient({ datasourceUrl: url })` | `Unknown property datasourceUrl` | Use adapter pattern instead |
| Missing `"type": "module"` in package.json | ESM import errors | Add `"type": "module"` |
| `import { PrismaClient } from '@prisma/client'` | Wrong export | Use `./generated/prisma/client.js` |
````

56. .agents\skills\prisma-postgres-setup\SKILL.md

```
---
name: prisma-postgres-setup
description: Set up a new Prisma Postgres database and connect it to a local project using the Management API. Use when asked to "set up a
database", "create a Prisma Postgres project", "get a connection string", "connect my app to Prisma Postgres", or "provision a database".
license: MIT
metadata:
  author: prisma
  version: "1.1.0"
---

# Prisma Postgres Setup

Procedural skill that guides you through provisioning a new Prisma Postgres database via the Management API and connecting it to a local
project.

## When to Apply

Use this skill when:

- Setting up a new Prisma Postgres database for a project
- Creating a Prisma Postgres project and connecting it locally
- Obtaining a connection string for Prisma Postgres
- Provisioning a database via the Management API (not the Console UI)

Do **not** use this skill when:

- Setting up CI/CD preview databases — use `prisma-postgres-cicd`
- Building multi-tenant database provisioning into an app — use `prisma-postgres-integrator`
- Working with a database that already exists and is connected (schema/migration tasks are standard Prisma CLI)

## Prerequisites

- Node.js 18+
- A Prisma Postgres workspace (create one at https://console.prisma.io if needed)
- A workspace service token (see `references/auth.md`)

## UX Guidelines

When presenting choices to the user (region selection, project deletion, etc.), **use your platform's interactive selection mechanism** (e.g.,
`ask` tool in Claude Code, structured prompts in other agents). Do not print static tables and ask the user to type a value — present
selectable options so the user can pick with minimal effort.

## Workflow

Follow these steps in order. Each step includes the API call to make and how to handle the response.

### Step 1: Authenticate

You need a service token. Try these methods in order:

**1a. Token in the user's prompt**

Check if the user included a service token in their initial message (e.g., "Set up Prisma Postgres with token eyJ..."). If so, use it
**exactly as provided** — do not truncate, re-encode, or round-trip it through a file. Store it in a shell variable for subsequent calls.

**1b. Token in the environment**

Check for `PRISMA_SERVICE_TOKEN` in the environment or `.env` file.

**1c. Ask the user to create one**

If no token is available, instruct the user:

> Create a service token in Prisma Console → Workspace Settings → Service Tokens.
> Copy the token and paste it here.

Read `references/auth.md` for details on service token creation.

Once you have a token, store it in a shell variable (`PRISMA_SERVICE_TOKEN`) and use it for all subsequent API calls.

### Step 2: List available regions

Fetch the list of available Prisma Postgres regions to let the user choose where to deploy.

```bash
curl -s -H "Authorization: Bearer $PRISMA_SERVICE_TOKEN" \
  https://api.prisma.io/v1/regions/postgres
```

The response contains an array of regions with `id`, `name`, and `status`. Only present regions where `status` is `available`.

Present the regions as an interactive menu — let the user pick from options rather than typing a region ID manually.

Read `references/endpoints.md` for the full response shape.

Step 3: Create a project with a database

```bash
curl -s -X POST https://api.prisma.io/v1/projects \
  -H "Authorization: Bearer $PRISMA_SERVICE_TOKEN" \
```
```

```
  -H "Content-Type: application/json" \
  -d '{
    "name": "<project-name>",
    "region": "<region-id>",
    "createDatabase": true
  }'
```

Use the current directory name as the project name by default.

The response is wrapped in `{ "data": { ... } }`. Extract:

- `data.id` — the project ID (prefixed with `proj_`)
- `data.database.id` — the database ID (prefixed with `db_`)
- `data.database.connections[0].endpoints.direct.connectionString` — the direct PostgreSQL connection string

Use the **direct** connection string (`endpoints.direct.connectionString`). Do not use the pooled or accelerate endpoints — those are for legacy Accelerate setups and not needed for new projects.

If the response status is `provisioning`, wait a few seconds and poll `GET /v1/databases/<database-id>` until `status` is `ready`.

If creation fails due to a database limit, list the user's existing projects and present them as an interactive menu for deletion. After the user picks one, delete it and retry.

Read `references/endpoints.md` for the full request/response shapes.

Step 4: Create a named connection (optional)

If you need a dedicated connection (e.g., per-developer or per-environment), create one:

```bash
curl -s -X POST https://api.prisma.io/v1/databases/<database-id>/connections \
  -H "Authorization: Bearer $PRISMA_SERVICE_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{ "name": "dev" }'
```

Extract the direct connection string from `data.endpoints.direct.connectionString`.

Step 5: Configure the local project

1. Install dependencies:

```bash
npm install prisma @prisma/client @prisma/adapter-pg pg dotenv
```

All five packages are required:
- `prisma` — CLI for migrations, schema push, client generation
- `@prisma/client` — the generated query client
- `@prisma/adapter-pg` — Prisma 7 driver adapter for direct PostgreSQL connections
- `pg` — Node.js PostgreSQL driver (used by the adapter)
- `dotenv` — loads `.env` variables for `prisma.config.ts`

2. Write the direct connection string to `.env`. **Append** to the file if it already exists — do not overwrite existing entries:

```
DATABASE_URL="<direct-connection-string>"
```

3. Verify `.gitignore` includes `.env`. Create `.gitignore` if it does not exist. Warn the user if `.env` is not gitignored.

4. Ensure `package.json` has `"type": "module"` set (Prisma 7 generates ESM output).

5. If `prisma/schema.prisma` does not exist, run `npx prisma init` to scaffold the project. This creates both `prisma/schema.prisma` and `prisma.config.ts`.

6. Ensure `schema.prisma` has the `postgresql` provider and **no** `url` or `directUrl` in the datasource block (Prisma 7 manages connection URLs in `prisma.config.ts`, not in the schema):

```prisma
datasource db {
  provider = "postgresql"
}
```

7. Ensure `prisma.config.ts` loads the connection URL from the environment:

```typescript
import path from 'node:path'
import { defineConfig } from 'prisma/config'
import 'dotenv/config'

export default defineConfig({
  earlyAccess: true,
  schema: path.join(import.meta.dirname, 'prisma', 'schema.prisma'),
  datasource: {
    url: process.env.DATABASE_URL!,
  },
})
```

Important Prisma 7 notes:

- Connection URLs go in `prisma.config.ts`, never in `schema.prisma`
- The provider in `schema.prisma` must be `"postgresql"` (not `"prismaPostgres"`)
- `dotenv/config` must be imported in `prisma.config.ts` to load `.env` variables

Step 6: Define schema and push

If the schema already has models, skip to pushing. Otherwise, **present these options as an interactive menu**:

1. **"I'll define my schema manually"** — Tell the user to edit `prisma/schema.prisma` and come back when ready. Wait for them before proceeding.
2. **"Give me a starter schema"** — Add a Blog starter schema (User, Post, Comment with relations) to `prisma/schema.prisma`. Show the user what was added and ask if they want to adjust it before pushing.
3. **"I'll describe what I need"** — Ask the user to describe their data model in natural language (e.g., "I'm building a task manager with projects, tasks, and team members"). Generate a schema from the description, show it, and ask for confirmation before pushing.

Once the schema has models and the user is ready, create a migration and generate the client:

```bash
npx prisma migrate dev --name init
```

This creates migration files in `prisma/migrations/` **and** generates the client in one step. Migration history is essential for CI/CD workflows (`prisma migrate deploy`) and production deployments.

Only use `npx prisma db push` if the user explicitly asks for prototyping-only mode (no migration history). In that case, follow it with `npx prisma generate`.

Step 7: Verify the connection

After generating the client, create and run a quick verification script to confirm everything works end-to-end. This is **critical** — do not skip this step.

Create a file named `test-connection.ts`:

```typescript
import 'dotenv/config'
import pg from 'pg'
import { PrismaPg } from '@prisma/adapter-pg'
import { PrismaClient } from './generated/prisma/client.js'

const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL })
const adapter = new PrismaPg(pool)
const prisma = new PrismaClient({ adapter })

const result = await prisma.$queryRawUnsafe('SELECT 1 as connected')
console.log('Connected to Prisma Postgres:', result)

await prisma.$disconnect()
await pool.end()
```

Run it:

```bash
npx tsx test-connection.ts
```

Prisma 7 client instantiation rules:
- Import from `./generated/prisma/client.js` (not `./generated/prisma`)
- Create a `pg.Pool` with the `DATABASE_URL` connection string
- Wrap it in a `PrismaPg` adapter
- Pass `{ adapter }` to the `PrismaClient` constructor
- Do **not** use `datasourceUrl` — that option does not exist in Prisma 7
- Do **not** use `new PrismaClient()` with no arguments — it will throw

After verification succeeds, delete `test-connection.ts`.

Then share links for the user to explore their database:

- **Prisma Studio (CLI):** `npx prisma studio` — opens a visual data browser locally
- **Console:** `https://console.prisma.io/<workspaceId>/<projectId>/<databaseId>/dashboard` — strip the prefixes (`wksp_`, `proj_`, `db_`) from the IDs returned in Step 3 to build this URL

Read `references/prisma7-client.md` for the full client instantiation reference.

Error Handling

Read `references/api-basics.md` for the full error reference. Key self-correction patterns:

HTTP Status	Error Code	Action
401	`authentication-failed`	Service token is invalid or expired. Ask the user to create a new one in Console → Workspace Settings → Service Tokens.
404	`resource-not-found`	Check that the resource ID includes the correct prefix (`proj_`, `db_`, `con_`).
422	`validation-error`	Check request body against the endpoint schema. Common: missing `name`, invalid `region`.
429	`rate-limit-exceeded`	Back off and retry after a few seconds.

Reference Files

Detailed API and usage information is in:

```
references/auth.md             — Service token creation and usage
```

```
references/api-basics.md       — Base URL, envelope, IDs, errors, pagination
references/endpoints.md        — Endpoint details for projects, databases, connections, regions
references/prisma7-client.md   — Prisma 7 client instantiation and usage patterns
```

57. .agents\skills\prisma-postgres\references\console-and-connections.md

````
# console-and-connections

Use Prisma Console workflows for project visibility, data inspection, and connection setup.

## Priority

HIGH

## Why It Matters

Many Prisma Postgres tasks are quickest in the Console: viewing Studio data, checking metrics, and retrieving connection details. This avoids unnecessary API or CLI work for simple operational tasks.

## Console workflow

1. Open `https://console.prisma.io`.
2. Select workspace and project.
3. Use dashboard metrics for usage and billing visibility.
4. Open the **Studio** tab in the sidebar to inspect and edit data.

## Local Studio

You can also inspect data locally:

```bash
npx prisma studio
```

## Linking an existing project

If the Prisma Postgres database already exists, link the local project instead of provisioning a new one:

```bash
prisma postgres link
```

For CI or non-interactive usage:

```bash
prisma postgres link --api-key "<your-api-key>" --database "db_..."
```

This command updates or creates `.env` with `DATABASE_URL`. If the project is already linked, use `--force` to re-link. After linking, run `prisma generate`, then `prisma migrate dev` if you need to apply the schema.

## Connection setup

For direct PostgreSQL tools and drivers:

- Generate/copy direct connection credentials from the project connection UI.
- Use the resulting PostgreSQL URL as `DATABASE_URL` for `pg` and `@prisma/adapter-pg`.
- For Prisma Postgres direct TCP, include `sslmode=require`.

Typical direct TCP format:

```env
DATABASE_URL="postgres://identifier:key@db.prisma.io:5432/postgres?sslmode=require"
```

Management API connection responses expose both `endpoints.direct` (`db.prisma.io:5432`) and `endpoints.pooled` (`pooled.db.prisma.io:5432`); prefer those fields over the deprecated flat `connectionString`. Connection secrets are shown once at creation (one-time view); store them immediately.

## Adapter choices

- Standard Node.js apps: prefer `@prisma/adapter-pg` with the direct TCP URL above.
- Edge/serverless runtimes: use `@prisma/adapter-ppg` with `@prisma/ppg` only when you specifically need the Prisma Postgres serverless driver.

## References

- [Prisma Postgres overview](https://www.prisma.io/docs/postgres/introduction/overview)
- [Viewing data](https://www.prisma.io/docs/postgres/integrations/viewing-data)
- [Direct connections](https://www.prisma.io/docs/postgres/database/direct-connections)
````

58. .agents\skills\prisma-postgres\references\create-db-cli.md

````
# create-db-cli

Use `create-db` for instant Prisma Postgres provisioning from the terminal.

## Priority

CRITICAL

## Why It Matters

`create-db` is the fastest way to get a working Prisma Postgres instance for development, demos, and CI previews. It can also emit machine-readable output and write env variables directly.

## Commands

```bash
npx create-db@latest
npx create-db@latest create [options]
npx create-db@latest regions
```

Aliases:

```bash
npx create-pg@latest
npx create-postgres@latest
```

## Command discovery (`--help`)

Always use `--help` first when integrating CLI commands:

```bash
npx create-db@latest --help
npx create-db@latest create --help
npx create-db@latest regions --help
```

Top-level commands currently exposed:

- `create` (default) to provision a database
- `regions` to list available regions

## `create` options

| Flag | Shorthand | Description |
|---|---|---|
| `--region [string]` | `-r` | Region choice: `ap-southeast-1`, `ap-northeast-1`, `eu-central-1`, `eu-west-3`, `us-east-1`, `us-west-1` |
| `--interactive [boolean]` | `-i` | Open region selector |
| `--json [boolean]` | `-j` | Output machine-readable JSON |
| `--env [string]` | `-e` | Write `DATABASE_URL` and `CLAIM_URL` into a target `.env` |
| `--ttl [string]` | `-t` | Auto-delete after a TTL like `30m` or `1h-24h` |
| `--copy [boolean]` | `-c` | Copy the connection string to the clipboard |
| `--quiet [boolean]` | `-q` | Only print the connection string |
| `--open [boolean]` | `-o` | Open the claim URL in your browser |

## Lifecycle and claim flow

- Databases are temporary by default.
- Unclaimed databases are auto-deleted after ~24 hours.
- Claim the database using the URL shown in command output to keep it permanently.

## Programmatic usage (library API)

You can also use `create-db` programmatically in Node.js/Bun instead of shelling out to the CLI.

Install:

```bash
npm install create-db
# or
bun add create-db
```

Create a database:

```ts
import { create, isDatabaseSuccess, isDatabaseError } from "create-db";

const result = await create({
  region: "us-east-1",
  userAgent: "my-app/1.0.0",
});

if (isDatabaseSuccess(result)) {
  console.log(result.connectionString);
  console.log(result.claimUrl);
  console.log(result.deletionDate);
}
````
````

````
if (isDatabaseError(result)) {
  console.error(result.error, result.message);
}
```

List regions programmatically:

```ts
import { regions } from "create-db";

const available = await regions();
console.log(available);
```

Programmatic `create()` defaults to `us-east-1` if no region is passed.

## Common patterns

```bash
# quick database
npx create-db@latest

# region-specific database
npx create-db@latest --region eu-central-1

# interactive region selection
npx create-db@latest --interactive

# write env vars for app bootstrap
npx create-db@latest --env .env

# auto-delete sooner
npx create-db@latest --ttl 2h

# copy connection string to clipboard
npx create-db@latest --copy

# print only the connection string
npx create-db@latest --quiet

# CI-friendly output
npx create-db@latest --json
```

## References

- [npx create-db docs](https://www.prisma.io/docs/postgres/introduction/npx-create-db)
````
````

59. .agents\skills\prisma-postgres\references\management-api-sdk.md

````
# management-api-sdk

Use `@prisma/management-api-sdk` for typed API integration with optional OAuth and token refresh.

## Priority

HIGH

## Why It Matters

The SDK provides typed endpoint methods and removes boilerplate around auth and refresh handling, which reduces errors in production
provisioning flows.

## Install

```bash
npm install @prisma/management-api-sdk
```

## Simple client (existing token)

```typescript
import { createManagementApiClient } from '@prisma/management-api-sdk'

const client = createManagementApiClient({ token: process.env.PRISMA_SERVICE_TOKEN! })
const { data: workspaces } = await client.GET('/v1/workspaces')
```

## Full SDK (OAuth + refresh)

```typescript
import { createManagementApiSdk, type TokenStorage } from '@prisma/management-api-sdk'

const tokenStorage: TokenStorage = {
  async getTokens() { return null },
  async setTokens(tokens) {},
  async clearTokens() {},
}

const api = createManagementApiSdk({
  clientId: process.env.PRISMA_CLIENT_ID!,
  redirectUri: 'https://your-app.com/auth/callback',
  tokenStorage,
})
```

## OAuth SDK flow

1. Call `getLoginUrl()` and persist `state` + `verifier`.
2. Redirect user to login URL.
3. Handle callback with `handleCallback()`.
4. Use `api.client` for typed endpoint calls.
5. Call `logout()` when needed.

## References

- [Management API SDK docs](https://www.prisma.io/docs/postgres/introduction/management-api-sdk)
````

60. .agents\skills\prisma-postgres\references\management-api.md

````
# management-api

Use Prisma Management API for programmatic provisioning and workspace/project/database management.

## Priority

CRITICAL

## Why It Matters

When you need backend automation, multi-tenant onboarding flows, or controlled resource provisioning, the Management API is the source of truth and is more reliable than interactive workflows.

## Base URL

```text
https://api.prisma.io/v1
```

## API exploration

- OpenAPI docs: `https://api.prisma.io/v1/doc`
- Swagger Editor: `https://api.prisma.io/v1/swagger-editor`

## Authentication methods

- Service token: best for server-to-server operations in your own workspace
- OAuth 2.0: best for acting on behalf of users across workspaces

## Service token flow

1. Create token in Prisma Console workspace settings.
2. Send token as Bearer auth:

```text
Authorization: Bearer $TOKEN
```

## OAuth flow summary

1. Redirect user to `https://auth.prisma.io/authorize` with `client_id`, `redirect_uri`, `response_type=code`, and scopes.
2. Receive `code` on callback.
3. Exchange code at `https://auth.prisma.io/token`.
4. Use returned access token in Management API requests.

## Resource model

Workspace -> Project -> Branch -> Database. Branches are a first-class resource: databases attach to a Branch, and branch-scoped env/databases are how preview isolation works.

## Common endpoints

- `GET /workspaces`
- `GET /projects`
- `POST /projects`
- `GET`/`POST /projects/:projectId/branches`
- `GET`/`PATCH`/`DELETE /branches/:branchId`
- Databases at both `/databases` (flat) and `/projects/:projectId/databases`
- Connection responses return structured `endpoints.direct` and `endpoints.pooled` objects; the flat top-level `connectionString` is deprecated in favor of `endpoints.direct.connectionString`/`endpoints.pooled.connectionString`
- Connection secrets are one-time view: revealed at creation, never re-readable from later GETs

## Notes

- Management API responses may include direct connection credentials for databases.
- Build PostgreSQL `DATABASE_URL` from direct connection values when needed.

## References

- [Management API docs](https://www.prisma.io/docs/postgres/introduction/management-api)
- [OpenAPI docs](https://api.prisma.io/v1/doc)
- [Swagger Editor](https://api.prisma.io/v1/swagger-editor)
````

61. .agents\skills\prisma-postgres\SKILL.md

````
---
name: prisma-postgres
description: Prisma Postgres setup and operations guidance across Console, create-db CLI, Management API, and Management API SDK. Use when
creating Prisma Postgres databases, working in Prisma Console, provisioning with create-db/create-pg/create-postgres, or integrating
programmatic provisioning with service tokens or OAuth.
license: MIT
metadata:
  author: prisma
  version: "7.7.0"
---

# Prisma Postgres

Guidance for creating, managing, and integrating Prisma Postgres across interactive and programmatic workflows.

## When to Apply

Reference this skill when:
- Setting up Prisma Postgres from Prisma Console
- Provisioning instant temporary databases with `create-db`
- Linking an existing local project with `prisma postgres link`
- Managing Prisma Postgres resources via Management API
- Using `@prisma/management-api-sdk` in TypeScript/JavaScript
- Handling claim URLs, connection strings, regions, and auth flows

## Rule Categories by Priority

| Priority | Category | Impact | Prefix |
|----------|----------|--------|--------|
| 1 | CLI Provisioning | CRITICAL | `create-db-cli` |
| 2 | Management API | CRITICAL | `management-api` |
| 3 | Management API SDK | HIGH | `management-api-sdk` |
| 4 | Console and Connections | HIGH | `console-and-connections` |

## Quick Reference

- `create-db-cli` - instant databases and current CLI flags (`--ttl`, `--copy`, `--quiet`, `--open`)
- `management-api` - service token and OAuth API workflows
- `management-api-sdk` - typed SDK usage with token storage
- `console-and-connections` - Console operations, `prisma postgres link`, direct TCP connections, and serverless-driver choices

## Core Workflows

### 1. Console-first workflow

Use Prisma Console for manual setup and operations:

- Open `https://console.prisma.io`
- Create/select workspace and project
- Use Studio in the project sidebar to view/edit data
- Retrieve direct connection details from the project UI

### 2. Quick provisioning with create-db

Use `create-db` when you need a database immediately:

```bash
npx create-db@latest
```

Aliases:

```bash
npx create-pg@latest
npx create-postgres@latest
```

For app integrations, you can also use the programmatic API (`create()` / `regions()`) from the `create-db` npm package.

Temporary databases auto-delete after ~24 hours unless claimed.

### 2b. Persistent databases with the Platform CLI

For databases that belong to a Project (not throwaway `create-db` databases), use `@prisma/cli`:

```bash
npx -y @prisma/cli@latest database create main --branch main
npx -y @prisma/cli@latest database list --json
npx -y @prisma/cli@latest database connection create db_123
npx -y @prisma/cli@latest database usage db_123
npx -y @prisma/cli@latest database backup list db_123
```

`database create` and `database connection create` print a one-time connection URL; store it immediately. Destructive commands (`remove`,
`restore`) require exact `--confirm <id>`.

### 3. Link an existing local project

Use `prisma postgres link` when the database already exists and you want to wire a local project to it:
````

```bash
prisma postgres link
```

For CI or other non-interactive environments:

```bash
prisma postgres link --api-key "<your-api-key>" --database "db_..."
```

This flow updates your local `.env` with `DATABASE_URL`, then you can run `prisma generate` and `prisma migrate dev`.

4. Programmatic provisioning with Management API

Use API endpoints on:

```text
https://api.prisma.io/v1
```

Explore the schema and endpoints using:

- OpenAPI docs: `https://api.prisma.io/v1/doc`
- Swagger Editor: `https://api.prisma.io/v1/swagger-editor`

Auth options:

- Service token (workspace server-to-server)
- OAuth 2.0 (act on behalf of users)

5. Type-safe integration with Management API SDK

Install and use:

```bash
npm install @prisma/management-api-sdk
```

Use `createManagementApiClient` for existing tokens, or `createManagementApiSdk` for OAuth + token refresh.

Rule Files

Detailed guidance lives in:

```
references/console-and-connections.md
references/create-db-cli.md
references/management-api.md
references/management-api-sdk.md
```

How to Use

Start with `references/create-db-cli.md` for fast setup, then switch to `references/management-api.md` or `references/management-api-sdk.md` when you need programmatic provisioning.

62. .agents\skills\prisma-upgrade-v7\references\accelerate-users.md

````
# Prisma Accelerate Users

Special migration instructions for users of Prisma Accelerate or Prisma Postgres with `prisma://` or `prisma+postgres://` URLs.

## Important

**Do NOT pass Accelerate URLs to driver adapters.**

Driver adapters (like `PrismaPg`) expect direct database connection strings. They will fail with `prisma://` or `prisma+postgres://` URLs.

## Correct v7 Setup for Accelerate

### 1. Keep your Accelerate URL

```env
# .env
DATABASE_URL="prisma://accelerate.prisma-data.net/?api_key=..."
# or
DATABASE_URL="prisma+postgres://accelerate.prisma-data.net/..."
```

### 2. Install Accelerate extension

```bash
npm install @prisma/extension-accelerate
```

### 3. Configure prisma.config.ts

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: {
    url: env('DATABASE_URL'),  // Accelerate URL works here
  },
})
```

### 4. Instantiate client with accelerateUrl

```typescript
import { PrismaClient } from '../generated/client'
import { withAccelerate } from '@prisma/extension-accelerate'

// Use accelerateUrl instead of adapter
export const prisma = new PrismaClient({
  accelerateUrl: process.env.DATABASE_URL,
}).$extends(withAccelerate())
```

## What NOT to Do

```typescript
// ❌ WRONG - Don't use adapter with Accelerate URL
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL  // This will fail with prisma://
})
```

## Migrations with Accelerate

For migrations, you may need a direct database connection:

### Option 1: Use Accelerate URL for everything

Accelerate URLs work with Prisma CLI commands:

```bash
# Works with Accelerate URL
prisma migrate deploy
prisma db push
```

### Option 2: Use direct URL for migrations

```env
DATABASE_URL="prisma+postgres://..."  # For app
DIRECT_DATABASE_URL="postgresql://..."  # For migrations
```

```typescript
// prisma.config.ts
export default defineConfig({
  datasource: {
    url: env('DIRECT_DATABASE_URL'),  // Direct URL for CLI
````
````

```
  },
})
```

Prisma Postgres (Cloud)

If using Prisma Postgres cloud database:

Same approach

```typescript
import { PrismaClient } from '../generated/client'
import { withAccelerate } from '@prisma/extension-accelerate'

export const prisma = new PrismaClient({
  accelerateUrl: process.env.DATABASE_URL,  // prisma+postgres:// URL
}).$extends(withAccelerate())
```

Switching Away from Accelerate

If you later switch to direct TCP connection:

```typescript
// Change from accelerateUrl to adapter
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL  // Direct postgres:// URL
})

export const prisma = new PrismaClient({ adapter })
```

Caching with Accelerate

The extension enables caching:

```typescript
const users = await prisma.user.findMany({
  cacheStrategy: {
    ttl: 60,  // Cache for 60 seconds
    swr: 120, // Stale-while-revalidate for 120 seconds
  },
})
```

Edge Runtime

Accelerate works great in edge runtimes:

```typescript
// Works in Vercel Edge, Cloudflare Workers, etc.
import { PrismaClient } from '../generated/client'
import { withAccelerate } from '@prisma/extension-accelerate'

export const prisma = new PrismaClient({
  accelerateUrl: process.env.DATABASE_URL,
}).$extends(withAccelerate())
```

63. .agents\skills\prisma-upgrade-v7\references\driver-adapters.md

````
# Driver Adapters

Prisma v7 requires driver adapters for SQL database connections. This is the standard SQL execution path in current Prisma releases.

MongoDB should not follow this path. There is no published MongoDB `@prisma/adapter-*` package, and MongoDB projects should remain on the latest Prisma 6.x release instead of trying to fit into the Prisma 7 SQL adapter model.

## Why Driver Adapters?

- No native engine binary in the Prisma Client SQL path
- Smaller bundle size
- Better serverless/edge compatibility
- Uses native Node.js database drivers
- More control over connection pooling

## Available Adapters

| Database | Adapter Package | Underlying Driver |
|----------|-----------------|-------------------|
| PostgreSQL | `@prisma/adapter-pg` | `pg` |
| MySQL / MariaDB | `@prisma/adapter-mariadb` | `mariadb` |
| SQLite | `@prisma/adapter-better-sqlite3` | `better-sqlite3` |
| Prisma Postgres (Node.js) | `@prisma/adapter-pg` | `pg` |
| Prisma Postgres (edge/serverless) | `@prisma/adapter-ppg` | `@prisma/ppg` |
| SQL Server | `@prisma/adapter-mssql` | `mssql` |
| Neon | `@prisma/adapter-neon` | `@neondatabase/serverless` |
| PlanetScale | `@prisma/adapter-planetscale` | `@planetscale/database` |
| Turso/libSQL | `@prisma/adapter-libsql` | `@libsql/client` |
| D1 (Cloudflare) | `@prisma/adapter-d1` | Cloudflare D1 |

## Installation

### PostgreSQL

```bash
npm install @prisma/adapter-pg
```

### MySQL

```bash
npm install @prisma/adapter-mariadb mariadb
```

### SQLite

```bash
npm install @prisma/adapter-better-sqlite3
```

### Prisma Postgres

```bash
npm install @prisma/adapter-pg pg
```

### SQL Server

```bash
npm install @prisma/adapter-mssql mssql
```

## Configuration

### PostgreSQL

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
})

const prisma = new PrismaClient({ adapter })
```

### MySQL

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaMariaDb } from '@prisma/adapter-mariadb'

const adapter = new PrismaMariaDb({
  host: 'localhost',
  port: 3306,
  connectionLimit: 5,
  user: process.env.MYSQL_USER,
  password: process.env.MYSQL_PASSWORD,
  database: process.env.MYSQL_DATABASE,
````
````

```
})

const prisma = new PrismaClient({ adapter })
```

SQLite

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaBetterSqlite3 } from '@prisma/adapter-better-sqlite3'

const adapter = new PrismaBetterSqlite3({
  url: process.env.DATABASE_URL || 'file:./dev.db'
})

const prisma = new PrismaClient({ adapter })
```

Neon (Serverless PostgreSQL)

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaNeon } from '@prisma/adapter-neon'

const adapter = new PrismaNeon({
  connectionString: process.env.DATABASE_URL
})

const prisma = new PrismaClient({ adapter })
```

Prisma Postgres

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL,
})

const prisma = new PrismaClient({ adapter })
```

Prisma Postgres serverless driver

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaPostgresAdapter } from '@prisma/adapter-ppg'

const prisma = new PrismaClient({
  adapter: new PrismaPostgresAdapter({
    connectionString: process.env.PRISMA_DIRECT_TCP_URL,
  }),
})
```

SQL Server

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaMssql } from '@prisma/adapter-mssql'

const adapter = new PrismaMssql({
  server: 'localhost',
  port: 1433,
  database: 'mydb',
  user: process.env.SQLSERVER_USER,
  password: process.env.SQLSERVER_PASSWORD,
  options: {
    encrypt: true,
    trustServerCertificate: true,
  },
})

const prisma = new PrismaClient({ adapter })
```

Connection Pool Configuration

Driver adapters use the underlying driver's pool settings, which differ from v6 defaults.

PostgreSQL with custom pool

```typescript
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL,
  // Pool configuration
  max: 10,                    // Maximum connections
  idleTimeoutMillis: 30000,   // Close idle connections after 30s
  connectionTimeoutMillis: 5000, // Connection timeout (v6 default was 5s)
```

````
})
```

### Matching v6 behavior

```typescript
const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL,
  connectionTimeoutMillis: 5000,  // v6 used 5 second timeout
})
```

## SSL Configuration

### Accept self-signed certificates

```typescript
const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    rejectUnauthorized: false  // Accept self-signed certs
  }
})
```

### Proper SSL configuration

```typescript
const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL,
  ssl: {
    ca: fs.readFileSync('/path/to/ca-cert.pem'),
    rejectUnauthorized: true
  }
})
```

## Migration from v6

### Before (v6)

```typescript
import { PrismaClient } from '@prisma/client'

const prisma = new PrismaClient({
  datasources: {
    db: { url: process.env.DATABASE_URL }
  }
})
```

### After (v7)

```typescript
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
})

const prisma = new PrismaClient({ adapter })
```

## Singleton Pattern

```typescript
// lib/prisma.ts
import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined
}

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL!
})

export const prisma = globalForPrisma.prisma ?? new PrismaClient({ adapter })

if (process.env.NODE_ENV !== 'production') {
  globalForPrisma.prisma = prisma
}
```
````
````

64. .agents\skills\prisma-upgrade-v7\references\env-variables.md

````
# Environment Variables

Prisma v7 no longer automatically loads environment variables. You must load them explicitly.

## The Change

### v6 Behavior

Prisma CLI automatically loaded `.env` files.

### v7 Behavior

You must manually load environment variables using `dotenv` or similar.

## Setup

### 1. Install dotenv

```bash
npm install dotenv
```

### 2. Import in prisma.config.ts

```typescript
import 'dotenv/config'  // Must be first import
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## Bun Users

Bun automatically loads `.env` files. No additional setup needed:

```typescript
// prisma.config.ts (Bun)
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## Multiple .env Files

### Using dotenv-cli

```bash
npm install -D dotenv-cli
```

```json
{
  "scripts": {
    "db:migrate": "dotenv -e .env.local -- prisma migrate dev",
    "db:push": "dotenv -e .env.development -- prisma db push"
  }
}
```

### Using dotenv with path

```typescript
// prisma.config.ts
import { config } from 'dotenv'
import path from 'path'

// Load specific .env file
config({ path: path.join(__dirname, '.env.local') })

import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

## Application Code

For your application, load env vars at startup:
````

Entry point

```typescript
// index.ts
import 'dotenv/config'

import { PrismaClient } from '../generated/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL!
})

const prisma = new PrismaClient({ adapter })
```

Or use dotenv explicitly

```typescript
import { config } from 'dotenv'
config()

// Now process.env.DATABASE_URL is available
```

Removed Environment Variables

These Prisma-specific env vars are removed in v7:

Removed Variable	Alternative
`PRISMA_CLI_QUERY_ENGINE_TYPE`	Not needed (no engines)
`PRISMA_CLIENT_ENGINE_TYPE`	Not needed (no engines)
`PRISMA_QUERY_ENGINE_BINARY`	Not needed
`PRISMA_QUERY_ENGINE_LIBRARY`	Not needed
`PRISMA_GENERATE_SKIP_AUTOINSTALL`	Not needed
`PRISMA_SKIP_POSTINSTALL_GENERATE`	Not needed
`PRISMA_GENERATE_IN_POSTINSTALL`	Not needed
`PRISMA_GENERATE_DATAPROXY`	Migrate to `prisma-client` with driver adapters
`PRISMA_GENERATE_NO_ENGINE`	Migrate to `prisma-client` with driver adapters
`PRISMA_CLIENT_NO_RETRY`	Configure on adapter
`PRISMA_MIGRATE_SKIP_GENERATE`	Not needed (auto-generate removed)
`PRISMA_MIGRATE_SKIP_SEED`	Not needed (auto-seed removed)

TypeScript env() Helper

The `env()` function from `prisma/config` provides type safety:

```typescript
import { env } from 'prisma/config'

// Type-safe environment variable access
const url = env('DATABASE_URL')  // string
```

Note: This only works within `prisma.config.ts`, not in your application code.

CI/CD Considerations

Ensure environment variables are set in your CI environment:

```yaml
# GitHub Actions
env:
  DATABASE_URL: ${{ secrets.DATABASE_URL }}

steps:
  - run: npx prisma migrate deploy
```

No need for dotenv in CI if variables are set directly.

65. .agents\skills\prisma-upgrade-v7\references\esm-support.md

````
# ESM and CommonJS Support

Prisma ORM v7 is ESM-first, but the `prisma-client` generator can target either ESM or CommonJS. Use ESM by default, and opt into CommonJS
with `moduleFormat = "cjs"` if your project still needs it.

## ESM Projects

Add `"type": "module"` to `package.json` and use an ESM-compatible `tsconfig.json`:

```json
{
  "type": "module",
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js"
  }
}
```

```json
{
  "compilerOptions": {
    "module": "ESNext",
    "moduleResolution": "bundler",
    "target": "ES2023",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "outDir": "dist"
  },
  "include": ["src/**/*", "prisma/**/*"]
}
```

## CommonJS Projects

If the rest of your app is still CommonJS, keep that setup and make the generated Prisma Client CommonJS too:

```json
{
  "compilerOptions": {
    "module": "CommonJS",
    "moduleResolution": "node",
    "target": "ES2022",
    "esModuleInterop": true
  }
}
```

```prisma
generator client {
  provider     = "prisma-client"
  output       = "../generated/prisma"
  moduleFormat = "cjs"
}
```

## Generator Fields That Matter

- `moduleFormat`: `esm` or `cjs`
- `runtime`: `nodejs`, `bun`, `deno`, `workerd`, `vercel-edge`, `react-native`
- `generatedFileExtension`: `ts`, `mts`, or `cts`
- `importFileExtension`: `ts`, `mts`, `cts`, `js`, `mjs`, `cjs`, or empty

Example:

```prisma
generator client {
  provider               = "prisma-client"
  output                 = "../generated/prisma"
  runtime                = "nodejs"
  moduleFormat           = "esm"
  generatedFileExtension = "ts"
  importFileExtension    = "ts"
}
```

## Import Paths

### Server Code

```typescript
import { PrismaClient } from '../generated/prisma/client'
```

### Browser-Safe Types

```typescript
import { Prisma } from '../generated/prisma/browser'
import { Role } from '../generated/prisma/enums'
````
````

````
import type { UserModel } from '../generated/prisma/models/User'
```

## File Extensions

With `moduleResolution: "Node16"` or `"NodeNext"`, use `.js`/`.mjs`/`.cjs` extensions that match your emitted files.

With `moduleResolution: "bundler"`, bare relative imports are usually fine.

## Minimum Versions

| Requirement | Minimum Version |
|-------------|-----------------|
| Node.js | 20.19.0 |
| TypeScript | 5.4.0 |

## Framework Considerations

### Next.js

Next.js works well with the default ESM output. If you need generated types in client components, import them from `browser`, `models`, or `enums`, not from `client`.

### Bun

Bun loads `.env` files automatically, so ESM plus `env()` is the smoothest default. You can still choose `moduleFormat = "cjs"` if the rest of your project requires it.

## Troubleshooting

### "ERR_REQUIRE_ESM"

Your generated client is ESM, but your app is requiring it as CommonJS. Either switch the project to ESM or set `moduleFormat = "cjs"` and regenerate.

### "Cannot use import statement outside a module"

Your app is still being executed as CommonJS. Add `"type": "module"` or use `moduleFormat = "cjs"` instead.

### TypeScript compilation errors

Ensure `module`, `moduleResolution`, and your generator's `moduleFormat` agree with one another.

## 66. .agents\skills\prisma-upgrade-v7\references\prisma-config.md

````
Prisma Config

Prisma v7 introduces `prisma.config.ts` as the central configuration file for the Prisma CLI.

Location

Place `prisma.config.ts` at your project root (next to `package.json`).

Basic Configuration

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
  },
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

Configuration Options

schema

Path to your Prisma schema file:

```typescript
schema: 'prisma/schema.prisma'
```

datasource.url

Database connection URL:

```typescript
datasource: {
  url: env('DATABASE_URL'),
}
```

datasource.directUrl

Direct connection URL (bypassing connection pooler):

```typescript
datasource: {
  url: env('DATABASE_URL'),
  directUrl: env('DIRECT_DATABASE_URL'),
}
```

datasource.shadowDatabaseUrl

Shadow database for migrations:

```typescript
datasource: {
  url: env('DATABASE_URL'),
  shadowDatabaseUrl: env('SHADOW_DATABASE_URL'),
}
```

migrations.path

Directory for migration files:

```typescript
migrations: {
  path: 'prisma/migrations',
}
```

migrations.seed

Seed command for `prisma db seed`:

```typescript
migrations: {
  path: 'prisma/migrations',
  seed: 'tsx prisma/seed.ts',
}
```

Full Example
````

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  // Schema location
  schema: 'prisma/schema.prisma',

  // Migration configuration
  migrations: {
    path: 'prisma/migrations',
    seed: 'tsx prisma/seed.ts',
  },

  // Database connection
  datasource: {
    url: env('DATABASE_URL'),
    directUrl: env('DIRECT_DATABASE_URL'),
    shadowDatabaseUrl: env('SHADOW_DATABASE_URL'),
  },
})
```

Environment Variables

The env() helper

Use `env()` to reference environment variables:

```typescript
import { env } from 'prisma/config'

datasource: {
  url: env('DATABASE_URL'),
}
```

This provides type safety but does NOT load .env files automatically.

Loading .env files

Install and import dotenv:

```bash
npm install dotenv
```

```typescript
import 'dotenv/config'  // Must be first import
import { defineConfig, env } from 'prisma/config'
```

Migrating from v6

Before (v6) - schema.prisma

```prisma
datasource db {
  provider  = "postgresql"
  url       = env("DATABASE_URL")
  directUrl = env("DIRECT_URL")
}
```

After (v7) - prisma.config.ts

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  datasource: {
    url: env('DATABASE_URL'),
    directUrl: env('DIRECT_URL'),
  },
})
```

And update schema.prisma:

```prisma
datasource db {
  provider = "postgresql"
  // URLs now in prisma.config.ts
}
```

Custom Config Path

Use `--config` flag with CLI commands:

```bash
```

```
prisma migrate dev --config ./config/prisma.config.ts
```

Monorepo Configuration

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'
import path from 'path'

export default defineConfig({
  schema: path.join(__dirname, 'packages/database/prisma/schema.prisma'),
  migrations: {
    path: path.join(__dirname, 'packages/database/prisma/migrations'),
  },
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

67. .agents\skills\prisma-upgrade-v7\references\removed-features.md

````
# Removed Features

Several features have been removed in Prisma v7. Here's how to migrate.

## Client Middleware

### Removed

```typescript
// ❌ No longer works in v7
prisma.$use(async (params, next) => {
  const before = Date.now()
  const result = await next(params)
  const after = Date.now()
  console.log(`Query took ${after - before}ms`)
  return result
})
```

### Use Client Extensions Instead

```typescript
// ✅ v7 approach
const prisma = new PrismaClient({ adapter }).$extends({
  query: {
    $allModels: {
      async $allOperations({ operation, model, args, query }) {
        const before = Date.now()
        const result = await query(args)
        const after = Date.now()
        console.log(`${model}.${operation} took ${after - before}ms`)
        return result
      },
    },
  },
})
```

### Common Middleware Patterns

#### Soft delete

```typescript
const prisma = new PrismaClient({ adapter }).$extends({
  query: {
    user: {
      async delete({ args, query }) {
        // Convert delete to soft delete
        return prisma.user.update({
          where: args.where,
          data: { deletedAt: new Date() },
        })
      },
      async findMany({ args, query }) {
        // Filter out soft-deleted records
        args.where = { ...args.where, deletedAt: null }
        return query(args)
      },
    },
  },
})
```

#### Logging

```typescript
const prisma = new PrismaClient({ adapter }).$extends({
  query: {
    $allModels: {
      async $allOperations({ operation, model, args, query }) {
        console.log(`${model}.${operation}`, JSON.stringify(args))
        return query(args)
      },
    },
  },
})
```

## Metrics

### Removed

The Metrics preview feature has been removed.

```typescript
// ❌ No longer works
const metrics = await prisma.$metrics.json()
```

### Alternatives
````

Custom counter with extensions

```typescript
let totalQueries = 0

const prisma = new PrismaClient({ adapter }).$extends({
  client: {
    async $totalQueries() {
      return totalQueries
    },
  },
  query: {
    $allModels: {
      async $allOperations({ query, args }) {
        totalQueries += 1
        return query(args)
      },
    },
  },
})

// Usage
const count = await prisma.$totalQueries()
```

Use driver-level metrics

Access metrics from the underlying driver adapter.

CLI Flags Removed

--skip-generate

Removed from `migrate dev` and `db push`.

```bash
# v6
prisma migrate dev --skip-generate

# v7 - generate is not run automatically
prisma migrate dev
prisma generate  # Run explicitly if needed
```

Local verification with Prisma `7.6.0` showed no generated client files emitted by `migrate dev` or `db push`, even though some CLI help text still says `migrate dev` "trigger[s] generators".

--skip-seed

Removed from `migrate dev`. More importantly, Prisma v7 no longer auto-runs seeds during `migrate dev` or `migrate reset`, so seed explicitly when you need it.

```bash
# v6
prisma migrate dev --skip-seed

# v7 - seed is not run automatically
prisma migrate dev
prisma db seed  # Run explicitly if needed
```

--schema and --url from db execute

```bash
# v6
prisma db execute --file ./script.sql --url "$DATABASE_URL"

# v7 - configure in prisma.config.ts
prisma db execute --file ./script.sql
```

migrate diff Options

Removed	Replacement
`--from-url`	`--from-config-datasource`
`--to-url`	`--to-config-datasource`
`--from-schema-datasource`	`--from-config-datasource`
`--to-schema-datasource`	`--to-config-datasource`
`--shadow-database-url`	Configure in `prisma.config.ts`

Example

```bash
# v6
prisma migrate diff --from-url "$DATABASE_URL" --to-schema schema.prisma

# v7
prisma migrate diff --from-config-datasource --to-schema schema.prisma
```

Automatic Behaviors Removed

Auto-generate after migrate

```bash
# v7 workflow
prisma migrate dev --name add_field
prisma generate  # Must run explicitly
```

Auto-seed after migrate

```bash
# v7 workflow
prisma migrate reset --force
prisma db seed  # Must run explicitly
```

Prisma.validator

The `prisma-client` generator no longer exposes `Prisma.validator`. Use TypeScript's `satisfies` operator instead.

```typescript
import { Prisma } from '../generated/prisma/client'

const userSelect = {
  id: true,
  email: true,
} satisfies Prisma.UserSelect
```

rejectOnNotFound

Removed in v5.0.0 (already deprecated).

```typescript
// ❌ Removed
const prisma = new PrismaClient({
  rejectOnNotFound: true,
})

// ✅ Use OrThrow methods
const user = await prisma.user.findUniqueOrThrow({
  where: { id: 1 },
})

const user = await prisma.user.findFirstOrThrow({
  where: { email: 'test@example.com' },
})
```

68. .agents\skills\prisma-upgrade-v7\references\schema-changes.md

````
# Schema Changes

Prisma v7 promotes `prisma-client` to the default generator. Update your generator block, output path, and imports accordingly.

This guide is for projects that are actually migrating to Prisma 7. Do not apply these schema changes to MongoDB projects; keep those on
Prisma 6.x.

## Generator Block (v7)

```prisma
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}
```

## Key Changes

### 1. Provider name

Use `prisma-client` in Prisma v7. The older `prisma-client-js` generator still exists in the repo for legacy setups, but `prisma-client` is
the default path for current projects.

### 2. Output is required

The `output` field is mandatory when using `prisma-client`. Prisma Client no longer generates to `node_modules` with this generator.

```prisma
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
}
```

### 3. engineType changed

Legacy Rust engine settings are gone. With `prisma-client`, the relevant value is `engineType = "client"` if you want to state it explicitly,
although it is typically inferred and can be omitted.

```prisma
generator client {
  provider   = "prisma-client"
  output     = "../generated/prisma"
  engineType = "client"
}
```

### 4. moduleFormat is explicit when needed

If you must stay on CommonJS:

```prisma
generator client {
  provider     = "prisma-client"
  output       = "../generated/prisma"
  moduleFormat = "cjs"
}
```

## Example Output Paths

### Standard project

```prisma
output = "../generated/prisma"
```

Creates files like:

```text
generated/prisma/
  client.ts
  browser.ts
  enums.ts
  models.ts
  models/
```

### Monorepo

```prisma
output = "../../packages/database/generated/prisma"
```

### Same directory as schema

```prisma
output = "./generated/prisma"
```
````

Creates: `prisma/generated/prisma/client.ts`

Datasource Block

The `url`, `directUrl`, and `shadowDatabaseUrl` fields in the `datasource` block are deprecated in Prisma v7. Move them to `prisma.config.ts` and keep only the provider in `schema.prisma`:

```prisma
datasource db {
  provider = "postgresql"
}
```

```typescript
export default defineConfig({
  datasource: {
    url: env('DATABASE_URL'),
    directUrl: env('DIRECT_URL'),
    shadowDatabaseUrl: env('SHADOW_DATABASE_URL'),
  },
})
```

After Schema Changes

1. Run `prisma generate`:
   ```bash
   npx prisma generate
   ```

2. Update imports throughout your codebase:
   ```typescript
   import { PrismaClient } from '../generated/prisma/client'
   ```

3. Update `.gitignore` if you manage this manually:
   ```
   /generated/prisma
   ```

4. Replace `Prisma.validator()` with TypeScript `satisfies` when using `prisma-client`:
   ```typescript
   import { Prisma } from '../generated/prisma/client'

   const userSelect = {
     id: true,
     email: true,
   } satisfies Prisma.UserSelect
   ```

Generated Entrypoints

- `client` - server-side Prisma Client and Prisma namespace
- `browser` - browser-safe types and enums without a real `PrismaClient`
- `enums` - slim enum-only entrypoint
- `models` - model types and derived helper types

Preview Features

Preview features still work as before:

```prisma
generator client {
  provider        = "prisma-client"
  output          = "../generated/prisma"
  previewFeatures = ["relationJoins", "fullTextSearch"]
}
```

Recent preview-feature examples also include `partialIndexes` for PostgreSQL, SQLite, SQL Server, and CockroachDB:

```prisma
generator client {
  provider        = "prisma-client"
  output          = "../generated/prisma"
  previewFeatures = ["partialIndexes"]
}
```

69. .agents\skills\prisma-upgrade-v7\SKILL.md

```
---
name: prisma-upgrade-v7
description: Complete migration guide from Prisma ORM v6 to v7 covering all breaking changes. Use when upgrading Prisma versions, encountering
v7 errors, or migrating existing projects. Triggers on "upgrade to prisma 7", "prisma 7 migration", "prisma-client generator", "driver adapter
required".
license: MIT
metadata:
  author: prisma
  version: "7.6.0"
---

# Upgrade to Prisma ORM 7

Complete guide for migrating from Prisma ORM v6 to v7. This upgrade introduces significant breaking changes around the new `prisma-client`
generator, driver adapters, `prisma.config.ts`, explicit environment loading, and generated client entrypoints.

## When to Apply

Reference this skill when:
- Upgrading from Prisma v6 to v7
- Updating to the `prisma-client` generator
- Setting up driver adapters
- Configuring `prisma.config.ts`
- Fixing import errors after upgrade

## Rule Categories by Priority

| Priority | Category | Impact | Prefix |
|----------|----------|--------|--------|
| 1 | Schema Migration | CRITICAL | `schema-changes` |
| 2 | Database Connectivity | CRITICAL | `driver-adapters` |
| 3 | Module System | CRITICAL | `esm-support` |
| 4 | Config and Env | HIGH | `prisma-config`, `env-variables` |
| 5 | Removed Features | HIGH | `removed-features` |
| 6 | Accelerate | HIGH | `accelerate-users` |

## Quick Reference

- `schema-changes` - generator migration, required output paths, generated entrypoints, and `Prisma.validator` replacement
- `driver-adapters` - required adapter installation for SQL providers, pool differences, and Prisma Postgres adapter choices
- `esm-support` - ESM-first setup plus CommonJS fallback with `moduleFormat = "cjs"`
- `prisma-config` - creating and using `prisma.config.ts`
- `env-variables` - explicit environment loading
- `removed-features` - removed middleware, metrics, and legacy CLI behavior
- `accelerate-users` - migration notes for Accelerate users

## Using MongoDB? This guide does not apply

Prisma 7 has no MongoDB connector. Do not apply any step in this guide to a project with
`provider = "mongodb"` — see the `prisma-mongodb-upgrade` skill for the actual decision
(stay on v6 deliberately vs migrate to Prisma Next).

## Important Notes

- **MongoDB projects should stay on Prisma 6.x or migrate to Prisma Next** - do not migrate MongoDB apps to Prisma 7's SQL client path (see
`prisma-mongodb-upgrade`)
- **Node.js 20.19.0+** required
- **TypeScript 5.4.0+** required
- **Latest stable Prisma ORM version**: `7.6.0`

## Upgrade Steps Overview

1. Update packages to v7
2. Choose your module format (`esm` by default, `cjs` if needed)
3. Update TypeScript configuration
4. Update the schema generator block
5. Create `prisma.config.ts`
6. Install and configure a driver adapter for SQL providers
7. Update Prisma Client imports
8. Update client instantiation
9. Replace deprecated helper patterns like `Prisma.validator`
10. Run `prisma generate` and test

## Quick Upgrade Commands

```bash
# Update packages
npm install @prisma/client@7
npm install -D prisma@7

# Install a driver adapter (PostgreSQL or Prisma Postgres via direct TCP)
npm install @prisma/adapter-pg pg

# Install dotenv for env loading
npm install dotenv

# Regenerate client
npx prisma generate
```
```

Breaking Changes Summary

Change	v6	v7
Module format	Implicit / mixed	ESM-first, `moduleFormat = "cjs"` supported
Generator provider	`prisma-client-js`	`prisma-client` is the default, while `prisma-client-js` still exists for legacy setups
Output path	Auto (node_modules)	Required explicit
Driver adapters	Optional	Required for SQL providers
Config file	`.env` + schema	`prisma.config.ts`
Env loading	Automatic	Manual (dotenv)
Generated entrypoints	Single package export	`client`, `browser`, `models`, `enums` entrypoints
Type-safe query fragments	`Prisma.validator()`	TypeScript `satisfies`
Middleware	`$use()`	Client Extensions
Metrics	Preview feature	Removed

Rule Files

Detailed migration guides for each breaking change:

```
references/esm-support.md        - ESM and CommonJS configuration
references/schema-changes.md     - Generator, output, imports, and generated entrypoints
references/driver-adapters.md    - Required driver adapter setup
references/prisma-config.md      - New configuration file
references/env-variables.md      - Environment variable loading
references/removed-features.md   - Middleware, metrics, and CLI flags
references/accelerate-users.md   - Special handling for Accelerate
```

Step-by-Step Migration

1. Update package.json for ESM-first projects

```json
{
  "type": "module"
}
```

If you need to stay on CommonJS, keep your app as CJS and set `moduleFormat = "cjs"` in the generator block instead of forcing ESM.

2. Update tsconfig.json

```json
{
  "compilerOptions": {
    "module": "ESNext",
    "moduleResolution": "bundler",
    "target": "ES2023",
    "strict": true,
    "esModuleInterop": true
  }
}
```

3. Update schema.prisma

```prisma
// Before (v6)
generator client {
  provider = "prisma-client-js"
}

// After (v7)
generator client {
  provider = "prisma-client"
  output   = "../generated/prisma"
  // Optional if you need CommonJS:
  // moduleFormat = "cjs"
}
```

4. Create prisma.config.ts

```typescript
import 'dotenv/config'
import { defineConfig, env } from 'prisma/config'

export default defineConfig({
  schema: 'prisma/schema.prisma',
  migrations: {
    path: 'prisma/migrations',
  },
  datasource: {
    url: env('DATABASE_URL'),
  },
})
```

5. Install a driver adapter (SQL providers only)

```bash
# PostgreSQL
```

```
npm install @prisma/adapter-pg pg

# MySQL
npm install @prisma/adapter-mariadb mariadb

# SQLite
npm install @prisma/adapter-better-sqlite3 better-sqlite3

# Prisma Postgres in standard Node.js apps (recommended)
npm install @prisma/adapter-pg pg

# Prisma Postgres serverless driver (edge/serverless)
npm install @prisma/adapter-ppg @prisma/ppg

# Neon
npm install @prisma/adapter-neon
```

MongoDB does not have a SQL `@prisma/adapter-*` package in the published Prisma 7.6.0 packages. If you're upgrading a MongoDB project, stop and keep that project on the latest Prisma 6.x release instead of following the standard Prisma 7 migration path.

6. Update client instantiation

```typescript
// Before (v6)
import { PrismaClient } from '@prisma/client'
const prisma = new PrismaClient()

// After (v7)
import { PrismaClient } from '../generated/prisma/client'
import { PrismaPg } from '@prisma/adapter-pg'

const adapter = new PrismaPg({
  connectionString: process.env.DATABASE_URL
})

const prisma = new PrismaClient({ adapter })
```

7. Replace Prisma.validator with satisfies

```typescript
import { Prisma } from '../generated/prisma/client'

const userSelect = {
  id: true,
  email: true,
  name: true,
} satisfies Prisma.UserSelect
```

8. Run migrations and generate

```bash
npx prisma generate
npx prisma migrate dev  # if needed
```

Troubleshooting

"Cannot find module" errors
- Check that the generator `output` path matches your import path
- Ensure `prisma generate` ran successfully

SSL certificate errors
- Add `ssl: { rejectUnauthorized: false }` to the adapter config if you need to preserve old behavior
- Or configure your certificates properly with `NODE_EXTRA_CA_CERTS` / OpenSSL CA settings

Connection timeout issues
- Driver adapters use the underlying driver's defaults, which differ from v6
- Configure pool settings explicitly on the adapter if needed

Resources

- [Official v7 Upgrade Guide](https://www.prisma.io/docs/orm/more/upgrades/to-v7)
- [Driver Adapters Documentation](https://www.prisma.io/docs/orm/core-concepts/supported-databases/database-drivers)
- [Prisma Config Reference](https://www.prisma.io/docs/orm/reference/prisma-config-reference)

How to Use

Follow `references/schema-changes.md` and `references/driver-adapters.md` first, then apply the remaining reference files based on your project setup.

70. .gitignore

```
# dependencies
node_modules/
.pnp
.pnp.*

# next.js (frontend)
.next/
frontend/.next/
frontend/out/

# production
/build

# misc
.DS_Store
*.pem

# debug
npm-debug.log*
yarn-debug.log*
yarn-error.log*
.pnpm-debug.log*

# env files (keep .env.example)
.env*
!.env.example

# vercel
.vercel

# typescript
*.tsbuildinfo
next-env.d.ts

# python
__pycache__/
*.pyc
backend/uploads/
backend/.venv/

# postgres local data (database/docker-compose.yml)
database/.pgdata/
```

71. AGENTS.md

```
<!-- BEGIN:nextjs-agent-rules -->
# This is NOT the Next.js you know

This version has breaking changes — APIs, conventions, and file structure may all differ from your training data. Read the relevant guide in
`node_modules/next/dist/docs/` before writing any code. Heed deprecation notices.
<!-- END:nextjs-agent-rules -->
```

72. backend\app__init__.py

73. backend\app\api__init__.py

74. backend\app\api\ai_routes.py

```
import json
from fastapi import APIRouter, HTTPException
from openai import AuthenticationError, RateLimitError, APIStatusError
from app.schemas.resume import (
    ImproveBulletRequest,
    GenerateSummaryRequest,
    SuggestSkillsRequest,
    TailorResumeRequest,
    AnalyzeJDRequest,
    JDAnalysis,
    SkillMatch,
)
from app.services import ai_service
from app.services.skill_synonyms import (
    canonical_skill,
    categorize_skill,
    match_skill_in_text,
)

router = APIRouter(prefix="/api/ai", tags=["ai"])


def _ai_error(e: Exception) -> HTTPException:
    if isinstance(e, AuthenticationError):
        return HTTPException(
            status_code=401,
            detail="Invalid or missing OPENROUTER_API_KEY. Get a free key at https://openrouter.ai/keys and add it to backend/.env.",
        )
    if isinstance(e, RateLimitError):
        code = getattr(e, "code", None) or (e.body or {}).get("code", "") if isinstance(e.body, dict) else getattr(e, "code", None)
        if code == "insufficient_quota" or "no credits" in str(getattr(e, "body", "")).lower():
            return HTTPException(
                status_code=402,
                detail="No credits on your OpenRouter account. Free models work with $0 balance, or add credits at
https://openrouter.ai/settings/credits",
            )
        return HTTPException(status_code=429, detail="AI rate limit reached. Please try again in a moment.")
    if isinstance(e, APIStatusError):
        return HTTPException(status_code=e.status_code, detail=f"AI service error: {e.message}")
    if isinstance(e, json.JSONDecodeError):
        return HTTPException(status_code=502, detail="AI returned invalid JSON. Please try again.")
    return HTTPException(status_code=500, detail=f"AI service error: {e}")


@router.post("/improve-bullet")
async def improve_bullet(req: ImproveBulletRequest):
    if not req.bullet:
        raise HTTPException(status_code=400, detail="Bullet point text is required")
    try:
        result = await ai_service.improve_bullet_point(req.bullet, req.role)
    except Exception as e:
        raise _ai_error(e)
    return result.model_dump()


@router.post("/generate-summary")
async def generate_summary(req: GenerateSummaryRequest):
    if not req.input:
        raise HTTPException(status_code=400, detail="Input text is required")
    try:
        result = await ai_service.generate_summary(req.input, req.targetRole)
    except Exception as e:
        raise _ai_error(e)
    return result.model_dump()


@router.post("/suggest-skills")
async def suggest_skills(req: SuggestSkillsRequest):
    if not req.targetRole:
        raise HTTPException(status_code=400, detail="Target role is required")
    try:
        result = await ai_service.suggest_skills(req.targetRole, req.currentSkills)
    except Exception as e:
        raise _ai_error(e)
    return result.model_dump()


@router.post("/tailor-resume")
async def tailor_resume(req: TailorResumeRequest):
    if not req.resumeJson or not req.jd:
        raise HTTPException(status_code=400, detail="Resume and job description are required")
    try:
        result = await ai_service.tailor_full_resume(req.resumeJson, req.jd)
    except Exception as e:
        raise _ai_error(e)
    return result.model_dump()


@router.post("/analyze-jd")
async def analyze_jd(req: AnalyzeJDRequest):
```

```
if not req.jd:
    raise HTTPException(status_code=400, detail="Job description is required")

try:
    extraction, _used = await ai_service.extract_jd_keywords(req.jd)
except Exception as e:
    raise _ai_error(e)

# Dedupe while preserving order, then semantically match against the resume.
all_skills: list[str] = []
for s in extraction.requiredSkills + extraction.preferredSkills:
    key = canonical_skill(s).lower()
    if all(key != canonical_skill(x).lower() for x in all_skills):
        all_skills.append(s)

resume_text = (req.resumeText or "").lower()
matched_skills: list[SkillMatch] = []
missing_skills: list[SkillMatch] = []

for skill in all_skills:
    matched = match_skill_in_text(skill, resume_text) if resume_text else False
    canonical = canonical_skill(skill)
    category = categorize_skill(canonical)
    item = SkillMatch(name=skill, matched=matched, category=category)
    (matched_skills if matched else missing_skills).append(item)

total = max(len(all_skills), 1)
match_pct = round((len(matched_skills) / total) * 100)

from app.services.skill_synonyms import CANONICAL_SKILL_HINTS

suggestions = [
    f'Click "Add" to insert "{s.name}" into your Skills section if you have experience with it.'
    for s in missing_skills[:5]
]
if not missing_skills and all_skills:
    suggestions.append("Your resume covers the JD keywords. Keep the wording ATS-friendly.")

analysis = JDAnalysis(
    matchedSkills=matched_skills,
    missingSkills=missing_skills,
    matchPercentage=match_pct,
    keywords=list(dict.fromkeys(extraction.keywords + extraction.hardSkills + extraction.softSkills)),
    suggestions=suggestions,
)
return analysis.model_dump()
```

75. backend\app\api\ats.py

```
from fastapi import APIRouter
from app.schemas.resume import ATSScoreRequest
from app.services.ats_scorer import analyze_ats_score

router = APIRouter(prefix="/api/ats", tags=["ats"])


@router.post("/score")
async def score_resume(req: ATSScoreRequest):
    analysis = analyze_ats_score(req.resumeData)
    return analysis.model_dump()
```

76. backend\app\api\export.py

```
from fastapi import APIRouter, HTTPException
from fastapi.responses import Response
from app.schemas.resume import ResumeData
from app.services.export_service import (
    generate_pdf,
    generate_docx,
    render_resume_html,
    render_resume_plain_text,
)

router = APIRouter(prefix="/api/export", tags=["export"])


@router.post("/pdf")
async def export_pdf(data: ResumeData):
    try:
        pdf_bytes = await generate_pdf(data)
        return Response(content=pdf_bytes, media_type="application/pdf", headers={"Content-Disposition": "attachment; filename=resume.pdf"})
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/text")
async def export_text(data: ResumeData):
    """Clean, copy-pasteable ATS-raw plain text for legacy ATS text boxes."""
    return {"text": render_resume_plain_text(data)}


@router.post("/docx")
async def export_docx(data: ResumeData):
    try:
        docx_bytes = generate_docx(data)
        return Response(
            content=docx_bytes,
            media_type="application/vnd.openxmlformats-officedocument.wordprocessingml.document",
            headers={"Content-Disposition": "attachment; filename=resume.docx"},
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


@router.post("/html")
async def export_html(data: ResumeData):
    html = render_resume_html(data)
    return Response(content=html, media_type="text/html")
```

77. backend\app\api\resume.py

```
from fastapi import APIRouter, UploadFile, File, HTTPException
from app.services.resume_parser import parse_resume_file, detect_sections
from app.services.ats_scorer import analyze_ats_score_from_text

router = APIRouter(prefix="/api/resume", tags=["resume"])


@router.post("/parse")
async def parse_resume(file: UploadFile = File(...)):
    if not file.filename or not (file.filename.endswith(".pdf") or file.filename.endswith(".docx")):
        raise HTTPException(status_code=400, detail="Unsupported file type. Please upload a PDF or DOCX file.")

    contents = await file.read()
    try:
        text, sections, layout = parse_resume_file(contents, file.content_type or "")
        detected = detect_sections(text)
        analysis = analyze_ats_score_from_text(
            text, detected,
            has_tables=layout["hasTables"],
            single_column=layout["singleColumn"],
        )
        return {
            "text": text,
            "sections": sections,
            "detectedSections": detected,
            "layout": layout,
            "analysis": analysis.model_dump(),
        }
    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
```

78. backend\app\api\resumes.py

```
from fastapi import APIRouter, Depends, HTTPException, Query
from sqlalchemy.exc import SQLAlchemyError
from sqlalchemy.ext.asyncio import AsyncSession

from app.database import get_session
from app.models import resume as repo
from app.schemas.resume import (
    CloneResumeRequest,
    InsertKeywordsRequest,
    SaveResumeRequest,
    UpdateResumeRequest,
)

router = APIRouter(prefix="/api/resumes", tags=["resumes"])

DATABASE_DOWN_DETAIL = (
    "Database is not available. Set DATABASE_URL to your Supabase Postgres URL, or "
    "run locally with the built-in SQLite fallback (see backend/.env.example)."
)


def _db_error(e: Exception) -> HTTPException:
    return HTTPException(status_code=503, detail=DATABASE_DOWN_DETAIL)


def _require_resume(resume: dict | None) -> dict:
    if resume is None:
        raise HTTPException(status_code=404, detail="Resume not found")
    return resume


@router.post("")
async def save_resume(req: SaveResumeRequest, session: AsyncSession = Depends(get_session)):
    if not req.userId:
        raise HTTPException(status_code=400, detail="userId is required")
    try:
        doc = await repo.create_resume(
            session=session,
            user_id=req.userId,
            data=req.data.model_dump(),
            title=req.title,
            template=req.template,
            target_role=req.targetRole,
        )
    except SQLAlchemyError as e:
        raise _db_error(e)
    return repo.serialize_resume(doc, req.data.model_dump())


@router.get("")
async def get_user_resumes(
    userId: str = Query(..., min_length=1),
    session: AsyncSession = Depends(get_session),
):
    try:
        return await repo.list_resumes(session, userId)
    except SQLAlchemyError as e:
        raise _db_error(e)


@router.get("/{resume_id}")
async def get_single_resume(
    resume_id: str,
    userId: str = Query(..., min_length=1),
    session: AsyncSession = Depends(get_session),
):
    if not userId:
        raise HTTPException(status_code=400, detail="userId is required to verify ownership")
    try:
        doc = await repo.get_resume_with_data(session, resume_id, userId)
    except SQLAlchemyError as e:
        raise _db_error(e)
    return _require_resume(doc)


@router.put("/{resume_id}")
async def update_single_resume(
    resume_id: str,
    req: UpdateResumeRequest,
    session: AsyncSession = Depends(get_session),
):
    if not req.userId:
        raise HTTPException(status_code=400, detail="userId is required to verify ownership")
    patch: dict = {}
    if req.title is not None:
        patch["title"] = req.title
    if req.template is not None:
        patch["template"] = req.template
    if req.score is not None:
        patch["score"] = req.score
```

```
    if req.targetRole is not None:
        patch["target_role"] = req.targetRole
    if req.data is not None:
        patch["data"] = req.data.model_dump()
    if not patch:
        raise HTTPException(status_code=400, detail="Nothing to update")
    try:
        doc = await repo.update_resume(session, resume_id, req.userId, patch)
    except SQLAlchemyError as e:
        raise _db_error(e)
    return _require_resume(doc)


@router.delete("/{resume_id}")
async def delete_single_resume(
    resume_id: str,
    userId: str = Query(..., min_length=1),
    session: AsyncSession = Depends(get_session),
):
    try:
        deleted = await repo.delete_resume(session, resume_id, userId)
    except SQLAlchemyError as e:
        raise _db_error(e)
    if not deleted:
        # Distinguish "not found" from "not owned" for security purposes without
        # leaking which resumes exist for other users.
        raise HTTPException(status_code=404, detail="Resume not found")
    return {"deleted": resume_id}


@router.post("/{resume_id}/clone")
async def clone_resume(
    resume_id: str,
    req: CloneResumeRequest,
    session: AsyncSession = Depends(get_session),
):
    """Create a tailored variant of an existing resume (owned by the same user).

    The copy records `parent_id` so variants can be traced back to the source.
    """
    if not req.userId:
        raise HTTPException(status_code=400, detail="userId is required")
    source = await repo.get_resume_with_data(session, resume_id, req.userId)
    _require_resume(source)
    if not source.get("data"):
        raise HTTPException(status_code=400, detail="Source resume has no data to clone")
    try:
        clone = await repo.create_resume(
            session=session,
            user_id=req.userId,
            data=source["data"],
            title=req.title or f"{source['title']} (variant)",
            template=source.get("template", "modern"),
            target_role=req.targetRole or source.get("targetRole"),
            parent_id=source["id"],
        )
    except SQLAlchemyError as e:
        raise _db_error(e)
    return {**repo.serialize_resume(clone), "data": source["data"]}


@router.post("/{resume_id}/insert-keywords")
async def insert_keywords(
    resume_id: str,
    req: InsertKeywordsRequest,
    session: AsyncSession = Depends(get_session),
):
    """One-click insertion of missing JD keywords into a resume's skills/summary."""
    if not req.userId:
        raise HTTPException(status_code=400, detail="userId is required")
    doc = await repo.get_resume_with_data(session, resume_id, req.userId)
    _require_resume(doc)
    data = doc.get("data") or {}
    keywords = [k.strip() for k in req.keywords if k and k.strip()]

    if req.targetSection == "skills":
        existing = [s.get("name", "").strip().lower() for s in data.get("skills", [])]
        added = []
        for kw in keywords:
            if kw.lower() not in existing:
                data.setdefault("skills", []).append({"id": "", "name": kw})
                existing.append(kw.lower())
                added.append(kw)
    elif req.targetSection == "summary":
        added = [kw for kw in keywords if kw.lower() not in (data.get("summary") or "").lower()]
        if added:
            suffix = ", ".join(added)
            data["summary"] = f"{data.get('summary', '').rstrip()} Core strengths include: {suffix}."
    else:
        raise HTTPException(status_code=400, detail="targetSection must be 'skills' or 'summary'")

    if not added:
        return {**doc, "insertedKeywords": [], "message": "All keywords already present."}
```

```
updated = await repo.update_resume(session, resume_id, req.userId, {"data": data})
_require_resume(updated)
return {**updated, "insertedKeywords": added, "message": f"Inserted {len(added)} keyword(s)."}
```

79. backend\app\database.py

```
import os
import logging
from dotenv import load_dotenv
from sqlalchemy.ext.asyncio import (
    AsyncEngine,
    AsyncSession,
    async_sessionmaker,
    create_async_engine,
)
from sqlalchemy.orm import DeclarativeBase

load_dotenv()

logger = logging.getLogger("uvicorn.error")

# Production (Supabase PostgreSQL) uses DATABASE_URL, e.g.:
#   postgresql+asyncpg://postgres:password@host:5432/postgres
# For local development / tests, fall back to a file-based async SQLite DB.
DEFAULT_DATABASE_URL = "sqlite+aiosqlite:///./resume_builder.db"

_engine: AsyncEngine | None = None
_session_factory: async_sessionmaker[AsyncSession] | None = None


class Base(DeclarativeBase):
    pass


def is_sqlite(url: str) -> bool:
    return url.startswith("sqlite")


def configure_database(url: str | None = None) -> None:
    """(Re)configure the engine. Exposed so tests can point at an in-memory SQLite."""
    global _engine, _session_factory
    url = url or os.getenv("DATABASE_URL", DEFAULT_DATABASE_URL)

    kwargs: dict = {"pool_pre_ping": True}
    if is_sqlite(url):
        kwargs["connect_args"] = {"check_same_thread": False}
    else:
        kwargs["pool_size"] = 5
        kwargs["max_overflow"] = 10

    _engine = create_async_engine(url, echo=False, **kwargs)
    _session_factory = async_sessionmaker(_engine, class_=AsyncSession, expire_on_commit=False)


def get_engine() -> AsyncEngine:
    if _engine is None:
        configure_database()
    assert _engine is not None
    return _engine


async def get_session():
    """FastAPI dependency that yields an async SQLAlchemy session."""
    if _session_factory is None:
        configure_database()
    assert _session_factory is not None
    async with _session_factory() as session:
        yield session


async def init_db() -> None:
    """Create tables + indexes if they do not exist. Safe to run on every boot."""
    from app.models import resume  # noqa: F401  (registers models on Base.metadata)

    engine = get_engine()
    async with engine.begin() as conn:
        await conn.run_sync(Base.metadata.create_all)


async def check_db_connection() -> bool:
    """Lightweight connectivity probe used by /api/health."""
    from sqlalchemy import text

    engine = get_engine()
    try:
        async with engine.connect() as conn:
            await conn.execute(text("SELECT 1"))
        return True
    except Exception:
        logger.exception("Database connectivity check failed")
        return False


async def close_db() -> None:
    global _engine, _session_factory
    if _engine is not None:
```

```
    await _engine.dispose()
_engine = None
_session_factory = None
```

80. backend\app\models__init__.py

81. backend\app\models\resume.py

```
import datetime
import uuid
from typing import Any

from sqlalchemy import JSON, DateTime, ForeignKey, Integer, String, Text, func
from sqlalchemy.dialects.postgresql import JSONB
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select, delete as sa_delete

from app.database import Base

# Generic JSON that becomes JSONB on PostgreSQL (Supabase), JSON on SQLite.
JSONColumn = JSON().with_variant(JSONB, "postgresql")

RESUMES_TABLE = "resumes"
RESUME_DATA_TABLE = "resume_data"
ATS_SCANS_TABLE = "ats_scans"
JOB_DESCRIPTIONS_TABLE = "job_descriptions"


def _new_id() -> str:
    return str(uuid.uuid4())


class Resume(Base):
    __tablename__ = RESUMES_TABLE

    id: Mapped[str] = mapped_column(String(36), primary_key=True, default=_new_id)
    user_id: Mapped[str] = mapped_column(String(64), nullable=False, index=True)
    title: Mapped[str] = mapped_column(String(200), nullable=False, default="Untitled Resume")
    target_role: Mapped[str | None] = mapped_column(String(200), nullable=True)
    template: Mapped[str] = mapped_column(String(50), nullable=False, default="modern")
    # When a resume is created from a clone/variant, parent_id points at the source resume.
    parent_id: Mapped[str | None] = mapped_column(String(36), nullable=True, index=True)
    score: Mapped[int | None] = mapped_column(Integer, nullable=True)
    created_at: Mapped[datetime.datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )
    updated_at: Mapped[datetime.datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), onupdate=func.now()
    )


class ResumeData(Base):
    __tablename__ = RESUME_DATA_TABLE

    id: Mapped[str] = mapped_column(String(36), primary_key=True, default=_new_id)
    resume_id: Mapped[str] = mapped_column(
        String(36), ForeignKey(f"{RESUMES_TABLE}.id", ondelete="CASCADE"), nullable=False, unique=True
    )
    # Structured JSONB payload: contact, summary, experience, education, skills, projects,
    # certifications - mirroring the ResumeData Pydantic schema.
    data: Mapped[dict[str, Any]] = mapped_column(JSONColumn, nullable=False, default=dict)
    created_at: Mapped[datetime.datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )
    updated_at: Mapped[datetime.datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), onupdate=func.now()
    )


class AtsScan(Base):
    __tablename__ = ATS_SCANS_TABLE

    id: Mapped[str] = mapped_column(String(36), primary_key=True, default=_new_id)
    resume_id: Mapped[str] = mapped_column(
        String(36), ForeignKey(f"{RESUMES_TABLE}.id", ondelete="CASCADE"), nullable=False, index=True
    )
    score: Mapped[int] = mapped_column(Integer, nullable=False)
    matched_keywords: Mapped[list[Any]] = mapped_column(JSONColumn, nullable=False, default=list)
    missing_keywords: Mapped[list[Any]] = mapped_column(JSONColumn, nullable=False, default=list)
    suggestions: Mapped[list[str]] = mapped_column(JSONColumn, nullable=False, default=list)
    created_at: Mapped[datetime.datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )


class JobDescription(Base):
    __tablename__ = JOB_DESCRIPTIONS_TABLE

    id: Mapped[str] = mapped_column(String(36), primary_key=True, default=_new_id)
    user_id: Mapped[str] = mapped_column(String(64), nullable=False, index=True)
    target_role: Mapped[str | None] = mapped_column(String(200), nullable=True)
    raw_text: Mapped[str] = mapped_column(Text, nullable=False, default="")
    parsed_skills: Mapped[dict[str, Any]] = mapped_column(JSONColumn, nullable=False, default=dict)
    created_at: Mapped[datetime.datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now()
    )
```

```
# ---------------------------------------------------------------------------
# Repository helpers (async, session-based so ownership is enforced per call)
# ---------------------------------------------------------------------------


def serialize_resume(resume: Resume, data: dict | None = None) -> dict:
    return {
        "id": resume.id,
        "title": resume.title,
        "targetRole": resume.target_role,
        "template": resume.template,
        "parentId": resume.parent_id,
        "score": resume.score,
        "data": data,
        "createdAt": resume.created_at,
        "updatedAt": resume.updated_at,
    }


async def create_resume(
    session: AsyncSession,
    user_id: str,
    data: dict,
    title: str = "Untitled Resume",
    template: str = "modern",
    target_role: str | None = None,
    parent_id: str | None = None,
) -> Resume:
    resume = Resume(user_id=user_id, title=title, template=template, target_role=target_role, parent_id=parent_id)
    session.add(resume)
    await session.flush()
    session.add(ResumeData(resume_id=resume.id, data=data))
    await session.commit()
    await session.refresh(resume)
    return resume


async def get_resume_for_user(session: AsyncSession, resume_id: str, user_id: str) -> Resume | None:
    result = await session.execute(
        select(Resume).where(Resume.id == resume_id, Resume.user_id == user_id)
    )
    return result.scalar_one_or_none()


async def get_resume_data(session: AsyncSession, resume_id: str) -> dict | None:
    result = await session.execute(select(ResumeData).where(ResumeData.resume_id == resume_id))
    doc = result.scalar_one_or_none()
    return doc.data if doc else None


async def get_resume_with_data(session: AsyncSession, resume_id: str, user_id: str) -> dict | None:
    resume = await get_resume_for_user(session, resume_id, user_id)
    if resume is None:
        return None
    data = await get_resume_data(session, resume_id)
    return serialize_resume(resume, data)


async def list_resumes(session: AsyncSession, user_id: str) -> list[dict]:
    result = await session.execute(
        select(Resume)
        .where(Resume.user_id == user_id)
        .order_by(Resume.updated_at.desc())
    )
    resumes = result.scalars().all()
    return [serialize_resume(r) for r in resumes]


async def update_resume(session: AsyncSession, resume_id: str, user_id: str, patch: dict) -> dict | None:
    resume = await get_resume_for_user(session, resume_id, user_id)
    if resume is None:
        return None

    for key in ("title", "template", "score", "target_role"):
        if key in patch and patch[key] is not None:
            setattr(resume, key, patch[key])

    if "data" in patch and patch["data"] is not None:
        result = await session.execute(select(ResumeData).where(ResumeData.resume_id == resume_id))
        row = result.scalar_one_or_none()
        if row is None:
            session.add(ResumeData(resume_id=resume_id, data=patch["data"]))
        else:
            row.data = patch["data"]

    await session.commit()
    return await get_resume_with_data(session, resume_id, user_id)


async def delete_resume(session: AsyncSession, resume_id: str, user_id: str) -> bool:
    resume = await get_resume_for_user(session, resume_id, user_id)
    if resume is None:
```

```
        return False
    await session.execute(sa_delete(Resume).where(Resume.id == resume_id))
    await session.commit()
    return True


async def create_ats_scan(
    session: AsyncSession,
    resume_id: str,
    score: int,
    matched_keywords: list,
    missing_keywords: list,
    suggestions: list[str],
) -> AtsScan:
    scan = AtsScan(
        resume_id=resume_id,
        score=score,
        matched_keywords=matched_keywords,
        missing_keywords=missing_keywords,
        suggestions=suggestions,
    )
    session.add(scan)
    await session.commit()
    await session.refresh(scan)
    return scan


async def list_ats_scans_for_resume(session: AsyncSession, resume_id: str) -> list[AtsScan]:
    result = await session.execute(select(AtsScan).where(AtsScan.resume_id == resume_id).order_by(AtsScan.created_at.desc()))
    return list(result.scalars().all())


async def save_job_description(
    session: AsyncSession,
    user_id: str,
    raw_text: str,
    parsed_skills: dict | None = None,
    target_role: str | None = None,
) -> JobDescription:
    jd = JobDescription(user_id=user_id, raw_text=raw_text, parsed_skills=parsed_skills or {}, target_role=target_role)
    session.add(jd)
    await session.commit()
    await session.refresh(jd)
    return jd
```

82. backend\app\schemas__init__.py

83. backend\app\schemas\resume.py

```
from pydantic import BaseModel, Field


class ContactInfo(BaseModel):
    fullName: str = ""
    email: str = ""
    phone: str = ""
    location: str = ""
    linkedin: str | None = None
    website: str | None = None


class WorkExperience(BaseModel):
    id: str = ""
    company: str = ""
    position: str = ""
    location: str = ""
    startDate: str = ""
    endDate: str = ""
    current: bool = False
    bullets: list[str] = []


class Education(BaseModel):
    id: str = ""
    institution: str = ""
    degree: str = ""
    field: str = ""
    startDate: str = ""
    endDate: str = ""
    gpa: str | None = None


class Skill(BaseModel):
    id: str = ""
    name: str = ""


class Project(BaseModel):
    id: str = ""
    name: str = ""
    description: str = ""
    url: str | None = None
    technologies: list[str] = []


class Certification(BaseModel):
    id: str = ""
    name: str = ""
    issuer: str = ""
    date: str = ""
    url: str | None = None


class ResumeData(BaseModel):
    contact: ContactInfo = ContactInfo()
    summary: str = ""
    experience: list[WorkExperience] = []
    education: list[Education] = []
    skills: list[Skill] = []
    projects: list[Project] = []
    certifications: list[Certification] = []


class ATSIssue(BaseModel):
    id: str
    severity: str
    category: str
    message: str
    fix: str


class ATSAnalysis(BaseModel):
    score: int
    issues: list[ATSIssue]
    passedChecks: list[str]
    summary: str


class SkillMatch(BaseModel):
    name: str
    matched: bool
    category: str = "uncategorized"  # "hard" | "soft" | "uncategorized"


class JDAnalysis(BaseModel):
    matchedSkills: list[SkillMatch]
    missingSkills: list[SkillMatch]
    matchPercentage: int
    keywords: list[str]
```

```
    suggestions: list[str]


class ImproveBulletRequest(BaseModel):
    bullet: str
    role: str | None = None


class GenerateSummaryRequest(BaseModel):
    input: str
    targetRole: str | None = None


class SuggestSkillsRequest(BaseModel):
    targetRole: str
    currentSkills: list[str] = []


class TailorResumeRequest(BaseModel):
    resumeJson: dict
    jd: str


class AnalyzeJDRequest(BaseModel):
    jd: str
    resumeText: str = ""


class ATSScoreRequest(BaseModel):
    resumeData: ResumeData


# ---------------------------------------------------------------------------
# AI structured outputs (Structured Outputs / guardrails)
# ---------------------------------------------------------------------------


class PassivePhraseFlag(BaseModel):
    phrase: str
    suggestion: str
    position: int = 0


class ImproveBulletResult(BaseModel):
    original: str
    improved: str
    usedModel: str = ""
    flags: list[PassivePhraseFlag] = Field(default_factory=list)


class GenerateSummaryResult(BaseModel):
    summary: str
    usedModel: str = ""


class SuggestSkillsResult(BaseModel):
    skills: list[str]
    usedModel: str = ""


class JDExtraction(BaseModel):
    requiredSkills: list[str] = Field(default_factory=list)
    preferredSkills: list[str] = Field(default_factory=list)
    keywords: list[str] = Field(default_factory=list)
    hardSkills: list[str] = Field(default_factory=list)
    softSkills: list[str] = Field(default_factory=list)


class TailorResult(BaseModel):
    tailored: ResumeData
    guardrailWarnings: list[str] = Field(default_factory=list)
    usedModel: str = ""


# ---------------------------------------------------------------------------
# Resume CRUD / variants / keyword insertion request models
# ---------------------------------------------------------------------------


class SaveResumeRequest(BaseModel):
    userId: str
    title: str = "Untitled Resume"
    data: ResumeData
    template: str = "modern"
    targetRole: str | None = None


class UpdateResumeRequest(BaseModel):
    userId: str
    title: str | None = None
    data: ResumeData | None = None
    template: str | None = None
    score: int | None = None
```

```
    targetRole: str | None = None


class CloneResumeRequest(BaseModel):
    userId: str
    title: str = "Untitled Resume (copy)"
    targetRole: str | None = None


class InsertKeywordsRequest(BaseModel):
    userId: str
    keywords: list[str]
    targetSection: str = "skills"  # "skills" | "summary"


class PersistJdRequest(BaseModel):
    userId: str
    rawText: str
    targetRole: str | None = None
    parsedSkills: dict | None = None
```

84. backend\app\services__init__.py

85. backend\app\services\ai_service.py

```
"""OpenRouter AI integration with structured outputs and guardrails.

- All AI endpoints parse & validate responses against Pydantic schemas.
- Model fallback chain: if the primary (often :free) model 429s / times out / 5xx,
  the next configured model is tried automatically.
- Anti-hallucination guardrails for /tailor-resume.
- Google XYZ formula + passive-phrase flagging for /improve-bullet.
"""

import asyncio
import json
import os
import re
from typing import Any, TypeVar

from openai import (
    APITimeoutError,
    APIStatusError,
    AsyncOpenAI,
    AuthenticationError,
    RateLimitError,
)
from pydantic import BaseModel, ValidationError

from app.schemas.resume import (
    GenerateSummaryResult,
    ImproveBulletResult,
    JDExtraction,
    PassivePhraseFlag,
    ResumeData,
    SuggestSkillsResult,
    TailorResult,
)

T = TypeVar("T", bound=BaseModel)

client = AsyncOpenAI(
    api_key=os.getenv("OPENROUTER_API_KEY", ""),
    base_url="https://openrouter.ai/api/v1",
    timeout=60.0,
    max_retries=0,
    default_headers={
        "HTTP-Referer": os.getenv("APP_URL", "https://localhost:3000"),
        "X-Title": "AI Resume Builder",
    },
)

PRIMARY_MODEL = os.getenv("OPENROUTER_MODEL", "deepseek/deepseek-chat-v3-0324:free")
FALLBACK_MODELS = [
    m.strip()
    for m in os.getenv("OPENROUTER_MODEL_FALLBACKS", "").split(",")
    if m.strip()
]
MODELS = [PRIMARY_MODEL, *FALLBACK_MODELS]

# ---------------------------------------------------------------------------
# Low-level transport (kept separate so tests can monkeypatch it)
# ---------------------------------------------------------------------------


async def _per_model_completion(model: str, system: str, prompt: str, max_tokens: int = 4096, json_mode: bool = False):
    kwargs: dict[str, Any] = {
        "model": model,
        "max_tokens": max_tokens,
        "messages": [
            {"role": "system", "content": system},
            {"role": "user", "content": prompt},
        ],
    }
    if json_mode:
        kwargs["response_format"] = {"type": "json_object"}
    response = await client.chat.completions.create(**kwargs)
    return response.choices[0].message.content or ""


async def _run_text(system: str, prompt: str, max_tokens: int = 4096) -> tuple[str, str]:
    """Call models in order; return (text, model_used). Raises on total failure."""
    last_error: Exception | None = None
    for i, model in enumerate(MODELS):
        try:
            text = await _per_model_completion(model, system, prompt, max_tokens=max_tokens)
            if text.strip():
                return text, model
            last_error = RuntimeError(f"Empty completion from {model}")
        except AuthenticationError:
            raise
        except (RateLimitError, APITimeoutError, APIStatusError, Exception) as e:  # noqa: PERF203
            last_error = e
            if i < len(MODELS) - 1:
                await asyncio.sleep(min(0.5 * (i + 1), 3))
```

```
    if isinstance(last_error, AuthenticationError):
        raise last_error
    raise last_error or RuntimeError("No AI model available")


def _extract_json(text: str) -> Any:
    cleaned = re.sub(r"```(?:json)?\s*", "", text).strip()
    cleaned = re.sub(r"```", "", cleaned).strip()
    try:
        return json.loads(cleaned)
    except json.JSONDecodeError:
        match = re.search(r"(\{.*\}|\[.*\])", cleaned, re.DOTALL)
        if match:
            return json.loads(match.group(1))
        raise


async def _run_json(system: str, prompt: str, model: type[T]) -> tuple[T, str]:
    """Request JSON, parse, then validate via Pydantic. Tries fallback models."""
    errors: list[str] = []
    for i, m in enumerate(MODELS):
        try:
            text = await _per_model_completion(m, system, prompt, max_tokens=4096, json_mode=True)
            data = _extract_json(text)
            return model.model_validate(data), m
        except AuthenticationError:
            raise
        except (RateLimitError, APITimeoutError, APIStatusError) as e:
            errors.append(f"{m}: {e}")
            if i < len(MODELS) - 1:
                await asyncio.sleep(min(0.5 * (i + 1), 3))
        except (json.JSONDecodeError, ValidationError, ValueError) as e:
            errors.append(f"{m}: schema/parse error ({e})")
            # Fall back to plain-text retry on the same model before switching.
            try:
                text = await _per_model_completion(m, system, prompt, max_tokens=4096, json_mode=False)
                data = _extract_json(text)
                return model.model_validate(data), m
            except (json.JSONDecodeError, ValidationError, ValueError):
                pass
    raise RuntimeError("AI returned no valid structured output: " + "; ".join(errors[:3]))


# ---------------------------------------------------------------------------
# Passive phrase detection + action verbs (Google XYZ formula)
# ---------------------------------------------------------------------------

PASSIVE_PHRASES: list[tuple[str, str]] = [
    ("responsible for", "Spearheaded"),
    ("helped with", "Drove"),
    ("worked on", "Delivered"),
    ("work on", "Build"),
    ("was responsible for", "Owned"),
    ("assisted with", "Supported"),
    ("in charge of", "Led"),
    ("duties included", "Executed"),
    ("tasked with", "Championed"),
    ("had to", "Achieved"),
    ("involved in", "Contributed to"),
    ("handled", "Managed"),
    ("made sure", "Ensured"),
    ("was tasked", "Owned"),
]

STRONG_ACTION_VERBS = (
    "Spearheaded, Optimized, Architected, Engineered, Led, Delivered, Drove, Built, "
    "Designed, Implemented, Launched, Scaled, Streamlined, Automated, Accelerated, "
    "Owned, Championed, Executed"
)


def flag_passive_phrases(bullet: str, role: str | None = None) -> list[PassivePhraseFlag]:
    flags: list[PassivePhraseFlag] = []
    lower = bullet.lower()
    for phrase, verb in PASSIVE_PHRASES:
        idx = lower.find(phrase)
        if idx != -1:
            flags.append(
                PassivePhraseFlag(
                    phrase=lower[idx : idx + len(phrase)],
                    suggestion=verb,
                    position=idx,
                )
            )
    return flags


# ---------------------------------------------------------------------------
# /api/ai/improve-bullet
# ---------------------------------------------------------------------------

IMPROVE_SYSTEM = """You are an expert resume coach and ATS optimization specialist.
```

```
Rewrite the given resume bullet point to follow the Google XYZ formula:

  "Accomplished [X] as measured by [Y], by doing [Z]"

Rules:
- Start with a strong action verb (avoid 'responsible for', 'helped with', 'worked on').
- Keep factual claims; NEVER invent metrics, numbers, tools, or technologies the user did not state.
- If the original bullet has no metric, phrase the "as measured by [Y]" part as the natural,
  believable outcome implied by the original text, or omit the metric rather than fabricate one.
- Output valid JSON only, using exactly these keys:
  {"improved": "<rewritten bullet>", "flags": [{"phrase": "<detected weak/passive phrase>",
   "suggestion": "<strong verb>"}], "note": "<short explanation, or empty string>"}
- "flags" must be empty when no weak phrases were present. Never include any text outside the JSON."""


async def improve_bullet_point(bullet: str, role: str | None = None) -> ImproveBulletResult:
    flags = flag_passive_phrases(bullet, role)
    prompt = f'Rewrite this resume bullet point following the Google XYZ formula. Preserve all client facts.\n\nBullet: "{bullet}"'
    if role:
        prompt += f"\n\nTarget role: {role}"
    prompt += "\n\nStrong action verbs to consider: " + STRONG_ACTION_VERBS

    try:
        parsed, used = await _run_json(IMPROVE_SYSTEM, prompt, _BulletSchema)
    except Exception:
        # Graceful local degradation when the model chain is unavailable.
        return ImproveBulletResult(
            original=bullet,
            improved=improve_bullet_point_locally(bullet, role),
            usedModel="local-fallback",
            flags=flags,
        )

    improved = (parsed.improved or "").strip()
    if not improved:
        raise RuntimeError("AI returned an empty rewritten bullet")

    # Merge locally detected flags with AI-reported flags (dedupe by phrase).
    merged: dict[str, PassivePhraseFlag] = {}
    for f in flags:
        merged[f.phrase] = f
    for f in parsed.flags:
        if f.phrase and f.phrase not in merged:
            merged[f.phrase] = PassivePhraseFlag(phrase=f.phrase, suggestion=f.suggestion or "Rewrite with a strong action verb")
    result = ImproveBulletResult(original=bullet, improved=improved, usedModel=used, flags=list(merged.values()))
    return result


class _BulletSchema(BaseModel):
    improved: str
    flags: list[PassivePhraseFlag] = []
    note: str = ""


# ---------------------------------------------------------------------------
# /api/ai/generate-summary
# ---------------------------------------------------------------------------

SUMMARY_SYSTEM = """You are an expert resume writer. Write a concise, achievement-oriented
professional summary (3-4 sentences). Use only facts present in the input. Do not invent
title history, employers, credentials, or metrics. Output valid JSON with key "summary"."""


async def generate_summary(raw_input: str, target_role: str | None = None) -> GenerateSummaryResult:
    prompt = f'Write a professional summary strictly based on this input:\n\n"{raw_input}"'
    if target_role:
        prompt += f"\n\nTarget role: {target_role}"
    prompt += '\n\nReturn JSON: {"summary": "<3-4 sentences>"}'
    parsed, used = await _run_json(SUMMARY_SYSTEM, prompt, _SummarySchema)
    if not parsed.summary.strip():
        raise RuntimeError("AI returned an empty summary")
    return GenerateSummaryResult(summary=parsed.summary.strip(), usedModel=used)


class _SummarySchema(BaseModel):
    summary: str


# ---------------------------------------------------------------------------
# /api/ai/suggest-skills
# ---------------------------------------------------------------------------

SKILLS_SYSTEM = """You are a career advisor. Return relevant skills for a role as a JSON object
with key "skills" (array of strings). Prefer canonical names and short, commonly searched
skill labels so ATS keyword matching works. Do not invent certifications."""


async def suggest_skills(target_role: str, current_skills: list[str] | None = None) -> SuggestSkillsResult:
    prompt = f'Suggest 10-15 relevant skills for a "{target_role}" position.'
    if current_skills:
        prompt += f"\n\nCurrent skills already listed (include/refine these if relevant): {', '.join(current_skills)}"
    prompt += '\n\nReturn JSON: {"skills": ["..."],'
    parsed, used = await _run_json(SKILLS_SYSTEM, prompt, _SkillsSchema)
```

```
    return SuggestSkillsResult(skills=[s for s in parsed.skills if s.strip()], usedModel=used)


class _SkillsSchema(BaseModel):
    skills: list[str] = []


# ---------------------------------------------------------------------------
# /api/ai/analyze-jd
# ---------------------------------------------------------------------------

JD_SYSTEM = """You are an ATS keyword-extraction specialist. Analyze the job description and
return a JSON object with exactly these keys:
  "requiredSkills": [..], "preferredSkills": [..], "keywords": [..],
  "hardSkills": [..], "softSkills": [..]
Split skills into hard (technologies, frameworks, tools, platforms) and soft
(methodologies, leadership, communication, agile, project management). Use canonical,
short skill labels."""


async def extract_jd_keywords(jd: str) -> tuple[JDExtraction, str]:
    parsed, used = await _run_json(JD_SYSTEM, f'Analyze this job description:\n\n"{jd}"', JDExtraction)
    return parsed, used


# ---------------------------------------------------------------------------
# /api/ai/tailor-resume  (anti-hallucination enforced)
# ---------------------------------------------------------------------------

TAILOR_SYSTEM = """You are an expert resume writer tailoring an existing resume to a job description.

STRICT RULES (violations are disqualifying):
1. DO NOT invent, add, or imply any technology, tool, framework, certification, company,
   school, degree, title, metric, or project that is NOT already present in the resume JSON.
2. You may only REPHRASE and REORDER the user's existing experience bullets, summary,
   projects, and skills to emphasize keywords from the job description.
3. Only surface a JD keyword as a skill if that skill already exists in the resume, or is an
   exact synonym (e.g. "React.js" -> "React", "K8s" -> "Kubernetes") of an existing skill.
4. Never change companies, positions, dates, institutions, degrees, or certifications.
5. Output valid JSON matching this exact shape and mirroring the input's key names:
   {
     "summary": "<rewritten summary or empty string>",
     "experience": [{"id": "<same as input>", "company": "<unchanged>", "position": "<unchanged>",
                     "location": "<same>", "startDate": "<same>", "endDate": "<same>",
                     "current": <same>, "bullets": ["<rephrased bullets only>"]}],
     "skills": [{"id": "<same-optional>", "name": "<existing or synonym only>"}]
   }
Return ONLY the JSON."""


async def tailor_full_resume(resume_json: dict, jd: str) -> TailorResult:
    original = ResumeData.model_validate(resume_json)

    original_companies = {e.company.strip().lower() for e in original.experience if e.company.strip()}
    original_institutions = {e.institution.strip().lower() for e in original.education if e.institution.strip()}
    original_skill_names = {s.name.strip().lower() for s in original.skills if s.name.strip()}
    _ = original_institutions

    prompt = f"""Tailor this resume to the job description below.

RESUME JSON:
{json.dumps(resume_json, indent=2, ensure_ascii=False)}

JOB DESCRIPTION:
"{jd}"
"""
    parsed, used = await _run_json(TAILOR_SYSTEM, prompt, _TailorSchema)

    warnings: list[str] = []
    # Factual entity layer is NEVER rewritten: restore contact, education, projects, certifications.
    tailored = original.model_copy(deep=True)
    tailored.summary = (parsed.summary or original.summary or "").strip() or original.summary

    # --- Experience: keep companies/positions/dates; only accept rephrased bullets ---
    orig_by_id = {e.id: e for e in original.experience}
    rebuilt_experience = []
    for ai_exp in parsed.experience:
        src = orig_by_id.get(ai_exp.id)
        if src is not None:
            if ai_exp.company.strip().lower() and ai_exp.company.strip().lower() not in original_companies:
                warnings.append(f'Removed fabricated company "{ai_exp.company}" from experience')
                ai_exp.company = src.company
            src.company = ai_exp.company
            if ai_exp.position:
                src.position = ai_exp.position
            src.bullets = [b for b in ai_exp.bullets if b.strip()] or src.bullets
            rebuilt_experience.append(src)
        else:
            # Model returned an entry with no id — only accept if company matches an original.
            if ai_exp.company.strip().lower() in original_companies:
                src = next((e for e in original.experience if e.company.strip().lower() == ai_exp.company.strip().lower()), None)
                if src is not None and ai_exp.bullets:
                    src.bullets = [b for b in ai_exp.bullets if b.strip()]
```

```
                    rebuilt_experience.append(src)
            else:
                warnings.append(f'Ignored invented experience entry "{ai_exp.company}"')
    # Preserve any original experiences the model dropped.
    used_src_ids = {e.id for e in rebuilt_experience}
    for e in original.experience:
        if e.id not in used_src_ids and e not in rebuilt_experience:
            rebuilt_experience.append(e)
    tailored.experience = rebuilt_experience

    # --- Skills: keep original only; the model may reorder or rename to synonyms ---
    accepted_skills = []
    for s in parsed.skills:
        name = s.name.strip()
        if not name:
            continue
        if name.lower() in original_skill_names or any(
            alias == name.lower() for alias in original_skill_names
        ):
            accepted_skills.append(s)
        else:
            warnings.append(f'Dropped skill "{name}" not present in the original resume (anti-hallucination)')

    final_skills: list[Any] = []
    seen: set[tuple[str, str]] = set()
    for s in list(original.skills) + accepted_skills:
        key = (s.id, s.name.strip().lower())
        if key not in seen:
            seen.add(key)
            final_skills.append(s)
    if len(final_skills) < len(original.skills):
        final_skills = list(original.skills)
    tailored.skills = final_skills

    return TailorResult(tailored=tailored, guardrailWarnings=warnings, usedModel=used)


class _TailorExperienceSchema(BaseModel):
    id: str = ""
    company: str = ""
    position: str = ""
    location: str = ""
    startDate: str = ""
    endDate: str = ""
    current: bool = False
    bullets: list[str] = []


class _TailorSkillSchema(BaseModel):
    id: str = ""
    name: str = ""


class _TailorSchema(BaseModel):
    summary: str = ""
    experience: list[_TailorExperienceSchema] = []
    skills: list[_TailorSkillSchema] = []


# ---------------------------------------------------------------------------
# Local degrade path (used when every model/network path fails)
# ---------------------------------------------------------------------------


def improve_bullet_point_locally(bullet: str, role: str | None = None) -> str:
    """Instant, no-network fallback: strip passive phrases and prefix a strong verb."""
    out = bullet.strip()
    low = out.lower()
    for phrase, verb in PASSIVE_PHRASES:
        idx = low.find(phrase)
        if idx != -1:
            out = f"{out[:idx].rstrip()} {verb} ".strip() + out[idx + len(phrase):].lstrip()
            if out and out[0].islower():
                out = out[0].upper() + out[1:]
            return out.strip()
    return out
```

86. backend\app\services\ats_scorer.py

```
import random
import re
import string
from app.schemas.resume import ATSAnalysis, ATSIssue, ResumeData

STANDARD_SECTIONS = ["Experience", "Education", "Skills", "Summary", "Projects", "Certifications"]

# Canonical headings the ATS parser recognizes; anything heading-like that is
# NOT one of these is flagged as a non-standard header.
STANDARD_HEADINGS = {
    "summary",
    "professional summary",
    "profile",
    "experience",
    "work experience",
    "employment",
    "work history",
    "education",
    "academic background",
    "skills",
    "technical skills",
    "core competencies",
    "projects",
    "personal projects",
    "professional projects",
    "certifications",
    "certificates",
    "licenses",
    "publications",
    "awards",
    "honors",
    "volunteer",
    "volunteering",
    "volunteer experience",
    "languages",
    "interests",
}

_MONTH_NAMES = (
    "january", "february", "march", "april", "may", "june",
    "july", "august", "september", "october", "november", "december",
)

DATE_FORMAT_PATTERN = re.compile(
    r"^(?:"                                          # either...
    r"(?P<m>\d{1,2})[/\\-](?P<y>\d{4})"            # MM/YYYY, M-YYYY
    r"|(?P<mn>[a-zA-Z]+)\.?\s+(?P<yn>\d{4})"       # Month YYYY / Mon. YYYY
    r"|(?P<yr>\d{4}))$",                       # bare year
    re.IGNORECASE,
)

HEADING_LIKE_LINE = re.compile(r"^[A-Z0-9][A-Za-z0-9 .\-&()/'+]+$")


def _generate_id() -> str:
    return ''.join(random.choices(string.ascii_lowercase + string.digits, k=8))


# ---------------------------------------------------------------------------
# Header & date standardizers (Step 3.3)
# ---------------------------------------------------------------------------


def normalize_date(value: str) -> dict:
    """Validate a resume date string. Returns status + recommended format."""
    value = (value or "").strip()
    if not value:
        return {"status": "empty", "recommended": ""}
    match = DATE_FORMAT_PATTERN.match(value)
    if match and match.group("m") and match.group("y"):
        return {"status": "valid", "recommended": f"{int(match.group('m')):02d}/{match.group('y')}"}
    if match and match.group("mn") and match.group("yn"):
        month = match.group("mn").strip(".").lower()
        if month in _MONTH_NAMES or month[:3] in _MONTH_NAMES:
            return {"status": "valid", "recommended": f"{month[:3].title()} {match.group('yn')}"}
        return {"status": "invalid", "recommended": "MM/YYYY"}
    if match and match.group("yr"):
        return {"status": "year_only", "recommended": "MM/YYYY"}
    return {"status": "invalid", "recommended": "MM/YYYY"}


def check_dates(dates: list[str], category: str) -> tuple[int, list[ATSIssue]]:
    """Returns (deduction, issues) for irregular formats. Skips empty values."""
    deduction = 0
    issues: list[ATSIssue] = []
    for value in dates:
        info = normalize_date(value)
        if info["status"] == "empty":
            continue
        if info["status"] in ("invalid", "year_only"):
```

```
            issues.append(ATSIssue(
                id=_generate_id(), severity="warning", category=category,
                message=f'Date "{value}" is not in a standard format (use MM/YYYY or Month YYYY, e.g. "06/2022" or "June 2022")',
                fix=f'Change to "{info["recommended"]}" so ATS parsers date-shift correctly',
            ))
            # Only deduct once per category bucket to avoid double-penalizing.
            if not any(i.category == category and "irregular date" in i.message for i in issues[1:]):
                pass
            deduction = max(deduction, 5)
    return deduction, issues


def detect_nonstandard_headers(text: str, detected_sections: list[str]) -> list[str]:
    """Heuristic scan for heading-like lines that are not standard resume headers."""
    offenders: list[str] = []
    for line in text.splitlines():
        stripped = line.strip()
        if not stripped or len(stripped) > 50:
            continue
        if stripped.endswith((".", "!", "?", ",")):
            continue
        if stripped.lower() in STANDARD_HEADINGS:
            continue
        if any(s.lower() == stripped.lower() for s in detected_sections):
            continue
        if not HEADING_LIKE_LINE.match(stripped):
            continue
        # Require heading-ish shape: all-caps, or mostly capitalized words, or ends with ':'
        words = stripped.split()
        if len(words) < 2:
            continue
        all_caps = stripped == stripped.upper() and len(stripped) > 3
        title_case = all(w[:1].isupper() for w in words if w[:1].isalpha())
        if all_caps or title_case or stripped.rstrip().endswith(":"):
            offenders.append(stripped)
    return offenders


# ---------------------------------------------------------------------------
# Structured resume scoring
# ---------------------------------------------------------------------------


def analyze_ats_score(data: ResumeData) -> ATSAnalysis:
    issues: list[ATSIssue] = []
    passed_checks: list[str] = []
    score = 100

    if data.contact.fullName:
        passed_checks.append("Contact information present")
    else:
        issues.append(ATSIssue(id=_generate_id(), severity="error", category="Contact", message="Full name is missing", fix="Add your full
name at the top of the resume"))
        score -= 10

    if data.contact.email:
        passed_checks.append("Email address present")
    else:
        issues.append(ATSIssue(id=_generate_id(), severity="error", category="Contact", message="Email address is missing", fix="Add a
professional email address"))
        score -= 10

    if data.contact.phone:
        passed_checks.append("Phone number present")
    else:
        issues.append(ATSIssue(id=_generate_id(), severity="error", category="Contact", message="Phone number is missing", fix="Add your phone
number with area code"))
        score -= 5

    if data.summary:
        passed_checks.append("Professional summary present")
    else:
        issues.append(ATSIssue(id=_generate_id(), severity="warning", category="Summary", message="Professional summary is missing", fix="Add
a 3-4 sentence summary highlighting your key qualifications"))
        score -= 10

    if data.experience:
        passed_checks.append("Work Experience section present")
        has_empty = any(len(exp.bullets) == 0 for exp in data.experience)
        if has_empty:
            issues.append(ATSIssue(id=_generate_id(), severity="warning", category="Experience", message="Some work experience entries have no
bullet points", fix="Add at least 3-5 bullet points for each role describing achievements"))
            score -= 5
        has_dates = all(exp.startDate for exp in data.experience)
        if not has_dates:
            issues.append(ATSIssue(id=_generate_id(), severity="error", category="Experience", message="Some work experience entries are
missing dates", fix="Add start and end dates for each position"))
            score -= 5

        exp_dates = [d for exp in data.experience for d in (exp.startDate, exp.endDate) if d]
        date_deduction, date_issues = check_dates(exp_dates, "Dates")
        issues.extend(date_issues)
        score -= date_deduction
```

```
    else:
        issues.append(ATSIssue(id=_generate_id(), severity="error", category="Experience", message="Work Experience section is missing or
empty", fix="Add your work history with relevant roles and achievements"))
        score -= 15

    if data.education:
        passed_checks.append("Education section present")
        edu_dates = [d for edu in data.education for d in (edu.startDate, edu.endDate) if d]
        date_deduction, date_issues = check_dates(edu_dates, "Dates")
        issues.extend(date_issues)
        score -= date_deduction
    else:
        issues.append(ATSIssue(id=_generate_id(), severity="warning", category="Education", message="Education section is missing", fix="Add
your educational background including degrees and institutions"))
        score -= 10

    if data.skills:
        passed_checks.append("Skills section present")
    else:
        issues.append(ATSIssue(id=_generate_id(), severity="warning", category="Skills", message="Skills section is missing", fix="Add
relevant technical and soft skills"))
        score -= 10

    if data.projects:
        passed_checks.append("Projects section present")

    if data.certifications:
        passed_checks.append("Certifications section present")

    total_bullets = sum(len(exp.bullets) for exp in data.experience)
    if total_bullets > 0:
        has_quantified = any(any(c.isdigit() for c in b) for exp in data.experience for b in exp.bullets)
        if not has_quantified:
            issues.append(ATSIssue(id=_generate_id(), severity="warning", category="Content", message="No quantified achievements found in
experience bullets", fix='Add numbers, percentages, or metrics to demonstrate impact (e.g., "Increased sales by 20%")'))
            score -= 5
        else:
            passed_checks.append("Quantified achievements present")

    # Passive phrase flagging (Step 4 UX: catch weak verbs)
    passive = ("responsible for", "helped with", "worked on", "duties included", "in charge of", "was tasked")
    weak_bullets = [b for exp in data.experience for b in exp.bullets if any(p in b.lower() for p in passive)]
    if weak_bullets:
        issues.append(ATSIssue(id=_generate_id(), severity="warning", category="Content", message="Some bullets use passive phrasing that
weakens impact", fix='Start bullets with strong action verbs (e.g., "Spearheaded", "Optimized", "Architected")'))
        score -= 5
    else:
        passed_checks.append("Strong action verbs used in bullets")

    if score < 0:
        score = 0

    if score >= 80:
        summary = "Your resume is well-structured and ATS-friendly."
    elif score >= 60:
        summary = "Your resume has some issues that may affect ATS parsing."
    else:
        summary = "Your resume needs significant improvements for ATS compatibility."

    return ATSAnalysis(score=score, issues=issues, passedChecks=passed_checks, summary=summary)


def analyze_ats_score_from_text(
    text: str,
    detected_sections: list[str],
    has_tables: bool | None = None,
    single_column: bool | None = None,
) -> ATSAnalysis:
    issues: list[ATSIssue] = []
    passed_checks: list[str] = []
    score = 100

    for section in STANDARD_SECTIONS:
        found = any(s.lower() == section.lower() for s in detected_sections)
        if found:
            passed_checks.append(f'"{section}" section detected')
        else:
            issues.append(ATSIssue(id=_generate_id(), severity="warning", category="Sections", message=f"Missing standard section: {section}",
fix=f'Add a "{section}" section with appropriate heading'))
            score -= 10

    if len(text) < 200:
        issues.append(ATSIssue(id=_generate_id(), severity="error", category="Content", message="Resume content is too short", fix="Add more
detailed content to your resume"))
        score -= 10
    else:
        passed_checks.append("Resume has sufficient content length")

    # Header standardizer: flag heading-like lines outside the canonical set.
    nonstandard = detect_nonstandard_headers(text, detected_sections)
    if nonstandard:
        issues.append(ATSIssue(
            id=_generate_id(), severity="warning", category="Sections",
```

```
            message=f'Non-standard section header(s) found: {", ".join(nonstandard[:3])}',
            fix='Rename non-standard headings to standard ones like "Experience", "Education", "Skills", "Summary" so ATS section detection
works',
        ))
        score -= 5
    else:
        passed_checks.append("Standard section headers only")

    if has_tables is not None:
        if has_tables:
            issues.append(ATSIssue(id=_generate_id(), severity="warning", category="Layout", message="Tables detected in resume", fix="Replace
tables with standard text sections - ATS parsers often fail to read table content"))
            score -= 5
        else:
            passed_checks.append("No tables detected (text-based resume)")

    if single_column is not None:
        if single_column:
            passed_checks.append("Single column layout (good for ATS)")
        else:
            issues.append(ATSIssue(id=_generate_id(), severity="warning", category="Layout", message="Multi-column layout detected", fix="Use
a single-column layout so ATS parsers read content in the correct order"))
            score -= 5

    if score < 0:
        score = 0

    if score >= 80:
        summary = "Your resume has all standard sections and is ATS-friendly."
    elif score >= 60:
        summary = "Your resume is missing some standard sections."
    else:
        summary = "Your resume is missing critical sections needed for ATS parsing."

    return ATSAnalysis(score=score, issues=issues, passedChecks=passed_checks, summary=summary)
```

87. backend\app\services\export_service.py

```
"""Export service.

PDF is produced from an ATS-friendly, single-column HTML+CSS template using a
headless rendering pipeline:

1. WeasyPrint (Linux / Render, best CSS fidelity, selectable text)
2. Playwright + Chromium (fallback for macOS/Windows where system GTK libs
   for WeasyPrint are unavailable)

Both produce standard A4/Letter PDFs with 0.5" margins, web-safe fonts, and a
real text layer (no flattened canvases), which ATS parsers can read reliably.
"""

import io
import logging
from jinja2 import Template
from app.schemas.resume import ResumeData

logger = logging.getLogger("uvicorn.error")

# ATS-compliant template: single column, web-safe fonts, standard 0.55in margins,
# predictable line boxes (no absolute coordinates, no flex tables that ATS breaks on).
RESUME_TEMPLATE = """<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<style>
  @page {
    size: Letter;
    margin: 0.55in 0.6in 0.55in 0.6in;
  }
  * { box-sizing: border-box; }
  html, body { margin: 0; padding: 0; }
  body {
    font-family: Arial, Helvetica, sans-serif;
    font-size: 10pt;
    line-height: 1.35;
    color: #1a1a1a;
  }
  .header { text-align: center; border-bottom: 1px solid #333; padding-bottom: 8px; margin-bottom: 12px; }
  .name { font-size: 22pt; font-weight: 700; margin: 0 0 4px 0; letter-spacing: 0.5px; }
  .contact-row { font-size: 9pt; color: #3a3a3a; margin: 0; }
  .section { margin: 12px 0 6px 0; padding-bottom: 3px; border-bottom: 1px solid #999; }
  .section-title { font-size: 11pt; font-weight: 700; text-transform: uppercase; letter-spacing: 1px; margin: 0; color: #111; }
  .summary { font-size: 10pt; margin: 0 0 6px 0; }
  .exp-item, .edu-item, .proj-item, .cert-item { margin-bottom: 8px; page-break-inside: avoid; }
  .line { display: block; }
  .company { font-size: 10.5pt; font-weight: 700; margin: 0; }
  .role-date { display: block; }
  .role { font-size: 10pt; font-style: italic; margin: 0; }
  .date { font-size: 9pt; color: #555; margin: 0; }
  .bullet { font-size: 10pt; margin: 1px 0 1px 12px; padding-left: 2px; }
  .degree { font-size: 10pt; margin: 0; }
  .skill-text { font-size: 10pt; margin: 0; }
  .proj-name { font-size: 10.5pt; font-weight: 700; margin: 0; display: block; }
  .proj-desc { font-size: 10pt; margin: 1px 0; }
  .proj-tech { font-size: 9pt; color: #555; margin: 0; }
  .cert-name { font-size: 10.5pt; font-weight: 700; margin: 0; display: block; }
  .cert-issuer { font-size: 9pt; color: #555; margin: 0; }
  .keep-together { page-break-inside: avoid; }
</style>
</head>
<body>
  <div class="header">
    <h1 class="name">{{ contact.fullName }}</h1>
    {% if contact.email or contact.phone or contact.location %}
    <p class="contact-row">{{ [contact.email, contact.phone, contact.location] | select | join(' | ') }}</p>
    {% endif %}
    {% if contact.linkedin or contact.website %}
    <p class="contact-row">{{ [contact.linkedin, contact.website] | select | join(' | ') }}</p>
    {% endif %}
  </div>

  {% if summary %}
  <div class="keep-together">
    <div class="section"><h2 class="section-title">Professional Summary</h2></div>
    <p class="summary">{{ summary }}</p>
  </div>
  {% endif %}

  {% if experience %}
  <div class="keep-together">
    <div class="section"><h2 class="section-title">Experience</h2></div>
    {% for exp in experience %}
    <div class="exp-item">
      <p class="company">{{ exp.company }}</p>
      <span class="role-date">
        <span class="role">{{ exp.position }}{% if exp.location %}{{ ' | ' + exp.location }}{% endif %}</span>
        <span class="date">{{ exp.startDate + ' - ' + ('Present' if exp.current else exp.endDate) }}</span>
      </span>
```

```
      {% for bullet in exp.bullets %}{% if bullet.strip() %}<p class="bullet">&bull; {{ bullet }}</p>{% endif %}{% endfor %}
    </div>
    {% endfor %}
  </div>
  {% endif %}

  {% if education %}
  <div class="keep-together">
    <div class="section"><h2 class="section-title">Education</h2></div>
    {% for edu in education %}
    <div class="edu-item">
      <p class="company">{{ edu.institution }}</p>
      <p class="degree">{{ edu.degree }}{% if edu.field %}{{ ' in ' + edu.field }}{% endif %}{% if edu.gpa %}{{ ' | GPA: ' + edu.gpa }}{%
endif %}</p>
      {% if edu.startDate or edu.endDate %}<p class="date">{{ [edu.startDate, edu.endDate] | select | join(' - ') }}</p>{% endif %}
    </div>
    {% endfor %}
  </div>
  {% endif %}

  {% if skills %}
  <div class="keep-together">
    <div class="section"><h2 class="section-title">Skills</h2></div>
    <p class="skill-text">{{ skills | map(attribute='name') | join(', ') }}</p>
  </div>
  {% endif %}

  {% if projects %}
  <div class="keep-together">
    <div class="section"><h2 class="section-title">Projects</h2></div>
    {% for proj in projects %}
    <div class="proj-item">
      <span class="proj-name">{{ proj.name }}{% if proj.url %}  <span style="font-weight:400;color:#2b5a9c">{{ proj.url }}</span>{% endif
%}</span>
      {% if proj.description %}<p class="proj-desc">{{ proj.description }}</p>{% endif %}
      {% if proj.technologies %}<p class="proj-tech">Technologies: {{ proj.technologies | join(', ') }}</p>{% endif %}
    </div>
    {% endfor %}
  </div>
  {% endif %}

  {% if certifications %}
  <div class="keep-together">
    <div class="section"><h2 class="section-title">Certifications</h2></div>
    {% for cert in certifications %}
    <div class="cert-item">
      <span class="cert-name">{{ cert.name }}</span>
      <p class="cert-issuer">{{ [cert.issuer, cert.date] | select | join(' | ') }}</p>
    </div>
    {% endfor %}
  </div>
  {% endif %}
</body>
</html>"""


def render_resume_html(data: ResumeData) -> str:
    template = Template(RESUME_TEMPLATE)
    return template.render(**data.model_dump())


async def _pdf_with_weasyprint(html: str) -> bytes | None:
    try:
        from weasyprint import HTML

        return bytes(HTML(string=html).write_pdf())
    except Exception as e:  # noqa: BLE001
        logger.warning("WeasyPrint unavailable (%s); trying Playwright.", e)
        return None


async def _pdf_with_playwright(html: str) -> bytes:
    from playwright.async_api import async_playwright

    async with async_playwright() as p:
        browser = await p.chromium.launch(args=["--no-sandbox"])
        try:
            page = await browser.new_page()
            await page.set_content(html, wait_until="load")
            pdf_bytes = await page.pdf(
                format="Letter",
                print_background=True,
                margin={"top": "0.55in", "bottom": "0.55in", "left": "0.6in", "right": "0.6in"},
            )
            return pdf_bytes
        finally:
            await browser.close()


async def generate_pdf(data: ResumeData) -> bytes:
    html = render_resume_html(data)

    pdf = await _pdf_with_weasyprint(html)
```

```
    if pdf is not None and pdf.startswith(b"%PDF"):
        return pdf

    try:
        return await _pdf_with_playwright(html)
    except Exception as e:  # noqa: BLE001
        logger.exception("All PDF renderers failed")
        raise RuntimeError(f"PDF rendering backend failed: {e}") from e


def render_resume_plain_text(data: ResumeData) -> str:
    """Clean, copy-pasteable plain text for legacy ATS text boxes."""
    lines: list[str] = []

    if data.contact.fullName:
        lines.append(data.contact.fullName)
    contact = " | ".join(
        x
        for x in [
            data.contact.email,
            data.contact.phone,
            data.contact.location,
            data.contact.linkedin,
            data.contact.website,
        ]
        if x
    )
    if contact:
        lines.append(contact)

    if data.summary:
        lines.append(f"\nSUMMARY\n{data.summary}")

    if data.experience:
        lines.append("\nEXPERIENCE")
        for exp in data.experience:
            header = " | ".join(x for x in [exp.company, exp.position, exp.location] if x)
            dates = f"{exp.startDate} - {'Present' if exp.current else exp.endDate}"
            lines.append(f"{header} ({dates})")
            for b in exp.bullets:
                if b.strip():
                    lines.append(f"- {b}")

    if data.education:
        lines.append("\nEDUCATION")
        for edu in data.education:
            degree = " ".join(x for x in [edu.degree, "in", edu.field] if x and x != "in")
            parts = [p for p in [edu.institution, degree or None, f"{edu.startDate} - {edu.endDate}"] if p]
            lines.append(" - ".join([edu.institution, degree]) if degree else edu.institution)

    if data.skills:
        lines.append("\nSKILLS")
        lines.append(", ".join(s.name for s in data.skills if s.name))

    if data.projects:
        lines.append("\nPROJECTS")
        for proj in data.projects:
            title = proj.name + (f" ({proj.url})" if proj.url else "")
            lines.append(title)
            if proj.description:
                lines.append(proj.description)
            if proj.technologies:
                lines.append(f"Technologies: {', '.join(proj.technologies)}")

    if data.certifications:
        lines.append("\nCERTIFICATIONS")
        for cert in data.certifications:
            lines.append(" - ".join(x for x in [cert.name, cert.issuer, cert.date] if x))

    return "\n".join(lines)


def generate_docx(data: ResumeData) -> bytes:
    from docx import Document
    from docx.shared import Pt

    doc = Document()

    if data.contact.fullName:
        doc.add_heading(data.contact.fullName, level=0).alignment = 1

    contact_parts = [p for p in [data.contact.email, data.contact.phone, data.contact.location] if p]
    if contact_parts:
        doc.add_paragraph(" | ".join(contact_parts)).alignment = 1

    links = [p for p in [data.contact.linkedin, data.contact.website] if p]
    if links:
        doc.add_paragraph(" | ".join(links)).alignment = 1

    def add_section(title: str):
        doc.add_heading(title, level=1)

    if data.summary:
```

```
    add_section("Professional Summary")
    doc.add_paragraph(data.summary)

if data.experience:
    add_section("Experience")
    for exp in data.experience:
        p = doc.add_paragraph()
        run = p.add_run(exp.company)
        run.bold = True
        if exp.position:
            p.add_run(f"  |  {exp.position}").font.italic = True
        date_str = f"{exp.startDate} - {'Present' if exp.current else exp.endDate}"
        if exp.location or date_str.strip(" -"):
            sub = doc.add_paragraph(f"{exp.location}{' | ' if exp.location else ''}{date_str}")
            sub.runs[0].font.size = Pt(9)
        for bullet in exp.bullets:
            if bullet.strip():
                doc.add_paragraph(bullet, style="List Bullet")

if data.education:
    add_section("Education")
    for edu in data.education:
        p = doc.add_paragraph()
        p.add_run(edu.institution).bold = True
        gpa_str = f" | GPA: {edu.gpa}" if edu.gpa else ""
        doc.add_paragraph(f"{' '.join(x for x in [edu.degree, 'in', edu.field] if x)}{gpa_str} | {edu.startDate} - {edu.endDate}")

if data.skills:
    add_section("Skills")
    doc.add_paragraph(", ".join(s.name for s in data.skills if s.name))

if data.projects:
    add_section("Projects")
    for proj in data.projects:
        p = doc.add_paragraph()
        p.add_run(proj.name).bold = True
        if proj.url:
            p.add_run(f"  |  {proj.url}").font.size = Pt(9)
        if proj.description:
            doc.add_paragraph(proj.description)
        if proj.technologies:
            sub = doc.add_paragraph(f"Technologies: {', '.join(proj.technologies)}")
            sub.runs[0].font.size = Pt(9)

if data.certifications:
    add_section("Certifications")
    for cert in data.certifications:
        p = doc.add_paragraph()
        p.add_run(cert.name).bold = True
        if cert.issuer or cert.date:
            doc.add_paragraph(" | ".join(x for x in [cert.issuer, cert.date] if x))

buf = io.BytesIO()
doc.save(buf)
return buf.getvalue()
```

88. backend\app\services\resume_parser.py

```
import io
import re
import pdfplumber
from docx import Document


SECTION_HEADINGS = [
    "summary", "professional summary", "profile",
    "experience", "work experience", "employment", "work history",
    "education", "academic background",
    "skills", "technical skills", "core competencies",
    "projects", "personal projects", "professional projects",
    "certifications", "certificates", "licenses",
    "publications", "awards", "honors",
    "volunteer", "volunteering", "volunteer experience",
    "languages", "interests",
]


def parse_resume_file(file_bytes: bytes, mime_type: str) -> tuple[str, dict[str, str], dict]:
    text = ""
    layout: dict = {"hasTables": False, "singleColumn": None}
    if "pdf" in mime_type:
        text, layout = _parse_pdf(file_bytes)
    elif "wordprocessingml" in mime_type or "docx" in mime_type:
        text = _parse_docx(file_bytes)
    else:
        raise ValueError("Unsupported file type. Please upload a PDF or DOCX file.")

    sections = extract_sections(text)
    return text, sections, layout


def _parse_pdf(file_bytes: bytes) -> tuple[str, dict]:
    text = ""
    has_tables = False
    x_starts: list[float] = []
    with pdfplumber.open(io.BytesIO(file_bytes)) as pdf:
        for page in pdf.pages:
            page_text = page.extract_text()
            if page_text:
                text += page_text + "\n"
            if page.find_tables():
                has_tables = True
            words = page.extract_words()
            x_starts.extend(w["x0"] for w in words)

    single_column = None
    if x_starts:
        ordered = sorted(x_starts)
        median = ordered[len(ordered) // 2]
        far_from_median = sum(1 for x in x_starts if abs(x - median) > 40)
        single_column = (far_from_median / len(x_starts)) < 0.05

    return text, {"hasTables": has_tables, "singleColumn": single_column}


def _parse_docx(file_bytes: bytes) -> str:
    doc = Document(io.BytesIO(file_bytes))
    text = "\n".join(p.text for p in doc.paragraphs)
    return text


def extract_sections(text: str) -> dict[str, str]:
    sections: dict[str, str] = {}
    lines = text.split("\n")
    current_section = "header"
    current_content: list[str] = []

    for line in lines:
        trimmed = line.strip()
        if not trimmed:
            continue

        heading_match = None
        lower = trimmed.lower()
        for h in SECTION_HEADINGS:
            if lower == h:
                heading_match = h
                break
            if lower.startswith(h):
                remainder = lower[len(h):]
                if remainder and remainder[0] in " :,;|-–\t":
                    heading_match = h
                    break

        if heading_match:
            if current_content:
                sections[current_section] = "\n".join(current_content).strip()
            current_section = heading_match
```

```
            current_content = []
        else:
            current_content.append(trimmed)

    if current_content:
        key = current_section
        if key in sections:
            sections[key] += "\n" + "\n".join(current_content)
        else:
            sections[key] = "\n".join(current_content).strip()

    return sections


def detect_sections(text: str) -> list[str]:
    detected = []
    lower_text = text.lower()
    for heading in SECTION_HEADINGS:
        if re.search(rf"\b{re.escape(heading)}\b", lower_text):
            formatted = heading[0].upper() + heading[1:]
            if formatted not in detected:
                detected.append(formatted)
    return detected
```

89. backend\app\services\skill_synonyms.py

```
"""Semantic skill matching.

Provides alias/synonym resolution (e.g. "React" == "React.js", "K8s" == "Kubernetes")
and categorization of skills into hard (technologies/tools) vs soft (methodologies).
"""

import re

# canonical name -> aliases (all matched case-insensitively)
SKILL_CANONICAL: dict[str, set[str]] = {
    # Frontend
    "React": {"react", "react.js", "reactjs", "react.js/js"},
    "React Native": {"react native", "reactnative"},
    "Next.js": {"next.js", "nextjs", "next"},
    "Vue.js": {"vue", "vue.js", "vuejs"},
    "Angular": {"angular", "angularjs", "angular 2+"},
    "TypeScript": {"typescript", "ts"},
    "JavaScript": {"javascript", "js", "es6", "ecmascript"},
    "HTML": {"html", "html5"},
    "CSS": {"css", "css3", "sass", "scss", "less", "styled-components", "tailwind", "tailwindcss"},
    "Redux": {"redux", "redux-toolkit"},
    "GraphQL": {"graphql", "apollo", "apollo client"},
    "Webpack": {"webpack", "vite", "rollup"},
    "Flutter": {"flutter", "dart/flutter"},
    "Swift": {"swift", "swiftui"},
    # Backend
    "Python": {"python", "python3", "py"},
    "FastAPI": {"fastapi", "fast api"},
    "Django": {"django", "django rest framework", "drf"},
    "Flask": {"flask"},
    "Node.js": {"node", "node.js", "nodejs"},
    "Go": {"go", "golang"},
    "Rust": {"rust", "rustlang"},
    "Java": {"java", "jdk"},
    "Kotlin": {"kotlin"},
    "C++": {"c++", "cplusplus", "cpp"},
    "C#": {"c#", "csharp", ".net", "dotnet", "asp.net"},
    "PHP": {"php", "laravel", "symfony"},
    "Ruby": {"ruby", "rails", "ruby on rails"},
    "Express": {"express", "express.js", "expressjs"},
    "NestJS": {"nestjs", "nest js"},
    # Databases
    "PostgreSQL": {"postgresql", "postgres", "psql", "supabase postgres"},
    "MySQL": {"mysql"},
    "SQLite": {"sqlite", "sqlite3"},
    "MongoDB": {"mongodb", "mongo", "mongoose"},
    "Redis": {"redis"},
    "SQL": {"sql", "structured query language", "t-sql", "pl/sql"},
    "Elasticsearch": {"elasticsearch", "elastic"},
    "DynamoDB": {"dynamodb", "aws dynamodb"},
    "Supabase": {"supabase", "supabase auth", "supabase storage"},
    "Firebase": {"firebase", "firestore", "firebase cloud functions"},
    "Prisma": {"prisma", "prisma orm"},
    "SQLAlchemy": {"sqlalchemy", "sql alchemy"},
    # Cloud / DevOps
    "AWS": {"aws", "amazon web services", "ec2", "s3", "lambda", "amazon s3"},
    "GCP": {"gcp", "google cloud", "google cloud platform", "gce"},
    "Azure": {"azure", "microsoft azure"},
    "Docker": {"docker", "docker compose", "docker-compose"},
    "Kubernetes": {"kubernetes", "k8s", "k3s", "eks", "gke", "aks"},
    "CI/CD": {"ci/cd", "ci cd", "jenkins", "github actions", "gitlab ci", "circleci", "travis"},
    "Terraform": {"terraform", "iac", "infrastructure as code"},
    "Git": {"git", "github", "gitlab", "bitbucket"},
    "Nginx": {"nginx", "caddy"},
    "Linux": {"linux", "unix", "ubuntu", "centos", "redhat"},
    "Nginx": {"nginx"},
    "Serverless": {"serverless", "lambda functions"},
    # Data / AI
    "pandas": {"pandas", "dataframes"},
    "NumPy": {"numpy", "np"},
    "Machine Learning": {"machine learning", "ml", "mlops"},
    "Deep Learning": {"deep learning", "dl", "cnn", "rnn", "transformers", "neural networks"},
    "TensorFlow": {"tensorflow", "tf", "keras"},
    "PyTorch": {"pytorch", "torch"},
    "LLMs": {"llm", "llms", "large language models", "gpt", "openai", "fine-tuning", "fine tuning"},
    "LangChain": {"langchain", "langchain4j"},
    "Prompt Engineering": {"prompt engineering", "prompt design"},
    "Airflow": {"airflow", "apache airflow"},
    "Spark": {"spark", "apache spark", "pyspark"},
    "Apache Kafka": {"kafka", "apache kafka"},
    "BigQuery": {"bigquery", "google bigquery"},
    "Snowflake": {"snowflake"},
    # Testing
    "Jest": {"jest"},
    "Playwright": {"playwright", "playwright test"},
    "pytest": {"pytest", "py test"},
    "JUnit": {"junit"},
    "Selenium": {"selenium"},
    "Cypress": {"cypress"},
    "TDD": {"tdd", "test driven development"},
```

```
    # Mobile
    "Expo": {"expo", "expo react native", "expo go"},
    # Other tools
    "Figma": {"figma"},
    "Jira": {"jira", "jira/atlassian"},
    "Agile": {"agile", "scrum", "kanban", "sprint planning", "agile methodology"},
    "AWS": {"aws", "amazon web services"},
    # Soft skills / methodologies
    "Communication": {"communication", "written communication", "verbal communication", "presentation skills"},
    "Team Leadership": {"leadership", "team leadership", "mentoring", "leading teams", "people management"},
    "Problem Solving": {"problem solving", "problem-solving", "analytical thinking", "critical thinking"},
    "Project Management": {"project management", "pm", "pmp", "project planning", "roadmapping"},
    "Stakeholder Management": {"stakeholder management", "stakeholder engagement", "stakeholders"},
    "Cross-functional Collaboration": {"cross-functional", "cross functional collaboration", "collaboration", "teamwork"},
    "Product Management": {"product management", "product strategy", "roadmap", "go-to-market"},
    "Time Management": {"time management", "prioritization", "prioritizing"},
    "Mentorship": {"mentorship", "coaching", "mentoring junior"},
    "Microservices": {"microservices", "micro-service", "microservice architecture", "soa"},
    "REST": {"rest", "restful", "rest api", "rest apis"},
    "Event-driven Architecture": {"event-driven", "event driven architecture", "event sourcing"},
    "OOP": {"oop", "object oriented", "object-oriented programming"},
    "System Design": {"system design", "systems design", "high-level design", "large scale systems"},
    "Unit Testing": {"unit testing", "unit tests"},
    "Code Review": {"code review", "code reviews"},
    "API Design": {"api design", "api development", "api integration"},
}

# Categorization of canonical names.
SOFT_SKILLS = {
    "Communication",
    "Team Leadership",
    "Problem Solving",
    "Project Management",
    "Stakeholder Management",
    "Cross-functional Collaboration",
    "Product Management",
    "Time Management",
    "Mentorship",
}

# Reverse lookup: every alias (lowercased) -> canonical name
_ALIAS_INDEX: dict[str, str] = {}
for _canonical, _aliases in SKILL_CANONICAL.items():
    for _alias in _aliases:
        _ALIAS_INDEX[_alias] = _canonical


def canonical_skill(name: str) -> str:
    """Return the canonical skill name for a given alias, falling back to the input."""
    key = name.strip().lower()
    if key in _ALIAS_INDEX:
        return _ALIAS_INDEX[key]
    return name.strip()


def categorize_skill(name: str) -> str:
    canonical = canonical_skill(name)
    if canonical in SOFT_SKILLS:
        return "soft"
    if canonical in SKILL_CANONICAL:
        return "hard"
    return "uncategorized"


def skill_aliases(name: str) -> list[str]:
    canonical = canonical_skill(name)
    return sorted(SKILL_CANONICAL.get(canonical, {canonical.lower()}))


def _word_bounded(pattern: str) -> re.Pattern:
    return re.compile(rf"(?<![A-Za-z0-9]){re.escape(pattern)}(?![A-Za-z0-9])", re.IGNORECASE)


def match_skill_in_text(skill: str, text: str) -> bool:
    """Check whether a skill (or any of its aliases) appears in the given text."""
    lower_text = text.lower()
    for alias in skill_aliases(skill):
        # Short aliases like "go" need word boundaries to avoid false positives.
        if len(alias) <= 3 and not any(c.isdigit() for c in alias):
            if _word_bounded(alias).search(lower_text):
                return True
        elif alias in lower_text:
            return True
    return False


def categorize_skill_batch(names: list[str]) -> dict[str, dict[str, str]]:
    """Return {raw_name: {"canonical": ..., "category": ...}} for a batch of names."""
    return {
        n: {"canonical": canonical_skill(n), "category": categorize_skill(n)}
        for n in names
    }
```

```
# A compact hint list used by the AI prompts so generations stay on-canonical names.
CANONICAL_SKILL_HINTS = "React, Next.js, Python, FastAPI, PostgreSQL, Kubernetes, Docker, AWS, TypeScript, GraphQL"
```

90. backend\main.py

```
import os
from contextlib import asynccontextmanager
from dotenv import load_dotenv
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from app.api.resume import router as resume_router
from app.api.resumes import router as saved_resumes_router
from app.api.ats import router as ats_router
from app.api.ai_routes import router as ai_router
from app.api.export import router as export_router
from app.database import init_db, close_db, check_db_connection

load_dotenv()


@asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()
    yield
    await close_db()


app = FastAPI(title="AI Resume Builder API", version="2.0.0", lifespan=lifespan)

# Allow the web frontend, Expo development clients, and the EAS-built app scheme.
app.add_middleware(
    CORSMiddleware,
    allow_origins=[
        "http://localhost:3000",
        "http://127.0.0.1:3000",
        *[o.strip() for o in os.getenv("CORS_ALLOWED_ORIGINS", "").split(",") if o.strip()],
    ],
    allow_origin_regex=(
        r"^(exp://|(?:http://|https://)(?:localhost|127\.0\.0\.1)(?::\d+)?$|"
        r"com\.resumebuilder\.app://.*$)"
    ),
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(resume_router)
app.include_router(saved_resumes_router)
app.include_router(ats_router)
app.include_router(ai_router)
app.include_router(export_router)


@app.get("/api/health")
async def health():
    db_ok = await check_db_connection()
    return {
        "status": "ok" if db_ok else "degraded",
        "database": "connected" if db_ok else "unavailable",
        "provider": "postgres" if os.getenv("DATABASE_URL", "").startswith("postgres") else "sqlite",
    }


if __name__ == "__main__":
    import uvicorn

    port = int(os.getenv("PORT", "8000"))
    host = os.getenv("HOST", "0.0.0.0")
    uvicorn.run("main:app", host=host, port=port, reload=True)
```

91. backend\render.yaml

```
services:
  - type: web
    name: ai-resume-builder-api
    runtime: python
    plan: starter
    buildCommand: "pip install --upgrade pip && pip install -r requirements.txt && playwright install --with-deps chromium"
    startCommand: "uvicorn main:app --host 0.0.0.0 --port $PORT"
    healthCheckPath: /api/health
    envVars:
      - key: DATABASE_URL
        fromDatabase:
          name: resume-builder-db
          property: connectionString
      - key: SUPABASE_URL
        sync: false
      - key: SUPABASE_KEY
        sync: false
      - key: OPENROUTER_API_KEY
        sync: false
      - key: OPENROUTER_MODEL
        value: deepseek/deepseek-chat-v3-0324:free
      - key: OPENROUTER_MODEL_FALLBACKS
        sync: false
      - key: CORS_ALLOWED_ORIGINS
        sync: false
      - key: APP_URL
        sync: false
      - key: PYTHON_VERSION
        value: 3.12.8

databases:
  - name: resume-builder-db
    plan: free
    databaseName: resume_builder
    user: resume_builder
```

92. backend\requirements.txt

```
# Web framework
fastapi==0.138.0
uvicorn[standard]==0.49.0
python-multipart==0.0.32

# Database (Supabase PostgreSQL via SQLAlchemy async; SQLite fallback for local dev/tests)
sqlalchemy==2.0.51
asyncpg==0.31.0
aiosqlite==0.22.1

# Auth / config (optional supabase-py for future Supabase Auth integration)
python-dotenv==1.0.1
supabase==2.20.1

# AI
openai==1.99.0
httpx==0.28.1

# Parsing / export
pdfplumber==0.11.10
python-docx==1.2.0
jinja2==3.1.6
# WeasyPrint on Linux/Render produces ATS-clean PDFs from the HTML template.
# On Windows/macOS we fall back to Playwright + Chromium (see export_service.py).
weasyprint==69.0
playwright==1.62.0

# Patterns / validation
pydantic==2.13.4

# Dev / test
pytest==8.4.2
```

93. backend\tests\conftest.py

```
import asyncio

import pytest
from fastapi.testclient import TestClient
from sqlalchemy.ext.asyncio import async_sessionmaker, create_async_engine
from sqlalchemy.pool import StaticPool

from app.database import Base, get_session
from main import app


def _run(coro):
    loop = asyncio.new_event_loop()
    try:
        return loop.run_until_complete(coro)
    finally:
        loop.close()


@pytest.fixture(scope="session")
def db_engine():
    """In-memory SQLite engine shared across the test session."""
    import app.models.resume  # noqa: F401  (register models on Base.metadata)

    engine = create_async_engine("sqlite+aiosqlite:///:memory:", poolclass=StaticPool)

    async def _create_tables():
        async with engine.begin() as conn:
            await conn.run_sync(Base.metadata.create_all)

    _run(_create_tables())
    yield engine

    async def _dispose():
        await engine.dispose()

    _run(_dispose())


@pytest.fixture()
def client(db_engine):
    factory = async_sessionmaker(db_engine, expire_on_commit=False)

    async def override_get_session():
        async with factory() as session:
            yield session

    app.dependency_overrides[get_session] = override_get_session

    with TestClient(app) as c:
        yield c

    app.dependency_overrides.clear()


def make_resume_payload(name: str = "Jane Doe", skills=None) -> dict:
    return {
        "contact": {
            "fullName": name,
            "email": f"{name.split()[0].lower()}@example.com",
            "phone": "555-1234",
            "location": "Remote",
        },
        "summary": "Product manager with 3 years of experience.",
        "experience": [
            {
                "id": "exp1",
                "company": "Tech Co",
                "position": "Product Manager",
                "location": "",
                "startDate": "01/2021",
                "endDate": "01/2024",
                "current": False,
                "bullets": ["Shipped 5 major releases", "Led a team of 4"],
            }
        ],
        "education": [{"id": "edu1", "institution": "State U", "degree": "BA", "field": "Business", "startDate": "09/2017", "endDate":
"05/2021"}],
        "skills": skills
        if skills is not None
        else [{"id": "sk1", "name": "SQL"}, {"id": "sk2", "name": "Figma"}],
        "projects": [],
        "certifications": [],
    }


def unique_user_id(prefix: str = "user") -> str:
    import uuid

    return f"{prefix}-{uuid.uuid4().hex[:8]}"
```

94. backend\tests\test_ai_service.py

```
import asyncio

import httpx
import pytest
from openai import RateLimitError

from app.schemas.resume import ResumeData
from app.services import ai_service
from app.services.skill_synonyms import (
    canonical_skill,
    categorize_skill,
    match_skill_in_text,
)
from app.services.ats_scorer import normalize_date, detect_nonstandard_headers


def run(coro):
    loop = asyncio.new_event_loop()
    try:
        return loop.run_until_complete(coro)
    finally:
        loop.close()


# ---------------------------------------------------------------------------
# Skill semantics
# ---------------------------------------------------------------------------


def test_synonym_canonicalization():
    assert canonical_skill("React.js") == "React"
    assert canonical_skill("K8s") == "Kubernetes"
    assert canonical_skill("Postgres") == "PostgreSQL"


def test_skill_categorization():
    assert categorize_skill("React") == "hard"
    assert categorize_skill("Kubernetes") == "hard"
    assert categorize_skill("Communication") == "soft"
    assert categorize_skill("Something Random 123") == "uncategorized"


def test_match_skill_in_text_with_alias():
    text = "I used React.js and K8s at work."
    assert match_skill_in_text("React", text)
    assert match_skill_in_text("Kubernetes", text)
    assert match_skill_in_text("PostgreSQL", "Postgres is my main db")
    assert not match_skill_in_text("PostgreSQL", "MongoDB is my main db")



# ---------------------------------------------------------------------------
# ATS date & header checks
# ---------------------------------------------------------------------------


def test_date_format_standardizer():
    assert normalize_date("06/2022")["status"] == "valid"
    assert normalize_date("June 2022")["status"] == "valid"
    assert normalize_date("2022")["status"] == "year_only"
    assert normalize_date("June22")["status"] == "invalid"
    assert normalize_date("")["status"] == "empty"


def test_nonstandard_header_detection():
    offenders = detect_nonstandard_headers(
        "SUMMARY\nExperienced.\nTEACHING EXPERIENCE\nTaught classes.\nMy Hobbies\nRunning.",
        ["Summary"],
    )
    assert "TEACHING EXPERIENCE" in offenders
    assert "My Hobbies" in offenders
    assert "SUMMARY" not in offenders


# ---------------------------------------------------------------------------
# AI structured output + guardrails (model is mocked)
# ---------------------------------------------------------------------------


def test_improve_bullet_structured_output_and_flags(monkeypatch):
    improved_text = "Achieved a 20% reduction in support tickets by automating triage workflows."

    async def fake_completion(model, system, prompt, max_tokens=4096, json_mode=False):
        return f'{{"improved": "{improved_text}", "flags": [{{"phrase": "worked on", "suggestion": "Delivered"}}], "note": "ok"}}'

    monkeypatch.setattr(ai_service, "_per_model_completion", fake_completion)

    result = run(ai_service.improve_bullet_point("Worked on support tickets", "Support Engineer"))
    assert result.improved == improved_text
    assert result.usedModel != ""
```

```
    # Local passive-phrase detection should flag the weak phrase too.
    assert any("worked on" in f.phrase for f in result.flags)


def test_improve_bullet_local_fallback(monkeypatch):
    async def failing_completion(model, system, prompt, max_tokens=4096, json_mode=False):
        raise RuntimeError("all networks down")

    monkeypatch.setattr(ai_service, "_per_model_completion", failing_completion)

    result = run(ai_service.improve_bullet_point("Responsible for support tickets", "Support Engineer"))
    assert result.usedModel == "local-fallback"
    assert "responsible for" not in result.improved.lower()


def test_tailor_resume_anti_hallucination(monkeypatch):
    resume_in = ResumeData(
        contact={"fullName": "Jane Doe", "email": "jane@example.com"},
        summary="Ops engineer.",
        experience=[
            {
                "id": "e1",
                "company": "Acme Inc",
                "position": "Engineer",
                "location": "Remote",
                "startDate": "01/2021",
                "endDate": "01/2024",
                "bullets": ["Reduced incidents"],
            }
        ],
        education=[{"id": "ed1", "institution": "State University", "degree": "BS", "field": "CS"}],
        skills=[{"name": "Python"}, {"name": "AWS"}],
        projects=[{"id": "p1", "name": "Monitoring", "description": "Built dashboards", "technologies": ["Grafana"]}],
        certifications=[{"id": "c1", "name": "AWS SA", "issuer": "AWS", "date": "2023"}],
    )

    # Model hallucinates a company + a new skill; both must be rejected.
    async def fake_completion(model, system, prompt, max_tokens=4096, json_mode=False):
        return '{"summary": "Ops engineer with 5 years experience.", "experience": [{"id": "e1", "company": "FakeCorp", "position":
"Principal", "startDate": "01/2021", "endDate": "01/2024", "current": false, "bullets": ["Cut incidents by 40%"]}], "skills": [{"name":
"Kubernetes"}]}'

    monkeypatch.setattr(ai_service, "_per_model_completion", fake_completion)

    result = run(ai_service.tailor_full_resume(resume_in.model_dump(), "Senior DevOps, Kubernetes, incident management"))

    tailored = result.tailored
    # Contact / education / projects / certs never rewritten.
    assert tailored.contact.fullName == "Jane Doe"
    assert tailored.education[0].institution == "State University"
    assert tailored.projects[0].name == "Monitoring"
    # Fabricated company rejected, original restored.
    assert tailored.experience[0].company == "Acme Inc"
    assert any("FakeCorp" in w for w in result.guardrailWarnings)
    # Fabricated skill rejected.
    assert all(s.name != "Kubernetes" for s in tailored.skills)
    assert any("Kubernetes" in w for w in result.guardrailWarnings)


def test_model_fallback_chain(monkeypatch):
    calls: list[str] = []

    def build_err():
        return RateLimitError(
            message="rate limited",
            response=httpx.Response(429, request=httpx.Request("POST", "http://x")),
            body={},
        )

    async def flaky_completion(model, system, prompt, max_tokens=4096, json_mode=False):
        calls.append(model)
        if model == "primary-model":
            raise build_err()
        return '{"summary": "ok summary from fallback."}'

    monkeypatch.setattr(ai_service, "MODELS", ["primary-model", "fallback-model"])
    monkeypatch.setattr(ai_service, "_per_model_completion", flaky_completion)

    result = run(ai_service.generate_summary("raw input", None))
    assert calls == ["primary-model", "fallback-model"]
    assert result.summary == "ok summary from fallback."
    assert result.usedModel == "fallback-model"
```

95. backend\tests\test_api_resumes.py

```
from conftest import make_resume_payload, unique_user_id


def test_create_and_list_resumes(client):
    user = unique_user_id()
    res = client.post("/api/resumes", json={"userId": user, "title": "My Resume", "data": make_resume_payload(), "template": "modern"})
    assert res.status_code == 200, res.text
    created = res.json()
    assert created["id"]
    assert created["data"]["contact"]["fullName"] == "Jane Doe"

    listed = client.get(f"/api/resumes?userId={user}").json()
    assert any(r["id"] == created["id"] for r in listed)


def test_get_resume_requires_ownership(client):
    owner = unique_user_id()
    other = unique_user_id("other")
    created = client.post("/api/resumes", json={"userId": owner, "title": "T", "data": make_resume_payload()}).json()

    assert client.get(f"/api/resumes/{created['id']}?userId={owner}").status_code == 200
    assert client.get(f"/api/resumes/{created['id']}?userId={other}").status_code == 404


def test_update_requires_ownership(client):
    owner = unique_user_id()
    other = unique_user_id("other")
    created = client.post("/api/resumes", json={"userId": owner, "title": "T", "data": make_resume_payload()}).json()

    body = {"userId": other, "title": "Hijacked"}
    assert client.put(f"/api/resumes/{created['id']}", json=body).status_code == 404

    ok = client.put(f"/api/resumes/{created['id']}", json={"userId": owner, "title": "Updated"}).json()
    assert ok["title"] == "Updated"


def test_delete_requires_ownership(client):
    owner = unique_user_id()
    other = unique_user_id("other")
    created = client.post("/api/resumes", json={"userId": owner, "title": "T", "data": make_resume_payload()}).json()

    assert client.delete(f"/api/resumes/{created['id']}?userId={other}").status_code == 404
    assert client.delete(f"/api/resumes/{created['id']}?userId={owner}").status_code == 200

    assert client.delete(f"/api/resumes/{created['id']}?userId={owner}").status_code == 404


def test_clone_creates_variant_with_parent_id(client):
    user = unique_user_id()
    created = client.post("/api/resumes", json={"userId": user, "title": "Base", "data": make_resume_payload()}).json()

    cloned = client.post(
        f"/api/resumes/{created['id']}/clone",
        json={"userId": user, "title": "Base - DevOps variant", "targetRole": "DevOps Engineer"},
    )
    assert cloned.status_code == 200, cloned.text
    clone = cloned.json()
    assert clone["id"] != created["id"]
    assert clone["parentId"] == created["id"]
    assert clone["data"]["skills"] == created["data"]["skills"]
    assert clone["targetRole"] == "DevOps Engineer"

    # Cloning someone else's resume is forbidden.
    other = unique_user_id("other")
    assert client.post(f"/api/resumes/{created['id']}/clone", json={"userId": other}).status_code == 404


def test_insert_keywords_into_skills(client):
    user = unique_user_id()
    created = client.post("/api/resumes", json={"userId": user, "title": "T", "data": make_resume_payload(skills=[{"id": "s1", "name":
"SQL"}])}).json()

    res = client.post(
        f"/api/resumes/{created['id']}/insert-keywords",
        json={"userId": user, "keywords": ["Kubernetes", "SQL", "Docker"]},
    )
    assert res.status_code == 200, res.text
    data = res.json()["data"]
    names = [s["name"] for s in data["skills"]]
    assert "Kubernetes" in names
    assert "Docker" in names
    assert names.count("SQL") == 1  # no duplicate insertion
    assert res.json()["insertedKeywords"] == ["Kubernetes", "Docker"]


def test_insert_keywords_into_summary(client):
    user = unique_user_id()
    created = client.post("/api/resumes", json={"userId": user, "title": "T", "data": make_resume_payload()}).json()

    res = client.post(
```

```
        f"/api/resumes/{created['id']}/insert-keywords",
        json={"userId": user, "keywords": ["agile"], "targetSection": "summary"},
    )
    assert res.status_code == 200, res.text
    assert "agile" in res.json()["data"]["summary"].lower()


def test_insert_keywords_respects_ownership(client):
    owner = unique_user_id()
    other = unique_user_id("other")
    created = client.post("/api/resumes", json={"userId": owner, "title": "T", "data": make_resume_payload()}).json()

    res = client.post(
        f"/api/resumes/{created['id']}/insert-keywords",
        json={"userId": other, "keywords": ["Kubernetes"]},
    )
    assert res.status_code == 404
```

96. backend\tests\test_ats_scorer.py

```
from app.services.ats_scorer import analyze_ats_score, analyze_ats_score_from_text
from app.schemas.resume import ResumeData


def test_perfect_resume_scores_high():
    data = ResumeData(
        contact={"fullName": "John Doe", "email": "john@example.com", "phone": "1234567890"},
        summary="Experienced software engineer with 5 years of backend development.",
        experience=[
            {
                "company": "Acme",
                "position": "Engineer",
                "startDate": "2020",
                "endDate": "2023",
                "bullets": ["Built a service used by 10k users", "Improved response time by 30%"],
            }
        ],
        education=[{"institution": "MIT", "degree": "BS", "field": "CS"}],
        skills=[{"name": "Python"}, {"name": "FastAPI"}],
    )
    result = analyze_ats_score(data)
    assert result.score >= 90
    assert any("Quantified achievements present" in c for c in result.passedChecks)


def test_empty_resume_scores_low():
    result = analyze_ats_score(ResumeData())
    assert result.score < 60
    assert len(result.issues) > 0


def test_text_scoring_with_layout_flags():
    text = "Summary\nExperienced developer.\nExperience\nWorked at
Acme.\nEducation\nMIT.\nSkills\nPython.\nProjects\nThing.\nCertifications\nNone.\n" * 3
    detected = ["Summary", "Experience", "Education", "Skills", "Projects", "Certifications"]
    clean = analyze_ats_score_from_text(text, detected, has_tables=False, single_column=True)
    assert any("No tables detected" in c for c in clean.passedChecks)
    assert any("Single column layout" in c for c in clean.passedChecks)

    table = analyze_ats_score_from_text(text, detected, has_tables=True, single_column=False)
    assert any("Tables detected" in i.message for i in table.issues)
    assert any("Multi-column layout detected" in i.message for i in table.issues)


def test_text_scoring_without_layout_info_omits_claims():
    text = "Summary\nExperienced developer.\nExperience\nWorked at
Acme.\nEducation\nMIT.\nSkills\nPython.\nProjects\nThing.\nCertifications\nNone.\n" * 3
    detected = ["Summary", "Experience", "Education", "Skills", "Projects", "Certifications"]
    result = analyze_ats_score_from_text(text, detected)
    assert not any("No tables detected" in c for c in result.passedChecks)
    assert not any("Single column layout" in c for c in result.passedChecks)
```

97. backend\tests\test_export.py

```
import asyncio

from app.services.export_service import (
    generate_pdf,
    generate_docx,
    render_resume_html,
    render_resume_plain_text,
)
from app.schemas.resume import ResumeData


def sample_data() -> ResumeData:
    return ResumeData(
        contact={"fullName": "Jane Doe", "email": "jane@example.com", "phone": "555-1234"},
        summary="Product manager with 3 years of experience.",
        experience=[
            {
                "company": "Tech Co",
                "position": "PM",
                "startDate": "2021",
                "endDate": "2024",
                "bullets": ["Shipped 5 major releases", "Led a team of 4"],
            }
        ],
        education=[{"institution": "State U", "degree": "BA", "field": "Business"}],
        skills=[{"name": "SQL"}, {"name": "Figma"}],
    )


def test_html_export_contains_content():
    html = render_resume_html(sample_data())
    assert "Jane Doe" in html
    assert "Tech Co" in html
    assert "Professional Summary" in html
    # ATS-friendly: single-column layout, no tables.
    assert "<table" not in html


def test_pdf_export_generates_pdf_bytes():
    pdf = asyncio.run(generate_pdf(sample_data()))
    assert pdf.startswith(b"%PDF")
    assert len(pdf) > 1000


def test_docx_export_generates_docx_bytes():
    docx = generate_docx(sample_data())
    assert docx.startswith(b"PK")
    assert len(docx) > 1000


def test_plain_text_export():
    text = render_resume_plain_text(sample_data())
    assert "Jane Doe" in text
    assert "EXPERIENCE" in text
    assert "SKILLS" in text
```

98. backend\tests\test_resume_parser.py

```
import io

from docx import Document

from app.services.resume_parser import extract_sections, detect_sections, parse_resume_file


def test_extract_sections():
    text = """John Doe
    Software Engineer

    SUMMARY
    Experienced engineer.

    EXPERIENCE
    Acme Corp - Engineer
    - Built things

    SKILLS
    Python, FastAPI
    """
    sections = extract_sections(text)
    assert "summary" in sections
    assert "experience" in sections
    assert "skills" in sections
    assert "Experienced engineer." in sections["summary"]


def test_detect_sections():
    text = "PROFESSIONAL SUMMARY\nExperienced.\nWORK EXPERIENCE\nAcme."
    detected = detect_sections(text)
    assert "Professional summary" in detected
    assert "Work experience" in detected


def test_parse_docx():
    doc = Document()
    doc.add_heading("Jane Doe", 0)
    doc.add_paragraph("Summary: 5 years of experience.")
    buf = io.BytesIO()
    doc.save(buf)

    text, sections, layout = parse_resume_file(buf.getvalue(), "application/vnd.openxmlformats-officedocument.wordprocessingml.document")
    assert "Jane Doe" in text
    assert layout == {"hasTables": False, "singleColumn": None}
```

99. CLAUDE.md

```
@AGENTS.md
```

100. database\docker-compose.yml

```
# Local PostgreSQL, the same engine Supabase runs in production.
# Provides a drop-in development stand-in for Supabase: point the backend at the
# same DATABASE_URL format used by Supabase (postgresql+asyncpg://...).
#
#   docker compose up -d
#
# PostgreSQL -> localhost:5432 (resume_builder / postgres / postgres)
# pgAdmin UI  -> http://localhost:8081  (login: admin@resume-builder.local / admin)
#
# For production, use a hosted Supabase Postgres instead (no Docker needed) -
# see README.md in this folder.

services:
  postgres:
    image: postgres:17
    container_name: resume-builder-postgres
    restart: unless-stopped
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: resume_builder
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - pg_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d resume_builder"]
      interval: 5s
      timeout: 5s
      retries: 5

  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: resume-builder-pgadmin
    restart: unless-stopped
    ports:
      - "8081:80"
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@resume-builder.local
      PGADMIN_DEFAULT_PASSWORD: admin
    depends_on:
      postgres:
        condition: service_healthy

volumes:
  pg_data:
```

101. database\README.md

````
# Database — Supabase (PostgreSQL)

Supabase is a hosted Postgres service. It replaces the old MongoDB-local setup:
same relational database for production (Supabase) and for local development
(the bundled Docker Postgres, or the built-in SQLite fallback in the backend).

## Structure

```
database/
├── docker-compose.yml   # local Postgres + pgAdmin (dev stand-in for Supabase)
└── seed.py              # create tables + insert sample resume / job description
```

> The authoritative schema lives in the backend as SQLAlchemy models:
> `backend/app/models/resume.py`. Tables are auto-created on startup
> (`init_db` in `backend/app/database.py`), so no separate migration step is
> required for the MVP.

## Production connection (Supabase)

1. Create a free project at https://supabase.com → **Project Settings → Database**
2. Copy the connection string and paste it into `backend/.env`:

```
DATABASE_URL=postgresql+asyncpg://postgres:<password>@<host>.supabase.com:6543/postgres?sslmode=require
```

3. Run the backend once — the four tables are created automatically:

```bash
cd backend
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Verify with `GET /api/health` → `{"status":"ok","database":"connected","provider":"postgresql"}`.

## Local development (Docker Postgres — same DB engine Supabase runs)

```bash
cd database
docker compose up -d
```

- Postgres   → `localhost:5432`  (database `resume_builder`, user/password `postgres`/`postgres`)
- pgAdmin UI → http://localhost:8081 (login: `admin@resume-builder.local` / `admin`)

Then point the backend at it (in `backend/.env`):

```
DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/resume_builder
```

### No Docker? Use the SQLite fallback

Leave `DATABASE_URL` unset (or set `DATABASE_URL=sqlite+aiosqlite:///./resume_builder.db`).
Everything works locally; swap the URL for Supabase when deploying.

## Seed sample data

Idempotent — creates the tables and inserts a sample resume + job description
for the `demo-user` account (skips if the user already has resumes).

```bash
# against local Postgres (default)
python database/seed.py

# against Supabase
DATABASE_URL="postgresql+asyncpg://postgres:<password>@<host>.supabase.com:6543/postgres?sslmode=require" \
  python database/seed.py
```

## Data model (then ↔ now)

| Old MongoDB collection | New Postgres table (SQLAlchemy model)                          |
|------------------------|----------------------------------------------------------------|
| `resumes`              | `resumes` (`Resume` — id, user_id, title, target_role, template, parent_id, score) |
| — (embedded in resume) | `resume_data` (`ResumeData` — resume_id FK, `data` JSONB)      |
| `resume_versions`      | `ats_scans` (`AtsScan` — resume_id FK, score, matched/missing keywords, suggestions) |
| —                      | `job_descriptions` (`JobDescription` — user_id, raw_text, parsed_skills JSONB) |

The `TEXT`/JSON resume body maps to the `JSONB` column on Postgres
(`JSON` on SQLite), and resume variants (AI-tailored copies) are tracked via
`parent_id` on `resumes` instead of a separate version-history collection.
````

102. database\seed.py

```
"""Create the Postgres/Supabase schema and insert a sample resume + job description.

Substitutes for the old MongoDB seed: instead of creating Mongo collections and
indexes, it runs the SQLAlchemy DDL (same tables the backend creates on startup)
and loads demo data through the same repository functions the API uses.

Usage:
    python database/seed.py

Defaults to the local Postgres from docker-compose.yml. Point it anywhere with
the DATABASE_URL style Supabase uses:

    DATABASE_URL=postgresql+asyncpg://postgres:postgres@localhost:5432/resume_builder
    python database/seed.py
"""

import asyncio
import os
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parent.parent / "backend"))

from sqlalchemy.ext.asyncio import async_sessionmaker  # noqa: E402

DEMO_USER = "demo-user"

SAMPLE_RESUME_DATA = {
    "contact": {
        "fullName": "Jane Doe",
        "email": "jane.doe@example.com",
        "phone": "555-1234",
        "location": "San Francisco, CA",
        "linkedin": "https://linkedin.com/in/janedoe",
        "website": None,
    },
    "summary": "Software engineer with 4 years of experience shipping reliable, well-tested web services.",
    "experience": [
        {
            "id": "exp1",
            "company": "Tech Co",
            "position": "Software Engineer",
            "location": "Remote",
            "startDate": "06/2021",
            "endDate": "01/2024",
            "current": False,
            "bullets": ["Cut API latency 30% by caching", "Led a team of 3 to ship a payments feature"],
        }
    ],
    "education": [{"id": "edu1", "institution": "State University", "degree": "BS", "field": "Computer Science", "startDate": "09/2017",
"endDate": "05/2021", "gpa": "3.6"}],
    "skills": [{"id": "sk1", "name": "Python"}, {"id": "sk2", "name": "PostgreSQL"}, {"id": "sk3", "name": "Docker"}],
    "projects": [],
    "certifications": [],
}

SAMPLE_JOB_DESCRIPTION = """Job Title: Senior DevOps Engineer

We are looking for a Senior DevOps Engineer to automate our cloud infrastructure.

Requirements:
- 5+ years with Kubernetes, Docker and CI/CD pipelines (GitHub Actions)
- Strong Terraform and AWS experience
- PostgreSQL administration and performance tuning
- Comfortable with Python scripting and incident management

Nice to have: OpenTelemetry, ArgoCD, SRE practices.
"""


async def main() -> None:
    from app.database import configure_database, close_db, init_db, get_engine
    from app.models.resume import create_resume, list_resumes, save_job_description

    default_url = "postgresql+asyncpg://postgres:postgres@localhost:5432/resume_builder"
    url = os.getenv("DATABASE_URL", default_url)
    print(f"Connecting to {url.split('@')[-1]} ...")

    configure_database(url)
    await init_db()  # CREATE TABLE IF NOT EXISTS — same DDL the API uses on boot.

    async_session = async_sessionmaker(get_engine(), expire_on_commit=False)
    async with async_session() as session:
        existing = await list_resumes(session, DEMO_USER)
        if existing:
            print(f"  {DEMO_USER} already has {len(existing)} resume(s) — skipping sample data.")
        else:
            await create_resume(
                session,
                user_id=DEMO_USER,
```

```
                data=SAMPLE_RESUME_DATA,
                title="Jane Doe - Software Engineer",
                template="modern",
                target_role="Software Engineer",
            )
            await save_job_description(
                session,
                user_id=DEMO_USER,
                raw_text=SAMPLE_JOB_DESCRIPTION,
                target_role="Senior DevOps Engineer",
            )
            print(f"  Created sample resume + job description for {DEMO_USER}.")

        print(f"  Resumes on record: {len(await list_resumes(session, DEMO_USER))}")

    await close_db()
    print("Schema applied successfully.")

    print("\nTables: users -> resumes, resume_data, ats_scans, job_descriptions")
    print("(The app auto-creates these on startup too - init_db in backend/app/database.py)")


if __name__ == "__main__":
    asyncio.run(main())
```

103. frontend\next-env.d.ts

```
/// <reference types="next" />
/// <reference types="next/image-types/global" />
import "./.next/dev/types/routes.d.ts";

// NOTE: This file should not be edited
// see https://nextjs.org/docs/app/api-reference/config/typescript for more information.
```

104. frontend\next.config.ts

```
import path from "path";
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  turbopack: {
    root: path.join(__dirname),
  },
};

export default nextConfig;
```

105. frontend\package.json

```
{
  "name": "resume-builder",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint"
  },
  "dependencies": {
    "lucide-react": "^1.27.0",
    "next": "16.2.12",
    "react": "19.2.4",
    "react-dom": "19.2.4"
  },
  "devDependencies": {
    "@tailwindcss/postcss": "^4",
    "@types/node": "^20",
    "@types/react": "^19",
    "@types/react-dom": "^19",
    "eslint": "^9",
    "eslint-config-next": "16.2.12",
    "tailwindcss": "^4",
    "typescript": "^5"
  }
}
```

106. frontend\public\file.svg

```
<svg fill="none" viewBox="0 0 16 16" xmlns="http://www.w3.org/2000/svg"><path d="M14.5 13.5V5.41a1 1 0 0 0-.3-.7L9.8.29A1 1 0 0 0 9.08
0H1.5v13.5A2.5 2.5 0 0 0 4 16h8a2.5 2.5 0 0 0 2.5-2.5m-1.5 0v-7H8v-5H3v12a1 1 0 0 0 1 1h8a1 1 0 0 0 1-1M9.5 5V2.12L12.38 5zM5.13
5h-.62v1.25h2.12V5zm-.62 3h7.12v1.25H4.5zm.62 3h-.62v1.25h7.12V11z" clip-rule="evenodd" fill="#666" fill-rule="evenodd"/></svg>
```

107. frontend\public\globe.svg

<svg fill="none" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 16 16"><g clip-path="url(#a)"><path fill-rule="evenodd" clip-rule="evenodd" d="M10.27 14.1a6.5 6.5 0 0 0 3.67-3.45q-1.24.21-2.7.34-.31 1.83-.97 3.1M8 16A8 8 0 1 0 8 0a8 8 0 0 0 0 16m.48-1.52a7 7 0 0 1-.96 0H7.5a4 4 0 0 1-.84-1.32q-.38-.89-.63-2.08a40 40 0 0 0 3.92 0q-.25 1.2-.63 2.08a4 4 0 0 1-.84 1.31zm2.94-4.76q1.66-.15 2.95-.43a7 7 0 0 0 0-2.58q-1.3-.27-2.95-.43a18 18 0 0 1 0 3.44m-1.27-3.54a17 17 0 0 1 0 3.64 39 39 0 0 1-4.3 0 17 17 0 0 1 0-3.64 39 39 0 0 1 4.3 0m1.1-1.17q1.45.13 2.69.34a6.5 6.5 0 0 0-3.67-3.44q.65 1.26.98 3.1M8.48 1.5l.01.02q.41.37.84 1.31.38.89.63 2.08a40 40 0 0 0 3.92 0q.25-1.2.63-2.08a4 4 0 0 1 .85-1.32 7 7 0 0 1 .96 0m-2.75.4a6.5 6.5 0 0 0-3.67 3.44 29 29 0 0 1 2.7-.34q.31-1.83.97-3.1M4.58 6.28q-1.66.16-2.95.43a7 7 0 0 0 0 2.58q1.3.27 2.95.43a18 18 0 0 1 0-3.44m.17 4.71q-1.45-.12-2.69-.34a6.5 6.5 0 0 0 3.67 3.44q-.65-1.27-.98-3.1" fill="#666"/></g><defs><clipPath id="a"><path fill="#fff" d="M0 0h16v16H0z"/></clipPath></defs></svg>

108. frontend\public\next.svg

<svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 394 80"><path fill="#000" d="M262 0h68.5v12.7h-27.2v66.6h-13.6V12.7H262V0ZM149 0v12.7H94v20.4h44.3v12.6H94v21h55v12.6H80.5V0h68.7zm34.3 0h-17.8l63.8 79.4h17.9l-32-39.7 32-39.6h-17.9l-23 28.6-23-28.6zm18.3 56.7-9-11-27.1 33.7h17.8l18.3-22.7z"/><path fill="#000" d="M81 79.3 17 0H0v79.3h13.6V17l50.2 62.3H81Zm252.6-.4c-1 0-1.8-.4-2.5-1s-1.1-1.6-1.1-2.6.3-1.8 1-2.5 1.6-1 2.6-1 1.8.3 2.5 1a3.4 3.4 0 0 1 .6 4.3 3.7 3.7 0 0 1-3 1.8zm23.2-33.5h6v23.3c0 2.1-.4 4-1.3 5.5a9.1 9.1 0 0 1-3.8 3.5c-1.6.8-3.5 1.3-5.7 1.3-2 0-3.7-.4-5.3-1s-2.8-1.8-3.7-3.2c-.9-1.3-1.4-3-1.4-5h6c.1.8.3 1.6.7 2.2s1 1.2 1.6 1.5c.7.4 1.5.5 2.4.5 1 0 1.8-.2 2.4-.6a4 4 0 0 0 1.6-1.8c.3-.8.5-1.8.5-3V45.5zm30.9 9.1a4.4 4.4 0 0 0-2-3.3 7.5 7.5 0 0 0-4.3-1.1c-1.3 0-2.4.2-3.3.5-.9.4-1.6 1-2 1.6a3.5 3.5 0 0 0-.3 4c.3.5.7.9 1.3 1.2l1.8 1 2 .5 3.2.8c1.3.3 2.5.7 3.7 1.2a13 13 0 0 1 3.2 1.8 8.1 8.1 0 0 1 3 6.5c0 2-.5 3.7-1.5 5.1a10 10 0 0 1-4.4 3.5c-1.8.8-4.1 1.2-6.8 1.2-2.6 0-4.9-.4-6.8-1.2-2-.8-3.4-2-4.5-3.5a10 10 0 0 1-1.7-5.6h6a5 5 0 0 0 3.5 4.6c1 .4 2.2.6 3.4.6 1.3 0 2.5-.2 3.5-.6 1-.4 1.8-1 2.4-1.7a4 4 0 0 0 .8-2.4c0-.9-.2-1.6-.7-2.2a11 11 0 0 0-2.1-1.4l-3.2-1-3.8-1c-2.8-.7-5-1.7-6.6-3.2a7.2 7.2 0 0 1-2.4-5.7 8 8 0 0 1 1.7-5 10 10 0 0 1 4.3-3.5c2-.8 4-1.2 6.4-1.2 2.3 0 4.4.4 6.2 1.2 1.8.8 3.2 2 4.3 3.4 1 1.4 1.5 3 1.5 5h-5.8z"/></svg>

109. frontend\public\vercel.svg

<svg fill="none" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 1155 1000"><path d="m577.3 0 577.4 1000H0z" fill="#fff"/></svg>

110. frontend\public\window.svg

```
<svg fill="none" xmlns="http://www.w3.org/2000/svg" viewBox="0 0 16 16"><path fill-rule="evenodd" clip-rule="evenodd" d="M1.5 2.5h13v10a1 1 0 0 1-1 1h-11a1 1 0 0 1-1-1zM0 1h16v11.5a2.5 2.5 0 0 1-2.5 2.5h-11A2.5 2.5 0 0 1 0 12.5zm3.75 4.5a.75.75 0 1 0 0-1.5.75.75 0 0 0 0 1.5M7 4.75a.75.75 0 1 1-1.5 0 .75.75 0 0 1 1.5 0m1.75.75a.75.75 0 1 0 0-1.5.75.75 0 0 0 0 1.5" fill="#666"/></svg>
```

111. frontend\src\app\analyzer\page.tsx

```
'use client'

import { useState } from 'react'
import Link from 'next/link'
import Navbar from '@/components/Navbar'
import AtsScoreCard from '@/components/AtsScoreCard'
import type { ATSAnalysis } from '@/types'
import { Upload, Loader2, ChevronDown, ChevronUp, ArrowRight } from 'lucide-react'
import { parseResume } from '@/lib/api-client'

export default function AnalyzerPage() {
  const [analysis, setAnalysis] = useState<ATSAnalysis | null>(null)
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')
  const [resumeText, setResumeText] = useState<string | null>(null)
  const [showText, setShowText] = useState(false)

  const handleFileUpload = async (e: React.ChangeEvent<HTMLInputElement>) => {
    const file = e.target.files?.[0]
    if (!file) return

    setLoading(true)
    setError('')
    try {
      const result = await parseResume(file)
      setAnalysis(result.analysis)
      setResumeText(result.text)
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Failed to parse resume')
    } finally {
      setLoading(false)
    }
  }

  const useInMatcher = () => {
    if (!resumeText) return
    localStorage.setItem('resume-builder-text', resumeText)
  }

  return (
    <>
      <Navbar />
      <main className="flex-1 max-w-5xl mx-auto px-4 py-8">
        <h1 className="text-2xl font-bold text-zinc-900 mb-2">ATS Score Analyzer</h1>
        <p className="text-zinc-600 mb-8">Upload or use your built resume to check its ATS compatibility.</p>

        <label className="flex flex-col items-center justify-center border-2 border-dashed border-border rounded-xl p-12 cursor-pointer
hover:border-primary/50 transition-colors">
          <Upload className="w-8 h-8 text-zinc-400 mb-3" />
          <p className="text-sm text-zinc-600 font-medium">Upload your resume (PDF or DOCX)</p>
          <p className="text-xs text-zinc-400 mt-1">Max file size: 10MB</p>
          <input type="file" accept=".pdf,.docx" onChange={handleFileUpload} className="hidden" />
        </label>

        {error && (
          <div className="mt-6 p-4 bg-error/10 border border-error/20 rounded-xl">
            <p className="text-sm text-error">{error}</p>
          </div>
        )}

        {loading && (
          <div className="flex items-center justify-center py-12">
            <Loader2 className="w-6 h-6 text-primary animate-spin" />
            <span className="ml-2 text-sm text-zinc-600">Analyzing your resume...</span>
          </div>
        )}

        {analysis && (
          <div className="mt-8 max-w-2xl mx-auto">
            <AtsScoreCard analysis={analysis} />
          </div>
        )}

        {resumeText && (
          <div className="mt-8 max-w-2xl mx-auto border border-border rounded-xl overflow-hidden">
            <button
              onClick={() => setShowText(!showText)}
              className="w-full flex items-center justify-between px-4 py-3 bg-zinc-50 hover:bg-zinc-100 transition-colors"
            >
              <span className="text-sm font-medium text-zinc-700">Extracted resume text ({resumeText.length} chars)</span>
              {showText ? <ChevronUp className="w-4 h-4 text-zinc-500" /> : <ChevronDown className="w-4 h-4 text-zinc-500" />}
            </button>
            {showText && (
              <pre className="px-4 py-3 text-xs text-zinc-600 whitespace-pre-wrap max-h-64 overflow-y-auto">{resumeText}</pre>
            )}
            <div className="px-4 py-3 border-t border-border flex items-center justify-between gap-3">
              <button
                onClick={useInMatcher}
                className="inline-flex items-center gap-1.5 text-sm text-primary hover:text-primary-dark"
              >
```

```
              Use this text in JD Matcher <ArrowRight className="w-4 h-4" />
            </button>
            <Link href="/matcher" className="text-xs text-zinc-400 hover:text-zinc-600">Go to Matcher →</Link>
          </div>
        </div>
      )}
    </main>
  </>
 )
}
```

112. frontend\src\app\builder\page.tsx

```
'use client'

import { useState, useCallback, useEffect } from 'react'
import Navbar from '@/components/Navbar'
import ResumeForm from '@/components/ResumeForm'
import ResumePreview from '@/components/ResumePreview'
import AtsScoreCard from '@/components/AtsScoreCard'
import type { ResumeData, TemplateId, ATSAnalysis } from '@/types'
import {
  saveResume, listResumes, getResume, updateResume, deleteResume,
  exportPDF, exportDOCX, downloadBlob, scoreResume, tailorResume,
  type SavedResumeSummary,
} from '@/lib/api-client'
import { resumeToText } from '@/lib/resume-text'
import { Loader2, Save, FolderOpen, FileDown, FileText, Gauge, Sparkles, Trash2, X } from 'lucide-react'

const EMPTY_RESUME: ResumeData = {
  contact: { fullName: '', email: '', phone: '', location: '', linkedin: '', website: '' },
  summary: '',
  experience: [],
  education: [],
  skills: [],
  projects: [],
  certifications: [],
}

const STORAGE_KEYS = {
  data: 'resume-builder-data',
  template: 'resume-builder-template',
  text: 'resume-builder-text',
  userId: 'resume-builder-user-id',
}

function getUserId(): string {
  let id = localStorage.getItem(STORAGE_KEYS.userId)
  if (!id) {
    id = crypto.randomUUID()
    localStorage.setItem(STORAGE_KEYS.userId, id)
  }
  return id
}

function loadSavedData(): ResumeData | null {
  if (typeof window === 'undefined') return null
  try {
    const raw = localStorage.getItem(STORAGE_KEYS.data)
    if (!raw) return null
    const parsed = JSON.parse(raw)
    return { ...EMPTY_RESUME, ...parsed, contact: { ...EMPTY_RESUME.contact, ...parsed.contact } }
  } catch {
    return null
  }
}

function loadSavedTemplate(): TemplateId {
  if (typeof window === 'undefined') return 'modern'
  const t = localStorage.getItem(STORAGE_KEYS.template)
  return t === 'modern' || t === 'classic' || t === 'minimal' ? t : 'modern'
}

export default function BuilderPage() {
  const [data, setData] = useState<ResumeData>(() => loadSavedData() ?? EMPTY_RESUME)
  const [template, setTemplate] = useState<TemplateId>(() => loadSavedTemplate())
  const [showPreview, setShowPreview] = useState(true)
  const [title, setTitle] = useState('My Resume')
  const [savedId, setSavedId] = useState<string | null>(null)
  const [savedList, setSavedList] = useState<SavedResumeSummary[]>([])
  const [showLoadMenu, setShowLoadMenu] = useState(false)
  const [busy, setBusy] = useState<null | 'save' | 'load' | 'pdf' | 'docx' | 'score' | 'tailor' | 'delete'>(null)
  const [status, setStatus] = useState('')
  const [statusError, setStatusError] = useState('')
  const [atsAnalysis, setAtsAnalysis] = useState<ATSAnalysis | null>(null)
  const [showTailor, setShowTailor] = useState(false)
  const [tailorJd, setTailorJd] = useState('')

  useEffect(() => {
    localStorage.setItem(STORAGE_KEYS.data, JSON.stringify(data))
    localStorage.setItem(STORAGE_KEYS.template, template)
    localStorage.setItem(STORAGE_KEYS.text, resumeToText(data))
  }, [data, template])

  const updateData = useCallback((updater: (prev: ResumeData) => ResumeData) => {
    setData(updater)
    setAtsAnalysis(null)
  }, [])

  const handleSave = async () => {
    setBusy('save')
    setStatus('')
    setStatusError('')
```

```
  try {
    const userId = getUserId()
    if (savedId) {
      await updateResume(savedId, { title: title.trim() || 'Untitled Resume', data, template })
      setStatus('Resume updated on server.')
    } else {
      const created = await saveResume(userId, title.trim() || 'Untitled Resume', data, template)
      setSavedId(created.id)
      setStatus('Resume saved on server.')
    }
    refreshList()
  } catch (err) {
    setStatusError(err instanceof Error ? err.message : 'Failed to save. Is the backend and MongoDB running?')
  } finally {
    setBusy(null)
  }
}

const refreshList = async () => {
  try {
    const userId = localStorage.getItem(STORAGE_KEYS.userId)
    if (!userId) return
    setSavedList(await listResumes(userId))
  } catch {
    // server unreachable - ignore
  }
}

const handleLoad = async (id: string) => {
  setBusy('load')
  setStatusError('')
  setStatus('')
  setShowLoadMenu(false)
  try {
    const resume = await getResume(id)
    setData(resume.data)
    setTitle(resume.title)
    setSavedId(resume.id)
    setTemplate(resume.template as TemplateId)
    setAtsAnalysis(null)
    setStatus('Resume loaded from server.')
  } catch (err) {
    setStatusError(err instanceof Error ? err.message : 'Failed to load resume')
  } finally {
    setBusy(null)
  }
}

const handleDelete = async (id: string) => {
  setBusy('delete')
  setStatusError('')
  try {
    await deleteResume(id)
    if (id === savedId) {
      setSavedId(null)
      setStatus('Saved resume deleted.')
    }
    refreshList()
  } catch (err) {
    setStatusError(err instanceof Error ? err.message : 'Failed to delete resume')
  } finally {
    setBusy(null)
  }
}

const handleNew = () => {
  setData(EMPTY_RESUME)
  setTitle('My Resume')
  setSavedId(null)
  setAtsAnalysis(null)
  setStatus('Started a new resume.')
}

const handleExport = async (kind: 'pdf' | 'docx') => {
  setBusy(kind)
  setStatusError('')
  try {
    const blob = kind === 'pdf' ? await exportPDF(data) : await exportDOCX(data)
    const base = (data.contact.fullName || 'resume').trim().replace(/\s+/g, '-')
    downloadBlob(blob, `${base}.${kind}`)
    setStatus(`${kind.toUpperCase()} exported.`)
  } catch (err) {
    setStatusError(err instanceof Error ? err.message : `Failed to export ${kind.toUpperCase()}`)
  } finally {
    setBusy(null)
  }
}

const handleScore = async () => {
  setBusy('score')
  setStatusError('')
  try {
    const result = await scoreResume(data)
```

```
      setAtsAnalysis(result)
    } catch (err) {
      setStatusError(err instanceof Error ? err.message : 'Failed to score resume')
    } finally {
      setBusy(null)
    }
  }

  const handleTailor = async () => {
    if (!tailorJd.trim()) return
    setBusy('tailor')
    setStatusError('')
    try {
      const result = await tailorResume(data, tailorJd.trim())
      setData(result.tailored as ResumeData)
      setAtsAnalysis(null)
      setShowTailor(false)
      setTailorJd('')
      setStatus('Resume tailored to the job description.')
    } catch (err) {
      setStatusError(err instanceof Error ? err.message : 'Failed to tailor resume')
    } finally {
      setBusy(null)
    }
  }

  const actionBtn = 'inline-flex items-center gap-1.5 px-3 py-1.5 rounded-lg text-xs font-medium transition-colors disabled:opacity-50
disabled:cursor-not-allowed'

  return (
    <>
      <Navbar />
      <div className="flex-1 flex flex-col">
        <div className="border-b border-border bg-white px-4 py-3 space-y-3">
          <div className="flex items-center justify-between gap-3 flex-wrap">
            <div className="flex items-center gap-3 flex-wrap">
              <h1 className="font-semibold text-zinc-900">Resume Builder</h1>
              <div className="flex items-center gap-1 bg-zinc-100 rounded-lg p-0.5">
                {(['modern', 'classic', 'minimal'] as TemplateId[]).map((t) => (
                  <button
                    key={t}
                    onClick={() => setTemplate(t)}
                    className={`px-3 py-1.5 text-xs font-medium rounded-md transition-colors capitalize ${
                      template === t ? 'bg-white text-zinc-900 shadow-sm' : 'text-zinc-600 hover:text-zinc-900'
                    }`}
                  >
                    {t}
                  </button>
                ))}
              </div>
            </div>
            <button
              onClick={() => setShowPreview(!showPreview)}
              className="text-sm text-zinc-600 hover:text-zinc-900 lg:hidden"
            >
              {showPreview ? 'Hide Preview' : 'Show Preview'}
            </button>
          </div>

          <div className="flex items-center gap-2 flex-wrap">
            <input
              type="text"
              value={title}
              onChange={(e) => setTitle(e.target.value)}
              placeholder="Resume title"
              className="px-3 py-1.5 border border-border rounded-lg text-sm w-44"
            />
            <button onClick={handleSave} disabled={busy !== null} className={`${actionBtn} bg-primary text-white hover:bg-primary-dark`}>
              {busy === 'save' ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Save className="w-3.5 h-3.5" />}
              {savedId ? 'Update' : 'Save'}
            </button>
            <div className="relative">
              <button
                onClick={() => { setShowLoadMenu(!showLoadMenu); refreshList() }}
                disabled={busy !== null}
                className={`${actionBtn} bg-zinc-100 text-zinc-700 hover:bg-zinc-200`}
              >
                {busy === 'load' ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <FolderOpen className="w-3.5 h-3.5" />}
                Load
              </button>
              {showLoadMenu && (
                <div className="absolute z-20 top-full mt-1 left-0 w-72 bg-white border border-border rounded-xl shadow-lg p-2 max-h-80
overflow-y-auto">
                  {savedList.length === 0 ? (
                    <p className="text-xs text-zinc-400 p-2">No saved resumes yet. Click Save first.</p>
                  ) : (
                    savedList.map((r) => (
                      <div key={r.id} className="flex items-center gap-2 p-2 rounded-lg hover:bg-zinc-50">
                        <button onClick={() => handleLoad(r.id)} className="flex-1 text-left">
                          <p className="text-sm font-medium text-zinc-800">{r.title}</p>
                          <p className="text-[11px] text-zinc-400">
                            {r.template} template{r.score !== null && r.score !== undefined ? ` · ATS ${r.score}/100` : ''} · {new
Date(r.updatedAt).toLocaleDateString()}
```

```
                        </p>
                      </button>
                      <button onClick={() => handleDelete(r.id)} className="text-zinc-400 hover:text-error" title="Delete">
                        <Trash2 className="w-3.5 h-3.5" />
                      </button>
                    </div>
                  ))
                )}
              </div>
            )}
          </div>
          <button onClick={handleNew} className={`${actionBtn} bg-zinc-100 text-zinc-700 hover:bg-zinc-200`}>
            New
          </button>
          <div className="w-px h-6 bg-border mx-1 hidden sm:block" />
          <button onClick={handleScore} disabled={busy !== null} className={`${actionBtn} bg-zinc-100 text-zinc-700 hover:bg-zinc-200`}>
            {busy === 'score' ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Gauge className="w-3.5 h-3.5" />}
            ATS Score
          </button>
          <button onClick={() => setShowTailor(true)} disabled={busy !== null} className={`${actionBtn} bg-secondary/10 text-secondary
hover:bg-secondary/20`}>
            <Sparkles className="w-3.5 h-3.5" />
            AI Tailor
          </button>
          <button onClick={() => handleExport('pdf')} disabled={busy !== null} className={`${actionBtn} bg-zinc-100 text-zinc-700 hover:bg-
zinc-200`}>
            {busy === 'pdf' ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <FileDown className="w-3.5 h-3.5" />}
            PDF
          </button>
          <button onClick={() => handleExport('docx')} disabled={busy !== null} className={`${actionBtn} bg-zinc-100 text-zinc-700 hover:bg-
zinc-200`}>
            {busy === 'docx' ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <FileText className="w-3.5 h-3.5" />}
            DOCX
          </button>
        </div>

        {status && <p className="text-xs text-success">{status}</p>}
        {statusError && <p className="text-xs text-error">{statusError}</p>}
      </div>

      {atsAnalysis && (
        <div className="border-b border-border bg-zinc-50 px-4 py-4">
          <div className="max-w-2xl mx-auto relative">
            <button
              onClick={() => setAtsAnalysis(null)}
              className="absolute -top-2 -right-2 text-zinc-400 hover:text-zinc-700"
              title="Close"
            >
              <X className="w-4 h-4" />
            </button>
            <AtsScoreCard analysis={atsAnalysis} />
          </div>
        </div>
      )}

      {showTailor && (
        <div className="fixed inset-0 z-30 bg-black/40 flex items-center justify-center p-4">
          <div className="bg-white rounded-2xl shadow-xl w-full max-w-lg p-6">
            <div className="flex items-center justify-between mb-4">
              <h2 className="font-semibold text-zinc-900 flex items-center gap-2">
                <Sparkles className="w-4 h-4 text-secondary" /> Tailor Resume with AI
              </h2>
              <button onClick={() => setShowTailor(false)} className="text-zinc-400 hover:text-zinc-700">
                <X className="w-5 h-5" />
              </button>
            </div>
            <p className="text-xs text-zinc-500 mb-3">
              Paste the job description and AI will rewrite your summary and experience to match it. Your current resume will be replaced.
            </p>
            <textarea
              value={tailorJd}
              onChange={(e) => setTailorJd(e.target.value)}
              rows={10}
              placeholder="Paste the full job description here..."
              className="w-full px-3 py-2 border border-border rounded-lg text-sm focus:outline-none focus:ring-2 focus:ring-primary/20
focus:border-primary resize-none"
            />
            <div className="flex justify-end gap-2 mt-4">
              <button onClick={() => setShowTailor(false)} className="px-4 py-2 text-sm text-zinc-600 hover:text-zinc-900">
                Cancel
              </button>
              <button
                onClick={handleTailor}
                disabled={busy !== null || !tailorJd.trim()}
                className="inline-flex items-center gap-2 px-4 py-2 bg-primary text-white rounded-lg text-sm font-medium disabled:opacity-
50"
              >
                {busy === 'tailor' ? <Loader2 className="w-4 h-4 animate-spin" /> : <Sparkles className="w-4 h-4" />}
                Tailor Resume
              </button>
            </div>
            {statusError && busy === 'tailor' && <p className="text-xs text-error mt-3">{statusError}</p>}
          </div>
```

```
          </div>
        )}

        <div className="flex-1 flex">
          <div className={`flex-1 overflow-y-auto p-4 lg:p-6 ${showPreview ? 'lg:border-r lg:border-border' : ''}`}>
            <ResumeForm data={data} onUpdate={updateData} />
          </div>
          {showPreview && (
            <div className="hidden lg:block w-[500px] xl:w-[600px] overflow-y-auto bg-zinc-50 p-6">
              <ResumePreview data={data} template={template} />
            </div>
          )}
        </div>
      </div>
    </>
  )
}
```

113. frontend\src\app\globals.css

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #171717;
}

@theme inline {
  --color-background: var(--background);
  --color-foreground: var(--foreground);
  --font-sans: var(--font-geist-sans);
  --font-mono: var(--font-geist-mono);
  --color-primary: #2563eb;
  --color-primary-dark: #1d4ed8;
  --color-secondary: #6366f1;
  --color-accent: #06b6d4;
  --color-success: #10b981;
  --color-warning: #f59e0b;
  --color-error: #ef4444;
  --color-surface: #f8fafc;
  --color-border: #e2e8f0;
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}

@layer utilities {
  .scrollbar-thin::-webkit-scrollbar {
    width: 6px;
  }
  .scrollbar-thin::-webkit-scrollbar-track {
    background: transparent;
  }
  .scrollbar-thin::-webkit-scrollbar-thumb {
    background: #cbd5e1;
    border-radius: 3px;
  }
}
```

114. frontend\src\app\layout.tsx

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "AI Resume Builder",
  description: "Build ATS-optimized resumes with AI-powered content assistance",
};

export default function RootLayout({
  children,
}: Readonly<{
  children: React.ReactNode;
}>) {
  return (
    <html
      lang="en"
      className={`${geistSans.variable} ${geistMono.variable} h-full antialiased`}
    >
      <body className="min-h-full flex flex-col">{children}</body>
    </html>
  );
}
```

115. frontend\src\app\matcher\page.tsx

```
'use client'

import { useState } from 'react'
import Navbar from '@/components/Navbar'
import type { JDAnalysis } from '@/types'
import { Loader2, CheckCircle2, XCircle, Sparkles, ClipboardPaste } from 'lucide-react'
import { analyzeJD } from '@/lib/api-client'

export default function MatcherPage() {
  const [jd, setJd] = useState('')
  const [resumeText, setResumeText] = useState('')
  const [analysis, setAnalysis] = useState<JDAnalysis | null>(null)
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')

  const loadFromBuilder = () => {
    const saved = localStorage.getItem('resume-builder-text')
    if (saved) {
      setResumeText(saved)
      setError('')
    } else {
      setError('No resume found. Build one in the Builder or upload it in the Analyzer first.')
    }
  }

  const handleAnalyze = async () => {
    if (!jd.trim()) return
    setLoading(true)
    setError('')

    try {
      const result = await analyzeJD(jd.trim(), resumeText.trim())
      setAnalysis(result)
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Failed to analyze. Please try again.')
    } finally {
      setLoading(false)
    }
  }

  return (
    <>
      <Navbar />
      <main className="flex-1 max-w-5xl mx-auto px-4 py-8">
        <h1 className="text-2xl font-bold text-zinc-900 mb-2">Job Description Matcher</h1>
        <p className="text-zinc-600 mb-8">Paste a job description to see how your resume matches up.</p>

        <div className="grid grid-cols-1 lg:grid-cols-2 gap-6 mb-6">
          <div>
            <label className="block text-sm font-medium text-zinc-700 mb-2">Job Description</label>
            <textarea
              value={jd}
              onChange={(e) => setJd(e.target.value)}
              placeholder="Paste the full job description here..."
              rows={12}
              className="w-full px-4 py-3 border border-border rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-primary/20
focus:border-primary resize-none"
            />
          </div>
          <div>
            <div className="flex items-center justify-between mb-2">
              <label className="block text-sm font-medium text-zinc-700">Your Resume (optional)</label>
              <button
                onClick={loadFromBuilder}
                className="inline-flex items-center gap-1 text-xs text-primary hover:text-primary-dark"
              >
                <ClipboardPaste className="w-3 h-3" /> Load from Builder
              </button>
            </div>
            <textarea
              value={resumeText}
              onChange={(e) => setResumeText(e.target.value)}
              placeholder="Paste your resume text here, or click 'Load from Builder'..."
              rows={12}
              className="w-full px-4 py-3 border border-border rounded-xl text-sm focus:outline-none focus:ring-2 focus:ring-primary/20
focus:border-primary resize-none"
            />
          </div>
        </div>

        <button
          onClick={handleAnalyze}
          disabled={loading || !jd.trim()}
          className="inline-flex items-center gap-2 bg-primary text-white px-6 py-2.5 rounded-xl font-medium hover:bg-primary-dark
disabled:opacity-50 disabled:cursor-not-allowed transition-colors"
        >
          {loading ? <Loader2 className="w-4 h-4 animate-spin" /> : <Sparkles className="w-4 h-4" />}
          {loading ? 'Analyzing...' : 'Analyze Match'}
        </button>
```

```
        {error && (
          <div className="mt-6 p-4 bg-error/10 border border-error/20 rounded-xl">
            <p className="text-sm text-error">{error}</p>
          </div>
        )}

        {analysis && (
          <div className="mt-8 space-y-6">
            <div className="flex items-center gap-4 p-6 bg-zinc-50 rounded-xl">
              <div className="text-4xl font-bold text-primary">{analysis.matchPercentage}%</div>
              <div>
                <p className="font-semibold text-zinc-900">Overall Match</p>
                <p className="text-sm text-zinc-600">Based on keyword analysis of the job description</p>
              </div>
            </div>

            <div className="grid grid-cols-1 lg:grid-cols-2 gap-6">
              <div>
                <h3 className="font-semibold text-zinc-900 mb-3">
                  Matched Skills ({analysis.matchedSkills.length})
                </h3>
                <div className="space-y-2">
                  {analysis.matchedSkills.map((skill, i) => (
                    <div key={i} className="flex items-center gap-2 text-sm">
                      <CheckCircle2 className="w-4 h-4 text-success shrink-0" />
                      <span className="text-zinc-700">{skill.name}</span>
                    </div>
                  ))}
                </div>
              </div>
              <div>
                <h3 className="font-semibold text-zinc-900 mb-3">
                  Missing Skills ({analysis.missingSkills.length})
                </h3>
                <div className="space-y-2">
                  {analysis.missingSkills.map((skill, i) => (
                    <div key={i} className="flex items-center gap-2 text-sm">
                      <XCircle className="w-4 h-4 text-error shrink-0" />
                      <span className="text-zinc-700">{skill.name}</span>
                    </div>
                  ))}
                </div>
              </div>
            </div>

            {analysis.suggestions.length > 0 && (
              <div>
                <h3 className="font-semibold text-zinc-900 mb-3">Suggestions</h3>
                <div className="space-y-2">
                  {analysis.suggestions.map((s, i) => (
                    <p key={i} className="text-sm text-zinc-700 flex items-start gap-2">
                      <span className="text-primary mt-0.5">•</span>
                      {s}
                    </p>
                  ))}
                </div>
              </div>
            )}
          </div>
        )}
      </main>
    </>
  )
}
```

116. frontend\src\app\page.tsx

```
'use client'

import Link from 'next/link'
import Navbar from '@/components/Navbar'
import { ArrowRight, FileText, BarChart3, Search, Sparkles } from 'lucide-react'

export default function Home() {
  return (
    <>
      <Navbar />
      <main className="flex-1">
        <section className="max-w-5xl mx-auto px-4 py-20 text-center">
          <div className="inline-flex items-center gap-2 px-4 py-1.5 rounded-full bg-primary/10 text-primary text-sm font-medium mb-6">
            <Sparkles className="w-4 h-4" />
            AI-Powered ATS Optimization
          </div>
          <h1 className="text-5xl font-bold tracking-tight text-zinc-900 mb-4">
            Build Resumes That<br />Pass the ATS Test
          </h1>
          <p className="text-lg text-zinc-600 max-w-2xl mx-auto mb-10">
            Create professional, ATS-optimized resumes with AI-powered content assistance.
            Get real-time scoring, match against job descriptions, and export in PDF/DOCX.
          </p>
          <div className="flex items-center justify-center gap-4">
            <Link
              href="/builder"
              className="inline-flex items-center gap-2 bg-primary text-white px-6 py-3 rounded-xl font-medium hover:bg-primary-dark
transition-colors"
            >
              Start Building
              <ArrowRight className="w-4 h-4" />
            </Link>
            <Link
              href="/analyzer"
              className="inline-flex items-center gap-2 border border-border text-zinc-700 px-6 py-3 rounded-xl font-medium hover:bg-zinc-50
transition-colors"
            >
              Check ATS Score
            </Link>
          </div>
        </section>

        <section className="max-w-5xl mx-auto px-4 pb-20">
          <div className="grid grid-cols-1 md:grid-cols-3 gap-6">
            {[
              {
                icon: FileText,
                title: 'Smart Resume Builder',
                desc: 'Multi-step form with live preview. Choose from ATS-safe templates.',
              },
              {
                icon: BarChart3,
                title: 'ATS Score Analyzer',
                desc: 'Upload or build a resume and get a detailed 0-100 ATS compatibility score.',
              },
              {
                icon: Search,
                title: 'JD Matcher',
                desc: 'Paste a job description and see how your resume measures up.',
              },
            ].map(({ icon: Icon, title, desc }) => (
              <div key={title} className="border border-border rounded-xl p-6 hover:shadow-sm transition-shadow">
                <div className="w-10 h-10 rounded-lg bg-primary/10 flex items-center justify-center mb-4">
                  <Icon className="w-5 h-5 text-primary" />
                </div>
                <h3 className="font-semibold text-zinc-900 mb-1">{title}</h3>
                <p className="text-sm text-zinc-600">{desc}</p>
              </div>
            ))}
          </div>
        </section>
      </main>
    </>
  )
}
```

117. frontend\src\components\AtsScoreCard.tsx

```
'use client'

import type { ATSAnalysis } from '@/types'
import { CheckCircle2, AlertTriangle, Info, XCircle } from 'lucide-react'

interface Props {
  analysis: ATSAnalysis
}

export default function AtsScoreCard({ analysis }: Props) {
  const scoreColor = analysis.score >= 80 ? 'text-success' : analysis.score >= 60 ? 'text-warning' : 'text-error'
  const scoreBg = analysis.score >= 80 ? 'bg-success/10' : analysis.score >= 60 ? 'bg-warning/10' : 'bg-error/10'

  const severityIcon = (severity: string) => {
    switch (severity) {
      case 'error': return <XCircle className="w-4 h-4 text-error mt-0.5 shrink-0" />
      case 'warning': return <AlertTriangle className="w-4 h-4 text-warning mt-0.5 shrink-0" />
      case 'info': return <Info className="w-4 h-4 text-primary mt-0.5 shrink-0" />
      default: return <Info className="w-4 h-4 text-zinc-400 mt-0.5 shrink-0" />
    }
  }

  return (
    <div className="space-y-6">
      <div className={`rounded-xl ${scoreBg} p-6 text-center`}>
        <div className={`text-5xl font-bold ${scoreColor}`}>{analysis.score}</div>
        <p className={`text-sm font-medium mt-1 ${scoreColor}`}>out of 100</p>
        <p className="text-sm text-zinc-600 mt-3">{analysis.summary}</p>
      </div>

      {analysis.issues.length > 0 && (
        <div>
          <h3 className="font-semibold text-zinc-900 mb-3">Issues Found ({analysis.issues.length})</h3>
          <div className="space-y-3">
            {analysis.issues.map((issue) => (
              <div key={issue.id} className="border border-border rounded-lg p-3">
                <div className="flex items-start gap-2">
                  {severityIcon(issue.severity)}
                  <div className="flex-1 min-w-0">
                    <p className="text-sm font-medium text-zinc-900">{issue.message}</p>
                    <p className="text-xs text-zinc-500 mt-0.5">{issue.fix}</p>
                  </div>
                </div>
              </div>
            ))}
          </div>
        </div>
      )}

      {analysis.passedChecks.length > 0 && (
        <div>
          <h3 className="font-semibold text-zinc-900 mb-3">Passed Checks ({analysis.passedChecks.length})</h3>
          <div className="space-y-2">
            {analysis.passedChecks.map((check, i) => (
              <div key={i} className="flex items-start gap-2">
                <CheckCircle2 className="w-4 h-4 text-success mt-0.5 shrink-0" />
                <span className="text-sm text-zinc-700">{check}</span>
              </div>
            ))}
          </div>
        </div>
      )}
    </div>
  )
}
```

118. frontend\src\components\Navbar.tsx

```
'use client'

import Link from 'next/link'
import { usePathname } from 'next/navigation'
import { FileText, BarChart3, Search, Sparkles } from 'lucide-react'

const NAV_ITEMS = [
  { href: '/builder', label: 'Builder', icon: FileText },
  { href: '/analyzer', label: 'ATS Analyzer', icon: BarChart3 },
  { href: '/matcher', label: 'JD Matcher', icon: Search },
]

export default function Navbar() {
  const pathname = usePathname()

  return (
    <nav className="border-b border-border bg-white">
      <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
        <div className="flex items-center justify-between h-16">
          <Link href="/" className="flex items-center gap-2 font-semibold text-lg text-primary">
            <Sparkles className="w-5 h-5" />
            <span>ResumeAI</span>
          </Link>
          <div className="flex items-center gap-1">
            {NAV_ITEMS.map(({ href, label, icon: Icon }) => (
              <Link
                key={href}
                href={href}
                className={`flex items-center gap-1.5 px-3 py-2 rounded-lg text-sm font-medium transition-colors ${
                  pathname === href || pathname.startsWith(href + '/')
                    ? 'bg-primary/10 text-primary'
                    : 'text-zinc-600 hover:text-zinc-900 hover:bg-zinc-50'
                }`}
              >
                <Icon className="w-4 h-4" />
                {label}
              </Link>
            ))}
          </div>
        </div>
      </div>
    </nav>
  )
}
```

119. frontend\src\components\ResumeForm.tsx

```
'use client'

import { useState } from 'react'
import type { ResumeData, SectionId, WorkExperience, Education, Project, Certification, Skill } from '@/types'
import { Plus, Trash2, Sparkles, Loader2, Wand2 } from 'lucide-react'
import { improveBullet, generateSummary, suggestSkills } from '@/lib/api-client'

interface Props {
  data: ResumeData
  onUpdate: (updater: (prev: ResumeData) => ResumeData) => void
}

const SECTIONS: { id: SectionId; label: string }[] = [
  { id: 'contact', label: 'Contact Info' },
  { id: 'summary', label: 'Professional Summary' },
  { id: 'experience', label: 'Work Experience' },
  { id: 'education', label: 'Education' },
  { id: 'skills', label: 'Skills' },
  { id: 'projects', label: 'Projects' },
  { id: 'certifications', label: 'Certifications' },
]

export default function ResumeForm({ data, onUpdate }: Props) {
  const [activeSection, setActiveSection] = useState<SectionId>('contact')

  const set = <K extends keyof ResumeData>(key: K, value: ResumeData[K]) => {
    onUpdate((prev) => ({ ...prev, [key]: value }))
  }

  return (
    <div className="max-w-2xl mx-auto">
      <div className="flex gap-2 flex-wrap mb-6">
        {SECTIONS.map(({ id, label }) => (
          <button
            key={id}
            onClick={() => setActiveSection(id)}
            className={`px-3 py-1.5 text-xs font-medium rounded-full transition-colors ${
              activeSection === id
                ? 'bg-primary text-white'
                : 'bg-zinc-100 text-zinc-700 hover:bg-zinc-200'
            }`}
          >
            {label}
          </button>
        ))}
      </div>

      <div className="space-y-6">
        {activeSection === 'contact' && (
          <ContactForm contact={data.contact} onChange={(c) => set('contact', c)} />
        )}
        {activeSection === 'summary' && (
          <SummaryForm
            value={data.summary}
            contact={data.contact}
            skills={data.skills}
            onChange={(v) => set('summary', v)}
          />
        )}
        {activeSection === 'experience' && (
          <ExperienceForm items={data.experience} onChange={(v) => set('experience', v)} />
        )}
        {activeSection === 'education' && (
          <EducationForm items={data.education} onChange={(v) => set('education', v)} />
        )}
        {activeSection === 'skills' && (
          <SkillsForm items={data.skills} onChange={(v) => set('skills', v)} />
        )}
        {activeSection === 'projects' && (
          <ProjectsForm items={data.projects} onChange={(v) => set('projects', v)} />
        )}
        {activeSection === 'certifications' && (
          <CertsForm items={data.certifications} onChange={(v) => set('certifications', v)} />
        )}
      </div>
    </div>
  )
}

function ContactForm({ contact, onChange }: { contact: ResumeData['contact']; onChange: (c: ResumeData['contact']) => void }) {
  const fields: { key: keyof typeof contact; label: string; placeholder: string }[] = [
    { key: 'fullName', label: 'Full Name', placeholder: 'John Doe' },
    { key: 'email', label: 'Email', placeholder: 'john@example.com' },
    { key: 'phone', label: 'Phone', placeholder: '+1 (555) 123-4567' },
    { key: 'location', label: 'Location', placeholder: 'San Francisco, CA' },
    { key: 'linkedin', label: 'LinkedIn URL', placeholder: 'linkedin.com/in/johndoe' },
    { key: 'website', label: 'Website', placeholder: 'johndoe.com' },
  ]

  return (
```

```
    <div className="space-y-4">
      <h2 className="font-semibold text-zinc-900">Contact Information</h2>
      {fields.map(({ key, label, placeholder }) => (
        <div key={key}>
          <label className="block text-sm font-medium text-zinc-700 mb-1">{label}</label>
          <input
            type="text"
            value={contact[key] || ''}
            onChange={(e) => onChange({ ...contact, [key]: e.target.value })}
            placeholder={placeholder}
            className="w-full px-3 py-2 border border-border rounded-lg text-sm focus:outline-none focus:ring-2 focus:ring-primary/20
focus:border-primary"
          />
        </div>
      ))}
    </div>
  )
}

function SummaryForm({ value, contact, skills, onChange }: { value: string; contact: ResumeData['contact']; skills: Skill[]; onChange: (v:
string) => void }) {
  const [targetRole, setTargetRole] = useState('')
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')

  const handleGenerate = async () => {
    setLoading(true)
    setError('')
    try {
      const fallback = [contact.fullName, `Skills: ${skills.map((s) => s.name).join(', ')}`].filter(Boolean).join(' - ')
      const input = value.trim() || fallback
      const result = await generateSummary(input, targetRole.trim() || undefined)
      onChange(result.summary)
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Failed to generate summary')
    } finally {
      setLoading(false)
    }
  }

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <h2 className="font-semibold text-zinc-900">Professional Summary</h2>
        <button
          type="button"
          onClick={handleGenerate}
          disabled={loading}
          className="inline-flex items-center gap-1 text-xs text-primary hover:text-primary-dark disabled:opacity-50"
        >
          {loading ? <Loader2 className="w-3 h-3 animate-spin" /> : <Sparkles className="w-3 h-3" />}
          {loading ? 'Generating...' : 'AI Generate'}
        </button>
      </div>
      {error && <p className="text-xs text-error">{error}</p>}
      <input
        type="text"
        value={targetRole}
        onChange={(e) => setTargetRole(e.target.value)}
        placeholder="Target role (e.g. Senior Frontend Engineer) - optional"
        className="w-full px-3 py-2 border border-border rounded-lg text-sm focus:outline-none focus:ring-2 focus:ring-primary/20
focus:border-primary"
      />
      <textarea
        value={value}
        onChange={(e) => onChange(e.target.value)}
        placeholder="Write a brief professional summary describing your key qualifications and career goals... (or leave blank and let AI
draft one from your contact info)"
        rows={5}
        className="w-full px-3 py-2 border border-border rounded-lg text-sm focus:outline-none focus:ring-2 focus:ring-primary/20
focus:border-primary resize-none"
      />
      {value && <p className="text-xs text-zinc-400">Tip: click AI Generate to polish what you wrote.</p>}
    </div>
  )
}

function ExperienceForm({ items, onChange }: { items: WorkExperience[]; onChange: (v: WorkExperience[]) => void }) {
  const [improvingIdx, setImprovingIdx] = useState<{ expId: string; bulletIdx: number } | null>(null)
  const [error, setError] = useState('')

  const add = () => {
    onChange([...items, { id: crypto.randomUUID(), company: '', position: '', location: '', startDate: '', endDate: '', current: false,
bullets: [''] }])
  }
  const update = (id: string, upd: Partial<WorkExperience>) => {
    onChange(items.map((i) => (i.id === id ? { ...i, ...upd } : i)))
  }
  const remove = (id: string) => onChange(items.filter((i) => i.id !== id))

  const handleImprove = async (exp: WorkExperience, bulletIdx: number) => {
    const bullet = exp.bullets[bulletIdx]
    if (!bullet.trim()) return
```

```
    setImprovingIdx({ expId: exp.id, bulletIdx })
    setError('')
    try {
      const result = await improveBullet(bullet, exp.position || undefined)
      const newBullets = [...exp.bullets]
      newBullets[bulletIdx] = result.improved
      update(exp.id, { bullets: newBullets })
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Failed to improve bullet')
    } finally {
      setImprovingIdx(null)
    }
  }

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <h2 className="font-semibold text-zinc-900">Work Experience</h2>
        <button onClick={add} className="inline-flex items-center gap-1 text-xs text-primary hover:text-primary-dark">
          <Plus className="w-3 h-3" /> Add Experience
        </button>
      </div>
      {error && <p className="text-xs text-error">{error}</p>}
      {items.map((exp) => (
        <div key={exp.id} className="border border-border rounded-lg p-4 space-y-3">
          <div className="flex justify-between">
            <div className="grid grid-cols-2 gap-3 flex-1">
              <input type="text" value={exp.company} onChange={(e) => update(exp.id, { company: e.target.value })} placeholder="Company"
 className="px-3 py-2 border border-border rounded-lg text-sm" />
              <input type="text" value={exp.position} onChange={(e) => update(exp.id, { position: e.target.value })} placeholder="Position"
 className="px-3 py-2 border border-border rounded-lg text-sm" />
              <input type="text" value={exp.location} onChange={(e) => update(exp.id, { location: e.target.value })} placeholder="Location"
 className="px-3 py-2 border border-border rounded-lg text-sm" />
              <div className="flex gap-2">
                <input type="text" value={exp.startDate} onChange={(e) => update(exp.id, { startDate: e.target.value })} placeholder="Start
 date" className="flex-1 px-3 py-2 border border-border rounded-lg text-sm" />
                {!exp.current && <input type="text" value={exp.endDate} onChange={(e) => update(exp.id, { endDate: e.target.value })}
 placeholder="End date" className="flex-1 px-3 py-2 border border-border rounded-lg text-sm" />}
              </div>
            </div>
            <button onClick={() => remove(exp.id)} className="text-zinc-400 hover:text-error ml-2"><Trash2 className="w-4 h-4" /></button>
          </div>
          <label className="flex items-center gap-2 text-sm text-zinc-600">
            <input type="checkbox" checked={exp.current} onChange={(e) => update(exp.id, { current: e.target.checked })} className="rounded"
 />
            Currently working here
          </label>
          <div className="space-y-2">
            {exp.bullets.map((bullet, bi) => (
              <div key={bi} className="flex gap-2">
                <input
                  type="text"
                  value={bullet}
                  onChange={(e) => {
                    const newBullets = [...exp.bullets]
                    newBullets[bi] = e.target.value
                    update(exp.id, { bullets: newBullets })
                  }}
                  placeholder="Describe an achievement or responsibility..."
                  className="flex-1 px-3 py-2 border border-border rounded-lg text-sm"
                />
                <button
                  onClick={() => handleImprove(exp, bi)}
                  disabled={improvingIdx?.expId === exp.id && improvingIdx?.bulletIdx === bi}
                  title="Improve with AI"
                  className="text-primary hover:text-primary-dark disabled:opacity-40"
                >
                  {improvingIdx?.expId === exp.id && improvingIdx?.bulletIdx === bi ? (
                    <Loader2 className="w-3.5 h-3.5 animate-spin" />
                  ) : (
                    <Wand2 className="w-3.5 h-3.5" />
                  )}
                </button>
                {exp.bullets.length > 1 && (
                  <button onClick={() => update(exp.id, { bullets: exp.bullets.filter((_, i) => i !== bi) })} className="text-zinc-400
 hover:text-error"><Trash2 className="w-3 h-3" /></button>
                )}
              </div>
            ))}
            <button
              onClick={() => update(exp.id, { bullets: [...exp.bullets, ''] })}
              className="text-xs text-primary hover:text-primary-dark"
            >
              + Add bullet point
            </button>
            {exp.bullets.some((b) => b.trim()) && (
              <p className="text-xs text-zinc-400">Tip: click the wand icon on any bullet to rewrite it with AI.</p>
            )}
          </div>
        </div>
      ))}
    </div>
  )
```

```
}

function EducationForm({ items, onChange }: { items: Education[]; onChange: (v: Education[]) => void }) {
  const add = () => onChange([...items, { id: crypto.randomUUID(), institution: '', degree: '', field: '', startDate: '', endDate: '' }])
  const update = (id: string, upd: Partial<Education>) => onChange(items.map((i) => (i.id === id ? { ...i, ...upd } : i)))
  const remove = (id: string) => onChange(items.filter((i) => i.id !== id))

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <h2 className="font-semibold text-zinc-900">Education</h2>
        <button onClick={add} className="inline-flex items-center gap-1 text-xs text-primary"><Plus className="w-3 h-3" /> Add
Education</button>
      </div>
      {items.map((edu) => (
        <div key={edu.id} className="border border-border rounded-lg p-4 space-y-3">
          <div className="flex justify-between">
            <div className="grid grid-cols-2 gap-3 flex-1">
              <input type="text" value={edu.institution} onChange={(e) => update(edu.id, { institution: e.target.value })}
placeholder="Institution" className="px-3 py-2 border border-border rounded-lg text-sm" />
              <input type="text" value={edu.degree} onChange={(e) => update(edu.id, { degree: e.target.value })} placeholder="Degree"
className="px-3 py-2 border border-border rounded-lg text-sm" />
              <input type="text" value={edu.field} onChange={(e) => update(edu.id, { field: e.target.value })} placeholder="Field of study"
className="px-3 py-2 border border-border rounded-lg text-sm" />
              <input type="text" value={edu.gpa || ''} onChange={(e) => update(edu.id, { gpa: e.target.value })} placeholder="GPA (optional)"
className="px-3 py-2 border border-border rounded-lg text-sm" />
              <input type="text" value={edu.startDate} onChange={(e) => update(edu.id, { startDate: e.target.value })} placeholder="Start
date" className="px-3 py-2 border border-border rounded-lg text-sm" />
              <input type="text" value={edu.endDate} onChange={(e) => update(edu.id, { endDate: e.target.value })} placeholder="End date"
className="px-3 py-2 border border-border rounded-lg text-sm" />
            </div>
            <button onClick={() => remove(edu.id)} className="text-zinc-400 hover:text-error ml-2"><Trash2 className="w-4 h-4" /></button>
          </div>
        </div>
      ))}
    </div>
  )
}

function SkillsForm({ items, onChange }: { items: Skill[]; onChange: (v: Skill[]) => void }) {
  const [input, setInput] = useState('')
  const [targetRole, setTargetRole] = useState('')
  const [loading, setLoading] = useState(false)
  const [error, setError] = useState('')

  const addSkill = () => {
    if (!input.trim()) return
    onChange([...items, { id: crypto.randomUUID(), name: input.trim() }])
    setInput('')
  }

  const handleSuggest = async () => {
    if (!targetRole.trim()) return
    setLoading(true)
    setError('')
    try {
      const result = await suggestSkills(targetRole.trim(), items.map((s) => s.name))
      const existing = new Set(items.map((s) => s.name.toLowerCase()))
      const fresh = result.skills.filter((s) => !existing.has(s.toLowerCase()))
      if (fresh.length === 0) {
        setError('AI suggestions already in your list.')
      } else {
        onChange([...items, ...fresh.map((name) => ({ id: crypto.randomUUID(), name }))])
      }
    } catch (err) {
      setError(err instanceof Error ? err.message : 'Failed to suggest skills')
    } finally {
      setLoading(false)
    }
  }

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <h2 className="font-semibold text-zinc-900">Skills</h2>
      </div>
      <div className="grid grid-cols-1 sm:grid-cols-2 gap-2">
        <input
          type="text"
          value={targetRole}
          onChange={(e) => setTargetRole(e.target.value)}
          placeholder="Target role for AI suggestions..."
          className="px-3 py-2 border border-border rounded-lg text-sm"
        />
        <button
          onClick={handleSuggest}
          disabled={loading || !targetRole.trim()}
          className="inline-flex items-center justify-center gap-1 px-3 py-2 bg-primary text-white rounded-lg text-sm font-medium
disabled:opacity-50"
        >
          {loading ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Sparkles className="w-3.5 h-3.5" />}
          {loading ? 'Suggesting...' : 'AI Suggest Skills'}
        </button>
```

```
      </div>
      {error && <p className="text-xs text-error">{error}</p>}
      <div className="flex gap-2">
        <input
          type="text"
          value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => { if (e.key === 'Enter') { e.preventDefault(); addSkill() } }}
          placeholder="Type a skill and press Enter..."
          className="flex-1 px-3 py-2 border border-border rounded-lg text-sm"
        />
        <button onClick={addSkill} className="px-3 py-2 bg-primary text-white rounded-lg text-sm font-medium">Add</button>
      </div>
      <div className="flex flex-wrap gap-2">
        {items.map((skill) => (
          <span key={skill.id} className="inline-flex items-center gap-1 px-3 py-1 bg-zinc-100 rounded-full text-sm">
            {skill.name}
            <button onClick={() => onChange(items.filter((s) => s.id !== skill.id))} className="text-zinc-400 hover:text-error"><Trash2
className="w-3 h-3" /></button>
          </span>
        ))}
      </div>
    </div>
  )
}

function ProjectsForm({ items, onChange }: { items: Project[]; onChange: (v: Project[]) => void }) {
  const add = () => onChange([...items, { id: crypto.randomUUID(), name: '', description: '', url: '', technologies: [] }])
  const update = (id: string, upd: Partial<Project>) => onChange(items.map((i) => (i.id === id ? { ...i, ...upd } : i)))
  const remove = (id: string) => onChange(items.filter((i) => i.id !== id))

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <h2 className="font-semibold text-zinc-900">Projects</h2>
        <button onClick={add} className="inline-flex items-center gap-1 text-xs text-primary"><Plus className="w-3 h-3" /> Add
Project</button>
      </div>
      {items.map((proj) => (
        <div key={proj.id} className="border border-border rounded-lg p-4 space-y-3">
          <div className="flex justify-between">
            <div className="grid grid-cols-2 gap-3 flex-1">
              <input type="text" value={proj.name} onChange={(e) => update(proj.id, { name: e.target.value })} placeholder="Project name"
className="px-3 py-2 border border-border rounded-lg text-sm" />
              <input type="text" value={proj.url || ''} onChange={(e) => update(proj.id, { url: e.target.value })} placeholder="URL
(optional)" className="px-3 py-2 border border-border rounded-lg text-sm" />
            </div>
            <button onClick={() => remove(proj.id)} className="text-zinc-400 hover:text-error ml-2"><Trash2 className="w-4 h-4" /></button>
          </div>
          <textarea value={proj.description} onChange={(e) => update(proj.id, { description: e.target.value })} placeholder="Project
description..." rows={2} className="w-full px-3 py-2 border border-border rounded-lg text-sm resize-none" />
          <input type="text" value={proj.technologies.join(', ')} onChange={(e) => update(proj.id, { technologies:
e.target.value.split(',').map(s => s.trim()) })} placeholder="Technologies (comma separated)" className="w-full px-3 py-2 border border-border
rounded-lg text-sm" />
        </div>
      ))}
    </div>
  )
}

function CertsForm({ items, onChange }: { items: Certification[]; onChange: (v: Certification[]) => void }) {
  const add = () => onChange([...items, { id: crypto.randomUUID(), name: '', issuer: '', date: '' }])
  const update = (id: string, upd: Partial<Certification>) => onChange(items.map((i) => (i.id === id ? { ...i, ...upd } : i)))
  const remove = (id: string) => onChange(items.filter((i) => i.id !== id))

  return (
    <div className="space-y-4">
      <div className="flex items-center justify-between">
        <h2 className="font-semibold text-zinc-900">Certifications</h2>
        <button onClick={add} className="inline-flex items-center gap-1 text-xs text-primary"><Plus className="w-3 h-3" /> Add
Certification</button>
      </div>
      {items.map((cert) => (
        <div key={cert.id} className="border border-border rounded-lg p-4 flex justify-between">
          <div className="grid grid-cols-3 gap-3 flex-1">
            <input type="text" value={cert.name} onChange={(e) => update(cert.id, { name: e.target.value })} placeholder="Certification name"
className="px-3 py-2 border border-border rounded-lg text-sm" />
            <input type="text" value={cert.issuer} onChange={(e) => update(cert.id, { issuer: e.target.value })} placeholder="Issuer"
className="px-3 py-2 border border-border rounded-lg text-sm" />
            <div className="flex gap-2">
              <input type="text" value={cert.date} onChange={(e) => update(cert.id, { date: e.target.value })} placeholder="Date"
className="flex-1 px-3 py-2 border border-border rounded-lg text-sm" />
              <button onClick={() => remove(cert.id)} className="text-zinc-400 hover:text-error"><Trash2 className="w-4 h-4" /></button>
            </div>
          </div>
        </div>
      ))}
    </div>
  )
}
```

120. frontend\src\components\ResumePreview.tsx

```
'use client'

import type { ResumeData, TemplateId } from '@/types'

interface Props {
  data: ResumeData
  template: TemplateId
}

export default function ResumePreview({ data, template }: Props) {
  return (
    <div className="bg-white shadow-sm border border-border rounded-lg overflow-hidden">
      <div className="p-8 min-h-[600px]">
        {template === 'modern' && <ModernTemplate data={data} />}
        {template === 'classic' && <ClassicTemplate data={data} />}
        {template === 'minimal' && <MinimalTemplate data={data} />}
      </div>
    </div>
  )
}

function Header({ data, variant }: { data: ResumeData; variant: 'modern' | 'classic' | 'minimal' }) {
  const contactLine = [data.contact.email, data.contact.phone, data.contact.location].filter(Boolean).join(' | ')
  const linksLine = [data.contact.linkedin, data.contact.website].filter(Boolean).join(' | ')

  if (variant === 'classic') {
    return (
      <div className="text-center border-b-4 border-double border-zinc-800 pb-4 mb-6">
        <h1 className="font-serif text-3xl font-bold text-zinc-900 tracking-wide">{data.contact.fullName}</h1>
        {contactLine && <p className="font-serif text-xs text-zinc-700 mt-1.5">{contactLine}</p>}
        {linksLine && <p className="font-serif text-xs text-zinc-500 mt-0.5">{linksLine}</p>}
      </div>
    )
  }

  if (variant === 'minimal') {
    return (
      <div className="mb-6">
        <h1 className="text-2xl font-extralight uppercase tracking-[0.3em] text-zinc-900">{data.contact.fullName}</h1>
        {contactLine && <p className="text-xs text-zinc-500 mt-2">{contactLine}</p>}
        {linksLine && <p className="text-xs text-zinc-400 mt-0.5">{linksLine}</p>}
      </div>
    )
  }

  return (
    <div className="text-center pb-5 mb-6 border-b border-zinc-200">
      <h1 className="text-2xl font-bold text-zinc-900">{data.contact.fullName}</h1>
      {contactLine && <p className="text-xs text-zinc-600 mt-1.5">{contactLine}</p>}
      {linksLine && <p className="text-xs text-zinc-500 mt-0.5">{linksLine}</p>}
    </div>
  )
}

function SectionTitle({ title, variant }: { title: string; variant: 'modern' | 'classic' | 'minimal' }) {
  if (variant === 'classic') {
    return (
      <h2 className="font-serif text-sm font-bold uppercase tracking-wider text-zinc-900 mb-2.5">
        {title}
      </h2>
    )
  }
  if (variant === 'minimal') {
    return (
      <h2 className="text-[11px] font-semibold uppercase tracking-[0.25em] text-zinc-400 mb-2.5">
        {title}
      </h2>
    )
  }
  return (
    <h2 className="text-xs font-bold uppercase tracking-wider text-primary border-b border-zinc-200 pb-1.5 mb-3">
      {title}
    </h2>
  )
}

function ExperienceBlock({ data, variant }: { data: ResumeData; variant: 'modern' | 'classic' | 'minimal' }) {
  return (
    <>
      {data.experience.map((exp) => (
        <div key={exp.id} className="mb-4 last:mb-0">
          <div className="flex justify-between items-start">
            <div>
              <p className={`font-semibold ${variant === 'classic' ? 'font-serif text-sm' : 'text-sm'} text-zinc-900`}>
                {exp.company}
              </p>
              <p className="text-xs text-zinc-600">{exp.position}{exp.location ? ` | ${exp.location}` : ''}</p>
            </div>
            <p className="text-xs text-zinc-500 whitespace-nowrap ml-4">{exp.startDate} - {exp.current ? 'Present' : exp.endDate}</p>
          </div>
```

```
          {exp.bullets.filter(b => b.trim()).map((bullet, i) => (
            <p key={i} className="text-xs text-zinc-700 mt-1 pl-3">• {bullet}</p>
          ))}
        </div>
      ))}
    </>
  )
}

function SkillsBlock({ data, variant }: { data: ResumeData; variant: 'modern' | 'classic' | 'minimal' }) {
  if (variant === 'minimal') {
    return (
      <p className="text-xs leading-relaxed text-zinc-700">
        {data.skills.map((s) => s.name).filter(Boolean).join(' · ')}
      </p>
    )
  }
  return (
    <div className="flex flex-wrap gap-1.5">
      {data.skills.map((skill) => (
        <span
          key={skill.id}
          className={
            variant === 'classic'
              ? 'text-xs text-zinc-700'
              : 'text-[11px] px-2 py-0.5 bg-primary/5 text-primary rounded-full'
          }
        >
          {skill.name}
        </span>
      ))}
    </div>
  )
}

function ModernTemplate({ data }: { data: ResumeData }) {
  return (
    <div>
      <Header data={data} variant="modern" />
      <div className="space-y-5">
        {data.summary && (
          <div>
            <SectionTitle title="Professional Summary" variant="modern" />
            <p className="text-xs leading-relaxed text-zinc-700">{data.summary}</p>
          </div>
        )}
        {data.experience.length > 0 && (
          <div>
            <SectionTitle title="Experience" variant="modern" />
            <ExperienceBlock data={data} variant="modern" />
          </div>
        )}
        {data.education.length > 0 && (
          <div>
            <SectionTitle title="Education" variant="modern" />
            {data.education.map((edu) => (
              <div key={edu.id} className="flex justify-between items-start mb-2 last:mb-0">
                <div>
                  <p className="text-sm font-semibold text-zinc-900">{edu.institution}</p>
                  <p className="text-xs text-zinc-600">{edu.degree} in {edu.field}{edu.gpa ? ` | GPA: ${edu.gpa}` : ''}</p>
                </div>
                <p className="text-xs text-zinc-500 whitespace-nowrap ml-4">{edu.startDate} - {edu.endDate}</p>
              </div>
            ))}
          </div>
        )}
        {data.skills.length > 0 && (
          <div>
            <SectionTitle title="Skills" variant="modern" />
            <SkillsBlock data={data} variant="modern" />
          </div>
        )}
        {data.projects.length > 0 && (
          <div>
            <SectionTitle title="Projects" variant="modern" />
            {data.projects.map((proj) => (
              <div key={proj.id} className="mb-3 last:mb-0">
                <p className="text-sm font-semibold text-zinc-900">{proj.name}{proj.url ? <span className="text-xs text-primary font-normal
ml-1">{proj.url}</span> : ''}</p>
                <p className="text-xs text-zinc-700 mt-0.5">{proj.description}</p>
                {proj.technologies.length > 0 && (
                  <p className="text-xs text-zinc-500 mt-0.5">Technologies: {proj.technologies.join(', ')}</p>
                )}
              </div>
            ))}
          </div>
        )}
        {data.certifications.length > 0 && (
          <div>
            <SectionTitle title="Certifications" variant="modern" />
            {data.certifications.map((cert) => (
              <div key={cert.id} className="mb-1.5 last:mb-0">
                <p className="text-sm font-semibold text-zinc-900">{cert.name}</p>
```

```
              <p className="text-xs text-zinc-600">{cert.issuer} | {cert.date}</p>
            </div>
          ))}
        </div>
      )}
    </div>
  </div>
  )
}

function ClassicTemplate({ data }: { data: ResumeData }) {
  return (
    <div className="font-serif">
      <Header data={data} variant="classic" />
      <div className="space-y-5">
        {data.summary && (
          <div>
            <SectionTitle title="Professional Summary" variant="classic" />
            <p className="text-xs leading-relaxed text-zinc-800">{data.summary}</p>
          </div>
        )}
        {data.experience.length > 0 && (
          <div>
            <SectionTitle title="Experience" variant="classic" />
            <ExperienceBlock data={data} variant="classic" />
          </div>
        )}
        {data.education.length > 0 && (
          <div>
            <SectionTitle title="Education" variant="classic" />
            {data.education.map((edu) => (
              <div key={edu.id} className="flex justify-between items-start mb-2 last:mb-0">
                <div>
                  <p className="font-serif text-sm font-semibold text-zinc-900">{edu.institution}</p>
                  <p className="font-serif text-xs text-zinc-700">{edu.degree} in {edu.field}{edu.gpa ? ` | GPA: ${edu.gpa}` : ''}</p>
                </div>
                <p className="font-serif text-xs text-zinc-500 whitespace-nowrap ml-4">{edu.startDate} - {edu.endDate}</p>
              </div>
            ))}
          </div>
        )}
        {data.skills.length > 0 && (
          <div>
            <SectionTitle title="Skills" variant="classic" />
            <SkillsBlock data={data} variant="classic" />
          </div>
        )}
        {data.projects.length > 0 && (
          <div>
            <SectionTitle title="Projects" variant="classic" />
            {data.projects.map((proj) => (
              <div key={proj.id} className="mb-3 last:mb-0">
                <p className="font-serif text-sm font-semibold text-zinc-900">{proj.name}{proj.url ? <span className="text-xs font-normal
text-zinc-500 ml-1">{proj.url}</span> : ''}</p>
                <p className="font-serif text-xs text-zinc-700 mt-0.5">{proj.description}</p>
                {proj.technologies.length > 0 && (
                  <p className="font-serif text-xs text-zinc-500 mt-0.5">Technologies: {proj.technologies.join(', ')}</p>
                )}
              </div>
            ))}
          </div>
        )}
        {data.certifications.length > 0 && (
          <div>
            <SectionTitle title="Certifications" variant="classic" />
            {data.certifications.map((cert) => (
              <div key={cert.id} className="mb-1.5 last:mb-0">
                <p className="font-serif text-sm font-semibold text-zinc-900">{cert.name}</p>
                <p className="font-serif text-xs text-zinc-600">{cert.issuer} | {cert.date}</p>
              </div>
            ))}
          </div>
        )}
      </div>
    </div>
  )
}

function MinimalTemplate({ data }: { data: ResumeData }) {
  return (
    <div>
      <Header data={data} variant="minimal" />
      <div className="space-y-6">
        {data.summary && (
          <div>
            <SectionTitle title="Professional Summary" variant="minimal" />
            <p className="text-xs leading-relaxed text-zinc-600">{data.summary}</p>
          </div>
        )}
        {data.experience.length > 0 && (
          <div>
            <SectionTitle title="Experience" variant="minimal" />
            <ExperienceBlock data={data} variant="minimal" />
```

```
          </div>
        )}
        {data.education.length > 0 && (
          <div>
            <SectionTitle title="Education" variant="minimal" />
            {data.education.map((edu) => (
              <div key={edu.id} className="flex justify-between items-start mb-2 last:mb-0">
                <div>
                  <p className="text-sm font-medium text-zinc-900">{edu.institution}</p>
                  <p className="text-xs text-zinc-500">{edu.degree} in {edu.field}{edu.gpa ? ` | GPA: ${edu.gpa}` : ''}</p>
                </div>
                <p className="text-xs text-zinc-400 whitespace-nowrap ml-4">{edu.startDate} - {edu.endDate}</p>
              </div>
            ))}
          </div>
        )}
        {data.skills.length > 0 && (
          <div>
            <SectionTitle title="Skills" variant="minimal" />
            <SkillsBlock data={data} variant="minimal" />
          </div>
        )}
        {data.projects.length > 0 && (
          <div>
            <SectionTitle title="Projects" variant="minimal" />
            {data.projects.map((proj) => (
              <div key={proj.id} className="mb-3 last:mb-0">
                <p className="text-sm font-medium text-zinc-900">{proj.name}{proj.url ? <span className="text-xs text-zinc-400 font-normal ml-
1">{proj.url}</span> : ''}</p>
                <p className="text-xs text-zinc-600 mt-0.5">{proj.description}</p>
                {proj.technologies.length > 0 && (
                  <p className="text-xs text-zinc-400 mt-0.5">Technologies: {proj.technologies.join(', ')}</p>
                )}
              </div>
            ))}
          </div>
        )}
        {data.certifications.length > 0 && (
          <div>
            <SectionTitle title="Certifications" variant="minimal" />
            {data.certifications.map((cert) => (
              <div key={cert.id} className="mb-1.5 last:mb-0">
                <p className="text-sm font-medium text-zinc-900">{cert.name}</p>
                <p className="text-xs text-zinc-500">{cert.issuer} | {cert.date}</p>
              </div>
            ))}
          </div>
        )}
      </div>
    </div>
  )
}
```

121. frontend\src\lib\api-client.ts

```
import type { ResumeData, ATSAnalysis } from '@/types'

const API_BASE = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000'

async function request<T>(path: string, options?: RequestInit): Promise<T> {
  const res = await fetch(`${API_BASE}${path}`, {
    headers: { 'Content-Type': 'application/json', ...options?.headers },
    ...options,
  })
  if (!res.ok) {
    const err = await res.json().catch(() => ({ detail: res.statusText }))
    throw new Error(err.detail || 'Request failed')
  }
  return res.json()
}

export async function parseResume(file: File) {
  const formData = new FormData()
  formData.append('file', file)
  const res = await fetch(`${API_BASE}/api/resume/parse`, { method: 'POST', body: formData })
  if (!res.ok) {
    const err = await res.json().catch(() => ({ detail: res.statusText }))
    throw new Error(err.detail || 'Failed to parse resume')
  }
  return res.json()
}

export async function scoreResume(data: unknown): Promise<ATSAnalysis> {
  return request<ATSAnalysis>('/api/ats/score', {
    method: 'POST',
    body: JSON.stringify({ resumeData: data }),
  })
}

export async function analyzeJD(jd: string, resumeText?: string) {
  return request<{
    matchedSkills: { name: string; matched: boolean }[]
    missingSkills: { name: string; matched: boolean }[]
    matchPercentage: number
    keywords: string[]
    suggestions: string[]
  }>('/api/ai/analyze-jd', {
    method: 'POST',
    body: JSON.stringify({ jd, resumeText: resumeText || '' }),
  })
}

export async function improveBullet(bullet: string, role?: string) {
  return request<{ original: string; improved: string }>('/api/ai/improve-bullet', {
    method: 'POST',
    body: JSON.stringify({ bullet, role }),
  })
}

export async function generateSummary(input: string, targetRole?: string) {
  return request<{ summary: string }>('/api/ai/generate-summary', {
    method: 'POST',
    body: JSON.stringify({ input, targetRole }),
  })
}

export async function suggestSkills(targetRole: string, currentSkills?: string[]) {
  return request<{ skills: string[] }>('/api/ai/suggest-skills', {
    method: 'POST',
    body: JSON.stringify({ targetRole, currentSkills }),
  })
}

export async function tailorResume(resumeJson: unknown, jd: string) {
  return request<{ tailored: unknown }>('/api/ai/tailor-resume', {
    method: 'POST',
    body: JSON.stringify({ resumeJson, jd }),
  })
}

export async function exportPDF(data: unknown): Promise<Blob> {
  const res = await fetch(`${API_BASE}/api/export/pdf`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
  })
  if (!res.ok) throw new Error('PDF export failed')
  return res.blob()
}

export async function exportDOCX(data: unknown): Promise<Blob> {
  const res = await fetch(`${API_BASE}/api/export/docx`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(data),
```

```
  })
  if (!res.ok) throw new Error('DOCX export failed')
  return res.blob()
}

export interface SavedResumeSummary {
  id: string
  title: string
  template: string
  score: number | null
  updatedAt: string
}

export interface SavedResume extends SavedResumeSummary {
  data: ResumeData
}

export async function saveResume(userId: string, title: string, data: ResumeData, template: string): Promise<SavedResumeSummary> {
  return request<SavedResumeSummary>('/api/resumes', {
    method: 'POST',
    body: JSON.stringify({ userId, title, data, template }),
  })
}

export async function listResumes(userId: string): Promise<SavedResumeSummary[]> {
  return request<SavedResumeSummary[]>(`/api/resumes?userId=${encodeURIComponent(userId)}`)
}

export async function getResume(id: string): Promise<SavedResume> {
  return request<SavedResume>(`/api/resumes/${id}`)
}

export async function updateResume(id: string, patch: { title?: string; data?: ResumeData; template?: string; score?: number }):
Promise<SavedResumeSummary> {
  return request<SavedResumeSummary>(`/api/resumes/${id}`, {
    method: 'PUT',
    body: JSON.stringify(patch),
  })
}

export async function deleteResume(id: string): Promise<unknown> {
  return request<unknown>(`/api/resumes/${id}`, { method: 'DELETE' })
}

export function downloadBlob(blob: Blob, filename: string) {
  const url = URL.createObjectURL(blob)
  const a = document.createElement('a')
  a.href = url
  a.download = filename
  document.body.appendChild(a)
  a.click()
  a.remove()
  URL.revokeObjectURL(url)
}
```

122. frontend\src\lib\resume-text.ts

```
import type { ResumeData } from '@/types'

export function resumeToText(data: ResumeData): string {
  const lines: string[] = []

  if (data.contact.fullName) lines.push(data.contact.fullName)
  const contact = [
    data.contact.email,
    data.contact.phone,
    data.contact.location,
    data.contact.linkedin,
    data.contact.website,
  ].filter(Boolean).join(' | ')
  if (contact) lines.push(contact)

  if (data.summary) {
    lines.push('')
    lines.push('SUMMARY')
    lines.push(data.summary)
  }

  if (data.experience.length) {
    lines.push('')
    lines.push('EXPERIENCE')
    for (const exp of data.experience) {
      lines.push(`${exp.company} - ${exp.position} (${exp.startDate} - ${exp.current ? 'Present' : exp.endDate})`)
      for (const b of exp.bullets) if (b.trim()) lines.push(`- ${b}`)
    }
  }

  if (data.education.length) {
    lines.push('')
    lines.push('EDUCATION')
    for (const edu of data.education) {
      lines.push(`${edu.institution} - ${edu.degree} in ${edu.field} (${edu.startDate} - ${edu.endDate})`)
    }
  }

  if (data.skills.length) {
    lines.push('')
    lines.push('SKILLS')
    lines.push(data.skills.map((s) => s.name).join(', '))
  }

  if (data.projects.length) {
    lines.push('')
    lines.push('PROJECTS')
    for (const proj of data.projects) {
      lines.push(`${proj.name}${proj.url ? ` (${proj.url})` : ''}`)
      if (proj.description) lines.push(proj.description)
      if (proj.technologies.length) lines.push(`Technologies: ${proj.technologies.join(', ')}`)
    }
  }

  if (data.certifications.length) {
    lines.push('')
    lines.push('CERTIFICATIONS')
    for (const cert of data.certifications) {
      lines.push(`${cert.name} - ${cert.issuer} (${cert.date})`)
    }
  }

  return lines.join('\n')
}
```

123. frontend\src\types\index.ts

```
export * from './resume'
```

124. frontend\src\types\resume.ts

```
export interface ContactInfo {
  fullName: string
  email: string
  phone: string
  location: string
  linkedin?: string
  website?: string
}

export interface WorkExperience {
  id: string
  company: string
  position: string
  location: string
  startDate: string
  endDate: string
  current: boolean
  bullets: string[]
}

export interface Education {
  id: string
  institution: string
  degree: string
  field: string
  startDate: string
  endDate: string
  gpa?: string
}

export interface Skill {
  id: string
  name: string
  level?: 'beginner' | 'intermediate' | 'advanced' | 'expert'
}

export interface Project {
  id: string
  name: string
  description: string
  url?: string
  technologies: string[]
}

export interface Certification {
  id: string
  name: string
  issuer: string
  date: string
  url?: string
}

export interface ResumeData {
  contact: ContactInfo
  summary: string
  experience: WorkExperience[]
  education: Education[]
  skills: Skill[]
  projects: Project[]
  certifications: Certification[]
}

export type TemplateId = 'modern' | 'classic' | 'minimal'

export type SectionId = 'contact' | 'summary' | 'experience' | 'education' | 'skills' | 'projects' | 'certifications'

export interface ATSIssue {
  id: string
  severity: 'error' | 'warning' | 'info'
  category: string
  message: string
  fix: string
}

export interface ATSAnalysis {
  score: number
  issues: ATSIssue[]
  passedChecks: string[]
  summary: string
}

export interface SkillMatch {
  name: string
  matched: boolean
  category?: string
}

export interface JDAnalysis {
  matchedSkills: SkillMatch[]
  missingSkills: SkillMatch[]
```

```
  matchPercentage: number
  keywords: string[]
  suggestions: string[]
}
```

125. frontend\tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2017",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": true,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "react-jsx",
    "incremental": true,
    "plugins": [
      {
        "name": "next"
      }
    ],
    "paths": {
      "@/*": ["./src/*"]
    }
  },
  "include": [
    "next-env.d.ts",
    "**/*.ts",
    "**/*.tsx",
    ".next/types/**/*.ts",
    ".next/dev/types/**/*.ts",
    "**/*.mts"
  ],
  "exclude": ["node_modules"]
}
```

126. mobile\.claude\settings.json

```
{
  "enabledPlugins": {
    "expo@claude-plugins-official": true
  }
}
```

127. mobile\.gitignore

```
# Learn more https://docs.github.com/en/get-started/getting-started-with-git/ignoring-files

# dependencies
node_modules/

# Expo
.expo/
dist/
web-build/
expo-env.d.ts

# Native
.kotlin/
*.orig.*
*.jks
*.p8
*.p12
*.key
*.mobileprovision

# Metro
.metro-health-check*

# debug
npm-debug.*
yarn-debug.*
yarn-error.*

# macOS
.DS_Store
*.pem

# local env files
.env*.local
.env

# typescript
*.tsbuildinfo

# generated native folders
/ios
/android
```

128. mobile\AGENTS.md

```
# Expo HAS CHANGED

Read the exact versioned docs at https://docs.expo.dev/versions/v57.0.0/ before writing any code.
```

129. mobile\app.config.js

```
export default {
  expo: {
    name: "AI Resume Builder",
    slug: "ai-resume-builder",
    scheme: "resumebuilder",
    version: "1.0.0",
    orientation: "portrait",
    icon: "./assets/icon.png",
    userInterfaceStyle: "light",
    ios: {
      supportsTablet: true,
      bundleIdentifier: "com.resumebuilder.app",
      infoPlist: {
        LSSupportsOpeningDocumentsInPlace: true,
        UIFileSharingEnabled: true,
      },
    },
    android: {
      package: "com.resumebuilder.app",
      adaptiveIcon: {
        backgroundColor: "#E6F4FE",
        foregroundImage: "./assets/android-icon-foreground.png",
        backgroundImage: "./assets/android-icon-background.png",
        monochromeImage: "./assets/android-icon-monochrome.png",
      },
      predictiveBackGestureEnabled: false,
    },
    web: {
      favicon: "./assets/favicon.png",
    },
    plugins: [
      "expo-sharing",
      [
        "expo-file-system",
        {
          supportsOpeningDocumentsInPlace: true,
          enableFileSharing: true,
        },
      ],
    ],
    extra: {
      apiUrl: process.env.EXPO_PUBLIC_API_URL ?? "http://localhost:8000",
    },
  },
};
```

130. mobile\App.tsx

```
import React from "react";

import { NavigationProvider, Shell, useNav } from "./src/navigation";
import { DashboardScreen } from "./src/screens/Dashboard";
import { BuilderScreen } from "./src/screens/Builder";
import { PreviewScreen } from "./src/screens/Preview";
import { MatcherScreen } from "./src/screens/Matcher";

function ActiveScreen() {
  const { screen } = useNav();
  switch (screen.name) {
    case "dashboard":
      return <DashboardScreen />;
    case "builder":
      return <BuilderScreen resumeId={screen.resumeId} title={screen.title} targetRole={screen.targetRole} data={screen.data} />;
    case "preview":
      return <PreviewScreen resumeId={screen.resumeId} title={screen.title} data={screen.data} unsaved={screen.unsaved} />;
    case "matcher":
      return <MatcherScreen resumeId={screen.resumeId} resume={screen.resume} />;
    default:
      return <DashboardScreen />;
  }
}

export default function App() {
  return (
    <NavigationProvider>
      <Shell>
        <ActiveScreen />
      </Shell>
    </NavigationProvider>
  );
}
```

131. mobile\CLAUDE.md

@AGENTS.md

132. mobile\eas.json

```
{
  "cli": {
    "version": ">= 14.0.0",
    "appVersionSource": "remote"
  },
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal",
      "env": {
        "ENVIRONMENT": "development"
      }
    },
    "preview": {
      "distribution": "internal",
      "env": {
        "ENVIRONMENT": "preview"
      }
    },
    "production": {
      "autoIncrement": true,
      "env": {
        "ENVIRONMENT": "production"
      }
    }
  },
  "submit": {
    "production": {}
  }
}
```

133. mobile\index.ts

```
import { registerRootComponent } from 'expo';

import App from './App';

// registerRootComponent calls AppRegistry.registerComponent('main', () => App);
// It also ensures that whether you load the app in Expo Go or in a native build,
// the environment is set up appropriately
registerRootComponent(App);
```

134. mobile\package.json

```
{
  "name": "mobile",
  "version": "1.0.0",
  "main": "index.ts",
  "dependencies": {
    "expo": "~57.0.20",
    "expo-file-system": "~57.0.6",
    "expo-sharing": "~57.0.18",
    "expo-status-bar": "~57.0.1",
    "react": "19.2.3",
    "react-native": "0.86.3"
  },
  "devDependencies": {
    "@types/react": "~19.2.2",
    "typescript": "~6.0.3"
  },
  "scripts": {
    "start": "expo start",
    "android": "expo start --android",
    "ios": "expo start --ios",
    "web": "expo start --web"
  },
  "private": true
}
```

135. mobile\src\api.ts

```
import { Platform } from "react-native";
import { fetch as expoFetch } from "expo/fetch";

import type {
  ATSAnalysis,
  GenerateSummaryResult,
  ImproveBulletResult,
  InsertKeywordsResult,
  JDAnalysis,
  ResumeData,
  ResumeMeta,
  SuggestSkillsResult,
  TailorResult,
} from "./types";

// Inlined by Expo CLI at build time (must be referenced with dot notation).
export const API_BASE = (process.env.EXPO_PUBLIC_API_URL ?? "http://10.0.2.2:8000").replace(/\/+$/, "");
export const API = `${API_BASE}/api`;

export function apiDoc(): string {
  return Platform.OS === "android" && API_BASE.includes("10.0.2.2")
    ? "Emulator → host backend. Set EXPO_PUBLIC_API_URL in .env for a device."
    : `Backend: ${API_BASE}`;
}

async function json<T>(path: string, init?: RequestInit): Promise<T> {
  const res = await fetch(`${API}${path}`, {
    ...init,
    headers: { "Content-Type": "application/json", ...(init?.headers ?? {}) },
  });
  if (!res.ok) {
    let detail = `HTTP ${res.status}`;
    try {
      const body = await res.json();
      detail = typeof body.detail === "string" ? body.detail : detail;
    } catch {
      // non-JSON error response
    }
    throw new Error(detail);
  }
  return res.json() as Promise<T>;
}

export const resumesApi = {
  list: (userId: string) => json<ResumeMeta[]>(`/resumes?userId=${encodeURIComponent(userId)}`),
  get: (userId: string, id: string) => json<ResumeMeta>(`/resumes/${encodeURIComponent(id)}?userId=${encodeURIComponent(userId)}`),
  save: (userId: string, title: string, data: ResumeData, template = "modern", targetRole?: string | null) =>
    json<ResumeMeta>("/resumes", {
      method: "POST",
      body: JSON.stringify({ userId, title, data, template, targetRole: targetRole ?? null }),
    }),
  update: (userId: string, id: string, patch: { title?: string; data?: ResumeData; score?: number | null; targetRole?: string | null }) =>
    json<ResumeMeta>(`/resumes/${encodeURIComponent(id)}`, {
      method: "PUT",
      body: JSON.stringify({ userId, ...patch }),
    }),
  remove: (userId: string, id: string) =>
    json<{ deleted: string }>(`/resumes/${encodeURIComponent(id)}?userId=${encodeURIComponent(userId)}`, { method: "DELETE" }),
  clone: (userId: string, id: string, title?: string, targetRole?: string | null) =>
    json<ResumeMeta & { data: ResumeData }>(`/resumes/${encodeURIComponent(id)}/clone`, {
      method: "POST",
      body: JSON.stringify({ userId, title, targetRole: targetRole ?? null }),
    }),
  insertKeywords: (userId: string, id: string, keywords: string[], targetSection: "skills" | "summary") =>
    json<InsertKeywordsResult>(`/resumes/${encodeURIComponent(id)}/insert-keywords`, {
      method: "POST",
      body: JSON.stringify({ userId, keywords, targetSection }),
    }),
};

export const aiApi = {
  improveBullet: (bullet: string, role?: string | null) =>
    json<ImproveBulletResult>("/ai/improve-bullet", { method: "POST", body: JSON.stringify({ bullet, role }) }),
  generateSummary: (input: string, targetRole?: string | null) =>
    json<GenerateSummaryResult>("/ai/generate-summary", { method: "POST", body: JSON.stringify({ input, targetRole }) }),
  suggestSkills: (targetRole: string, currentSkills: string[] = []) =>
    json<SuggestSkillsResult>("/ai/suggest-skills", { method: "POST", body: JSON.stringify({ targetRole, currentSkills }) }),
  tailorResume: (resumeData: ResumeData, jd: string) =>
    json<TailorResult>("/ai/tailor-resume", { method: "POST", body: JSON.stringify({ resumeJson: resumeData, jd }) }),
  analyzeJd: (jd: string, resumeText: string = "") =>
    json<JDAnalysis>("/ai/analyze-jd", { method: "POST", body: JSON.stringify({ jd, resumeText }) }),
};

export const atsApi = {
  score: (resumeData: ResumeData) => json<ATSAnalysis>("/ats/score", { method: "POST", body: JSON.stringify({ resumeData }) }),
};

export const exportApi = {
  text: (data: ResumeData) => json<{ text: string }>("/export/text", { method: "POST", body: JSON.stringify(data) }),
  async pdfBytes(data: ResumeData): Promise<Uint8Array> {
```

```
    const res = await expoFetch(`${API}/export/pdf`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(data),
    });
    if (!res.ok) {
      throw new Error(`PDF export failed (HTTP ${res.status})`);
    }
    return res.bytes();
  },
};
```

136. mobile\src\components.tsx

```
import React from "react";
import {
  ActivityIndicator,
  Pressable,
  StyleSheet,
  Text,
  TextInput,
  TextInputProps,
  View,
  ViewStyle,
} from "react-native";

import { colors, radius, spacing } from "./theme";

type Variant = "primary" | "secondary" | "outline" | "danger" | "ghost";

interface ButtonProps {
  title: string;
  onPress: () => void;
  variant?: Variant;
  loading?: boolean;
  disabled?: boolean;
  small?: boolean;
  style?: ViewStyle;
}

export function Button({ title, onPress, variant = "primary", loading, disabled, small, style }: ButtonProps) {
  const palette: Record<Variant, { bg: string; fg: string; border?: string }> = {
    primary: { bg: colors.primary, fg: "#FFFFFF" },
    secondary: { bg: colors.surfaceMuted, fg: colors.text },
    outline: { bg: "transparent", fg: colors.primary, border: colors.border },
    danger: { bg: colors.danger, fg: "#FFFFFF" },
    ghost: { bg: "transparent", fg: colors.primary },
  };
  const p = palette[variant];
  const isDisabled = disabled || loading;
  return (
    <Pressable
      accessibilityRole="button"
      onPress={onPress}
      disabled={isDisabled}
      style={{[
        styles.button,
        { backgroundColor: p.bg, borderColor: p.border ?? "transparent", borderWidth: p.border ? 1 : 0 },
        small && styles.buttonSmall,
        isDisabled && styles.disabled,
        style,
      ]}
    >
      {loading ? <ActivityIndicator color={p.fg} /> : <Text style={[styles.buttonText, { color: p.fg }, small && styles.buttonTextSmall]}>
{title}</Text>}
    </Pressable>
  );
}

interface TextFieldProps extends TextInputProps {
  label?: string;
  editable?: boolean;
}

export function TextField({ label, editable = true, style, ...props }: TextFieldProps) {
  return (
    <View style={styles.field}>
      {label ? <Text style={styles.label}>{label}</Text> : null}
      <TextInput
        {...props}
        editable={editable}
        placeholderTextColor={colors.textMuted}
        style={[styles.input, editable === false && styles.inputDisabled, style]}
      />
    </View>
  );
}

export function Card({ children, style }: { children: React.ReactNode; style?: ViewStyle }) {
  return <View style={[styles.card, style]}>{children}</View>;
}

export function SectionHeading({ title, subtitle }: { title: string; subtitle?: string }) {
  return (
    <View style={styles.sectionHeading}>
      <Text style={styles.sectionTitle}>{title}</Text>
      {subtitle ? <Text style={styles.sectionSubtitle}>{subtitle}</Text> : null}
    </View>
  );
}

export function Chip({ label, onPress, tone = "default" }: { label: string; onPress?: () => void; tone?: "default" | "match" | "missing" }) {
  const bg = tone === "match" ? "#DCFCE7" : tone === "missing" ? "#FEE2E2" : colors.surfaceMuted;
  const fg = tone === "match" ? colors.success : tone === "missing" ? colors.danger : colors.text;
```

```
  return (
    <Pressable disabled={!onPress} onPress={onPress} style={[styles.chip, { backgroundColor: bg }]}>
      <Text style={[styles.chipText, { color: fg }]}>{label}</Text>
    </Pressable>
  );
}

export function Badge({ score }: { score: number }) {
  const color = score >= 85 ? colors.success : score >= 70 ? colors.warning : colors.danger;
  return (
    <View style={[styles.badge, { backgroundColor: color }]}>
      <Text style={styles.badgeText}>{score}</Text>
    </View>
  );
}

export function ErrorBanner({ message }: { message: string }) {
  if (!message) return null;
  return (
    <View style={styles.errorBanner}>
      <Text style={styles.errorText}>{message}</Text>
    </View>
  );
}

export function Loading({ label = "Loading…" }: { label?: string }) {
  return (
    <View style={styles.loading}>
      <ActivityIndicator color={colors.primary} />
      <Text style={styles.loadingText}>{label}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  button: {
    borderRadius: radius.md,
    paddingVertical: spacing.md,
    paddingHorizontal: spacing.lg,
    alignItems: "center",
    justifyContent: "center",
    minHeight: 48,
  },
  buttonSmall: {
    minHeight: 36,
    paddingVertical: spacing.sm,
    paddingHorizontal: spacing.md,
    borderRadius: radius.sm,
  },
  buttonText: { fontSize: 16, fontWeight: "600" },
  buttonTextSmall: { fontSize: 13, fontWeight: "600" },
  disabled: { opacity: 0.5 },
  field: { marginBottom: spacing.md },
  label: { fontSize: 13, fontWeight: "600", color: colors.textMuted, marginBottom: spacing.xs },
  input: {
    borderWidth: 1,
    borderColor: colors.border,
    borderRadius: radius.sm,
    paddingHorizontal: spacing.md,
    paddingVertical: spacing.sm,
    fontSize: 15,
    color: colors.text,
    backgroundColor: colors.surface,
    minHeight: 44,
  },
  inputDisabled: { backgroundColor: colors.surfaceMuted, color: colors.textMuted },
  card: {
    backgroundColor: colors.surface,
    borderRadius: radius.lg,
    padding: spacing.lg,
    borderWidth: 1,
    borderColor: colors.border,
  },
  sectionHeading: { marginBottom: spacing.sm },
  sectionTitle: { fontSize: 18, fontWeight: "700", color: colors.text },
  sectionSubtitle: { fontSize: 13, color: colors.textMuted, marginTop: 2 },
  chip: {
    borderRadius: 999,
    paddingVertical: spacing.xs,
    paddingHorizontal: spacing.md,
    marginRight: spacing.sm,
    marginBottom: spacing.sm,
  },
  chipText: { fontSize: 13, fontWeight: "600" },
  badge: {
    borderRadius: 999,
    paddingVertical: 2,
    paddingHorizontal: spacing.md,
    alignSelf: "flex-start",
  },
  badgeText: { color: "#FFFFFF", fontWeight: "700", fontSize: 13 },
  errorBanner: {
    backgroundColor: "#FEE2E2",
```

```
    borderColor: "#FECACA",
    borderWidth: 1,
    borderRadius: radius.sm,
    padding: spacing.md,
    marginBottom: spacing.md,
  },
  errorText: { color: colors.danger, fontSize: 13 },
  loading: { alignItems: "center", paddingVertical: spacing.xl },
  loadingText: { marginTop: spacing.sm, color: colors.textMuted, fontSize: 14 },
});
```

137. mobile\src\navigation.tsx

```
import React, { createContext, useContext, useState } from "react";
import { Pressable, StyleSheet, Text, View } from "react-native";
import { StatusBar } from "expo-status-bar";

import { colors, radius, spacing } from "./theme";
import type { ResumeData } from "./types";

export type Screen =
  | { name: "dashboard" }
  | {
      name: "builder";
      resumeId?: string;
      title?: string;
      targetRole?: string | null;
      data?: ResumeData;
    }
  | { name: "preview"; resumeId?: string; title: string; data: ResumeData; unsaved?: boolean }
  | { name: "matcher"; resumeId?: string; resume?: ResumeData };

interface NavContextValue {
  screen: Screen;
  navigate: (next: Screen) => void;
}

const NavContext = createContext<NavContextValue>({ screen: { name: "dashboard" }, navigate: () => {} });

export function useNav(): NavContextValue {
  return useContext(NavContext);
}

const TITLES: Record<Screen["name"], string> = {
  dashboard: "My Resumes",
  builder: "Edit Resume",
  preview: "Preview",
  matcher: "ATS Match",
};

export function NavigationProvider({ children }: { children: React.ReactNode }) {
  const [screen, setScreen] = useState<Screen>({ name: "dashboard" });
  return <NavContext.Provider value={{ screen, navigate: setScreen }}>{children}</NavContext.Provider>;
}

export function Shell({ children }: { children: React.ReactNode }) {
  const { screen, navigate } = useNav();
  return (
    <View style={styles.root}>
      <StatusBar style="dark" />
      <View style={styles.header}>
        {screen.name !== "dashboard" ? (
          <Pressable onPress={() => navigate({ name: "dashboard" })} hitSlop={12} style={styles.back}>
            <Text style={styles.backText}>‹</Text>
          </Pressable>
        ) : null}
        <Text style={styles.headerTitle}>{TITLES[screen.name]}</Text>
      </View>
      <View style={styles.body}>{children}</View>
    </View>
  );
}

const styles = StyleSheet.create({
  root: { flex: 1, backgroundColor: colors.background },
  header: {
    flexDirection: "row",
    alignItems: "center",
    backgroundColor: colors.surface,
    paddingTop: 56,
    paddingBottom: spacing.md,
    paddingHorizontal: spacing.lg,
    borderBottomWidth: 1,
    borderBottomColor: colors.border,
  },
  back: { marginRight: spacing.md, width: 28 },
  backText: { fontSize: 34, color: colors.primary, lineHeight: 26 },
  headerTitle: { fontSize: 20, fontWeight: "700", color: colors.text, flex: 1 },
  body: { flex: 1 },
});
```

138. mobile\src\pdf.ts

```
import { Directory, File, Paths } from "expo-file-system";
import * as Sharing from "expo-sharing";

import { exportApi } from "./api";
import type { ResumeData } from "./types";

function slugify(title: string): string {
  const cleaned = (title || "resume").toLowerCase().replace(/[^a-z0-9]+/g, "-").replace(/(^-|-$)/g, "");
  return cleaned || "resume";
}

/**
 * Generates a PDF via the backend, saves it to the device cache and opens the
 * system share sheet so the user can preview/send the file.
 */
export async function exportPdfAndShare(data: ResumeData, title: string): Promise<string> {
  const bytes = await exportApi.pdfBytes(data);
  const dir = new Directory(Paths.cache, "exports");
  dir.create({ intermediates: true, idempotent: true });
  const file: File = dir.createFile(`${slugify(title)}.pdf`, "application/pdf");
  file.write(bytes);

  if (await Sharing.isAvailableAsync()) {
    await Sharing.shareAsync(file.uri, {
      mimeType: "application/pdf",
      UTI: "com.adobe.pdf",
      dialogTitle: "Share resume PDF",
    });
  }
  return file.uri;
}
```

139. mobile\src\resumeText.ts

```
import type { ResumeData } from "./types";

// Heuristics tuned to the backend export template: US Letter, 10pt, ~0.55/0.6in margins.
const LINE_CHARS = 95;
const LINES_PER_PAGE = 46;

function lineCount(text: string): number {
  if (!text) return 0;
  return Math.max(1, Math.ceil(text.length / LINE_CHARS));
}

/**
 * Estimates how many A4/Letter pages the resume would occupy when exported,
 * and where the first page boundary falls in the plain-text preview.
 */
export function estimatePages(text: string): { pages: number; pageOneEndIndex: number } {
  const lines = text.split("\n");
  let totalLines = 0;
  let consumed = 0;
  let pageOneEndIndex = -1;

  for (let i = 0; i < lines.length; i++) {
    const len = lines[i].length;
    const count = lineCount(len === 0 ? "\n" : lines[i]);
    const before = totalLines;
    if (before < LINES_PER_PAGE && before + count >= LINES_PER_PAGE) {
      const partial = LINES_PER_PAGE - before;
      let idx = consumed + (len === 0 ? 1 : Math.round((partial / count) * len));
      idx = Math.min(idx, consumed + len);
      const nl = text.indexOf("\n", idx);
      pageOneEndIndex = nl === -1 && idx < text.length ? text.length : nl === -1 ? text.length : nl + 1;
      break;
    }
    totalLines += count;
    consumed += len + 1;
  }

  if (pageOneEndIndex === -1) pageOneEndIndex = text.length;
  return { pages: Math.max(1, Math.ceil(totalLines / LINES_PER_PAGE)), pageOneEndIndex: Math.min(pageOneEndIndex, text.length) };
}

function formatRange(start: string, end: string, current: boolean): string {
  if (current || !end) return `${start || ""} – Present`;
  return `${start || ""} – ${end || ""}`;
}

/**
 * Builds an ATS-friendly plain-text layout locally (mirrors the server's
 * /api/export/text output closely enough for preview + one-page estimation).
 */
export function resumeToPlainText(d: ResumeData): string {
  const parts: string[] = [];

  if (d.contact.fullName) parts.push(d.contact.fullName.toUpperCase());
  const contactBits = [
    d.contact.email,
    d.contact.phone,
    d.contact.location,
    d.contact.linkedin ?? "",
    d.contact.website ?? "",
  ].filter(Boolean);
  if (contactBits.length) parts.push(contactBits.join(" | "));

  const sections: string[] = [];

  if (d.summary.trim()) sections.push(`SUMMARY\n${d.summary.trim()}`);

  if (d.experience.length) {
    const exp = d.experience
      .map((e) => {
        const lines = [`${e.position}${e.company ? ` - ${e.company}` : ""}`, e.location, formatRange(e.startDate, e.endDate, e.current)];
        e.bullets.forEach((b) => b.trim() && lines.push(`- ${b.trim()}`));
        return lines.filter(Boolean).join("\n");
      })
      .join("\n\n");
    sections.push(`EXPERIENCE\n${exp}`);
  }

  if (d.education.length) {
    const edu = d.education
      .map((e) => {
        const line = [e.degree, e.field].filter(Boolean).join(" in ");
        const partsArr = [e.institution, line, formatRange(e.startDate, e.endDate, false), e.gpa ? `GPA: ${e.gpa}` : ""].filter(Boolean);
        return partsArr.join(" - ");
      })
      .join("\n");
    sections.push(`EDUCATION\n${edu}`);
  }

  if (d.skills.length) sections.push(`SKILLS\n${d.skills.map((s) => s.name).join(", ")}`);
```

```
  if (d.projects.length) {
    const proj = d.projects
      .map((p) => {
        const lines = [p.name, p.description, p.technologies.length ? `Technologies: ${p.technologies.join(", ")}` : "", p.url ??
"""].filter(Boolean);
        return lines.join("\n");
      })
      .join("\n\n");
    sections.push(`PROJECTS\n${proj}`);
  }

  if (d.certifications.length) {
    const cert = d.certifications.map((c) => `${c.name}${c.issuer ? ` - ${c.issuer}` : ""}${c.date ? ` (${c.date})` : ""}`).join("\n");
    sections.push(`CERTIFICATIONS\n${cert}`);
  }

  return parts.concat(sections).join("\n\n");
}

/** Raw text fed into summary-generation / keyword extraction. */
export function buildSourceText(d: ResumeData): string {
  return [
    d.summary.trim(),
    ...d.experience.map((e) => [e.position, e.company, e.location, formatRange(e.startDate, e.endDate, e.current),
...e.bullets].filter(Boolean).join(" ").trim()),
    ...d.education.map((e) => [e.institution, e.degree, e.field].filter(Boolean).join(" ").trim()),
    ...d.skills.map((s) => s.name).join(", "),
  ]
    .filter(Boolean)
    .join("\n");
}
```

140. mobile\src\screens\Builder.tsx

```
import React, { useState } from "react";
import { Alert, ScrollView, StyleSheet, Switch, Text, View } from "react-native";

import { aiApi, resumesApi } from "../api";
import { Button, Card, Chip, ErrorBanner, SectionHeading, TextField } from "../components";
import { useNav } from "../navigation";
import { buildSourceText } from "../resumeText";
import { colors, radius, spacing } from "../theme";
import { emptyResumeData, nextId, ResumeData, WorkExperience } from "../types";
import { getUserId } from "../userStore";

type Tab = "contact" | "summary" | "experience" | "education" | "skills" | "projects" | "certs";

const TABS: { key: Tab; label: string }[] = [
  { key: "contact", label: "Contact" },
  { key: "summary", label: "Summary" },
  { key: "experience", label: "Experience" },
  { key: "education", label: "Education" },
  { key: "skills", label: "Skills" },
  { key: "projects", label: "Projects" },
  { key: "certs", label: "Certifications" },
];

function linesToBullets(value: string): string[] {
  return value
    .split("\n")
    .map((l) => l.trim())
    .filter(Boolean);
}

function bulletsToLines(bullets: string[]): string {
  return bullets.join("\n");
}

interface BuilderProps {
  resumeId?: string;
  title?: string;
  targetRole?: string | null;
  data?: ResumeData;
}

export function BuilderScreen(props: BuilderProps) {
  const { navigate } = useNav();
  const [tab, setTab] = useState<Tab>("contact");
  const [title, setTitle] = useState(props.title ?? "Untitled Resume");
  const [targetRole, setTargetRole] = useState(props.targetRole ?? "");
  const [data, setData] = useState<ResumeData>(props.data ?? emptyResumeData());
  const [saving, setSaving] = useState(false);
  const [busyLabel, setBusyLabel] = useState("");
  const [error, setError] = useState("");

  const patch = (p: Partial<ResumeData>) => setData((d) => ({ ...d, ...p }));

  const save = async () => {
    setSaving(true);
    setError("");
    try {
      const userId = await getUserId();
      if (props.resumeId) {
        await resumesApi.update(userId, props.resumeId, { title, data, targetRole: targetRole || null });
      } else {
        await resumesApi.save(userId, title || "Untitled Resume", data, "modern", targetRole || null);
      }
      navigate({ name: "dashboard" });
    } catch (e) {
      setError(e instanceof Error ? e.message : String(e));
      setSaving(false);
    }
  };

  const scatter = async (label: string, fn: () => Promise<void>) => {
    setBusyLabel(label);
    setError("");
    try {
      await fn();
    } catch (e) {
      setError(e instanceof Error ? e.message : String(e));
    } finally {
      setBusyLabel("");
    }
  };

  const aiSummary = () =>
    void scatter("Generating summary…", async () => {
      const source = buildSourceText(data);
      if (!source.trim()) throw new Error("Add some experience/education first so the AI has raw material.");
      const result = await aiApi.generateSummary(source, targetRole || null);
      Alert.alert("Generated summary", result.summary, [
        { text: "Discard", style: "cancel" },
        { text: "Use this", onPress: () => patch({ summary: result.summary.trim() }) },
```

```
      ]);
    });

  const aiSkills = () =>
    void scatter("Suggesting skills…", async () => {
      const role = targetRole.trim() || (data.experience[0]?.position ?? "");
      if (!role) throw new Error("Set a target role (top of screen) so the AI knows what to suggest.");
      const current = data.skills.map((s) => s.name);
      const result = await aiApi.suggestSkills(role, current);
      const missing = result.skills.filter((s) => !current.some((c) => c.toLowerCase() === s.toLowerCase()));
      if (!missing.length) {
        Alert.alert("Nothing new", "The suggestions are already in your skills list.");
        return;
      }
      Alert.alert("Suggested skills", missing.join(", "), [
        { text: "Skip", style: "cancel" },
        { text: "Add all", onPress: () => patch({ skills: [...data.skills, ...missing.map((name) => ({ id: nextId("sk"), name }))] }) },
      ]);
    });

  const aiSharpenBullets = (index: number) =>
    void scatter("Sharpening bullets…", async () => {
      const exp = data.experience[index];
      const improved: string[] = [];
      for (const bullet of exp.bullets) {
        const result = await aiApi.improveBullet(bullet, targetRole || exp.position || null);
        improved.push(result.improved.charAt(0).toUpperCase() + result.improved.slice(1));
      }
      Alert.alert("AI-suggested bullets", improved.map((b) => `• ${b}`).join("\n\n"), [
        { text: "Keep mine", style: "cancel" },
        { text: "Replace", onPress: () => setData((d) => replaceExperienceBullets(d, index, improved)) },
      ]);
    });

  const addExperience = () =>
    patch({
      experience: [
        ...data.experience,
        { id: nextId("exp"), company: "", position: "", location: "", startDate: "", endDate: "", current: false, bullets: [] },
      ],
    });

  const renderTab = () => {
    switch (tab) {
      case "contact":
        return (
          <ContactTab data={data} patch={patch} />
        );
      case "summary":
        return (
          <View>
            <SectionHeading title="Summary" subtitle="2–4 crisp sentences. Avoid “responsible for”." />
            <TextField label="Professional summary" value={data.summary} onChangeText={(v) => patch({ summary: v })} multiline style=
{styles.textarea} />
            <Button title="✨ Generate with AI" variant="secondary" onPress={aiSummary} loading={busyLabel === "Generating summary…"} />
          </View>
        );
      case "experience":
        return (
          <View>
            <SectionHeading title="Experience" subtitle="Each bullet: Action verb + What you did + Result (Google XYZ)." />
            {data.experience.map((exp, i) => (
              <Card key={exp.id} style={styles.card}>
                <TextField label="Company" value={exp.company} onChangeText={(v) => setData((d) => updateExperience(d, i, { company: v }))} />
                <TextField label="Position" value={exp.position} onChangeText={(v) => setData((d) => updateExperience(d, i, { position: v }))}
/>
                <View style={styles.row}>
                  <View style={styles.rowHalf}>
                    <TextField label="Start (MM/YYYY)" value={exp.startDate} onChangeText={(v) => setData((d) => updateExperience(d, i, {
startDate: v }))} />
                  </View>
                  <View style={styles.rowHalf}>
                    <TextField label="End (MM/YYYY)" value={exp.endDate} editable={!exp.current} onChangeText={(v) => setData((d) =>
updateExperience(d, i, { endDate: v }))} />
                  </View>
                </View>
                <View style={styles.switchRow}>
                  <Text style={styles.switchLabel}>Current role</Text>
                  <Switch value={exp.current} onValueChange={(v) => setData((d) => updateExperience(d, i, { current: v }))} />
                </View>
                <TextField
                  label="Bullets (one per line)"
                  value={bulletsToLines(exp.bullets)}
                  onChangeText={(v) => setData((d) => updateExperience(d, i, { bullets: linesToBullets(v) }))}
                  multiline
                  style={styles.textareaMax}
                />
                <View style={styles.rowSpace}>
                  <Button title="🔎 Sharpen with AI" variant="secondary" small onPress={() => aiSharpenBullets(i)} loading={busyLabel ===
"Sharpening bullets…"} />
                  <Button title="Remove" variant="danger" small onPress={() => patch({ experience: data.experience.filter((e) => e.id !==
exp.id) })} />
                </View>
```

```
              </Card>
            ))}
            <Button title="+ Add experience" variant="outline" onPress={addExperience} />
          </View>
        );
      case "education":
        return (
          <View>
            <SectionHeading title="Education" />
            {data.education.map((edu, i) => (
              <Card key={edu.id} style={styles.card}>
                <TextField label="Institution" value={edu.institution} onChangeText={(v) => setData((d) => updateEducation(d, i, {
institution: v }))} />
                <View style={styles.row}>
                  <View style={styles.rowHalf}>
                    <TextField label="Degree" value={edu.degree} onChangeText={(v) => setData((d) => updateEducation(d, i, { degree: v }))} />
                  </View>
                  <View style={styles.rowHalf}>
                    <TextField label="Field" value={edu.field} onChangeText={(v) => setData((d) => updateEducation(d, i, { field: v }))} />
                  </View>
                </View>
                <View style={styles.row}>
                  <View style={styles.rowHalf}>
                    <TextField label="Start" value={edu.startDate} onChangeText={(v) => setData((d) => updateEducation(d, i, { startDate: v
}))} />
                  </View>
                  <View style={styles.rowHalf}>
                    <TextField label="End" value={edu.endDate} onChangeText={(v) => setData((d) => updateEducation(d, i, { endDate: v }))} />
                  </View>
                </View>
                <TextField label="GPA (optional)" value={edu.gpa ?? ""} onChangeText={(v) => setData((d) => updateEducation(d, i, { gpa: v
}))} />
                <Button title="Remove" variant="danger" small onPress={() => patch({ education: data.education.filter((e) => e.id !== edu.id)
})} />
              </Card>
            ))}
            <Button
              title="+ Add education"
              variant="outline"
              onPress={() => patch({ education: [...data.education, { id: nextId("edu"), institution: "", degree: "", field: "", startDate:
"", endDate: "", gpa: null }] })}
            />
          </View>
        );
      case "skills":
        return (
          <View>
            <SectionHeading title="Skills" subtitle="List the ones that appear in job descriptions. Synonyms count." />
            <View style={styles.chips}>
              {data.skills.map((s) => (
                <Chip key={s.id} label={s.name} onPress={() => patch({ skills: data.skills.filter((x) => x.id !== s.id) })} />
              ))}
            </View>
            <SkillAdder onAdd={(name) => patch({ skills: [...data.skills, { id: nextId("sk"), name }] })} />
            <Button title="✨ Suggest skills for role" variant="secondary" onPress={aiSkills} loading={busyLabel === "Suggesting skills…"} />
          </View>
        );
      case "projects":
        return (
          <View>
            <SectionHeading title="Projects" />
            {data.projects.map((p, i) => (
              <Card key={p.id} style={styles.card}>
                <TextField label="Project name" value={p.name} onChangeText={(v) => setData((d) => updateProject(d, i, { name: v }))} />
                <TextField label="Description" value={p.description} onChangeText={(v) => setData((d) => updateProject(d, i, { description: v
}))} multiline style={styles.textarea} />
                <TextField label="Technologies (comma separated)" value={p.technologies.join(", ")} onChangeText={(v) => setData((d) =>
updateProject(d, i, { technologies: v.split(",").map((x) => x.trim()).filter(Boolean) }))} />
                <TextField label="URL (optional)" value={p.url ?? ""} onChangeText={(v) => setData((d) => updateProject(d, i, { url: v }))} />
                <Button title="Remove" variant="danger" small onPress={() => patch({ projects: data.projects.filter((x) => x.id !== p.id) })}
/>
              </Card>
            ))}
            <Button title="+ Add project" variant="outline" onPress={() => patch({ projects: [...data.projects, { id: nextId("prj"), name: "",
description: "", url: null, technologies: [] }] })} />
          </View>
        );
      case "certs":
        return (
          <View>
            <SectionHeading title="Certifications" />
            {data.certifications.map((c, i) => (
              <Card key={c.id} style={styles.card}>
                <TextField label="Name" value={c.name} onChangeText={(v) => setData((d) => updateCert(d, i, { name: v }))} />
                <View style={styles.row}>
                  <View style={styles.rowHalf}>
                    <TextField label="Issuer" value={c.issuer} onChangeText={(v) => setData((d) => updateCert(d, i, { issuer: v }))} />
                  </View>
                  <View style={styles.rowHalf}>
                    <TextField label="Date" value={c.date} onChangeText={(v) => setData((d) => updateCert(d, i, { date: v }))} />
                  </View>
                </View>
                <Button title="Remove" variant="danger" small onPress={() => patch({ certifications: data.certifications.filter((x) => x.id
```

```
!== c.id) })} />
              </Card>
            ))}
            <Button title="+ Add certification" variant="outline" onPress={() => patch({ certifications: [...data.certifications, { id:
nextId("cert"), name: "", issuer: "", date: "", url: null }] })} />
          </View>
        );
    }
  };

  return (
    <View style={styles.container}>
      <ScrollView horizontal showsHorizontalScrollIndicator={false} style={styles.tabs} contentContainerStyle={styles.tabRow}>
        {TABS.map((t) => (
          <Chip key={t.key} label={t.label} onPress={() => setTab(t.key)} tone={tab === t.key ? "match" : "default"} />
        ))}
      </ScrollView>
      <ScrollView style={styles.body} keyboardShouldPersistTaps="handled" contentContainerStyle={styles.scrollContent}>
        <TextField label="Resume title" value={title} onChangeText={setTitle} />
        <TextField label="Target role (for AI + matching)" value={targetRole} onChangeText={setTargetRole} />
        <ErrorBanner message={error} />
        {renderTab()}
      </ScrollView>
      <View style={styles.footer}>
        <Button title={busyLabel || "Save"} onPress={save} loading={saving || busyLabel !== ""} />
      </View>
    </View>
  );
}

function replaceExperienceBullets(d: ResumeData, index: number, bullets: string[]): ResumeData {
  return { ...d, experience: d.experience.map((e, i) => (i === index ? { ...e, bullets } : e)) };
}

function updateExperience(d: ResumeData, index: number, patch: Partial<WorkExperience>): ResumeData {
  return { ...d, experience: d.experience.map((e, i) => (i === index ? { ...e, ...patch } : e)) };
}

function updateEducation(d: ResumeData, index: number, patch: Partial<ResumeData["education"][number]>): ResumeData {
  return { ...d, education: d.education.map((e, i) => (i === index ? { ...e, ...patch } : e)) };
}

function updateProject(d: ResumeData, index: number, patch: Partial<ResumeData["projects"][number]>): ResumeData {
  return { ...d, projects: d.projects.map((p, i) => (i === index ? { ...p, ...patch } : p)) };
}

function updateCert(d: ResumeData, index: number, patch: Partial<ResumeData["certifications"][number]>): ResumeData {
  return { ...d, certifications: d.certifications.map((c, i) => (i === index ? { ...c, ...patch } : c)) };
}

function ContactTab({ data, patch }: { data: ResumeData; patch: (p: Partial<ResumeData>) => void }) {
  const c = data.contact;
  return (
    <View>
      <SectionHeading title="Contact" />
      <TextField label="Full name" value={c.fullName} onChangeText={(v) => patch({ contact: { ...c, fullName: v } })} />
      <TextField label="Email" value={c.email} keyboardType="email-address" onChangeText={(v) => patch({ contact: { ...c, email: v } })} />
      <TextField label="Phone" value={c.phone} keyboardType="phone-pad" onChangeText={(v) => patch({ contact: { ...c, phone: v } })} />
      <TextField label="Location" value={c.location} onChangeText={(v) => patch({ contact: { ...c, location: v } })} />
      <TextField label="LinkedIn (optional)" value={c.linkedin ?? ""} onChangeText={(v) => patch({ contact: { ...c, linkedin: v } })} />
      <TextField label="Website (optional)" value={c.website ?? ""} onChangeText={(v) => patch({ contact: { ...c, website: v } })} />
    </View>
  );
}

function SkillAdder({ onAdd }: { onAdd: (name: string) => void }) {
  const [value, setValue] = useState("");
  return (
    <View style={styles.skillAdder}>
      <TextField
        label="Add a skill"
        value={value}
        onChangeText={setValue}
        onSubmitEditing={() => {
          const name = value.trim();
          if (name) onAdd(name);
          setValue("");
        }}
        style={{ flex: 1 }}
      />
      <Button
        title="Add"
        variant="secondary"
        onPress={() => {
          const name = value.trim();
          if (name) onAdd(name);
          setValue("");
        }}
      />
    </View>
  );
}
```

```
const styles = StyleSheet.create({
  container: { flex: 1 },
  tabs: { flexGrow: 0, backgroundColor: colors.surface, borderBottomWidth: 1, borderBottomColor: colors.border },
  tabRow: { paddingHorizontal: spacing.lg, paddingVertical: spacing.sm, flexDirection: "row" },
  body: { flex: 1 },
  scrollContent: { padding: spacing.lg, paddingBottom: spacing.xl },
  footer: { padding: spacing.lg, borderTopWidth: 1, borderTopColor: colors.border, backgroundColor: colors.surface },
  textarea: { minHeight: 90, textAlignVertical: "top" },
  textareaMax: { minHeight: 120, textAlignVertical: "top" },
  card: { marginBottom: spacing.lg },
  row: { flexDirection: "row", gap: spacing.md },
  rowHalf: { flex: 1 },
  rowSpace: { flexDirection: "row", alignItems: "center", justifyContent: "space-between", marginTop: spacing.sm, gap: spacing.sm },
  switchRow: { flexDirection: "row", alignItems: "center", justifyContent: "space-between", marginBottom: spacing.md },
  switchLabel: { fontSize: 14, color: colors.textMuted, fontWeight: "600" },
  chips: { flexDirection: "row", flexWrap: "wrap", marginBottom: spacing.md },
  skillAdder: { flexDirection: "row", alignItems: "flex-end", gap: spacing.sm, marginBottom: spacing.md },
});
```

141. mobile\src\screens\Dashboard.tsx

```
import React, { useCallback, useEffect, useState } from "react";
import { Alert, FlatList, Pressable, RefreshControl, StyleSheet, Text, View } from "react-native";

import { apiDoc, resumesApi } from "../api";
import { Badge, Button, Card, Loading } from "../components";
import { useNav } from "../navigation";
import { colors, radius, spacing } from "../theme";
import { emptyResumeData, ResumeMeta } from "../types";
import { getUserId } from "../userStore";

export function DashboardScreen() {
  const { navigate } = useNav();
  const [userId, setUserId] = useState<string | null>(null);
  const [resumes, setResumes] = useState<ResumeMeta[]>([]);
  const [loading, setLoading] = useState(true);
  const [refreshing, setRefreshing] = useState(false);
  const [error, setError] = useState("");

  const load = useCallback(
    async (refresh = false) => {
      if (refresh) setRefreshing(true);
      else setLoading(true);
      setError("");
      try {
        const uid = userId ?? (await getUserId());
        setUserId(uid);
        const items = await resumesApi.list(uid);
        setResumes(items);
      } catch (e) {
        setError(e instanceof Error ? e.message : String(e));
      } finally {
        setLoading(false);
        setRefreshing(false);
      }
    },
    [userId],
  );

  useEffect(() => {
    void load();
  }, [load]);

  const createNew = () =>
    navigate({ name: "builder", title: "Untitled Resume", data: emptyResumeData(), targetRole: null });

  const openResume = (r: ResumeMeta) => {
    if (!r.data) {
      getFull(r);
      return;
    }
    navigate({ name: "builder", resumeId: r.id, title: r.title, targetRole: r.targetRole, data: r.data });
  };

  const getFull = async (r: ResumeMeta) => {
    try {
      const full = await resumesApi.get(userId!, r.id);
      navigate({ name: "builder", resumeId: full.id, title: full.title, targetRole: full.targetRole, data: full.data ?? emptyResumeData() });
    } catch (e) {
      setError(e instanceof Error ? e.message : String(e));
    }
  };

  const preview = (r: ResumeMeta) => {
    if (r.data) {
      navigate({ name: "preview", resumeId: r.id, title: r.title, data: r.data });
    } else {
      void (async () => {
        try {
          const full = await resumesApi.get(userId!, r.id);
          navigate({ name: "preview", resumeId: full.id, title: full.title, data: full.data ?? emptyResumeData() });
        } catch (e) {
          setError(e instanceof Error ? e.message : String(e));
        }
      })();
    }
  };

  const cloneResume = (r: ResumeMeta) =>
    Alert.alert("Duplicate resume", "Create a variant you can tailor for a specific job?", [
      { text: "Cancel", style: "cancel" },
      {
        text: "Duplicate",
        onPress: async () => {
          try {
            const clone = await resumesApi.clone(userId!, r.id);
            await load(true);
            if (clone.data) navigate({ name: "builder", resumeId: clone.id, title: clone.title, data: clone.data });
          } catch (e) {
            setError(e instanceof Error ? e.message : String(e));
          }
```

```
        },
      },
    ]);

  const deleteResume = (r: ResumeMeta) =>
    Alert.alert("Delete resume", `Delete "${r.title}"?`, [
      { text: "Cancel", style: "cancel" },
      {
        text: "Delete",
        style: "destructive",
        onPress: async () => {
          try {
            await resumesApi.remove(userId!, r.id);
            await load(true);
          } catch (e) {
            setError(e instanceof Error ? e.message : String(e));
          }
        },
      },
    ]);

  return (
    <View style={styles.container}>
      <Text style={styles.hint}>{apiDoc()}</Text>
      <Button title="+ New Resume" onPress={createNew} style={{ marginBottom: spacing.lg }} />
      {error ? (
        <View style={styles.errorBox}>
          <Text style={styles.errorText}>{error}</Text>
          <Button title="Retry" variant="outline" small onPress={() => void load()} style={{ marginTop: spacing.sm }} />
        </View>
      ) : null}
      {loading ? (
        <Loading />
      ) : (
        <FlatList
          data={resumes}
          keyExtractor={(r) => r.id}
          refreshControl={<RefreshControl refreshing={refreshing} onRefresh={() => void load(true)} />}
          ListEmptyComponent={<Text style={styles.empty}>No resumes yet. Tap “+ New Resume” to build your first one.</Text>}
          renderItem={({ item }) => (
            <Card style={styles.row}>
              <Pressable onPress={() => openResume(item)} style={styles.rowMain}>
                <Text style={styles.title} numberOfLines={1}>
                  {item.title}
                </Text>
                {item.targetRole ? <Text style={styles.role}>{item.targetRole}</Text> : null}
                {item.parentId ? <Text style={styles.variant}>variant</Text> : null}
              </Pressable>
              {item.score != null ? <Badge score={item.score} /> : null}
              <View style={styles.actions}>
                <Button title="Preview" variant="ghost" small onPress={() => preview(item)} />
                <Button title="Duplicate" variant="outline" small onPress={() => cloneResume(item)} />
                <Button title="Delete" variant="ghost" small onPress={() => deleteResume(item)} />
              </View>
            </Card>
          )}
        />
      )}
      <View style={styles.footer}>
        <Button title="ATS Match" variant="secondary" onPress={() => navigate({ name: "matcher", resumeId: undefined, resume: undefined })} />
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  container: { flex: 1, padding: spacing.lg },
  hint: { fontSize: 12, color: colors.textMuted, marginBottom: spacing.md },
  errorBox: {
    backgroundColor: "#FEE2E2",
    borderRadius: radius.sm,
    padding: spacing.md,
    marginBottom: spacing.md,
  },
  errorText: { color: colors.danger, fontSize: 13 },
  empty: { textAlign: "center", color: colors.textMuted, marginTop: spacing.xl, paddingHorizontal: spacing.xl },
  row: { marginBottom: spacing.md },
  rowMain: { marginBottom: spacing.sm },
  title: { fontSize: 17, fontWeight: "700", color: colors.text },
  role: { fontSize: 13, color: colors.textMuted, marginTop: 2 },
  variant: {
    fontSize: 11,
    color: colors.primary,
    backgroundColor: "#DBEAFE",
    alignSelf: "flex-start",
    borderRadius: 999,
    paddingHorizontal: spacing.sm,
    paddingVertical: 1,
    marginTop: spacing.xs,
    overflow: "hidden",
  },
  actions: { flexDirection: "row", flexWrap: "wrap", gap: spacing.sm, marginTop: spacing.sm },
```

```
  footer: { marginTop: spacing.md },
});
```

142. mobile\src\screens\Matcher.tsx

```
import React, { useEffect, useState } from "react";
import { Alert, ScrollView, StyleSheet, Text, View } from "react-native";

import { aiApi, atsApi, resumesApi } from "../api";
import { Button, Card, Chip, ErrorBanner, Loading, SectionHeading, TextField } from "../components";
import { useNav } from "../navigation";
import { resumeToPlainText } from "../resumeText";
import { colors, spacing } from "../theme";
import { JDAnalysis, ResumeData } from "../types";
import { getUserId } from "../userStore";

interface MatcherProps {
  resumeId?: string;
  resume?: ResumeData;
}

export function MatcherScreen({ resumeId, resume }: MatcherProps) {
  const { navigate } = useNav();
  const [data, setData] = useState<ResumeData | null>(resume ?? null);
  const [jd, setJd] = useState("");
  const [analysis, setAnalysis] = useState<JDAnalysis | null>(null);
  const [resumeRefId, setResumeRefId] = useState<string | null>(resumeId ?? null);
  const [busy, setBusy] = useState("");
  const [error, setError] = useState("");

  useEffect(() => {
    void (async () => {
      try {
        const userId = await getUserId();
        if (data) return;
        if (resumeId) {
          const full = await resumesApi.get(userId, resumeId);
          setResumeRefId(full.id);
          setData(full.data);
          return;
        }
        const list = await resumesApi.list(userId);
        if (list.length > 0) {
          const full = list[0].data ? list[0] : await resumesApi.get(userId, list[0].id);
          setResumeRefId(full.id);
          setData(full.data);
        }
      } catch (e) {
        setError(e instanceof Error ? e.message : String(e));
      }
    })();
  }, [resumeId, resume, data]);

  const analyze = () =>
    void (async () => {
      if (!jd.trim()) {
        setError("Paste a job description first.");
        return;
      }
      setBusy("Matching against JD…");
      setError("");
      try {
        const result = await aiApi.analyzeJd(jd, data ? resumeToPlainText(data) : "");
        setAnalysis(result);
      } catch (e) {
        setError(e instanceof Error ? e.message : String(e));
      } finally {
        setBusy("");
      }
    });

  const insertMissing = () => {
    if (!analysis || !analysis.missingSkills.length || !resumeRefId) return;
    const names = analysis.missingSkills.map((s) => s.name);
    void (async () => {
      setBusy("Inserting keywords…");
      setError("");
      try {
        const userId = await getUserId();
        const result = await resumesApi.insertKeywords(userId, resumeRefId, names, "skills");
        setData(result.data);
        Alert.alert("Keywords inserted", `Added ${result.insertedKeywords.length} keyword(s): ${result.insertedKeywords.join(", ")}`);
      } catch (e) {
        setError(e instanceof Error ? e.message : String(e));
      } finally {
        setBusy("");
      }
    })();
  };

  const reScore = () => {
    if (!data) return;
    void (async () => {
      setBusy("Scoring…");
      setError("");
```

```
      try {
        const a = await atsApi.score(data);
        Alert.alert(`ATS score: ${a.score}/100`, a.summary + (a.issues.length ? `\n\n${a.issues.map((i) => `• ${i.message}`).join("\n")}` :
""));
      } catch (e) {
        setError(e instanceof Error ? e.message : String(e));
      } finally {
        setBusy("");
      }
    })();
  };

  const tailor = () => {
    if (!data) return;
    void (async () => {
      setBusy("Tailoring to this JD…");
      setError("");
      try {
        const result = await aiApi.tailorResume(data, jd);
        if (result.guardrailWarnings.length) {
          Alert.alert("Guardrails applied", result.guardrailWarnings.join("\n"));
        }
        const roleGuess = guessTargetRole(jd);
        navigate({
          name: "preview",
          title: `${data.contact.fullName || "Resume"} — tailored${roleGuess ? ` (${roleGuess})` : ""}`,
          data: result.tailored,
          unsaved: true,
        });
      } catch (e) {
        setError(e instanceof Error ? e.message : String(e));
      } finally {
        setBusy("");
      }
    })();
  };

  const hasMissing = (analysis?.missingSkills.length ?? 0) > 0;

  return (
    <View style={styles.container}>
      <ScrollView contentContainerStyle={styles.scrollContent} keyboardShouldPersistTaps="handled">
        {!data ? (
          <Loading label="Loading a resume to match…" />
        ) : (
          <>
            <TextField label="Job description (paste below)" value={jd} onChangeText={setJd} multiline style={styles.textarea}
placeholder="Paste the full JD text here…" />
            <Button title={busy || "🔎 Analyze JD & match skills"} onPress={analyze} loading={busy !== ""} />

            <ErrorBanner message={error} />

            {analysis ? (
              <View style={styles.results}>
                <Card style={styles.matchCard}>
                  <Text style={styles.matchPct}>{analysis.matchPercentage}%</Text>
                  <Text style={styles.matchLabel}>keyword match</Text>
                  <Text style={styles.matchDetail}>
                    {analysis.matchedSkills.length} of {analysis.matchedSkills.length + analysis.missingSkills.length} JD skills found in
resume
                  </Text>
                </Card>

                {analysis.matchedSkills.length > 0 ? (
                  <>
                    <SectionHeading title="Matched" />
                    <View style={styles.chips}>
                      {analysis.matchedSkills.map((s) => (
                        <Chip key={s.name} label={`${s.name} (${s.category})`} tone="match" />
                      ))}
                    </View>
                  </>
                ) : null}

                {hasMissing ? (
                  <>
                    <SectionHeading title="Missing — add these if truthful" />
                    <View style={styles.chips}>
                      {analysis.missingSkills.map((s) => (
                        <Chip key={s.name} label={`${s.name} (${s.category})`} tone="missing" />
                      ))}
                    </View>
                    <Button title="➕ Insert missing keywords into Skills" variant="secondary" onPress={insertMissing} loading={busy ===
"Inserting keywords…"} />
                  </>
                ) : null}

                {analysis.keywords.length > 0 ? (
                  <View style={styles.keywordBox}>
                    <Text style={styles.keywordLabel}>JD keywords to mirror in your wording:</Text>
                    <Text style={styles.keywordText}>{analysis.keywords.slice(0, 20).join(", ")}</Text>
                  </View>
                ) : null}
```

```
                <View style={styles.rowSpace}>
                  <Button title="🩺 Re-score ATS" variant="outline" onPress={reScore} loading={busy === "Scoring…"} />
                  <Button title="✨ Tailor to this JD" onPress={tailor} loading={busy === "Tailoring to this JD…"} disabled={!jd.trim()} />
                </View>
              </View>
            ) : null}
          </>
        )}
      </ScrollView>
    </View>
  );
}

function guessTargetRole(jd: string): string | null {
  const firstLine = jd
    .split("\n")
    .map((l) => l.trim())
    .find((l) => l && !/^(about|job|the|we|position|location|company|who|what|role|responsib)/i.test(l));
  if (!firstLine) return null;
  return firstLine.length <= 60 ? firstLine : null;
}
const styles = StyleSheet.create({
  container: { flex: 1 },
  scrollContent: { padding: spacing.lg, paddingBottom: spacing.xl },
  textarea: { minHeight: 140, textAlignVertical: "top" },
  results: { marginTop: spacing.xl },
  matchCard: { alignItems: "center", paddingVertical: spacing.xl, marginBottom: spacing.xl },
  matchPct: { fontSize: 44, fontWeight: "800", color: colors.primary },
  matchLabel: { fontSize: 13, color: colors.textMuted, marginTop: 2 },
  matchDetail: { fontSize: 13, color: colors.textMuted, marginTop: spacing.sm, textAlign: "center" },
  chips: { flexDirection: "row", flexWrap: "wrap", marginBottom: spacing.lg },
  keywordBox: {
    backgroundColor: colors.surfaceMuted,
    borderRadius: 10,
    padding: spacing.md,
    marginBottom: spacing.lg,
  },
  keywordLabel: { fontSize: 12, fontWeight: "600", color: colors.textMuted, marginBottom: 4 },
  keywordText: { fontSize: 13, color: colors.text, lineHeight: 18 },
  rowSpace: { flexDirection: "row", gap: spacing.sm },
});
```

143. mobile\src\screens\Preview.tsx

```
import React, { useEffect, useMemo, useState } from "react";
import { Alert, ScrollView, StyleSheet, Text, View } from "react-native";

import { atsApi, resumesApi } from "../api";
import { Badge, Button, ErrorBanner, Loading, SectionHeading } from "../components";
import { useNav } from "../navigation";
import { exportPdfAndShare } from "../pdf";
import { estimatePages, resumeToPlainText } from "../resumeText";
import { colors, radius, spacing } from "../theme";
import { ResumeData } from "../types";
import { getUserId } from "../userStore";

interface PreviewProps {
  resumeId?: string;
  title: string;
  data: ResumeData;
  unsaved?: boolean;
}

export function PreviewScreen({ resumeId, title, data, unsaved }: PreviewProps) {
  const { navigate } = useNav();
  const [score, setScore] = useState<{ score: number; label: string } | null>(null);
  const [issues, setIssues] = useState<{ severity: string; message: string }[]>([]);
  const [busy, setBusy] = useState("");
  const [error, setError] = useState("");

  const text = useMemo(() => resumeToPlainText(data), [data]);
  const { pages, pageOneEndIndex } = useMemo(() => estimatePages(text), [text]);
  const isNew = unsaved === true;

  useEffect(() => {
    let mounted = true;
    void (async () => {
      try {
        const analysis = await atsApi.score(data);
        if (!mounted) return;
        const label = analysis.score >= 85 ? "Strong" : analysis.score >= 70 ? "Good" : "Needs work";
        setScore({ score: analysis.score, label });
        setIssues(analysis.issues.map((i) => ({ severity: i.severity, message: i.message })));
      } catch (e) {
        if (mounted) setError(e instanceof Error ? e.message : String(e));
      }
    })();
    return () => {
      mounted = false;
    };
  }, [data]);

  const sharePdf = () =>
    void (async () => {
      setBusy("Preparing PDF…");
      setError("");
      try {
        await exportPdfAndShare(data, title);
      } catch (e) {
        setError(e instanceof Error ? e.message : String(e));
      } finally {
        setBusy("");
      }
    });

  const saveTailored = () =>
    Alert.alert("Save as new resume", "Save this tailored variant as a separate resume?", [
      { text: "Cancel", style: "cancel" },
      {
        text: "Save",
        onPress: () =>
          void (async () => {
            setBusy("Saving…");
            setError("");
            try {
              const userId = await getUserId();
              await resumesApi.save(userId, title, data);
              navigate({ name: "dashboard" });
            } catch (e) {
              setError(e instanceof Error ? e.message : String(e));
              setBusy("");
            }
          }),
      },
    ]);

  const before = text.slice(0, pageOneEndIndex);
  const after = text.slice(pageOneEndIndex);

  return (
    <View style={styles.container}>
      <ScrollView contentContainerStyle={styles.scrollContent}>
        {error ? (
          <ErrorBanner message={error} />
```

```
        ) : (
          <>
            <View style={styles.summaryRow}>
              <View style={{ flex: 1 }}>
                <Text style={styles.resumeTitle}>{title}</Text>
                <Text style={styles.role}>{data.contact.fullName || "Untitled resume"}</Text>
              </View>
              {score ? <Badge score={score.score} /> : <Loading label="Scoring…" />}
            </View>

            {score ? (
              <View style={styles.scoreMetric}>
                <Text style={styles.scoreText}>
                  ATS score: {score.score}/100 — {score.label}
                </Text>
                {issues.length > 0 ? (
                  <Text style={styles.issueCount}>
                    {issues.filter((i) => i.severity === "error").length} errors · {issues.filter((i) => i.severity === "warning").length}
warnings
                  </Text>
                ) : null}
              </View>
            ) : null}

            <View style={[styles.pageBanner, pages > 1 ? styles.pageWarn : styles.pageOk]}>
              <Text style={[styles.pageBannerText, { color: pages > 1 ? colors.warning : colors.success }]}>
                {pages > 1 ? `≈ ${pages} pages — try to keep it to 1 for ATS-friendly length` : "Fits on 1 page 🎉"}
              </Text>
            </View>

            <SectionHeading title="Preview" subtitle="Roughly how the exported PDF lays out." />
            <View style={styles.paper}>
              <Text style={styles.paperText}>{before}</Text>
              {pages > 1 ? (
                <View style={styles.pageBreak}>
                  <View style={styles.pageBreakLine} />
                  <Text style={styles.pageBreakLabel}>page 1 ↑ ends here · page 2 ↓</Text>
                  <View style={styles.pageBreakLine} />
                </View>
              ) : null}
              <Text style={styles.paperText}>{after}</Text>
            </View>

            {issues.length > 0 ? (
              <View style={styles.issues}>
                <SectionHeading title="ATS issues" subtitle="Fix these to score higher." />
                {issues.map((issue, i) => (
                  <Text key={i} style={styles.issueText}>
                    • {issue.message}
                  </Text>
                ))}
              </View>
            ) : null}
          </>
        )}
      </ScrollView>

      <View style={styles.footer}>
        {isNew ? (
          <Button title="💾 Save as new resume" onPress={saveTailored} loading={busy === "Saving…"} />
        ) : (
          <Button title={busy || "Export PDF & Share"} onPress={sharePdf} loading={busy === "Preparing PDF…"} />
        )}
        <View style={{ marginTop: spacing.sm }}>
          <Button title="📝 Edit in builder" variant="secondary" onPress={() => navigate({ name: "builder", resumeId, title, targetRole:
null, data })} />
        </View>
      </View>
    </View>
  );
}
const styles = StyleSheet.create({
  container: { flex: 1 },
  scrollContent: { padding: spacing.lg, paddingBottom: spacing.xl },
  summaryRow: { flexDirection: "row", alignItems: "center", marginBottom: spacing.md },
  resumeTitle: { fontSize: 20, fontWeight: "700", color: colors.text },
  role: { fontSize: 14, color: colors.textMuted, marginTop: 2 },
  scoreMetric: { marginBottom: spacing.md },
  scoreText: { fontSize: 14, fontWeight: "600", color: colors.text },
  issueCount: { fontSize: 12, color: colors.textMuted, marginTop: 2 },
  pageBanner: {
    borderRadius: radius.sm,
    paddingVertical: spacing.sm,
    paddingHorizontal: spacing.md,
    marginBottom: spacing.lg,
    borderWidth: 1,
  },
  pageWarn: { backgroundColor: "#FEF3C7", borderColor: "#FDE68A" },
  pageOk: { backgroundColor: "#DCFCE7", borderColor: "#BBF7D0" },
  pageBannerText: { fontSize: 14, fontWeight: "600" },
  paper: {
    backgroundColor: colors.surface,
    borderRadius: radius.md,
```

```
    padding: spacing.lg,
    borderWidth: 1,
    borderColor: colors.border,
    marginBottom: spacing.xl,
  },
  paperText: { fontSize: 12, color: colors.text, lineHeight: 17, fontFamily: "monospace" },
  pageBreak: { alignItems: "center", marginVertical: spacing.md },
  pageBreakLine: { width: "100%", borderTopWidth: 1.5, borderTopColor: colors.pageLine, borderStyle: "dashed" },
  pageBreakLabel: { fontSize: 11, color: colors.pageLine, fontWeight: "600", marginVertical: 4 },
  issues: { marginTop: spacing.sm },
  issueText: { fontSize: 13, color: colors.textMuted, marginBottom: spacing.xs, lineHeight: 18 },
  footer: {
    padding: spacing.lg,
    borderTopWidth: 1,
    borderTopColor: colors.border,
    backgroundColor: colors.surface,
  },
});
```

144. mobile\src\theme.ts

```
export const colors = {
  background: "#F5F7FA",
  surface: "#FFFFFF",
  surfaceMuted: "#EEF1F6",
  border: "#D7DDE5",
  text: "#1B2330",
  textMuted: "#5C6775",
  primary: "#2563EB",
  primaryDark: "#1D4ED8",
  success: "#16A34A",
  warning: "#D97706",
  danger: "#DC2626",
  pageLine: "#DC2626",
};

export const spacing = {
  xs: 4,
  sm: 8,
  md: 12,
  lg: 16,
  xl: 24,
};

export const radius = {
  sm: 6,
  md: 10,
  lg: 16,
};

export const fontSize = {
  xs: 12,
  sm: 14,
  md: 16,
  lg: 20,
  xl: 26,
};

export const font = {
  regular: "System",
  medium: "System",
  semibold: "System",
  bold: "System",
};
```

145. mobile\src\types.ts

```
export interface ContactInfo {
  fullName: string;
  email: string;
  phone: string;
  location: string;
  linkedin?: string | null;
  website?: string | null;
}

export interface WorkExperience {
  id: string;
  company: string;
  position: string;
  location: string;
  startDate: string;
  endDate: string;
  current: boolean;
  bullets: string[];
}

export interface Education {
  id: string;
  institution: string;
  degree: string;
  field: string;
  startDate: string;
  endDate: string;
  gpa?: string | null;
}

export interface Skill {
  id: string;
  name: string;
}

export interface Project {
  id: string;
  name: string;
  description: string;
  url?: string | null;
  technologies: string[];
}

export interface Certification {
  id: string;
  name: string;
  issuer: string;
  date: string;
  url?: string | null;
}

export interface ResumeData {
  contact: ContactInfo;
  summary: string;
  experience: WorkExperience[];
  education: Education[];
  skills: Skill[];
  projects: Project[];
  certifications: Certification[];
}

export interface ResumeMeta {
  id: string;
  title: string;
  targetRole?: string | null;
  template: string;
  parentId?: string | null;
  score?: number | null;
  data: ResumeData | null;
  createdAt?: string | null;
  updatedAt?: string | null;
}

export interface PassivePhraseFlag {
  phrase: string;
  suggestion: string;
  position: number;
}

export interface ImproveBulletResult {
  original: string;
  improved: string;
  usedModel: string;
  flags: PassivePhraseFlag[];
}

export interface GenerateSummaryResult {
  summary: string;
  usedModel: string;
}
```

```ts
export interface SuggestSkillsResult {
  skills: string[];
  usedModel: string;
}

export interface TailorResult {
  tailored: ResumeData;
  guardrailWarnings: string[];
  usedModel: string;
}

export interface SkillMatch {
  name: string;
  matched: boolean;
  category: string;
}

export interface JDAnalysis {
  matchedSkills: SkillMatch[];
  missingSkills: SkillMatch[];
  matchPercentage: number;
  keywords: string[];
  suggestions: string[];
}

export interface ATSIssue {
  id: string;
  severity: string;
  category: string;
  message: string;
  fix: string;
}

export interface ATSAnalysis {
  score: number;
  issues: ATSIssue[];
  passedChecks: string[];
  summary: string;
}

export interface InsertKeywordsResult extends ResumeMeta {
  insertedKeywords: string[];
  message: string;
}

export function emptyResumeData(): ResumeData {
  return {
    contact: {
      fullName: "",
      email: "",
      phone: "",
      location: "",
      linkedin: null,
      website: null,
    },
    summary: "",
    experience: [],
    education: [],
    skills: [],
    projects: [],
    certifications: [],
  };
}

export function nextId(prefix = "id"): string {
  return `${prefix}-${Math.random().toString(36).slice(2, 8)}${Date.now().toString(36).slice(-4)}`;
}
```

146. mobile\src\userStore.ts

```
import { File, Paths } from "expo-file-system";

let cached: string | null = null;

function generateUserId(): string {
  return `device-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 10)}`;
}

function readFile(): string | null {
  try {
    const file = new File(Paths.document, "user.json");
    if (!file.exists) return null;
    const raw = file.textSync();
    return raw ? (JSON.parse(raw).userId as string) : null;
  } catch {
    return null;
  }
}

function writeFile(userId: string): void {
  try {
    const file = new File(Paths.document, "user.json");
    file.create({ overwrite: true });
    file.write(JSON.stringify({ userId }));
  } catch {
    // Persistence is best-effort; the in-memory id still works for the session.
  }
}

export async function getUserId(): Promise<string> {
  if (cached) return cached;
  cached = readFile();
  if (!cached) {
    cached = generateUserId();
    writeFile(cached);
  }
  return cached;
}

export function resetCachedUserId(): void {
  cached = null;
}
```

147. mobile\tsconfig.json

```
{
  "extends": "expo/tsconfig.base",
  "compilerOptions": {
    "strict": true
  }
}
```

148. README.md

````
# AI Resume Builder with ATS Optimization

A full-stack web app to build professional, ATS-optimized resumes with AI-powered content assistance. Build or upload a resume, get scored
against ATS rules, match against job descriptions, and export to PDF or DOCX.

## Project Structure

```
AI RESUME BUILDER WITH ATS/
├── backend/     # Python FastAPI server (all business logic + API)
├── mobile/      # Expo / React Native app (iOS + Android, EAS builds)
└── database/    # Supabase PostgreSQL: local dev compose + seed + docs
```

> A legacy `frontend/` (Next.js) also exists in this repo but is no longer the
> maintained UI — the Expo app in `mobile/` is the current frontend.

| Layer       | Folder       | Technology                                    |
|-------------|--------------|-----------------------------------------------|
| Mobile      | `mobile/`    | Expo/React Native, TypeScript, EAS Build      |
| Backend     | `backend/`   | Python FastAPI, OpenRouter API (free models)  |
| Database    | `database/`  | Supabase PostgreSQL (SQLAlchemy async)        |
| PDF Export  | `backend/`   | WeasyPrint / Playwright (Chromium)            |
| DOCX Export | `backend/`   | python-docx                                   |
| Parsing     | `backend/`   | pdfplumber (PDF), python-docx (DOCX)          |

## Getting Started

### 1. Database (Supabase PostgreSQL)

Required only for the server-side Save/Load feature. Set `DATABASE_URL` to your
Supabase connection string (see `database/README.md`). Everything else works
without it — the backend falls back to a local SQLite file.

For local development, either use the bundled Docker Postgres (same engine as
Supabase) or skip the database entirely:

```bash
cd database
docker compose up -d   # local Postgres + pgAdmin (Supabase stand-in)
```

### 2. Backend (Python FastAPI)

```bash
cd backend
pip install -r requirements.txt
```

Create `backend/.env` (or copy `backend/.env.example`) and set your keys:

```
DATABASE_URL=sqlite+aiosqlite:///./resume_builder.db   # local; see database/README.md for Supabase
OPENROUTER_API_KEY=sk-or-...
```

Run the API server:

```bash
uvicorn main:app --reload --port 8000
```

API docs at [http://localhost:8000/docs](http://localhost:8000/docs)

### 3. Mobile app (Expo / EAS)

```bash
cd mobile
npm install
cp .env.example .env      # set EXPO_PUBLIC_API_URL to your backend
npx expo start            # dev in Expo Go
# or build a binary:
npx eas build --profile development
```

### 4. Tests

```bash
cd backend
python -m pytest tests -q
```

## Features

- **Resume Builder** â€” Tabbed form with live preview, 3 distinct ATS-safe templates (modern / classic / minimal)
- **AI Content Assistant** (needs a FREE OpenRouter key from [openrouter.ai/keys](https://openrouter.ai/keys) — free models need $0 balance)
  - Generate / polish your professional summary
  - Improve any experience bullet with the wand icon
  - Suggest skills for a target role
  - Tailor your whole resume to a pasted job description (AI Tailor button)
````

- **ATS Score Analyzer** â€” Upload PDF/DOCX, get a 0â€“100 score with itemized fixes and real layout checks (table / multi-column detection)
- **JD Matcher** â€” Paste a job description, extract keywords via AI, see matched/missing skills; load your built resume text with one click
- **Save / Load** â€” Auto-saves locally on-device; server-side save/load/delete/clone for your saved resumes when `DATABASE_URL` points at Supabase (or the local SQLite fallback)
- **Export** â€” PDF (ATS-clean HTML rendering) or DOCX, shared from the mobile app

API Endpoints

Method	Path	Description
POST	`/api/resume/parse`	Upload PDF/DOCX â†’ structured text + layout analysis
POST	`/api/ats/score`	Rule-based ATS scoring on structured resume data
POST	`/api/ai/analyze-jd`	Extract JD keywords + match against resume text
POST	`/api/ai/improve-bullet`	Rewrite bullet point with AI
POST	`/api/ai/generate-summary`	Generate professional summary
POST	`/api/ai/suggest-skills`	Suggest skills for a role
POST	`/api/ai/tailor-resume`	Rewrite resume for a JD
POST	`/api/export/pdf`	Export resume as PDF
POST	`/api/export/docx`	Export resume as DOCX
GET	`/api/health`	Health check
POST	`/api/resumes`	Save resume (needs DATABASE_URL)
GET	`/api/resumes?userId=`	List saved resumes
GET	`/api/resumes/{id}`	Get one saved resume
PUT	`/api/resumes/{id}`	Update saved resume
DELETE	`/api/resumes/{id}`	Delete saved resume

149. skills-lock.json


```
{
  "version": 1,
  "skills": {
    "prisma-cli": {
      "source": "prisma/skills",
      "sourceType": "github",
      "skillPath": "prisma-cli/SKILL.md",
      "computedHash": "f41a5af7f25a686631a0e2a8b7031f3c57e0e06b81932f6389736d930d05fa54"
    },
    "prisma-client-api": {
      "source": "prisma/skills",
      "sourceType": "github",
      "skillPath": "prisma-client-api/SKILL.md",
      "computedHash": "7825288518e0722b497de03367a1aee477fc61b0b488d456b9b6257294c005ae"
    },
    "prisma-compute": {
      "source": "prisma/skills",
      "sourceType": "github",
      "skillPath": "prisma-compute/SKILL.md",
      "computedHash": "b2e1d4b2fda6f523b2eb4a38e25c12b7800e4a2801a97b750e95df8131addec0"
    },
    "prisma-database-setup": {
      "source": "prisma/skills",
      "sourceType": "github",
      "skillPath": "prisma-database-setup/SKILL.md",
      "computedHash": "0911d9454bd48df3badd23a50fd90a280eabb15dcbd472c14e7b729075dadefb"
    },
    "prisma-driver-adapter-implementation": {
      "source": "prisma/skills",
      "sourceType": "github",
      "skillPath": "prisma-driver-adapter-implementation/SKILL.md",
      "computedHash": "86034220752575ac0551aa3d579ed0bdfb90616000ae5703bf11e47b655ec7d1"
    },
    "prisma-mongodb-upgrade": {
      "source": "prisma/skills",
      "sourceType": "github",
      "skillPath": "prisma-mongodb-upgrade/SKILL.md",
      "computedHash": "f9ba440e88ca4cec9801d04762296e29e99ca08cb8a8156e4f783ecf90279c8f"
    },
    "prisma-postgres": {
      "source": "prisma/skills",
      "sourceType": "github",
      "skillPath": "prisma-postgres/SKILL.md",
      "computedHash": "66dc44132bdb85cfa369e21a8512c6427c574f5186c8631d345aca9b111091c1"
    },
    "prisma-postgres-setup": {
      "source": "prisma/skills",
      "sourceType": "github",
      "skillPath": "prisma-postgres-setup/SKILL.md",
      "computedHash": "c89d3aa91285d8e4964fcaa5238c99a3d3c20b617e032e731cf632330a85a5a6"
    },
    "prisma-upgrade-v7": {
      "source": "prisma/skills",
      "sourceType": "github",
      "skillPath": "prisma-upgrade-v7/SKILL.md",
      "computedHash": "dcc6c71adca6b22f37c5bda5ac7fd63c3bb59495596a22e06a29c7c85371558f"
    }
  }
}
```